Python 程序设计基础

同济大学计算机基础教研室

2026 年 3 月

目　录

第一章　Python 语言概述 1
1.1 程序与程序设计语言 1
1.2 为什么要学习程序设计 4
1.3 Python 语言简介 5
1.4 简单的 Python 程序 7
1.5 Python 开发环境 9
1.5.1 IDLE 简介 9
1.5.2 Anaconda 简介 11
1.5.3 第三方库 12
1.6 程序调试与获取帮助 15
1.6.1 程序调试 15
1.6.2 获取帮助 17
习题 19
第 2 章 语言基础 21
2.1 数据类型 21
2.1.1 引例—计算地球质量的问题 21
2.1.2 内置数据类型 22
2.1.3 常量、变量和标识符 23
2.2 数值数据的表示和处理 24
2.2.1 数值数据常量 24
2.2.2 算术表达式 25
2.2.3 常用数值类函数 28
2.3 字符串类型和处理 31
2.3.1 字符串类型 31
2.3.2 字符串的处理 32
2.4 布尔类型和处理 36
2.4.1 关系运算 37
2.4.2 逻辑运算 37
2.5 组合数据类型简介 39
2.5.1 列表和元组 39
2.5.2 字典和集合 40
2.6 数据的输入和输出 42
2.6.1 数据的输入 42
2.6.2 数据的输出 43
2.6.3 格式化输出 43
2.7 综合应用 45
习题 47
第 3 章 基本的控制结构 52
3.1 顺序结构 52
3.1.1 引例—人民币与美元兑换 52
3.1.2 简单程序的结构 53
3.2 选择结构 53

3.2.1 引例－体质指数 BMI 的计算.....54
3.2.2 选择结构语句.....55
3.2.3 选择结构的嵌套.....58
3.3 循环结构.....59
3.3.1 引例－批量数据的处理.....59
3.3.2 for 语句.....60
3.3.3 while 语句.....63
3.3.4 循环结构的嵌套.....65
3.3.5 循环结构的其他语句.....66
3.4 综合应用.....69
习题.....72
第四章 函数.....77
4.1 函数定义与调用.....77
4.1.1 引例-多边形面积的计算.....77
4.1.2 函数的定义和调用.....78
4.2 参数及传递.....81
4.2.1 位置参数及默认值参数.....81
4.2.2 可变长参数.....82
4.2.3 形参对实参的影响.....85
4.2.4 变量的作用域.....86
4.3 lambda 函数.....88
4.4 递归函数.....88
4.4.1 什么是递归.....89
4.4.2 递归函数的设计.....90
4.5 综合应用.....93
习题.....99
第五章 列表、元组、字典与集合.....103
5.1 列表与元组.....103
5.1.1 引例-学生成绩计算.....103
5.1.2 列表操作.....104
5.1.3 列表对象方法.....107
5.1.4 操作列表的内置函数.....111
5.1.5 列表推导式.....114
5.1.6 元组.....116
5.2 字典与集合.....117
5.2.1 字典的创建.....117
5.2.2 字典项操作.....118
5.2.3 集合运算.....120
5.3 综合应用.....121
习题.....128
第六章 文件.....131
6.1 概述.....131
6.1.1 引例——统计学生成绩.....131
6.1.2 文件和文件路径.....132

6.1.3 文件分类.....133
6.1.4 文本文件编码.....134
6.1.5 文件访问流程.....135
6.2 文本文件.....136
6.2.1 文本文件的打开与关闭.....136
6.2.2 文件文件的读写.....137
6.2.4 异常处理.....139
6.2.4 CSV 格式.....141
6.3 jieba 中文分词.....142
6.4 图像文件.....144
6.4.1 图像基本处理.....145
6.4.2 图像的滤波和增强.....149
6.5 综合应用.....150
习题.....153
第七章 面向对象程序设计基础.....159
7.1 面向对象程序设计概述.....159
7.1.1 面向对象的基本概念.....159
7.1.2 类的基本特征.....160
7.2 类和对象.....161
7.2.1 类的定义.....161
7.2.2 构造函数.....163
7.2.3 类的实例及其成员的访问.....164
7.3 继承和派生.....165
7.4 迭代器和生成器.....168
7.4.1 迭代器.....168
7.4.2 生成器.....169
7.5 模块、库、包.....170
7.6 综合应用.....171
习题.....172
第 8 章 数据可视化.....175
8.1 Python 图表绘制基础.....175
8.1.1 引例—销售量柱状图.....175
8.1.2 Matplotlib 绘图基础.....176
8.1.3 图表绘制函数.....181
8.2 二维图表.....183
8.2.1 绘制线条.....183
8.2.2 绘制饼图.....188
8.2.3 绘制散点图.....189
8.2.4 绘制直方图.....190
8.3 三维图表.....192
8.3.1 三维图表基础.....192
8.3.2 绘制三维图表.....194
8.4 综合应用.....197
习题.....202

实验一　Python 程序设计基础 205
实验二　顺序结构 206
实验三　选择结构 208
实验四　循环结构 210
实验五　函数 213
实验六　列表、元组、字典、集合 215
实验七　文件 216
实验八　面向对象程序设计基础 217
实验九　数据可视化 218

第一章　Python 语言概述

计算机是通过执行程序完成各种各样的任务，而程序是用程序设计语言编写的。自 1954 年 FORTRAN 语言诞生以来，程序设计语言姹紫嫣红、繁花似锦，据统计发布了上千种语言，Python 是其中最华美绚丽的的一朵。

1.1 程序与程序设计语言

1．程序

计算机系统之所以能完成各种各样的工作，是因为有了程序，那么什么是程序？程序又是如何设计的？

在日常生活中，大家都知道做任何事情要有个先后次序，这些按一定的顺序安排的工作被称为程序。例如，某学校颁奖大会的程序：

① 主持人宣布颁奖会开始，介绍出席颁奖会的领导
② 校长讲话
③ 领导宣布获奖名单
④ 领导颁奖
⑤ 获奖代表发言
⑥ 主持人宣布大会结束

可见，程序的概念是很普遍的，例如用汉语撰写的菜谱、会议的日程等。随着计算机的出现和普及后，程序这一概念成了计算机的专用名词，用于描述了计算机解决问题的过程。

计算机程序是为了解决某个特定问题使用某种程序设计语言编写的一组指令序列。例如，用 Python 编写的计算机角度转换弧度程序如下：

```
import math                                  #导入 math 模块
angle=float(input("请输入角度："))           #输入角度，并且转换为 float 类型
radian=math.pi/180*angle                     #引用 math 库中的 pi，计算弧度
print("弧度=",radian)                        #输出弧度
```

尽管上述程序非常简单，但是反映了程序的基本运算模式：数据输入、数据处理、数据输出，如图 1.1.1 所示。不管是小程序还是大程序，一般都遵循这种朴素的基本运算模式，称为 IPO 模式。

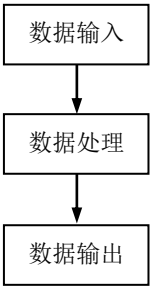


图 1.1.1　程序的基本运算模式

（1）数据输入

数据输入是一个程序的开始。一个程序的数据输入途径有：控制台输入、随机产生、图形用户界面输入、文件输入等。

（2）数据处理

数据处理是一个程序最重要的部分，处理的方法被称为算法。

（3）数据输出

数据输出是程序不可或缺的部分，没有输出的程序是没有意义的。数据输出有控制台输出，图形用户界面输出、文件输出等。

2．程序设计语言

自然语言是人与人之间进行交流的语言，如汉语、英语、日语等。迄今为止，计算机技术以及人工智能的发展还没有达到人与计算机之间完全使用自然语言进行交流的水平。

程序设计语言是人与计算机进行交流的语言。利用程序设计语言编写计算机程序实现人与计算机的交流。若没有程序设计语言，则计算机几乎没有任何价值。

（1）程序设计语言的分类

发布的程序设计语言有上千种，最常用的不过十多种。按照程序设计语言发展的过程，程序设计语言大概分为三类：机器语言、汇编语言和高级语言。

① 机器语言

机器语言是由 0 和 1 二进制代码按一定规则组成的、能被机器直接理解和执行的指令集合。如某型机器上由机器语编写的计算 15+10 的程序：10110000 00001111 00101100 00001010。

机器语言编写的程序像“天书”，程序设计成了一件复杂而只能由专家完成的工作，现在已经没有人用机器语言直接编程了。

② 汇编语言

为了克服上述机器语言缺点，人们用英文助记符来表示机器指令。例如，计算 15+10 的汇编语言程序如下：

```
MOV     A，15       ：把 15 放入累加器 A 中
ADD     A，10       ：10 与累加器 A 中相加
```

用英文助记符表示的指令称为汇编指令，汇编指令的集合称为汇编语言，用汇编语言编写的程序称为汇编语言程序。汇编语言程序不能直接在计算机上运行，需要将其翻译成为机器语言程序才能执行。如图 1.1.2 所示。



图.1.1.2　汇编语言程序翻译成机器语言程序

汇编语言在一定程度上克服了机器语言难读、难改的缺点，但是还没有从根本解决问题，这推动了高级语言的出现。

③ 高级语言

为了从根本上改变上述缺陷，使计算机语言更接近于自然语言并力求使语言脱离具体机器，达到程序可移植的目的，在 20 世纪 50 年代出现了高级语言。高级语言是一种接近自然语言和数学公式的程序设计语言。高级语言之所以“高级”，就是因为它使程序员可以不用与计算机的硬件打交道，可以不必了解机器的指令系统，这样，程序员就可以集中精力来解决问题本身而不必受机器制约，极大地提高了编程的效率。图 1.1.3 表示了上述三类语言在机

器和人的自然语言之间的紧密关系程度，通常机器语言和汇编语言称为低级语言。从计算机技术和程序设计语言发展的角度来看，程序设计语言的目标是让计算机直接理解人的自然语言，不需要计算机语言，但这个过程是漫长的。

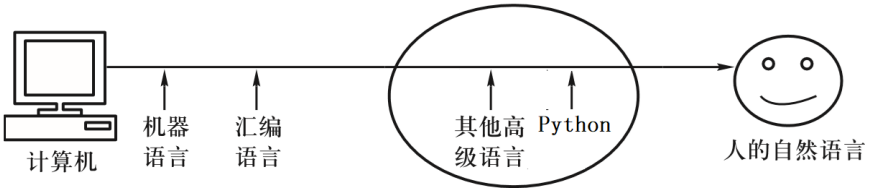


图 1.1.3　程序设计语言在机器和人的自然语言之间的紧密关系程度

（2）解释方式和编译方式

高级语言翻译程序是将高级语言编写的源程序翻译成目标程序的工具。翻译程序有两种工作方式：解释方式和编译方式，相应的翻译工具也分别称为解释程序和编译程序。

① 解释方式

解释方式是对源程序进行逐句解释执行，即翻译一句立即执行一句，如同“同声翻译”。解释方式不生成目标程序，其工作过程如图 1.1.4 所示。采用解释方式的有 Python、Visual Basic 等。

② 编译方式

编译方式是将整个程序翻译成目标程序，这种方式如同“笔译”。目标程序还不能直接执行，因为程序中还会调用一些其他语言编写的程序或库函数，所有这些程序通过连接程序将目标程序和有关的程序库组合成一个完整的可执行程序。整个过程如图 1.1.5 所示。编译方式执行速度快，但每次修改源程序，都必须重新编译。大多数高级语言都是采用编译方式，如 C/C++、Java 等。

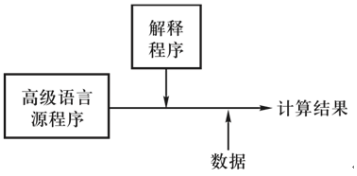


图 1.1.4　解释方式的工作过程

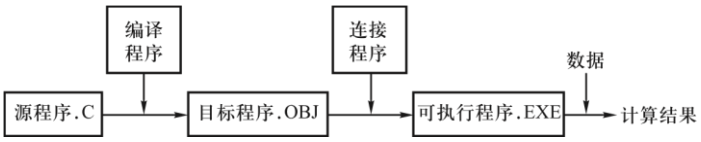


图 1.1.5　C 语言编译方式的工作过程

（3）面向过程语言与面向对象语言

从描述客观系统来看，程序设计语言可以分为面向过程语言和面向对象语言。

① 面向过程程序设计语言

以“数据结构+算法”程序设计范式构成的程序设计语言，称为面向过程语言。典型的有 BASIC、C 语言等。

③ 面向对象程序设计语言

面向对象程序设计语言是一类以对象作为基本程序结构单位的程序设计语言，设计的程序是以对象为核心，在程序运行时对象是基本成分。

面向对象程序设计语言提供了对象、类、方法、实例、变量、消息、派生类和继承等成分。

面向对象程序设计语言在目前属于主流，典型的有 Python、C++、Visual Basic、Java 等。

1.2 为什么要学习程序设计

程序设计又称编程，是计算机为解决某个问题而使用某种程序设计语言编写程序代码的过程。近年来，随着人工智能、大数据等新一代信息技术融合到各个领域，以 Python 为代表的高级程序设计语言越来越火热，越来越多的学校开设了相应的课程，甚至编程也将纳入中小学课程的设置。那么为什么从前相对比较小众的编程得到了如此强烈的追捧呢？作为大学生又为什么要学习编程呢？

1．培养逻辑思维能力

程序设计与逻辑思维息息相关。我国著名数理逻辑学家莫绍揆曾经指出：“事实上，它们（程序设计）或者就是数理逻辑，或者是用计算机语言书写的数理逻辑，或者是数理逻辑在计算机中的应用。”荷兰著名计算机科学家 E.W.Dijkstra 曾感慨地说：“搞了这么多年软件，错误不知犯了多少，现在觉悟了。我想，假如我早年在数理逻辑上好好下点功夫的话，我就不会犯这么多的错误。”在使用程序设计语言进行程序设计时要把各种情况、各种可能性丝毫不漏地考虑到，防止在计算过程中出现错误，即使出现错误了也能很快地发现并及时改正，计算机才能依此运行，如稍有错误，轻则得到毫无意义的错误结果，重则产生严重的后果。由此可见，必须有足够的逻辑训练，熟悉推理过程，才能从事程序设计。

学习程序设计能反过来培养逻辑思维。程序设计的工作就是让计算机按照人的意愿去做事。在完成一件复杂的事情时，需要不断的分解、组装，这个过程中，只有自己的逻辑思维清楚了，计算机才能正确的实现你的命令。在计算机的世界中，一切都遵循着非 0 即 1 的基本原则，它教会我们当一个错误出现时，不要试图不加修正，重新执行一次程序，我们就会得到正确的结果，让我们不再心存侥幸，直面问题。

2．提高计算思维能力

学习程序设计可以提高计算思维能力。美国计算机科学家周以真认为，“计算思维是运用计算机科学的基础概念进行问题求解、系统设计、以及人类行为理解等涵盖计算机科学之广度的一系列思维活动。计算思维的本质是抽象和自动化。”计算思维的根本目的是解决问题，而解决问题的核心是算法设计，这也是程序设计的主要任务。尽管计算思维不仅仅是程

序设计，但是计算思维最终需要程序设计去实现。显然，学习程序设计时中，可以潜移默化提高计算思维能力。

3．提高综合素质

近年来，STEAM 开始颠覆传统的教育。STEAM 由科学（Science）、技术（Technology）、工程（Engineering）、艺术（Arts）、数学（Mathematical）五个部分组成。STEAM 的主要理念就是通过多学科融合，培养人全方位的能力，充分挖掘人自己未知的潜力。通过学习程序设计就可以培养上面所有的能力。程序设计首先是一门工程学，把创意想法变成真实的作品需要学习科学技术，按照工程学的方法论规划实施，同样的，优秀的软件也离不开优雅的艺术设计，最后，如果我们想进一步探索计算机的奥秘，当我们深入其基本原理时，我们又必须提升自己的数学能力。

4．提高专业能力

学习程序设计还可以提高专业能力。一方面，各专业与新一代信息技术深化融合，在知识结构上不断完善和拓展，特别是随着大数据技术逐渐开始落地应用，对于数据分析能力的要求也会越来越高，而掌握一定的程序设计技术对于提高数据分析能力是具有较大帮助的；另一方面，人工智能技术走进日常办公领域产生了智慧办公，智慧办公不仅能够降低工作难度，同时也能够提升办公效率，要想充分利用智慧办公系统，就需要具备一定的程序设计能力。学习程序设计不仅能提高自己的专业能力，并且也可以能更快的适应未来的生活和工作环境。

1.3 Python 语言简介

1．Python 的发展

Python 是由荷兰 Guido van Rossum 设计的解释性的、高级的、通用程序设计语言。

1989 年，Guido 为了开发一个新的脚本解释程序，在 ABC 语言基础上，吸收 Modula-3 优秀思想，结合了 Unix shell 和 C 的习惯而设计了 Python，1991 年发布第一版本。之所以取名 Python，是因为他所挚爱的电视喜剧《Monty Python's Flying Circus》。

2000 年 10 月，Python 2.0 发布。它修复了许多缺陷和错误，开始了广泛应用。

2008 年 12 月，Python 3.0 发布。它打破向后兼容性进行了革命性的修改，让 Python 向着最好的程序设计语言迈出的坚实一步。

目前，Python 的两个系列版本 2.X 和 3.X 均在使用，但是 3.X 版本是主流。Python 2.X 的最后一个版本是 2.7，2020 年将被终止支持。

2．Python 的特点

Python 有 4 个显著的优点：

（1）简单易学

Python 程序不需要类型说明，简化了 python 的语法，摈弃了 C 语言非常复杂的指针，特别容易学习，特别适合非计算机类专业的人士，是一个面向应用的程序设计语言。

（2）简洁优美

Python 代码简洁、优美。Python 采用缩进来标识代码块，通过减少无用的大括号，去除语句末尾的分号等视觉杂乱，使得代码的可读性显著提高，也提高了开发效率。

（3）自由／开源软件

Python 是自由／开源软件。也就是说，可以自由地发布它的拷贝、阅读它的源代码、对它做改动、把它的一部分用于新的自由软件中。

（4）丰富强大的库

Python 不仅标准库庞大，而且还拥有大量第三方库的支持。例如经典的科学计算库：NumPy、SciPy 和 Matplotlib 以及著名的计算机视觉库 OpenCV、三维可视化库 VTK、医学图像处理库 ITK。

与 C/C++相比，Python 不可忽视的缺点运行速度慢。

3．应用领域广泛

（1）常规软件开发

Python 支持面向过程编程和面向对象编程，能够承担任何种类软件的开发工作，因此常规的软件开发、脚本编写、网络编程等可以使用 Python。

（2）科学计算

随着 NumPy，SciPy，Matplotlib，Enthoughtlibrarys 等众多程序库的开发，Python 越来越适合于做科学计算、绘制高质量的 2D 和 3D 图像。和科学计算领域最流行的商业软件 Matlab 相比，Python 是一门通用的程序设计语言，比 Matlab 所采用的脚本语言的应用范围更广泛，有更多的程序库的支持。

（3）自动化运维

这几乎是 Python 应用的自留地，作为运维工程师首选的编程语言，Python 在自动化运维方面已经深入人心，比如 Saltstack 和 Ansible 都是大名鼎鼎的自动化平台。

（4）云计算

开源云计算解决方案 OpenStack 就是基于 Python 开发的，搞云计算的同学都懂的。

（5）WEB 开发

基于 Python 的 Web 开发框架不要太多，比如耳熟能详的 Django，还有 Tornado，Flask。其中的 Python+Django 架构，应用范围非常广，开发速度非常快，学习门槛也很低，能够帮助你快速的搭建起可用的 WEB 服务。

（6）网络爬虫

网络爬虫是大数据行业获取数据的核心工具。没有网络爬虫自动地、不分昼夜地、高智能地在互联网上爬取免费的数据，许多大数据公司将无法生存。能够编写网络爬虫的编程语言有不少，但 Python 绝对是其中的主流之一，其 Scrapy 爬虫框架应用非常广泛。

（7）数据分析

在大量数据的基础上，结合科学计算、机器学习等技术，对数据进行清洗、去重、规格化和针对性的分析是大数据行业的基石。Python 是数据分析的主流语言之一。

（8）人工智能

Python 在人工智能大范畴领域内的机器学习、神经网络、深度学习等方面都是主流的编程语言，得到广泛的支持和应用。

1.4 简单的 Python 程序

下面通过一个简单的实例，说明 Python 程序的基本组成。

【例 1.1】角度转换弧度程序。输入角度，求弧度。

程序：

```
import math                                  #导入 math 模块
angle=float(input("请输入角度："))           #输入角度，并且转换为 float 类型
radian=math.pi/180*angle                     #引用 math 库中的 pi，计算弧度
print("弧度=",radian)                        #输出弧度
```

这是一个简单的 Python 源程序，运行结果如图 1.1.6 所示。在图中，数据 45 是用户从键盘输入的，而结果 0.7853981633974483 是程序输出。

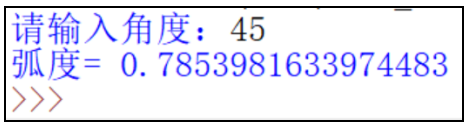


图 1.1.6　程序运行结果

说明：

① 一行中以“#”开始的内容称为注释。它的作用是对程序进行说明，提高程序的可读性。

② 语句 import math 用于导入 math 模块，因为下面的代码需要使用 math 模块中定义的符号常量 pi，代表 3.141592653589793。

③ input 函数用于读入角度，读到的数据是字符串类型，使用 float 函数转换成 float 类型。

④ 语句 radian=math.pi/180*angle 的作用是使用 math 中的符号常量 pi 根据公式计算弧度，并且赋值给变量 radian。

⑤ 语句 print("弧度=",radian)输出计算结果，输出格式为：

弧度=XXX.XXXXXXX

【例 1.2】 求一元二次方程 $ax^2+bx+c=0$ 的根。

程序：

```
import math                                  #导入 math 模块
a=float(input("请输入系数 a："))              #输入系数 a，转换成 float 类型，并且赋值给变量 a
b=float(input("请输入系数 b："))              #输入系数 b，转换成 float 类型，并且赋值给变量 b
c=float(input("请输入系数 c："))              #输入系数 c，转换成 float 类型，并且赋值给变量 c
delta=b*b-4*a*c                              #计算 b2-4ac，并且赋值给变量 delta
if delta>=0:                                 #判断 delta 是否≥0，若是则执行下面缩进的语句
    root1=(-b+math.sqrt(delta))/(2*a)        #计算第 1 个根，并且赋值给变量 root1
    root2=(-b-math.sqrt(delta))/(2*a)        #计算第 2 个根，并且赋值给变量 root2
    print("root1=",root1,"\nroot2=",root2)   #输出两个根
else:
    print("方程没有实根。")                   #若 delta≥0 不成立，若是输出“方程没有实根。”
```

程序运行结果如图 1.1.7 所示。其中 1、-3 和 3 是用户输入的数据。

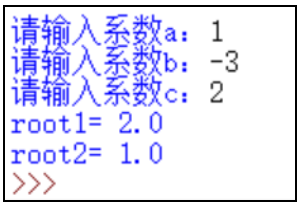


图 1.1.7　程序运行结果

说明：

① 若 delta>=0 成立，则执行下面缩进的三条语句。语句增加缩进表示语句块的开始，减少缩进表示语句块的结束，相同的缩进表示语句处于同一个级别。若后面的三条语句不缩进表示与 if 语句同级别，语法上会出错。

② 若 delta>=0 不成立，则执行 else 下面缩进的 print("方程没有实根。")语句。

③ sqrt 函数是 math 模块中的求平方根函数。

从上面两个实例，可以看到 Python 程序的组成及书写规则为：

（1）Python 程序由一个或多个语句组成，从上到下按次序执行。

（2）注释是从“#”开始到本行结束的内容。除了增加程序的可读性之外，它没有其它任何作用。若是多行注释，则应用三个单引号“''”括起来。例如，

```
'''
这是一个验证程序，用于验证用多行注释命令
文件名为 Test.py
'''
print("Hello,World!")
```

（3）一般情况下，一行只写一个语句。若一行太长需要换行，则应在换行处加上“\”表示换行。一行写多个语句时，语句之间“;”分隔。

（4）缩进是 Python 简洁的特征之一。虽然可以缩进，但也应按编辑器的指示规范缩进，而不能任意缩进。

（5）Python 中大小写字母是有区别的，这一点初学者要特别注意。

1.5 Python 开发环境

在 Windows 下，Python 的常用开发环境（Integrated Development Environment，简称 IDE）有五个：

① IDLE　Python 内置的一个简洁的集成开发环境，使用第三方库时需要安装；

② Anaconda　严格来说，Anaconda 是一个开源项目，其中的 Spder 才是集成开发环境。由于使用 Anaconda 主要是使用 Spder，所以常常将 Spder 等同于 Anaconda。Anaconda 下已经集成了大量常用的第三方库，一般情况不需要安装新的第三方库。

③ Jupyter Notebook　基于网页的用于交互计算的应用程序，可以随 Anaconda 一起安装。可以网页的形式打开、编写和运行代码，并且运行结果也会直接在代码块下显示。

④ PyCharm　由 JetBrains 开发的 Python IDE，支持 Django 和 Google App Engine。

⑤ VSCode　是 Visual Studio Code 的简称，微软开发的现代 Web 和云应用的跨平台源代码编辑器，在安装相应的扩展包后可具备强大的编程开发能力。

本书推荐使用 IDLE 和 Anaconda。

1.5.1 IDLE 简介

IDLE 是一个 Python 自带的集成开发环境。安装 Python 时，IDLE 会自动安装。

1．IDLE 下载

直接在官网（https://www.python.org/）选择合适的版本下载安装包，下载完成后根据提示直接安装。图 1.1.8 是在 Python 官网主页中鼠标指向“Downloads丨Windows”后看到的界面，在其中可以下载最新版本的 Python for Windows。

下载安装包时需要考虑三个因素：一、操作系统是 Windows、Mac OS X 还是其他操作系统；二、下载 32 位还是 64 位的 Python。若操作系统是 32 位的，则 Python 只能下载 32 位的，否则下载 32 位和 64 位的都可以；三、Python 版本号，并不是最新版本是最好的，某些情况下旧的版本可能更适合自己的操作系统。



图 1.1.8　Python 官网“Downloads”主页

安装时应选定“Add Python 3.7 to PATH”复选框，如图 1.1.9 所示，将 Python 的安装目录路径添加到环境变量 PATH 中，方便以后安装第三方库。安装后在 Windows“开始”菜单中出现了 Python 程序组，如图 1.1.10 所示。选择程序组中“IDLE”命令就进入了开发环境。

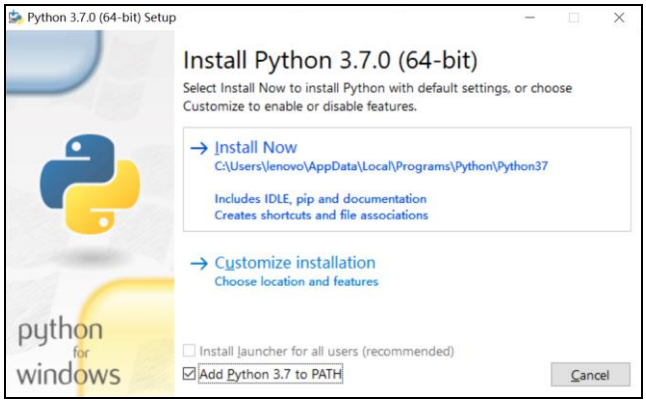


图 1.1.9　Python 安装界面

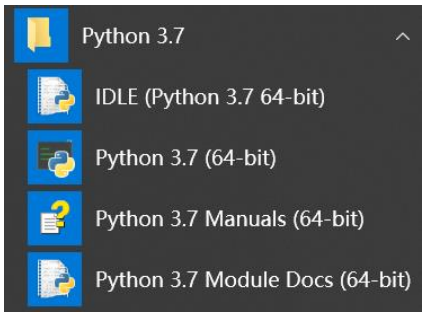


图 1.1.10　“开始”菜单中的 Python 程序组

2．运行方式

进入 IDLE 开发环境后就可以编辑和运行 Python 程序。有两种运行方式：Shell 交互方式和文件直接执行方式。

① Shell 交互方式

Shell 实质是一种交互式的命令解释器。它的执行过程为先输入一条命令，解释器解释并执行这条命令，执行完毕就显示相应的结果。例如，图 1.1.11 是输入“print("同学，你好！")”命令后，Python 解释器执行并显示结果：同学，你好！。

需要注意的是“>>>”是命令提示符。

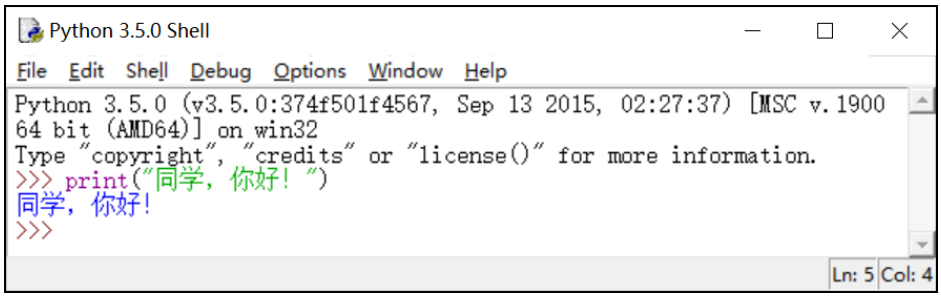


图 1.1.11　Shell 交互方式

② 文件执行方式

这种方式是创建一个以.py 为扩展名的 Python 程序文件，Python 解释器执行后在 Shell 窗口中显示结果。

在 Shell 中选择“File｜New File”命令，出现如图 1.1.12 所示的文件编辑窗口，在其中输入程序代码，然后执行“Run｜Run Module”命令，则程序执行结果如图 1.1.13 所示。

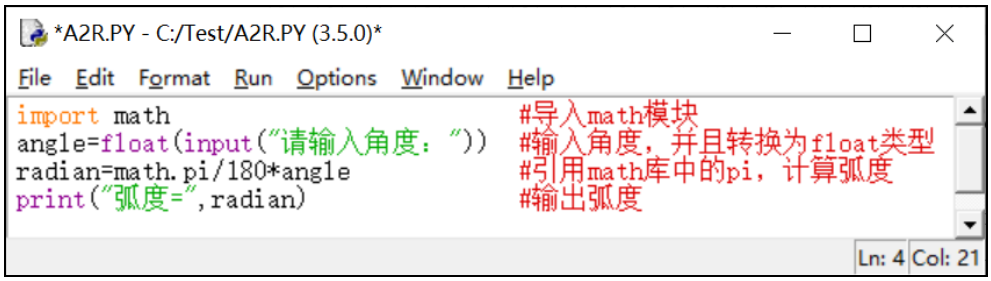


图 1. 1. 12　文件编辑窗口

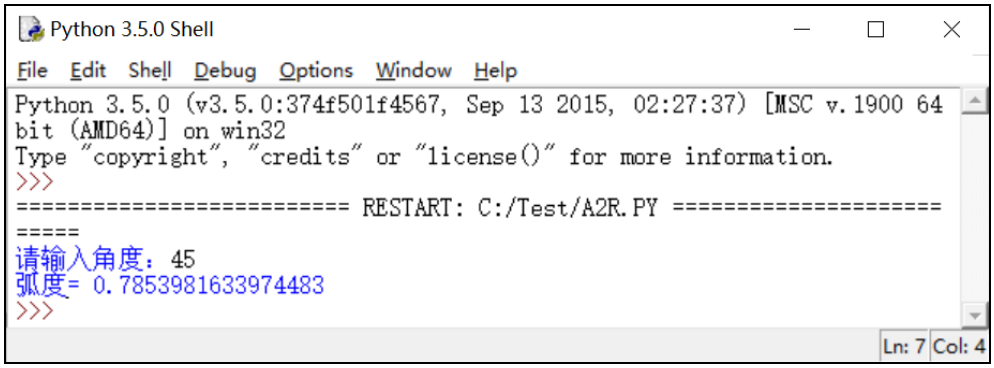


图 1.1.13　Shell 窗口中显示程序文件执行结果

在程序执行开始前，会提示保存程序文件。数据输入也是在 Shell 窗口中进行的。

1.5.2 Anaconda 简介

Anaconda 集成了大量常用的第三方库，安装后一般不再需要特别去安装，避免了安装第三方包的麻烦，深受用户欢迎。

1．Anaconda 的安装

直接在官网（https://www.anaconda.com/）选择合适的版本下载 Anaconda 安装包，下载完成后根据提示直接安装，安装后在 Windows 菜单中出现如图 1.1.14 所示的程序组，其中 Spyder 就是 Anaconda 的集成开发环境。



图 1.1.14　“开始”菜单中的 Anaconda 程序组

2．执行方式

选择“开始”菜单中的“Anaconda | Spyder”命令后出现如图所示 1.1.15 所示的界面，在其中就可以编辑、执行程序。

Anaconda 程序的执行方式也分为两种：一、Shell 执行方式。窗口左下角的 Console 窗口类似 IDLE 的 Shell，在其中可以一条一条输入、运行命令。命令提示符不是“>>>”，而是 In[*n*]，其中“*n*”是一个数字，代表命令的序号；二是文件执行方式。首先在左边的编辑窗口中输入程序，然后执行“Run | Run”命令（或者直接选择工具栏上的“Run file”按钮，图标为“ ▶ ”），就可以执行程序了。

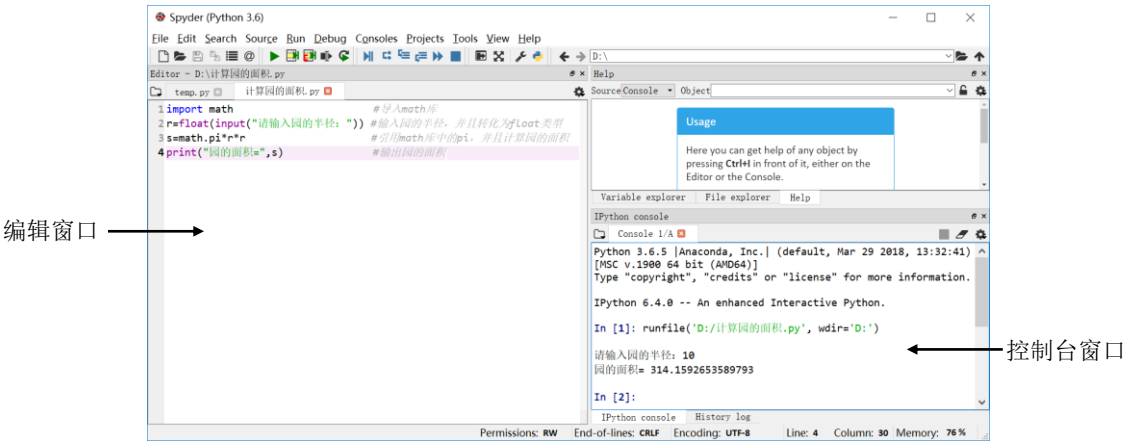


图 1.1.15　Spyder 界面

1.5.3 第三方库

Python 除了拥有随 Python 解释器一起安装的标准库之外，还拥有丰富的第三方库，这是 Python 功能强大而深受欢迎的原因之一。

1．第三方库的安装

Python 的第三方库在使用之前必须安装。一般来说，有两种安装方法：直接安装和下载 whl 文件后安装。

（1）使用 pip 命令直接安装

使用 pip 命令直接安装，语法格式如下：

```
pip install jieba                                    #假定需要安装的第三方库为 jieba
```

判断是否已经安装的方法是执行 import 命令尝试导入，若没有显示错误信息则表示已经安装成功。例如，下面的信息表示 jieba 没有安装成功。

```
>>> import jieba
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    import jieba
ModuleNotFoundError: No module named 'jieba'
```

1）IDLE 中安装第三方库

由于 IDLE 仅仅安装了标准库，若要使用第三方库则一定要安装。假如 Python 安装时

选定了“Add Python 3.7 to PATH”复选框已经将安装目录路径添加到了环境变量 PATH 中，则安装步骤为：

① 进入命令提示符界面；

② 执行安装命令，如图 1.1.16 所示。

```
C:\Users\lenovo>pip install jieba                    #假定需要安装的第三方库为 jieba
```



图 1.1.16　安装第三方库（选定“Add Python 3.7 to PATH”情况下）

假如 Python 安装时没有选定“Add Python 3.7 to PATH”复选框，则安装有点麻烦。具体过程是：

① 设法确定 Python 的安装目录。假定 Python 的安装目录是 C:\Python37，pip.exe 在 Python37 文件夹下 Scripts 子目录中。

② 进入命令提示符界面；

③ 使用命令进入 C:\Python37\Scripts 目录：

```
C:\Users\lenovo>CD C:\Python37\Scripts
```

④ 执行命令，如图 1.1.17 所示。

```
C:\ Python35\Scripts>pip install jieba                    #假定需要安装的第三方库为 jieba
```



图 1.1.17　安装第三方库（没有选定“Add Python 3.7 to PATH”情况下）

需要注意的是：有可能因为 pip 版本太低而安装失败，此时应先升级，升级的具体命令：

```
python -m pip install --upgrade pip
```

pip 升级命令在安装失败时会提示，只要仔细观察即可。

2）Anaconda 中安装第三方库

由于 Anaconda 已经集成了常用的第三方库，一般不需要安装。若要安装特殊的库，则执行“开始”菜单中的“Anaconda | Anaconda Prompt”命令后，然后运行 pip 命令，如图 1.1.18 所示。

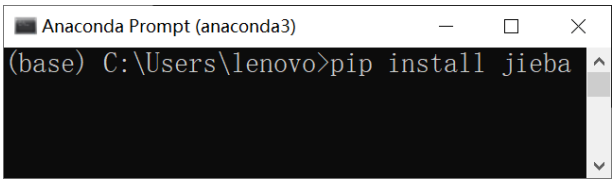


图 1.1.18　Anaconda 中安装第三方库

（2）安装 whl 文件

许多 Python 第三方库是以 whl 文件的形式提供的，此时就需要下载 whl 文件后再安装，具体步骤如下：

① 下载 whl 文件。

whl 文件主要有两个网站：

◆ pypi 网站：https://pypi.python.org/pypi/

◆ 国内 whl 集合网：https://www.lfd.uci.edu/~gohlke/pythonlibs/

② 若是 Anaconda 则，执行“Anaconda Command Prompt”；若是 IDLE，则进入命令提示符界面（路径添加到环境变量 Path 中）。

③ 执行 pip 安装 whl 文件，使用形式为

```
pip  install  whl 文件名
```

例如，若有 MyLib.whl 文件放在 D:\Test 盘跟目录下，则命令为：

```
pip  install  D:\Test\MyLib.whl
```

2．常用的第三方库

对 Python 初学者来说，常用的第三方库如表 1.1.1 所示。

表 1.1.1　常用的第三方库

第三方库	说　明
Jieba	中文分词库
wordcloud	词云图生成器
PIL	图像处理库
Matplotlib	数据可视化，可绘制各种图形
Numpy	数值计算库
Pandas	数据分析包
Scikit-learn	机器学习库
TensorFlow	深度学习库
win32com	有关 Windows 系统操作、Office（Word、Excel、Access 等）文件读写等的综合应用库

3．查看已安装的第三方库

有时需要查看 Python 环境中安装了什么库，常用方法是在在命令提示符界面中执行命令：

```
C:\Users\lenovo>pip list
```

结果如图 1.1.19 所示。

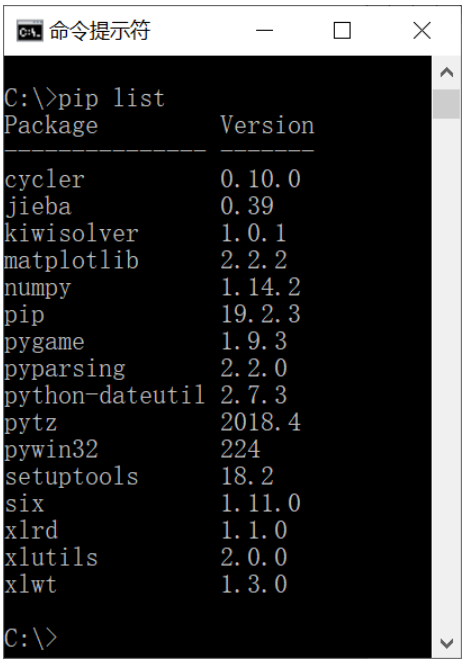


图 1.1.19　查看已安装的第三方库

1.6 程序调试与获取帮助

1.6.1 程序调试

在程序设计过程中，调试是一个不可缺少的环节。俗话说，“三分编程七分调试”，说明调试的工作要比编程大得多。

程序的错误可以分为两种：一、语法错误，如变量名大小写错误，解释器可以发现，用户可以根据提示信息修改；二、逻辑错误，如将 c=a+b 写成 c=a-b，解释器发现不了，用户往往需要通过设置断点，跟踪程序的运行过程，才能发现错误。

1．语法错误

程序若有语法错误，系统则会在输出窗口中显示错误信息。错误信息的形式为：

File "文件名", line 行号, in <module>
错误代码
错误类型: 错误内容

例如，

```
File "C:/Test/A2R.PY", line 1, in <module>
import Math                          #导入 math 模块
ModuleNotFoundError: No module named 'Math'
```

表示在“C:/Test/A2R.PY”文件的第 1 行处有一个 ModuleNotFoundError 错误，没有 Math 模块。因为 Python 中大小写是敏感的，即大小写字母是不同的，因此 math 不能写成 Math。

常见错误有：

① 变量没有定义

```
x=2019                  # 此处 x 是小写
print(X)                # 此处 X 是大写，与 x 是不同的，导致出现变量没有定义错误
```

错误信息为：NameError: name 'X' is not defined

② 没有指定引用库

```
import math
angle=float(input("请输入角度："))
radian= pi/180*angle
print("弧度=",radian)
```

错误信息为：NameError: name 'pi' is not defined，实质是没有指定引用库。

③ 没有圆括号

```
    print "弧度=",radian            #输出弧度
```

④ 没有导入模块错误

```
x=sqrt(5)               # 使用 sqrt 之前没有导入 math 模块
print(x)
```

⑤ 数据类型错误

```
x= input("请 x 的值：")
y=x*x                   # 通过 input 输入的数据是一个字符串，应转换成数值类型
print(y)
```

此错误信息为：TypeError: can't multiply sequence by non-int of type 'str'

修改语法错误时要注意：

① 有时候系统并不能很准确地定位错误所在的位置，这是因为代码中的某个错误，在解释器看来实际是另外一种错误。一般在定位到的位置如果没有发现错误的话，这个错误应该就在附近。

② 一个错误可能会产生若干条错误信息，但是第一条信息最能反映错误的位置和类型，所以务必先根据第一条错误信息进行修改，修改后立即进行执行是否还有错误信息。即使有多个错误，也要一个个地修改，即修改一处执行一次。

2．逻辑错误

一般来说，逻辑错误很难发现，常用的有效方法三种：添加 print 语句、设置断点、单步运行。下面 Anaconda 的调试方法。

（1）添加 print 语句

在怀疑可能是错误的前后，添加 print 语句输出变量或表达式的值，根据输出情况发现错误。

（2）设置断点

设置断点并让程序运行到该断点，然后在“Variable explorer”窗口观察各变量的值，从中发现错误。

① 设置断点方法

在需要设置断点的行首双击设置断点，或者首先将光标移动到需要设置断点的行上，然后选择“Debug丨Set/Clear breakpoint”命令。断点设置后，行首有一个实心圆点。

② 运行到断点

选择“Debug丨Debug”命令或者选择按钮，进入调试状态，标志是 Console 窗口出现“ipdb>”提示符。

在“ipdb>”下可以使用下列命令：

- c 命令：程序执行到下一个断点
- q 命令：退出调试
- !(python 语句)：在 pdb 提示符下执行 python 语句，可以用来查看变量值或者给变量临时指定值

（3）单步执行

程序进入调试状态后，可以单步执行语句。单步运行命令如图 1.1.20 所示。

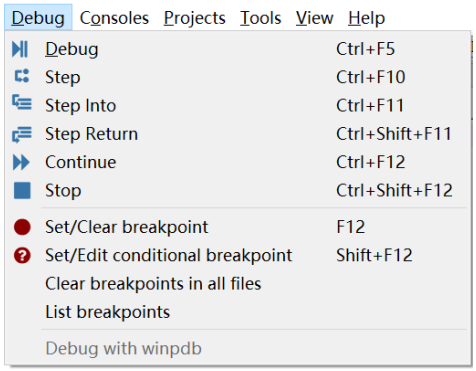


图 1.1.20　Debug 菜单

Debug：开始调试

Step：开始单步执行

Step Info：单步运行进入函数体内

Step Return：单步运行跳出函数体

Continue：继续

Stop：停止调试

一般来说，逻辑错误解释器很难发现的，但对程序来说很可能是致命的。为了避免这样的错误，这就要求编写代码的时候要遵循一定的规范，并且要对程序做足够的测试工作。

1.6.2 获取帮助

程序设计时，即使是高级程序员也会遇到问题，并且需要得到帮助。Python 中获取帮助可以有两个途径：一、访问 https://docs.python.org，它提供了各个版本、各种语言的帮助文档；二、Python 自带了一个内置的帮助系统，通过 help、dir、__doc__获得帮助。

1．help 函数

通过 help 函数可以获对象的信息，其使用形式为

```
help(对象)
```

例如，如果要了解 sqrt 的功能和用法，则执行下面的语句：

```
>>> import math                    # 导入 math 模块后才能查看 math 对象的帮助
```

```
>>>help(math.sqrt)
Help on built-in function sqrt in module math:
sqrt(x, /)                                    # “/”表示函数参数 x 不可以是负的
    Return the square root of x.
```

2．dir 函数

dir 函数可以用来查看对象的所有属性及方法，其使用形式为：

```
dir(对象)
```

说明：在 python 中任何东西都是对象，一种数据类型，一个模块等，都有自己的属性和方法，都可以通过 dir 函数查。例如，如果要了解 math 中所有的函数和常数，则执行下面的语句。

```
>>> import math
>>> dir(math)
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan',
'atan2', 'atanh', 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs',
'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan',
'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'nextafter', 'perm', 'pi', 'pow',
'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
```

3．__doc__

Python 中有一个特色——文档字符串（DocStrings），是指模块、类、函数中等添加的说明性文字，可以通过 Python 自带的标准方法——__doc__将这些信息输出。其使用方法为

```
对象.__doc__
```

例如，如果得到关于 math 的描述性文字信息，则执行下面的语句：

```
>>> import math
>>> math.__doc__
'This module provides access to the mathematical functions\ndefined by the C standard.'
```

注意：当不是函数、方法、模块等调用 doc 时，而是具体对象调用时，会显示此对象从属的类型的构造函数的文档字符串。

习题

一、选择题

1．在下列程序设计语言中，不属于面向对象程序设计语言的是_____。

A．Python　　B．VB.NET

C．C　　D．Java

2．在下列程序设计语言中，属于解释执行的_____。

A．Python　　B．C++

C．C　　D．Java

3．Python 自带的开发环境是_____。

A．PyCharm　　B．TensorFlow

C．Anaconda　　D．IDLE

4．在下列程序设计语言中，属于自由/开源软件（Free/Libre and Open Source Software，简称 FLOSS）的是_____。

A．C/C++　　B．Python

C．VB.NET　　D．Java

5．执行下列程序时输入 1 和 2，则输出是_____。

```
a=input()
b=input()
c=a+b
print(c)
```

A．3　　B．12

C．4950　　D．99

6．下列不属于调试手段的_____。

A．谨慎地设置输出语句　　B．设置断点

C．单步执行　　D．查阅文献

二、填空题

1．能被机器直接理解和执行的语言是_____。

2．程序的基本运算模式：数据输入、数据处理、数据输出，这种朴素的运算模式被称为_____。

3．_____是 Python 内置的一个简洁的集成开发环境。

4．选定“Add Python 3.7 to PATH”复选框，可以将 Python 的安装目录路径添加到环境变量_____中。

5．第三方库最常见的安装命令是_____。

6．Python 功能强大而深受欢迎的原因之一是拥有_____、第三方库和自定义模块。

7．查看已安装的第三方库的完整命令是_____。

8．程序的错误可以分为语法错误和_____。

三、问答题

1．Python 有什么特点？

2．什么是开源软件？什么是自由软件？

3．Python 程序的书写格式有什么特点？

4．请简述 IDLE 和 Anaconda 两个编程环境的主要区别。

5．请查阅有关资料，说明自己所在学科常用的 Python 库有哪些？

6．程序错误分成哪几类？在 Python 中如何调试程序？

第 2 章 语言基础

计算机程序是通过计算来解决实际问题的。程序是用程序设计语言所描述的解决问题的方法和步骤。从组成上讲，程序包括数据和对数据的操作两部分。数据是程序加工处理的对象，操作则反映了对数据的处理方法，体现了程序的功能。计算机能处理的数据是多种多样的，程序设计语言定义了不同的数据类型，规定了各种类型数据的存储和所能进行的处理。

本章介绍 Python 语言提供的数据类型、各类型数据的表示方法和运算规则等语言基础知识。

2.1 数据类型

2.1.1 引例—计算地球质量的问题

【例 2.1】 利用计算机来解决初等数学问题。编写一个计算地球质量的程序，地球体积的计算公式为：$V=\frac{4}{3}\pi r^3$，地球半径 r 为 6356.91 千米，平均密度为每立方米 5.52 吨。要求地球半径 r 从键盘输入，地球密度为常量。

程序如下：

```
r=input("输入地球半径 r：")
v=4/3*3.14159*(r*1000)**3                    # 体积，单位为立方米
w=v*5.52                                     # 求得地球质量，单位为吨
print("地球质量 w=",w)
```

程序运行和输入：

```
Traceback (most recent call last):
  File "D:/py/2-1.py", line 2, in <module>
    v=4/3*3.14159*(r*1000)**3                # 体积，单位为立方米
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int
'>>>
```

程序第 2 行有错误，原因是变量 r 在第 1 行通过 input 函数输入后获得的是字符串类型“6356.91”，字符串类型是不能进行乘方（**）运算。

程序代码若修改为：

```
r=int(input("输入地球半径 r："))          # int()函数将输入的数字字符串转换成整数类型
```

程序运行输入 6356，能有运行结果。但程序运行若输入 6356.91，则出现如下错误信息：

```
输入地球半径 r：6536.91
Traceback (most recent call last):
  File "D:/py/2-1.py", line 1, in <module>
    r=int(input("输入地球半径 r："))
ValueError: invalid literal for int() with base 10: '6539.91'
```

原因是输入的 6356.91，而 int()函数只能将整数形式的字符串转化为整数。

要解决此问题，则要调用 float()函数，即 r=float(input("输入地球半径 r："))，程序运行结果如下：

```
输入地球半径 r：6356.91
地球质量 w= 5.939713652967326e+21
```

说明：对于大数，系统自动以指数形式显示，最后的“e+21”表示 10 的 21 次方；若要保留三位小数，则可用如下格式输出：

```
print("地球质量 w=%5.3e"%(w))        # 以指数形式显示，保留 3 位小数
```

显示：

```
地球重量 w=5.940e+21
```

从上面引例中可看出：在程序设计语言中，对要处理的数据规定数据的类型、所能进行的运算、数据的输入和输出的方式和格式控制等，在本章将作逐一介绍。

2.1.2 内置数据类型

每一种程序设计语言都会预先设置一些数据类型，称为内置数据类型，在程序中可以直接使用。Python 提供的内置数据类型如图 1.2.1 所示，包括数值型（整型、浮点型和复数类型）、字符串型、布尔型等一般语言都具有的数据类型；还提供了列表、元组、字典、集合等 Python 特有的组合数据类型。数据类型可分为两类：基本数据类型和组合数据类型。

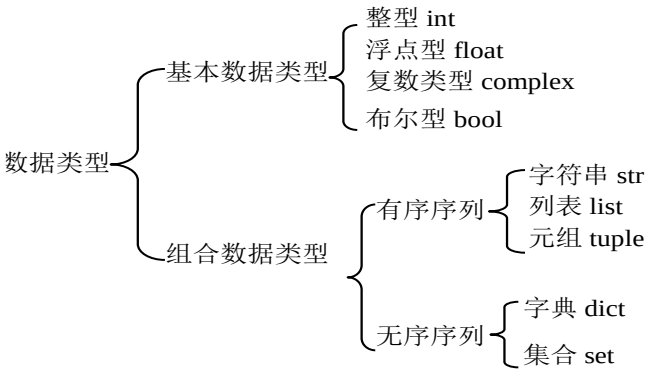


图 1.2.1 Python 的内置数据类型

说明：

（1）有序序列是一维向量，元素之间存在次序关系，可通过索引访问元素。字符串可以方便地表示文本数据，列表和元组可表示数据的序列。

（2）无序序列是指元素之间不存在次序关系，因此不能用索引来访问。集合也可表述数据的组合，但是其中的元素不重复出现；字典使用键-值(key-value)进行存储，查找速度极快，即通过键来获得对应值。

2.1.3 常量、变量和标识符

计算机程序是通过计算来解决实际问题的。计算中涉及的数据在程序中以常量或变量的形式出现。变量的值可以变化，而常量则表示固定不变的数据。如在例 2.1 中求圆体积 v 的公式为：

```
v=4/3*3.14159*(r*1000)**3
```

其中 r、v 是变量，4、3、3.14159 和 1000 是常量。假如当 r 的值为 6356.91 时，v 获得相应的计算结果 1.0760350820592982e+21。

1.常量

常量是在程序运行中固定不变的数据，通过常数值直接反映了其数据类型。例如，123 是整数、123.0 是浮点数、’123’是字符串，[123]是列表等。

2.变量

变量是在程序运行过程中其值可以变化的量。变量具有变量名、类型和值三要素。变量通过变量名访问，变量名必须满足标识符命名规则。在 Python 中，不需要事先声明变量名及类型，通过赋值运算符可以创建任意类型的变量，其数据类型与所赋的值一致，通过 type(变量)函数获得变量的数据类型；不仅变量的值可以改变，变量的数据类型也可改变。

为什么变量的数据类型可以变化？Python 作为面向对象的编程语言，一切都是对象，包括数据类型、变量、函数、方法、类等。Python 中变量并不直接存储数据而是个引用到实际对象的名称，赋值运算只是改变了变量的引用对象，可以使用函数 id(变量)获取引用对象的存储地址。因此 Python 属于动态类型的程序设计语言。

2. 变量

变量是在程序运行过程中其值可以变化的量。变量名必须符合标识符的命名规则。在 Python 中变量不需要事先声明类型就可以使用，而且变量类型是可以变化的。例如：

```
x=10                                # 没有声明直接使用，变量为整型
x="Tongji University"               # 变量从整型改变为字符串类型
```

为什么变量的数据类型可以变化？因为在 Python 中变量与值的关系是引用关系，赋值运算只是改变了变量的引用对象。Python 因为变量类型可以改变所以属于动态类型程序设计语言，如图 1.2.2 所示。

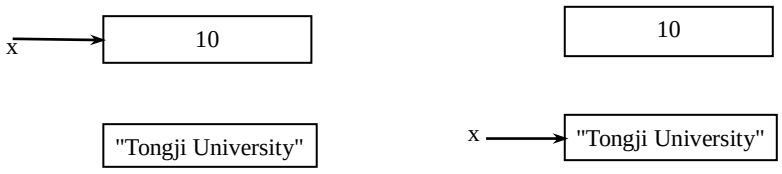


图 1.2.2 Python 动态类型变量示意

3.标识符

标识符是指在程序中为一些要处理的对象起个名字，包括变量名、函数名、类名等。

在 Python 中，标识符的命名规则：以字母或下划线开头，后面可跟字母、数字和下划线的序列，大小写区分。在中文系统中，标识符中任意位置可以出现汉字。

例如：area，score_list，x123、学号、姓名等都是合法的标识符；而 3x，x-y，x y 等是非法的标识符。

注意：在 Python 中大小写区分的，即 x123、X123 是两个不同的变量名；关键字不能用于命名变量；也要避免将 Python 的内置的数据类型、函数名等用作变量名，以免引起混淆和带来不必要的麻烦。

4.关键字

Python 语言保留某些单词用做特殊用途，这些单词被称为关键字，也叫作保留字；在不同的编程开发环境中系统会以不同的颜色标识关键字来突出语法表示。用户定义的标识符（变量名、函数名等）不能与关键字相同，否则在运行时会出现异常。

【例 2.2】显示 Python 中用到的关键字。

要显示 Python 中用到的关键字，必须导入 keyword 库。

```
>>> import keyword                #导入 keyword 库
>>> print(keyword.kwlist)
```

['False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

Python 3.X 拥有 33 个关键字，可以通过 help()命令进入帮助系统查看相关关键字说明。

```
>>> help()                        #进入帮助系统
help> if                          #查看 if 关键字说明，help>是帮助系统提示符
help>quit                         #退出帮助系统
```

2.2 数值数据的表示和处理

2.2.1 数值数据常量

Python 的数值数据包含了整数、浮点数和复数。常量表示形式见表 1.2.1

表 1.2.1　数值类型常量举例

数据类型	表示形式	说明	举例
整型	①十进制：±n ②二进制：0Bn 或 0bn ③八进制：0O n 或 0on ④十六进制：0X n 或 0x n	n 为 0～9 数字组成 0B 或 0b 二进制标记，n 为 0～1 数字组成 0O 或 0o 八进制标记，n 为 0～7 数字组成 0X 或 0x 十六进制标记,n 为 0～9 数字、A～F 或 a～f 字母组成	1234、-101 0B1110100 0O144 0X5B
浮点型	①小数形式 ±m.n ②指数形式：±m[.n]E±k	m、n、k 为 0～9 数字组成 E 或 e 表示指数符号	123.45 0.12345E+3
复数型	实部+虚部 J	实部（real）和虚部（imag）可以是整数或浮点数，J 或 j 是后缀	12.3+45.6J

1.整数

整数是不带小数点和指数符号的数。整数有四种表示形式，分别为：十进制、二进制整数、八进制整数和十六进制整数，除十进制整数外都有标记，如表 1.2.1 所示，常用十进制整数。

例如，123、+123、0B1111011、0O173 和 0X7B 都是合法的整数，后三个分别为二、八、和十六进制整数，并与十进制整数 123 等值。

整数特点数值精确表示；在 Python 中，整数类型没有取值范围的限制。

2.浮点数

浮点数又称实数，有两种表示形式，分别为：小数形式和指数形式，如表 1.2.1 所示。

例如，123.0、0.123e3、1.23E2、123e0 等都是合法的浮点数，后三个是指数形式并与 123.0 小数形式等值。一般使用小数形式比较直观，而对于特大或特小的浮点数使用指数形式。

要注意的是浮点数特点数值不能精确表示，这样在计算时可能会出现误差。

【例 2.3】浮点数运算精度问题和解决的方法

```
>>> x=1.2
>>> x-1.0
0.19999999999999996
>>> 1.2-1.0==0.2                    # 两浮点数相减判断结果不正确，这是浮点数精度引起的
False
>>> abs(x-1.0-0.2)<1e-5             # 解决的方法是将相等比较改为绝对值之差是否足够小来比较
True
```

3. 复数

Python 提供了复数类型，复数由实部和虚部构成，形式：实部+虚部 j 或 J。通过表 1.2.4 中所列出的 complex(x1[,x2])函数可以将 1 个或者 2 个数值转换成复数类型。

【例 2.4】复数表示。

```
>>> c=12.3+45.6j
>>> c                               # 显示 (12.3+45.6j)
>>> c.real                          # 显示 12.3
>>> c.imag                          # 显示 45.6
>>> c1=complex(10)
>>> c1                              # 显示 (10+0j)
>>> c2=complex(10.0,20.2)
>>> c2                              # 显示 (10+20.2j)
```

2.2.2 算术表达式

要进行各种复杂的运算，就需要各种运算的符号，通过运算符和操作数组合成表达式，实现程序编制中所需的大量操作。

1.算术运算符

在 Python 中，为数值类型提供了通常的算术运算和运算规则，运算符及其含义如表 1.2.2 所示（若已知 a=3）。

表 1.2.2　算术运算符

运算符	含义	优先级	实例	结果	说明
-	负号	1	-a	-3	单目运算
**	幂运算	2	27**(1/3)	3.0	乘方
*	乘	3	a*a*a	27	
/	除	3	10/a	3.3333333333333335	整数相除结果为浮点数
//	整除	3	10//a	3	结果为不大于商的最大整数
%	取余数	3	10 % a	1	也称为模运算
+	加	4	10.0+a	13.0	浮点数与整数运算结果为浮点数
-	减	4	a-10	-7	

说明：

（1）“-”是单目运算，对操作数取负；其余都是双目运算，即运算符旁有两个操作数。

（2）运算优先级表示当表达式中含有多个操作符时，先执行哪个操作符。

（3）与 C 语言不同，Python 中两个整数相除返回的类型是浮点数。如：4/2，结果为 2.0，而“//”是整数相除得整数，5//2 得 2。

（4）整数和浮点数运算，结果类型为浮点数。如：10+5.0 结果为 15.0。

【例 2.5】利用运算符，输入秒数，以小时、分、秒形式显示。

```
x =int(input("输入秒数"))
s = x % 60                    # 求得秒
m = x //60 % 60               # 求得分钟
h = x // 3600                 # 求得小时
print(h,":",m,":",s)
```

程序运行：

```
输入秒数 200000
55:33:20
>>>
```

思考：如何将一个三位数倒置，例如 n=345，倒置后为 543。

2.赋值运算符

程序中的变量在进行具体的运算之前，都需要有明确的值，Python 中对变量的赋值提供了多种方式。

（1）简单赋值

这是一般程序设计语言最常见和常用的格式，即：

变量=表达式

作用：计算右边表达式的值，赋值给左边的变量。

例如：y=x**2+6

注意：赋值号的左边只能是变量，不能是常量、表达式。下面均为错误的赋值语句：

```
>>> sin(x)=x+y          #左边是表达式（函数）
>>> 5=sqrt(s)+x+y       #左边是常量
>>> x+y=z               #左边是表达式
```

（2）复合赋值

Python 中还为上述运算符（表 1.2.2 中除负号单目运算符外）提供了与之对应的复合赋值（或称增强赋值）运算符：+=、-=、*=、/=、//=、%=、**=。复合赋值语句的形式为：

变量 复合赋值运算符 表达式

作用：计算赋值运算符右边表达式的值，然后与左边的变量进行相应的运算，最后赋值给变量。例如：

```
a*=b+3       等价于：a=a*(b+3)
```

复合赋值常用于累加、计数和连乘运算，例如：

①累加 sum += x 等价于 sum = sum+x

②计数 n += 1 等价于 n = n+1

③连乘 t *=x 等价于 t=t*x

（3）链式赋值

链式赋值将同一个值赋给多个变量。例如：

```
x=y=z=10.3                    # x、y、z 三个变量获得相同值 10.3
```

（4）序列解包赋值

序列解包赋值指用同一个赋值号“=”，给不同的变量分别赋值。例如：

```
x,y,z=1,2,3                   # x、y、z 依次获得 1、2、3 的值
```

序列解包还可以方便实现交换变量的值，例如：

```
x,y=y,x                       # 变量 x 与 y 交换值
```

3.数据类型的转换

在算术运算中，如果操作数具有不同的数据类型，则使用隐式或显式进行类型转换。

（1）隐式转换

在源代码中不需要任何特殊语法自动进行，称隐式转换。隐式转换规则是整型向浮点型转换、浮点型向复数类型转换，可避免在转换时造成的误差损失。

（2）显式转换

利用转换函数来实现类型强制转换，数值类型转换函数使用的是 int()、float()和 complex()，见表 1.2.4 所示。

【例 2.6】如下代码显示了隐式转换与显式转换。

```
>>> 3.0+4              # 浮点数与整数运算，整数类型隐式转换成浮点数类型
```

```
7.0
>>> int(7.5)          # 通过 int()函数，将浮点数转换成整数
7
```

2.2.3 常用数值类函数

程序设计语言中的函数如同数学函数的概念，完成某个特定功能并获得某种返回。Python 功能强大之处在于提供了丰富的库函数，以模块的方式，便于管理和使用。

Python 中的函数分为四类：

- 内置函数：内置模块_ _builtins_ _中，可以直接调用；
- 标准库函数：先导入模块再调用函数，常用的标准库有 math 库和 random 库；；
- 第三方库：先安装库再导入模块然后才能调用函数，例如 numy 库等；
- 用户自定义函数：是用户自己编写的函数，这将在第 6 章介绍。

本章主要介绍前两类函数，使用函数提高代码重用和编程效率。函数调用形式：

函数名（参数列表）

注意：可以使用 help (函数名)查看某个函数的详细用法。

1.内置的数值类型函数

内置的数值类型函数见表 1.2.3 所示：

表 1.2.3　内置的数值类型函数

函数	含　义	实　例	结　果
abs(x)	取 x 的绝对值	abs(-3.5)	3.5
divmod(x,y)	结果为一元组：(x//y,x%y)	divmod(10,3)	(3,1)
pow(x,y)	幂运算，即 x 的 y 次幂	pow(3,4)	81
round(x[,n])	对 x 四舍五入，保留 n 位小数 n 可以缺省，表示无小数位。	round(10.4456)	10
max(x_1,x_2,…x_n)	获得所有参数的最大值	max(13,56,89,33,65)	89
min(x_1,x_2,…x_n)	获得所有参数的最小值	min(13,56,89,33,65)	13

2.内置的数据类型转换函数

内置的数据类型转换函数见表 1.2.4 所示。

表 1.2.4　内置的数据类型转换

函数名	描述	实例	结果
int(x)	将 x 浮点数取整，即去除小数部分	int(2.5) int("23")	2 23
float(x)	将 x 转换成浮点数	float(3) float("23.4")	3.0 23.4
complex(x1[,x2])	将 x1、x2 转换成复数 缺省 x2，虚部为 0	complex(3,4) complex(3)	(3+4j) (3+0j)
str(x)	将 x 转换成字符串	str(23.4) str(True)	'23.4' 'True'

bool(x)	将 x 转换成布尔类型。若 x 为数值，非 0 为 True，0 为 False；若 x 为字符串，非空为 True，空为 False。	bool("23.4") bool(0) bool("")	True False False
eval(str)	将字符串 str 当成有效表达式来求值，并返回计算结果。	eval('3+4') eval('True') eval('3,4,5')	7 True (3,4,5)

说明：eval(str)是功能很强的函数，根据其函数的特点功能，可拓展如下常见的应用。

（1）利用该函数的功能，计算表达式的值。

【例 2.7】利用 eval 函数编写通用、简单计算的程序。

```
from math import *
x=eval(input("输入表达式："))
print("计算结果=",x)
```

程序运行：

```
输入表达式：log(10)
计算结果= 2.302585092994046
>>>
输入表达式：sin(3.14/2)
计算结果= 0.9999996829318346
>>>
```

（2）用作类型转换，即将任意字符串智能转化为其合理的对应值。

```
>>> eval('True')                    # True        将字符串转换为布尔类型
>>> eval ('1.2e4')                  # 12000.0     将字符串转换为浮点数
```

（3）多个数值型变量的输入

```
>> x,y,z=eval(input("输入 x,y,z："))
输入 x,y,z：3,4,5
>>> x,y,z
(3, 4, 5)
```

3. math 库

使用数学库函数，需导入 math 模块。

【例 2.8】输入角度，显示对应的正弦值和余弦值。

```
import math
x=int(input("输入角度:"))
y= math.sin(x*math.pi/180)              # 输入的是角度，函数要求是弧度
print("x=",x,"sin(x)=",y)
```

程序运行：

```
输入角度:30
x= 30 sin(x)= 0.49999999999999994
```

注意：若用 from math import * 方式导入 math 模块，则调用时可省略模块名，即：

```
y=sin(x*pi/180)
```

常用的数学函数和常数如表 1.2.5 所示。

表 1.2.5　常用的数学函数和常数

函数	含　义	实　例	结　果
pi	常数 π	pi	3.141592653589793
e	常数 e	e	2.718281828459045
cos(x)	返回 x 弧度的余弦值	cos(0)	1.0
exp(x)	返回以 e 为底的幂，即 e^x	exp(3)	20.085536923187668
gcd(m,n)	返回 m,n 两数的最大公约数	gcd(12,8)	4
log(x)	返回自然对数（以 e 为底）	log(10)	2.302585092994046
log10(x)	返回常用对数（以 10 为底）	log10(10)	1.0
sin(x)	返回 x 弧度的正弦值	sin(0)	0.0
sqrt(x)	求 x 的平方根	sqrt(9)	3.0
tan(x)	返回 x 弧度的正切	tan(0)	0.0

4. random 库

random 库中包含了可以生成不同类型、范围的随机数函数，表 1.2.6 列出常用的随机数生成函数。同样调用 random 库中函数，必须导入 random 模块。

表 1.2.6 常用的随机数生成函数

函数	描述	示例	结果
random()	随机产生[0,1)区间内的一个浮点数	random()	0.6412506684965525
randint(a,b)	随机产生[a,b]区间内的一个整数	randint(65,90)	75
randrange(m,n[,k])	随机产生[m, n)之间以 k 为步长的随机整数，缺省 k 为 1	randrange(1,101,10)	41
sample(seq,k)	从序列 seq 中随机抽取 k 个元素组成列表，k≤len(seq)	sample(range(1,10),5)	[3, 2, 1, 9, 5]

【例 2.9】随机产生 10 个大写字母。

分析：大写字母的编码值区间为 65~90，已知编码值转换成字母的函数是 chr()。

程序如下：

```
from random import *
lst=[]
for i in range(0,10):
    lst+=chr(randint(65,90))
print(lst)
```

运行三次结果：

```
['E', 'O', 'O', 'Y', 'F', 'W', 'M', 'X', 'L', 'N']
['N', 'Z', 'Z', 'C', 'O', 'I', 'V', 'Y', 'M', 'I']
['B', 'V', 'I', 'Z', 'M', 'X', 'S', 'W', 'N', 'M']
```

说明；每次运行程序，产生不同的序列，若要为了程序调试需要，产生相同的序列，在程序的循环结构前加语句：seed(任意整数)，表示种子数不变。则运行三次结果：

```
['E', 'O', 'O', 'Y', 'F', 'W', 'M', 'X', 'L', 'N']
['E', 'O', 'O', 'Y', 'F', 'W', 'M', 'X', 'L', 'N']
['E', 'O', 'O', 'Y', 'F', 'W', 'M', 'X', 'L', 'N']
```

函数形式：seed([n])，作用初始化随机数种子。n 为整数，生成序列固定；缺省参数 n 或参数为 None、非整数，则为当前系统时间，生成序列可变。

2.3 字符串类型和处理

2.3.1 字符串类型

字符串是程序设计中非常常见的数据类型，它是由字符组成的序列。例如，"Python 程序设计"、'2020'等。

Python 中全面支持中文，在统计字符串长度时，中文字符和西文字符都视为一个字符来对待；默认对字符串采用的是 Unicode 编码，任一字符用两字节表示。

1.字符串常量

字符串常量是由字符串定界符括起的字符序列，在 Python 中，字符串定界符由三种表示形式：

（1）用一对单引号包括的单行字符序列。

（2）用一对双引号包括的单行字符序列。

（3）用两边都是 3 个单引号或 3 个双引号包括的多行字符序列。

对于单行字符串用单引号或双引号没有区别，主要是若字符串中包含了双引号，则用一对单引号括起字符串，反之亦然。

例如：'他说:"Python 语言学习很容易!" '、"what's your name? "等都是字符串常量。

2.转义字符

问题的提出：除了可显示的字符外，还有一些控制字符如何表示？若单引号作为定界符，而字符序列中有单引号又如何表示？

例如：'what's your name? '怎么书写？

为表示控制符等一些具有特殊功能的符号，Python 提供了一种称为转义表示的方法：就是以反斜杠加上其它特定字符来表示一个特殊功能符号，称该符号为转义字符。例如，'what\'s your name? '，这里“\'”就是转义字符，表示单引号。

常用的转义字符如表 1.2.7 所示：

表 1.2.7 常用的转义字符

转义字符	描述	转义字符	描述
\\	反斜杠符号	\n	换行
\'	单引号	\r	回车
\"	双引号	\t	横向制表符

说明：

（1）对于“**\t**”横向制表符所占的列数与开发环境有关。例如 IDLE 环境占 8 列、Spyder 占 4 列；所以同样的代码在不同的开发环境输出的效果将不同。

（2）Python 语言还允许在字符串前加 r，即：r"内容"或 r'内容'，表示字符串内的内容不转义。

【例 2.10】 应用举例。在 IDLE 开发环境下，利用字符串常量、转义字符和操作符将学生信息以表格形式输出。

```
print(25*"-")                                    #“*”为对字符串复制，即显示 25 个下划线字符
print("|学号\t|姓名\t|计算机\t| ")  #“\t”跳到下一个制表位（默认一个制表位占 8 列）
print(25*"-")
print("|180001\t|张三\t|优秀\t|")
print(25*"-")
print("|180002\t|李小明\t|良\t|")
print(25*"-")
print("|180003\t|王平平\t|不及格\t|\r")
print(25*"-","\n")                              # 本身 print 结束要换行，加了“\n”实质是空一行
print(r"|180003\t|王平平\t|不及格\t|\r")      #r"字符串"内容不转义，照样输出
```

程序运行如图 1.2.3 所示：

```
-------------------------
|学号    |姓名    |计算机  |
-------------------------
|180001 |张三    |优秀    |
-------------------------
|180002 |李小明 |良      |
-------------------------
|180003 |王平平 |不及格 |
-------------------------

|180003\t|王平平\t|不及格\t|\r
```

图 1.2.3 转义符应用举例

2.3.2 字符串的处理

字符串的处理包括对字符串的基本操作、常用函数和方法。

1.字符串基本操作符

Python 语言提供了 5 个字符串的基本操作，如表 1.2.8 所示。

假定 st1="cde",st2="abcdefg"

表 1.2.8 字符串的基本操作符

操作符	描述	实例	结果
s1+s2	连接两个字符串	st1+st2	'cdeabcdefg'
n*s 或 s*n	对字符串 s 复制 n 次	3*st1 或 st1*3	'cdecdecde'
s1 in s2	子串测试。s1 若是 s2 的子串返回 True,否则 False	st1 in st2	True
s[i]	索引操作,取索引为 **i** 的字符	st2[3] 或 st2[-4]	'd'
s[m:n:k]	切片操作,返回索引号 m 到 n 但不包括 n 的子串；省略 m 表示从头，省略 n 表示取到结束；k 为步长，省略为 1	st2[:-4] st2[:6:2] st2[3:] st2[:]	'abc' 'ace' 'defg' 'abcdefg'

说明：

（1）索引

字符串中的每个字符都是有索引号的，通过索引号和索引操作符“[]”可以获取单个字符。有两种表示索引方式：正向递增索引和反向递减索引，习惯用正向。例如 st2 变量的字符串，正向和方向索引如图 1.2.4 所示。st2[3]与 st2[-4]都表示通过索引获取到字符：'d'。

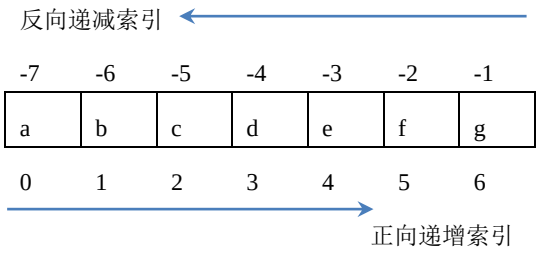


图 1.2.4 序列的正向和反向索引

（2）切片

利用[m:n:k]操作符，可以对字符串任意切片获得所需的子串。

例如：st2[3:]、st2[3:7] 都表示通过切片获取到子串：'defg'；而 st2[:3]结果为：'abc'。

注意：切片操作常用于取子串，一般缺省 k 为 1，取**[m:n)**区间的子串，即左闭右开区间的子串。

例如：st2[:3]结果为'abc'、st2[:6:2]的结果为'ace'、st2[3:4]的结果为'd'、st2[3:3]的结果为"空串。

【例 2.11】提取星期几。已知 s= "星期一星期二星期三星期四星期五星期六星期日"，根据输入的数字 （1-7），输出星期几。

分析：s 字符串是有规律的，每个星期几占 3 位；根据输入数字，确定取的起始位即可

获取对应的星期。

代码如下：

```
s="星期一星期二星期三星期四星期五星期六星期日"
n=int(input("请输入数字 1-7："))
start=(n-1)*3
weekDay=s[start:start+3]
print(weekDay)
```

程序运行：

```
请输入数字 1-7：3
星期三
```

2.内置的字符串函数和方法

（1）字符串函数

Python 提供了一些内置的字符串函数，常用的函数见表 1.2.9 所示：

表 1.2.9 常用内置函数

函数	描述	实例	结果
len(s)	返回 s 字符串长度	len("abcdefg")	7
chr(n)	返回字符编码 **n** 对应的单字符	chr(65)	'A'
ord(s)	返回 s 单字符对应的编码	ord('A')	65

说明：

①在 Python 中，默认对字符串采用的是 unicode 编码的 str 类型来表示，任一字符用两字节表示。

②常用的西文字符：'0'~'9' 编码值为 48~57，'A'~'Z' 编码值为 65~90，'a'~'z' 编码值为 97~122。

【例 2.12】输入一字符串，显示其对应的 Unicode 码，再根据 Unicode 码显示其字符作验证。

分析：利用 len()函数获得输入的字符串长度，然后利用循环结构依次取出每个字符，调用 ord()函数获得 Unicode 码，再调用 chr()函数作验证。

代码如下：

```
s=input("请输入字符串：")
n=len(s)                              # 调用 len()函数求得字符串长度
for i in range(n):                    # 利用循环结构依次取每一个字符
    c_ord=ord(s[i])                   # 调用 ord()函数获得字符对应的编码值
    ord_c=chr(c_ord)                  # 已知编码值调用 chr()函数获得对应的字符
    print(s[i]," ord=",c_ord," ord_c=",ord_c)   # 显示字符、字符对应的编码值和对应字符
```

程序运行如图 1.2.5 所示。

```
请输入字符串：Python程序设计
P   c_ord= 80   ord_c= P
y   c_ord= 121   ord_c= y
t   c_ord= 116   ord_c= t
h   c_ord= 104   ord_c= h
o   c_ord= 111   ord_c= o
n   c_ord= 110   ord_c= n
程   c_ord= 31243   ord_c= 程
序   c_ord= 24207   ord_c= 序
设   c_ord= 35774   ord_c= 设
计   c_ord= 35745   ord_c= 计
```

图 1.2.5 运行效果

从该例看出 ord()函数与 chr()函数是一对反函数，即 chr(ord("p"))="p"，同样 ord(chr(65))=65。

（2）字符串方法

Python 中每一个数据类型都有一个相对应的内置类，每新建一个数据，实际就是初始化并生成一个对象。系统为对象提供了一些方法便于功能处理。

方法引用形式：字符串.方法（参数）

要注意：方法作用于对象，方法引用结果获得一个处理结果，但不改变引用对象。

系统提供了很多方法，在编辑环境中，在字符串变量的后面输入“.”后，稍等会自动弹出字符串对象的所有方法，如图 1.2.6 所示，移动光标选择所需的方法。

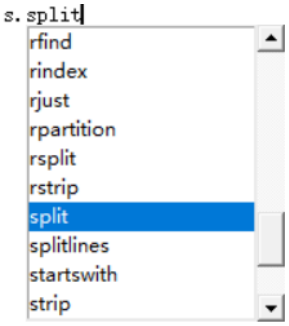


图 1.2.6 字符串对象的方法

常用的方法见表 1.2.10 所示。假定如下举例中均以 s 变量操作：s1="ABC DEFG BC"，“实例”列是对方法的引用，返回结果见最右列，不改变原字符串对象 s 的值。

表 1.2.10 常用内置方法

方法	描述	实例	结果
find(s)	在字符串中找第一个出现 s 子串的位置，找不到返回-1	s1.index("BB")	-1
lower()	全部转换成字母小写	s1.lower()	'abc defg bc'
isnumeric()	字符串全部是数字返回 True，否则 False	s1.isnumeric() "123".isnumeric()	False True
isalpha()	字符串全部是字母返回 True，否则 False	s1.isalpha() "asDF". isalpha()	False True
index(s)	在字符串中找第一个出现 s 子串的位置，找不到抛出异常	s1.index("BC")	1
replace(old,new)	将字符串中用 new 子串替换 old 子串	s1.replace("BC","WYZ")	'AWYZ DEFG WYZ'
split([s])	将字符串按照 s 分隔符分离，返回结果是列表。	若 s2="a 12 34" s2.split()	['a', '12', '34']

方法	描述	实例	结果
	缺省 s 按空格、制表符或换行符分隔	s2.split(" ")	['a', ' ', '12','34']
startswith(s [,start[,end]])	字符串(在可选的范围内)是否是以 s 开头，若有 end，表示不包含 end 的整数	s1.startswith("AB") s1.startswith("BC,1,3")	True True
strip(s)	去掉字符串两旁的 s 子串，一般用于去掉空格	" 123 ".strip()	'123'
upper()	全部字母大写	"abcd".upper()	'ABCD'

说明：对于 split()函数，若以 split(" ")单空格分离，当字符串中连续出现多个空格时，分离的效果比较差。改进的方法为缺省参数，系统就会优化处理，即当连续出现多个空格作为一个空格分离，效果比较好。

【例 2.13】对 s1 字符串引用 replace 方法，运行如下：

```
>>> s1="ABC DEFG BC"
>>> s1.replace ("BC","WYZ")
'AWYZ DEFG WYZ'
 >>> s1
'ABC DEFG BC'
```

【例 2.14】利用 split 方法对输入的时间按小时、分、秒分离。

```
s=input("输入时间：小时：分：秒  ")
hour,minute,second=s.split(":")
print(hour,"小时，",minute,"分，",second,"秒")
```

程序运行：

```
输入时间：小时：分：秒  12:23:54
12 小时，23 分，54 秒
```

2.4 布尔类型和处理

布尔数据类型用于描述判断的结果，具有真和假两个值。在 Python 中用 True 和 False 表示真和假，注意首字母大写其余小写。

以真和假为值的表达式称为布尔表达式，用于表示某种条件成立与否，一般在选择结构和循环结构中用于分支和循环条件判断。

注意：在 Python 中，布尔类型值也可以当作数值来使用。当参与数值表达式计算时，True 自动转化为 1，False 转化为 0。例如：

```
>>> x=3+True
>>> x
4
```

2.4.1 关系运算

关系运算的作用是判断两个表达式值得大小关系，，结果为布尔类型 True 或 False。

关系运算表达式形式：

操作数 1 关系运算符 操作数 2

操作数：可以是数值型和字符串，按其数值或字符串的编码值大小比较。

关系运算符有：==、>、>=、<、<=、!=，分别表示等于、大于、大于等于、小于、小于等于、不等于。

注意：

（1）等号“==”与赋值号“=”的区别

```
>>> x=20          # x 通过赋值号运算获得值 20
>>> x==20         #  根据 x 的值与 20 进行比较是否相等？结果为 True 或 False
```

（2）浮点数的相等比较

计算机中的浮点数有时是近似值存储数据（因浮点数十进制与二进制互转因素），当两个浮点数进行相等比较时结果会不正确，应采用两个数之间的差值小于给定的精度。

【例 2.15】 两浮点数相等比较。

```
>>> 3.5-2.7==0.8
False
>>> 3.5-2.7
0.7999999999999998
>>> abs((3.5-2.7)-0.8)<0.000000001
True
```

（3）字符串的比较

两个字符串进行比较，则按字符的 ASCII 码值从左到右逐一进行比较，即首先比较两个字符串中的第 1 个字符，其 ASCII 码值大的字符串为大；如果第 1 个字符相同，则比较第 2 个字符，以此类推，直到出现不同的字符时为止。

【例 2.16】字符串的比较例。

```
>>> "ABCDE" > "ABR"
False
>>> "BC"<"bc"
True
```

2.4.2 逻辑运算

常用的关系运算只能进行简单的逻辑判断。当判断的条件复杂时，需要逻辑运算符将多个条件进行组合。逻辑表达式通过逻辑运算符，可以将关系表达式、布尔型常量或变量进行逻辑运算，运算结果是 True 或 False。除 not 是单目运算符外（即仅右边有操作数），其余是双目运算符。表 1.2.11 列出 Python 中的常用逻辑运算符。

表 1.2.11 常用逻辑运算符

运算符	含义	优先级	举例	说　　明
not	取反	1	not x	当 x 的值为 False 时，逻辑表达式的值为 True 当 x 的值为 True 时，逻辑表达式的值为 False
and	与	2	x and y	当 x、y 有一个 False 时，逻辑表达式的值为 False 只有当 x、y 均为 True 时，逻辑表达式的值才为 True
or	或	3	x or y	当 x、y 有一个 True，逻辑表达式的值为 True 只有当 x、y 均为 False 时，逻辑表达式的值才为 False

注意：

（1）在 Python 中也支持数值或字符串的表达式参与逻辑运算，规则是对数值型非 0 为 True，0 为 False；对于字符串非空为 True，空为 False。同样，当逻辑量进行算术运算时，True 自动转换成 1，False 转换为 0，然后参与运算。

例如：

```
>>> True+3
4
>>> False+3
3
```

（2）对数学上常用于表示值域范围的表达式，在 Python 的逻辑运算中，则可采用与数学上相同书写的逻辑表达式。

例如，要表示 x 的值域范围 0≤x<10，则简化的逻辑表达式为：0<=x<10，这在其他程序设计语言中是不支持的，必须写成：0<=x and x<10。

【例 2.17】输入成绩，判断该生成绩是否优秀，优秀生成绩为 90~100。

```
>>> mark=int(input("输入成绩："))
输入成绩：93
>>> 90<=mark<=100
True
```

【例 2.18】输入三角形三边，判断其能否构成三角形。

```
>>> a, b, c = eval(input("输入 a,b,c："))
输入 a,b,c：2, 4, 5
>>> a+b>c and b+c>a and a+c>b
True
```

说明：eval 函数将字符串中去掉引号后，其中的内容看作表达式进行计算，其实质是前面赋值运算符中介绍的序列解包赋值。该函数是 Python 中很有特色的函数，作用见表 1.2.4 所示，在以后多变量的输入中常使用。

（3）运算优先级

当一个表达式中，多种不同类型的运算符出现时，运算符优先级如下：

算术运算符>关系运算符>逻辑运算符

当然，算术运算符、逻辑运算符各自都有不同的优先级，关系运算符优先级相同。

注意：对于多种运算符并存的表达式，可增加圆括号，改变计算的顺序也可使表达式更清晰。

【例 2.19】 选拔优秀学生。若选拔优秀生的条件为：年龄(Age)小于 19 岁，三门课总分(Total)高于 285 分且其中有一门为 100 分，如果其表达式写为：

```
age<19 and total >285 and mark1 ==100 or mark2 ==100 or mark3==100
```

有何问题？应如何改正?请读者考虑。

2.5 组合数据类型简介

为了更好地描述和处理客观世界中的成批数据，满足不同的需要，Python 还提供了列表、元组、字典和集合等组合数据类型。本节简要介绍这些数据类型的创建，便于大家建立对 Python 数据类型的整体认识和使用，更详细的用法在第 4 章中介绍。

2.5.1 列表和元组

Python 中的字符串、列表和元组是 Python 中常用的有序类型，它们的每个元素按照位置编号即索引来顺序存取，如同其他程序设计语言的数组。但数组通常只能存储相同数据类型的元素，而列表和元组则可以存储不同类型的元素。

列表和元组在很多操作上是一样的，如索引、切片、各种数据的统计等，不同的是表示形式不同，前者是方括号“[]”，后者是圆括号“()”，更大区别是列表是可修改的，元组是不可修改的，元组可以看成是只读的列表。

1.创建列表

创建列表最简单的最简单的方法是利用列表表示形式直接创建。

【例 2.20】利用列表表示形式直接创建。

```
>>> fibs=[0,1,1,2,3,5,8,13]                    # 建立了一个斐波那契数列列表
>>> stud=[["黄小明",87],["李平",79],["章一",99]]  # 建立由 3 名学生的姓名成绩构成的嵌套列表
>>> num1=["一二三四五六"]                       # 建立了仅有一个元素的列表["一二三四五六"]
>>> nl=[]                                      # 建立空列表
```

2.创建元组

创建元组的方法类似创建列表，当利用元组表示形式直接创建时只要用圆括号(可以缺省)代替方括号。

【例 2.21】直接建立元组。

```
>>> tfibs=(0,1,1,2,3,5,8,13)                    # 建立元组：(0,1,1,2,3,5,8,13)
>>> tfibs=0,1,1,2,3,5,8,13                      # 与上式等价，建立元组：(0,1,1,2,3,5,8,13)
>>> tstud=(("黄小明",87),("李平",79),("章一",99))# 建立由 3 名学生的姓名成绩构成的嵌套元组
>>> tnl=()                                      # 建立空元组
```

注意：当建立的元组仅有一个元素时，要在后面加逗号，不然建立的就不是元组 tuple。

例如：

```
>>> tone=(123,)                    # 是元组：(123,)
>>> tone=(123)                     # 不是元组，是整数 123
```

3.通用序列操作

通用序列操作指的是对有序序列类型包括列表、元组和字符串其操作是一致的。

这些操作符包括索引、切片、序列相加、乘法运算，在字符串类型节已经介绍，如图 1.2.4 和表 1.2.8 所示。

【例 2.22】列表的序列操作。

```
>>> fibs=[0,1,1,2,3,5,8,13]      # 建立 fibs 列表：[0,1,1,2,3,5,8,13]
>>> fibs[4]                      # 通过索引访问列表中元素值：3
>>> fibs+[21]                    # 两个列表加操作结果：[0, 1, 1, 2, 3, 5, 8, 13, 21]
>>> fibs*2                       # 将列表复制 2 次：[0, 1, 1, 2, 3, 5, 8, 13, 0, 1, 1, 2, 3, 5, 8, 13]
>>> fibs[3:5]                    # 通过切片访问列表中元素：[2, 3]
```

4.元组应用场合

元组除了用于表达固定数据外，还常用如下场合，例如多变量同步赋值即序列解包、函数返回多个值等。

说明：把一个序列（列表、元组、字符串等）直接赋给多个变量，此时会把序列中的各个元素依次赋值给每个变量，但是元素的个数必须和变量个数相同，这称为序列解包。

【例 2.23】 元组常用场合示例。

```
>>>x,y,z=(10,20,30)               # 元组序列赋值给多个变量，括号可省
>>>x,y=y,x                        # 元组(y,x)的元素依次赋值给 x,y 变量，实现两数交换
>>>print("x=%d,y=%d"%(x,y))        # 格式输出中的输出元素是元组
>>>def f(x):
        return x**2,x**(1/2)      # 函数返回两个值，默认是元组类型
```

2.5.2 字典和集合

字典和集合都属于无序的数据类型，因此不能像列表和元组通过位置索引来访问数据元素。

1.字典

实际使用的字典工具书以字或词为单位以及含义信息组成，按一定顺序排列，通过字或词获得其含义信息。Python 中的字典与字典工具书类似，由 key-value 键值对组成，通过 key 可以快速地找到其对应的值。

（1）创建字典

通过字典表示形式直接创建，即由大括号“{}”括起、逗号分隔、键值对构成的，键必须唯一。键值对形式为“键:值”。

【例 2.24】建立各星期与其英文缩写的字典。

```
>>> week1={1:'Mon',2:'Tues',3:'Wed',4:'Thur',5:'Fri',6:'Sat',7:'Sun'}        #字典表示形式创建
>>> week1
```

```
{1: 'Mon', 2: 'Tues', 3: 'Wed', 4: 'Thur', 5: 'Fri', 6: 'Sat', 7: 'Sun'}
```

（2）字典应用

Python 中的字典是内置的映射类型，即通过键获得对应的值。利用这样的特点，可以方便地进行字典中元素的检索、插入、删除和统计等操作。

在例 2.24 中，若要显示周三的英文缩写，则语句如下：

```
>>> week1[3]
'Wed'
```

2.集合

集合类型与数学中的集合概念是一致的，其主要作用是消除重复元素、进行集合运算。

（1）.创建集合

通过集合表示形式直接创建，即大括号“{}”括起、逗号分隔的数据项。

【例 2.25】直接建立集合。

```
>>> sfibs={0,1,1,2,3,5,8,13}                    # 建立 sfibs 集合{0, 1, 2, 3, 5, 8, 13}
>>> sh={"hello!"}                               # 直接建立 sh 集合{"hello!"}
>>> sh2=set("hello!")                           # 利用函数建立 sh2 集合{'!', 'h', 'l', 'e', 'o'}
```

注意：字典和集合表示形式都是一对“{}”括起，那么如下语句，到底建立的是空字典还是空集合：

```
>>> week1[3]
'Wed'
>>> s={}
>>> s
{}
>>> type(s)
<class 'dict'>
>>>
```

通过交互方式运行可知，创建的是空字典，若要创建空集合，必须通过 set()函数，如下：

```
>>> s=set()
>>> type (s)
<class 'set'>
>>> s
set()
```

（2）集合应用

【例 2.26】利用集合特点过滤重复数据。

```
>>> score=["计算机","数学","外语","计算机","物理"]        # 建立课程 score 列表
>>> score
['计算机', '数学', '外语', '计算机', '物理']
>>> s=set(score)                                  # 列表转化为集合，过滤掉重复的课程
>>> s
{'数学', '计算机', '外语', '物理'}
```

说明：组合数据类型可以通过类型转换函数进行相互转换，类型转换函数分别为 list()、

tuple()、set()和 dict()。

【例 2.27】已知建立了斐波那契数列列表，分别进行类型转换，代码和执行效果如下：

```
>>> fibs=[0,1,1,2,3,5,8,13]
>>> tuple(fibs)                          # 列表转换成元组
(0, 1, 1, 2, 3, 5, 8, 13)
>>> set(fibs)                            # 列表转换成集合
{0, 1, 2, 3, 5, 8, 13}
>>> dict(fibs)                           # 列表转换成字典出错，银字典每个元素为键值对
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    dict(fibs)
TypeError: cannot convert dictionary update sequence element #0 to a sequence
>>>
```

2.6 数据的输入和输出

程序的一般结构是输入数据，处理数据、输出结果。程序的输入/输出分为两大类：一类是人—机交互，把人们可以识别的形式（字符串、数）进行输入和输出；另一类是程序之间以文件形式数据传送。本节介绍第一类。

2.6.1 数据的输入

Python 语言利用 input()函数实现输入，调用形式为：

input([提示字符串])

说明：input 函数返回的是字符串类型。可通过 int()、float()函数进行所需类型转换；也可通过 eval()函数对多个变量进行输入。

【例 2.28】各种类型数据的输入

```
>>> x=input("输入 x:")                          # x 获得的是字符串类型
输入 x:123
>>> x
'123'
>>> y=int(input("输入 y:"))                     # 通过 int()函数强制类型转换，y 获得的是整数类型
输入 y:123
>>> y
123
>>> n,x,s=eval(input("请输入 n,x,s:"))          # eval()根据输入的值智能转换成相应的类型
请输入 n,x,s:10,67.8,"成绩"
>>> n,x,s
(10, 67.8, '成绩')
```

2.6.2 数据的输出

Python 语言有两种输出方式：表达式方式和 print()函数方式。

1.表达式的方式

在 IDLE 交互状态下，输入表达式可直接输出表达式的值，这种方式常用于数据的检查。例如：

```
>>> x=100
>>> x /3
33.333333333333336
```

2. print()函数方式

使用 print 函数方式可以控制输出的格式，这是常用的方式。形式：

```
print([输出项列表][, sep=分隔符][, end=结束符])
```

print()函数参数全部可以缺省，若没有参数，则输出一个空行。参数说明如下：

- 输出项列表 是以逗号分隔的表达式；
- sep 表示各输出项间的分隔，缺省为空格；
- end 表示输出的结束符，默认值为换行符，也就是 print 默认输出后换行。

【例 2.29】 输出 1~9 的数及平方数。

```
>>> for i in range(1,10):                    # for 是循环结构语句，range(1,10)产生[1~9]间的序列
        print(i,i*i,sep=",",end=";")
1,1;2,4;3,9;4,16;5,25;6,36;7,49;8,64;9,81;
```

若要每行输出一对，用默认方式空格分隔、每对结束换行，输出 3 对，则代码如下：

```
>>> for i in range(1,4):
        print(i,i*i)
1 1
2 4
3 9
```

若要每个数据项占 5 位，如何实现?则涉及到下面介绍的格式化输出。

2.6.3 格式化输出

为了使得输出的结果更美观、规范化，例如保留小数位数、每项输出宽度等，可使用格式化控制输出的格式。Python 语言有两种方式控制格式输出：

（1）“%”方式（如同 C 语言的格式控制符）；

（2）format()方法。

Python 后续的版本中不再使用“%”方式，将主要使用 format()方法。

1. “%”方式

【例 2.30】输入 a、b、c 三个系数，计算一元两次方程的两个实根，保留三位小数。

```
from math import *
a,b,c=eval(input("输入 a,b,c：  "))
d=b*b-4*a*c
if d>=0 :
    x1=(-b+sqrt(d))/(2*a)
    x2=(-b-sqrt(d))/(2*a)
    print("x1=%8.3f\nx2=%8.3f"%(x1,x2))        # %8.3f 浮点数输出宽度 8 位，小数位数 3 位
else:
     print("无实根")
```

程序运行：

```
输入 a,b,c：  2,7,5
x1=∪∪-1.000                                    # ∪表示空一格
x2=∪∪-2.500
```

从上例可以看出，格式化输出时用操作符“%”分隔格式字符串与输出项，即：

格式控制字符串 %（输出项列表）

其中：

①格式控制字符串作为模板，模板中有显示的原样字符（包括转义符）和格式说明符，格式说明符为输出值预留位置；输出序列将多个值依次传递给模板中对应格式说明符。

②格式说明符常用形式如下：

% [m][.n]类型符

其中：m 为输出最小宽度；n 为小数位数；类型符为对应输出数据类型。常用的格式说明符见表 1.2.12 所示：

表 1.2.12 常用的格式说明符

格式说明符	%d	%c	%s	%f 或%F	%e 或%E
类型符	d	c	s	f 或 F	e 或 E
对应输出数据类型	整数	单个字符	字符串	浮点数	指数形式

【例 2.31】字符串格式化操作符%输出应用。

```
已知 x=1234.567，n=1234567，s="pythonOk"，则：
>>>print("x=%6.2fs=%sn=%d"%(x,s,n))
x=1234.57s=pythonOkn=1234567
>>>print("x=%13.5fs=%13sn=%13d s[1]=%c"%(x,s,n,s[1]))
x=∪∪∪1234.56700s=∪∪∪∪∪pythonOkn=∪∪∪∪∪∪1234567∪s[1]=y
```

注意：

（1）当输出值整数位数多于最小宽度，按实际值输出；输出值小数位数多于精度位数，则四舍五入按精度位数输出；输出小数位数少于精度位数，右补 0；输出值位数少于最小宽度，则右对齐左补空格。

（2）当输出项的数据类型与格式符不一致时，若格式符是%s，输出项是非字符串类型，

自动转换为字符串再输出；反之若输出项是字符串，而格式符是非%s，则会产生错误信息。例：

```
>>>print("x=%13.5fa=%13sn=%13d"%(x,n,a))                    #a 字符串变量对应格式符%13d
Traceback (most recent call last):
  File "<pyshell#11>", line 1, in <module>
    print("x=%13.5fa=%13sn=%13d"%(x,n,a))
TypeError: %d format: a number is required, not str
```

2. format 方法

format()方法更清晰地表示对字符串进行格式化处理。形式：

格式模板.format（输出项列表）

格式模板中有显示的原样字符（包括转义符）和用若干个“{}”也称为“槽”作为格式占位符，为输出值预留位置；输出项列表将各个值依次传递给模板对应的“{}”。

“{}”的形式：{[序号:[m.n]类型符]}

其中：

①“{ }”中可以是带格式的串，也可以空；序号与后面的格式控制符之间用“:”分隔；同样 m 为输出最小宽度；n 为小数位数，类型符同表 1.2.12。

②序号为输出项列表的顺序号，第 1 个元素序号为 0，写在{ }中的起始位置；若无序号，则按参数顺序对应。

【例 2.32】format 方法输出应用。

已知 x=1234.567，n=1234567，s="pythonOk"，则：

```
>>> print("x={}s={}n={}".format(x,s,n))                    #最简单的占位符按默认格式输出
x=1234.567s=pythonOkn=1234567
>>> print("x={0:12.2f}s={1:10s}n={2:10d}".format(x,s,n))          #按占位符设置的格式输出
x=∪∪∪∪∪1234.57s=pythonOk∪∪n=∪∪∪1234567
```

说明：从输出的两种方式看，“ %”方式当设置的最小宽度大于输出值的宽度，数值和字符串类型默认均右对齐左补空；而 format()方式默认数值类型右对齐左补空、字符串类型默认左对齐右补空。

2.7 综合应用

本章我们围绕着数据和数据处理进行了介绍，包括数据类型、变量的声明、常量的表示；各种数据类型、运算符、函数和方法等。这些内容都是后面几章学习所必需的基础，对初学者一下子不可能理解和掌握这些内容，通过实践很多问题就能迎刃而解。

【例 2.33】输入姓名，利用输出格式设计制作生日贺卡，程序运行如图 1.2.7 所示：



图 1.2.7 程序运行效果

分析：输出有规律的图案，利用循环结构、字符串复制操作符“*”来实现。程序代码如下：

```
kg="  "
ss="★"
name=input("输入好友的姓名：")
n=len(name)
print("{}好友{}：生日快乐！".format(5*kg,name))
print("{}{}{}{}{}".format(9*kg,2*ss,kg,2*ss,9*kg))
print("{}{}{}{}".format(8*kg,7*ss,8*kg))
for i in range(1,5):                                    # 循环语句执行 print 函数 4 次输出图案下部
    print("{}{}{}".format((7+i)*kg,(9-2*i)*ss,(7+i)*kg))
```

说明：对于有规律的字符图输出用到了 for 循环结构，将在下一章详细介绍。

习题

一、单选题

1.下面_____不是 Python 的基本数据类型。

A.float　　B.type

C.bool　　D.int

2.下面_____不是 Python 的组合数据类型。

A.list 和 tuple　　B.str

C.set 和 dict　　C.complex

3. 下面_____是 Python 合法的变量名。

A._yz　　B.123xyz

C.int　　D.x-y

4.下面_____是不合法的整型常数。

A.100　　B.0O100

C.0x100　　D.%100

5.下面_____是不合法的字符串常数。

A.abc　　B."abc"

C.'abc'　　D.'"abc"'

6. 数学关系 $3 \le x < 10$ 表示成 Python 表达式，错误的是_____。

A.3<=x<10　　B.3<=x and x<10

C.10>x>=3　　D.3<=x or x<10

7. random()函数不可能产生的值是_____。

A.0　　B.1

C.0.1234　　D.0.0005

8. 已知 s="12345678"，则表达式 s[3:6]的值为_____。

A."3456"　　B."345"

C."456"　　D."45"

9. input()函数返回值的类型是_____。

A.整型数值　　B.字符串

C.由用户输入的值决定　　D.浮点数

10.下面_____不属于 print()函数中的格式类型符。

A.d　　B.f 和 e

C. s　　D.b

11. 以下注释语句中，不正确的是_____。

A. #Python 注释　　B. '''Python 注释'''

C. """Python 注释"""　　D. // Python 注释

12. 表达式 'ab' + '''甲乙\new''' * 2 的结果字符串长度是_____。

A. 12　　B. 14

C. 16　　D. 18

13. 以下关系表达式中，其值为 False 的是_____。

A."ABC"> "AbC"　　B."PYTHON"== "python".upper()

C.True>False　　D."123"<"23"

14. 下面正确的赋值语句是_____。

A.x+y=30　　B.x,y,z=1,2,3

C.10=y=x　　D.3y=x

15. 下列各项中，不能使用索引运算的是_____

A.列表　　B.元组

C.集合　　D.字符串

16.tuple(range(1,9,2))的返回结果是_____。

A.(1, 3, 5, 7)　　B.[1, 3, 5, 7]

C.(2, 4, 6, 8)　　D.(1, 3, 5, 7,9)

17.以下不能建立集合的语句是_____。

A.s=set()　　B.s=set{}

C.s= {1,2,3}　　D.s=set([1,2,3])

18.以下不能建立字典的语句是_____。

A.d={}　　B.d={1:'甲',2:'T 乙',3:'丙',4:'丁'}

C.s=dict(1:'甲',2:'T 乙',3:'丙',4:'丁)　　D.d=dict()

19.执行语句：x,y=eval(input( ))时，从键盘上输入数据格式错误的是_____。

A.10　20　　B.10, 20

C.(10,20)　　D.[10, 20]

20.已知 x=100,y=15，执行语句：print("x/y={0:5.2f}x % y={1:3d}".format(x/y,x % y))的结

果是_____。

A.x/y=∪6.67x∪%∪y=∪10

B.6.6710

C.x/y={0:5.2f}x∪%∪y={1:3d}

D.6.67=∪6.6710=∪10

21．假定变量有两个变量 x 和 y，它们的值分别为 20 和 30，则下面语句中合法的是_____。

A．print('x is %10d, y is %6d'%(x,y))

B．print('x is %10d, y is %6d',%(x,y))

C．print('x is %10d, y is %6d',x,y)

D．print('x is %10d, y is %6d',%x,%y)

22．执行下列语句后，程序运行的结果是_____。

s1="湖南人"

s2="不怕"

s3="辣"

print("{0}{2}{1}".format(s1,s2,s3))

A．湖南人不怕辣　　B．湖南人辣不怕

C．辣不怕湖南人　　D．湖南人怕不辣

23.若 string='Hello_World'，则 string[:-6]*2 的值是_____。

A.'Hello_Hello_'　　B.'World_World_'

C.'HelloHello'　　D.执行时报错

24.print(r'\\')和 print('\\')结果是_____。

A.\和\\　　B.\\和\\

C.\和\　　D.\\和\

二、填空题

1. 把下列数学表达式写成 Python 表达式。

(1) $|x+y|+z^5$ 的 Python 表达式为_____。

(2) $\frac{10x+\sqrt{3y}}{xy}$ 的 Python 表达式为_____。

(3) $\frac{-b+\sqrt{b^2-4ac}}{2a}$ 的 Python 表达式为_____。

(4) $\sin 45^{\circ}+\frac{e^{10}+\ln 10}{\sqrt{x+y+1}}$ 的 Python 表达式为_____。

2. 根据条件写一个 Python 表达式：

(1) 表示 *x* 是 5 或 7 的倍数的表达式为_____。

(2) 表示 10≤*x*＜20 的表达式为_____。

(3) *x*，*y* 中有一个小于 *z* *的*表达式为_____。

3. 已知 st='Python Programming'，则

st[3] 结果：_____

st[-1] 结果：_____

st[2:7] 结果：_____

st[2:-1] 结果：_____

st[2:] 结果：_____

st[:7] 结果：_____

st[1::3] 结果：_____

st[4]*4 结果：_____

4. 执行下列操作后，lst2 的值是_____

```
lst1=[1,,3,5,7]
lst2= lst1
lst1[2]=6
print(lst2)
```

三、程序填空题

1.输入浮点数 x 的值，计算 y=sin(x)+|2x|，输出计算的结果。

```
import ____（1）____
x=_____（2）_____(input("输入 x"))
y=___(3)___.sin(x)+abs(2*x)
print("y=",y)
```

2.输入一个 6 位的正整数，将其分离成 3 个 2 位的整数。例如输入：123456，分离出 12，34，56。

```
n=int(input("输入一个 6 位的正整数 n"))
n3=n ____（1）____
n2=_____（2）_____ % 100
n1=n // ____（3）____
print(n1,n2,n3)
```

四、简答题

1． 简述 Python 标识符的命名规则。

2． 下列符号中，哪些是 Python 合法的变量名？

(1) a123　　(2) a12_3　　(3) 123_a

(4) a-123　　(5) int　　(6) XYZ

(7) False　　(8) sin(x)　　(9) sinx

3．Python 中基本的数据类型有哪些？

4．Python 中组合数据类型有哪些？

5．字符串的定界符有几种？

6．转义符的作用是什么？

7．格式化输出有几种形式？

第 3 章 基本的控制结构

结构化程序设计的基本思想之一是程序代码编写采用三种基本结构，即顺序结构、选择结构和循环结构组成；它们的特点是都是“单入口和单出口”，即每种控制结构可用一个入口和一个出口的流程图表示。据此可编写出结构良好、易于阅读和调试的程序。

3.1 顺序结构

顺序结构是最基本和常用的程序结构，它最符合人们的书写习惯。Python 的 IDLE 交互状态下执行的程序实质就是顺序结构。顺序结构是按照语句出现的先后顺序依次执行，如图 1.3.1 所示。程序执行了“语句块 1”后再执行“语句块 2”。这里的语句块可以是简单语句、控制结构语句等。

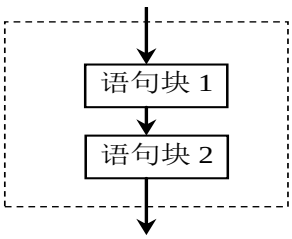


图 1.3.1　顺序结构流程图

3.1.1 引例—人民币与美元兑换

【例 3.1】人民币与美元之间兑换，假定汇率为：1 人民币=0.1556 美元，1 美元=6.4250 人民币。要求：输入人民币，转换成美元，显示保留两位小数。

分析：该程序比较简单，通过前两章的学习编程是很容易的，关键要搞清楚语句顺序问题。

人民币兑换成美元的程序代码如下：

```
dollar=CNY*0.1556                                    #语句①
CNY=int(input("输入人民币："))                        #语句②
print("人民币：%d 元  兑换为美元为：%.2f"%(CNY,dollar))
```

程序运行：

```
Traceback (most recent call last):
  File "D:/py/3-1.py", line 1, in <module>
    dollar=CNY*0.1556
NameError: name 'CNY' is not defined
```

错误原因是语句①和②顺序不正确，程序执行到语句①，变量 CNY 没有定义没有值，不能参加运算。改正后程序为：

```
CNY=int(input("输入人民币："))                        #语句①
dollar=CNY*0.1556                                    #语句②
print("人民币：%d 元  兑换为美元为：%.2f"%(CNY,dollar))
```

程序运行：

```
输入人民币：100
人民币：100 元 兑换为美元为：15.56
```

对于美元兑换为人民币，请读者自行完成。

3.1.2 简单程序的结构

从上例可以看出，程序的执行顺序是自上而下，依次执行。简单的程序一般由三部分组成：输入数据、对数据进行处理、输出结果，如图 1.3.2 所示：

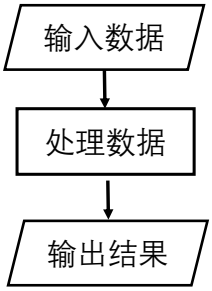


图 1.3.2　程序的基本结构

（1）在 Python 中，输入少量数据一般通过 input()函数，测试程序的批量数据可以通过随机数生成，实际应用中批量数据一般通过文件读入。

（2）进行处理要看解决问题的复杂度。例 3.1 是最简单的计算处理，即将计算表达式的值赋值给变量。

（3）输出结果通常使用 print()函数。

以上这些语句都介绍过，读者可以编写完整的 Python 程序了。

【例 3.2】输入三角形三条边长，计算面积并显示结果。

分析：已知三条边边长，计算三角形面积利用海伦公式即可求得。相应的程序代码和运行如下：

```
#输入三角形三条边的边长，求三角形的面积
import math                                  #   导入 math 模块
a,b,c=eval(input("输入 a,b,c: "))
t=(a+b+c)/2
area=math.sqrt(t*(t-a)*(t-b)*(t-c))          #   调用 math 模块的 sqrt 函数
print("三角形面积=",area)
```

程序运行：

```
输入 a,b,c: 3,4,5
三角形面积= 6.0
输入 a,b,c: 1,2,3
三角形面积= 0.0
```

注意：当程序输入三条边长分别为 1,2,3 时，不能构成三角形，故显示面积为 0。为防止此类问题出现，好的程序应在计算面积前首先判断能否构成三角形。这就要涉及到下一节的选择结构。

3.2 选择结构

计算机要处理的问题往往是复杂多变的，仅采用顺序结构是不够的。必须利用选择结构等来解决实际应用中的各种问题。选择结构的特点是在程序执行时，根据不同的“条件”，选

择执行相应的语句。

Python 中提供的 if 条件语句有多种形式：单分支、双分支和多分支等。

3.2.1 引例－体质指数 BMI 的计算

【例 3.2】体质指数 BMI(Body Mass Index)的计算公式为：

BMI=体重(kg)/身高(m)^2

根据 BMI 值可分成 4 类，如图 1.3.3 所示。

$$分类=\begin{cases}偏瘦 & bmi<18.5\\ 正常 & 18.5<bmi<24\\ 偏胖 & 24\leq bmi<28\\ 肥胖 & bmi\geqslant 28\end{cases}$$

图 1.3.3 BMI 分类

编写输入体重和身高，显示所属类型和提示信息。

单分支实现程序代码如下：

```
w,h=eval(input("输入体重(kg)，身高(m)："))
bmi=w/(h*h)
if 18.5<=bmi<=24:
    print("bmi=%5.2f,正常指数，恭喜你！"%(bmi))
```

程序运行：

```
输入体重(kg)，身高(m)：61,1.6
bmi=23.83,正常指数，恭喜你！
>>>
```

程序运行：

```
输入体重(kg)，身高(m)：62,1.6
>>>
```

即计算的 bmi 值不在正常范围内，程序没有任何信息显示，不够完整。

【例 3.3】要分别考虑在正常范围内和不在正常范围内两种情况和相应的处理。用双分支结构实现，程序代码如下：

```
w,h=eval(input("输入体重(kg)，身高(m)："))
bmi=w/(h*h)
if 18.5<=bmi<=24:
    print("bmi=%5.2f,正常指数，恭喜你！"%(bmi))
else:
    print("bmi=%5.2f,不正常指数! "%(bmi))
```

【例 3.4】进一步思考，对计算的 BMI 值，希望能给出所属的分类信息。这就要多分支结构实现，程序代码如下：

```
w,h=eval(input("输入体重(kg)，身高(m)："))
bmi=w/(h*h)
if  bmi<18.5:
    print("bmi=%5.2f,低于正常指数范围 18.5~24，偏瘦! "%(bmi))
elif 18.5<=bmi<=24:
    print("bmi=%5.2f,正常指数，恭喜你！"%(bmi))
```

```
    elif 24<=bmi<=27:
        print("bmi=%5.2f,高于正常指数范围 18.5~24，偏胖!"%(bmi))
    else:
        print("bmi=%5.2f,高于正常指数范围 18.5~24，肥胖! "%(bmi))
```

通过对 BMI 值的计算和分析，显示了 Python 提供的 if 语句构成的三种选择结构，即单分支、双分支和多分支。

3.2.2 选择结构语句

从上例可以看到，选择结构有三种形式。

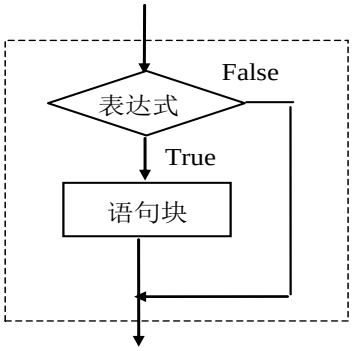


图 1.3.4　单分支结构

1.单分支结构—if 语句

语句形式：

```
if 表达式：
    语句块
```

说明：

①表达式表示条件，结果有两种情况 True 和 False；表达式可以是算术、字符串、关系和逻辑表达式，若为算术或字符串表达式，则非零或非空为 True，零或空为 False。

②语句块为 if 语句的内嵌语句，由一条或多条语句构成，同一语句块内的语句必须在同一列上进行缩进；否则表示语句块的结束。

③if 语句的作用是当“表达式”值为 True 时执行“语句块”，若为 False，则跳过语句块；然后继续执行 if 语句后的下一语句，流程图如图 1.3.4 所示。

注意：语句中用冒号表示下面是内嵌语句块，语句块由相同缩排的语句组成，空格数量必须相同（默认为 4 个），可以使用 Tab 键只要确保对齐即可。语句块的使用和书写规则在下面选择结构的 elif、else 子句，循环结构的循环体、函数定义中的函数体等都相同，不再重复说明。

2. 双分支结构—if…else 语句

语句形式：

```
if  表达式:
    语句块 1
else:
    语句块 2
```

该语句的作用是当表达式的值为 True 时，执行语句块 1，否则执行 else 后面的语句块 2，其流程

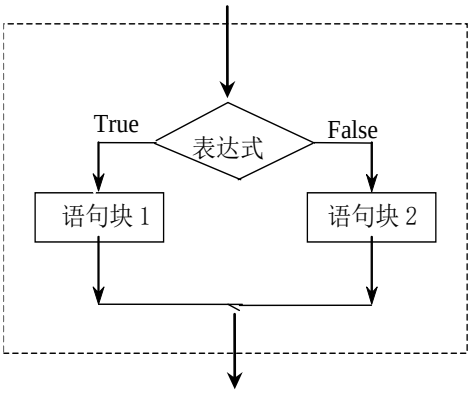


图 1.3.5　双分支 if 语句流程图

参见图 1.3.5。

【例 3.5】已知分段函数 $$y=\begin{cases}\sin x+\sqrt{x^2+1} & x\neq 0\\ \cos x-x^3+3x & x=0\end{cases}$$。编程，输入任意 x，计算对应的 y 值。

分析：最直观的使用双分支结构实现，当然也可使用单分支结构实现。

双分支结构实现

```
from  math  import *
x=float(input("输入 x："))
if x!=0:
    y=sin(x)+sqrt(x*x+1)
else:
    y=cos(x)-x**3+3*x
print("x={}y={:5.3f}".format(x,y))
```

单分支结构实现

```
from  math  import *
x=float(input("输入 x："))
y=cos(x)-x**3+3*x
if x!=0:
    y=sin(x)+sqrt(x*x+1)
print("x={}y={:5.3f}".format(x,y))
```

程序运行显示：

```
输入 x：2.3
x=2.3y=3.254
```

思考：若单分支结构实现代码次序改为如下，请问能实现吗？为什么？

```
if x!=0:
    y=sin(x)+sqrt(x*x+1)
y=cos(x)-x**3+3*x
```

3.多分支结构— if…elif…else 语句

双分支结构只能根据条件的 True 和 False 决定处理两个分支中的其中一个。当实际处理的问题有多种条件时，就要用到多分支结构。

语句形式如下：

```
if   表达式 1:
     语句块 1
elif   表达式 2:
      语句块 2
     …
[else:
      语句块  n+1]
```

该语句的作用是根据不同的表达式值确定执行哪个语句块，测试条件的顺序为表达式 1、表达式 2…一旦遇到表达式值为 True，则执行该条件下的语句块，然后执行多分支结构的下一语句。其流程参见图 1.3.6。

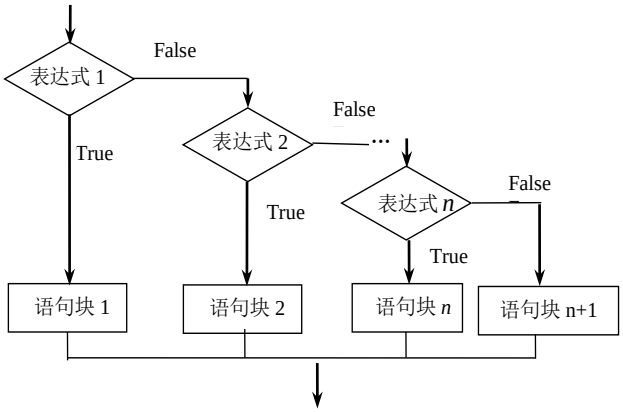


图 1.3.6　多分支结构

【例 3.6】已知字符变量 ch 中存放了一个输入的字符，判断该字符是字母字符、数字字符还是其他字符，并作相应的显示，程序代码如下：

```
#输入字符，显示输入的是何种类型字符
ch = input("输入一个字符:")
if   'a'<=ch.lower() <= 'z':                    #   大小写字母均考虑
      print(ch + "是字母字符")
elif   '0'<= ch <= '9':                         # 表示是数字字符
      print(ch + "是数字字符")
else:
       print(ch +"是其他字符")                  # 除上述字符以外的字符
```

程序运行：

```
输入一个字符:e
e 是字母字符
```

注意：

（1）不管有几个分支，程序执行了一个分支后，其余分支不再执行。

（2）当多分支中有多个表达式同时满足，则只执行第一个与之匹配的语句块。因此，要注意多分支中表达式的书写次序，防止某些值的过滤。

【例 3.7】计算轨道交通票价。某城市轨道交通实行按里程计价的多级票价，起步 6 公里内每人次 3 元，6-12 公里每人次 4 元，12-32 公里每 10 公里加 1 元，32 公里以上每 20 公里加 1 元，票价不封顶。输入公里数，显示对应票价。程序代码如下：

```
km=eval(input("输入公里："))
if   km<=6:
      s = 3
elif   km<=12:
      s = 4
elif   km<=32:
      s = 4 + (km-12)/10
else :
      s=6+(km-32)/20                  # 6 为 0~32 公里的票价
if int(s)!=s:                         # 考虑票价以元为单位
```

```
        s=int(s)+1
print("某城市地铁票价：",s)
```

3.2.3 选择结构的嵌套

选择结构的嵌套是指把一个选择结构放入另一个选择结构之内，也就是选择结构中的语句块也是选择结构。

例如嵌套的形式如下：

```
双分支均内嵌单分支
if  表达式 1：
    if  表达式 11：
        语句块 11
else：
    if  表达式 21：
        语句块 21
```

```
双分支的 if 块内嵌双分支
if  表达式 1：
    if  表达式 11：
        语句块 11
    else：
        语句块 12
else：
    语句块 2
```

使用嵌套的 if 结构时，一定要严格控制好不同层次语句块的缩进量，因为这决定了不同语句块的从属关系以及要处理的问题是否被正确地实现、是否能够被 Python 正确地理解和执行。

【例 3.8】 已知 x、y、z 三个数，比较它们的大小并进行排序，使得 x>y>z。

分析：在计算机中无法直接同时对三个数比较，只能依次通过多次两两相比较来实现。下面程序是通过一个 if 语句和一个嵌套 if 语句来实现，流程图如图 1.3.7 所示。

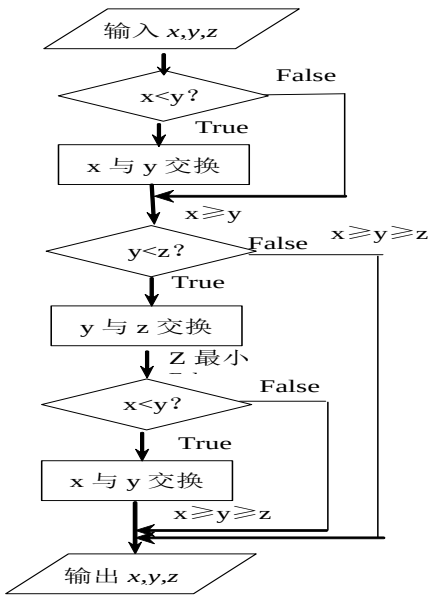


图 1.3.7　三个数排序流程图

程序如下：

```
x,y,z=eval(input("输入 x，y，z："))
if x<y :
    x,y=y,x             # x<y，交换，使得 x>y
if y<z:
    y,z=z,y             # y<z,交换，此时 z 最小
    if x<y :            # 但还要考虑 x 与 y 大小
        x,y=y,x         # x<y，交换，使得 x>y
print(x,">",y,">",z)
```

程序运行：

```
输入 x，y，z：3,45,9
45 > 9 > 3
>>>
```

思考：

（1）若不用嵌套 if 语句，而用三个 if 语句，程序如何编写？

（2）使用系统提供的 max()和 min()函数是否不需要使用 if 语句就可对 3 个数据进行排

序了？

（3）若要对四个数进行排序，程序又如何编写？若要对大量数据进行排序呢？

3.3 循环结构

计算机最擅长的功能之一就是按规定的条件，重复执行某些操作。例如，按照人口增长率统计人口数；根据各课程的学分、绩点和学生的成绩，统计每个学生的平均绩点等。这类问题都可通过循环结构来方便地实现。

3.3.1 引例－批量数据的处理

例 3.4 实现了 1 个人的 BMI 体质指数计算，若要对一个部门若干人的 BMI 体质指数进行计算，通常有以下两种方法。

【例 3.9】要统计分析一个部门 100 个职工的 BMI 体质指数，如何实现？只要在计算一个人的基础上增加一个 for 语句即可实现。程序如下：

```
for i in range(1,101):
    w,h=eval(input("输入第"+str(i)+"个人的体重，身高："))
    bmi=w/(h*h)
    if 18.5<=bmi<=24:
        print("正常")
    elif bmi<18.5:
        print("偏瘦")
    else:
        print("偏胖")
```

程序运行：

```
输入第 1 个人的体重，身高：62,1.62
正常
输入第 2 个人的体重，身高：80,1.78
偏胖
输入第 3 个人的体重，身高：78,1.87
正常
输入第 4 个人的体重，身高：50,1.6
正常
输入第 5 个人的体重，身高：50,1.7
偏瘦
输入第 6 个人的体重，身高：
......
>>>
```

【例 3.10】若要处理若干个职工的信息，当输入体重或身高为 **0 时**结束，如何实现？则利用 while 语句来实现，程序如下：

```
i=1
w,h=eval(input("输入第"+str(i)+"个人的体重，身高："))
```

```
while w!=0 and h!=0:                                    # 输入的体重和身高均不为 0，进行计算和显示
    bmi=w/(h*h)
    if 18.5<=bmi<=24:
        print("正常")
    elif bmi<18.5:
        print("偏瘦")
    else:
        print("偏胖")
    i=i+1                                                # 一个人处理完成
    w,h=eval(input("输入第"+str(i)+"个人的体重，身高："))    # 继续输入下一个人信息
```

程序运行：

```
输入第 1 个人的体重，身高：61,1.7
正常
输入第 2 个人的体重，身高：51,1.7
偏瘦
输入第 3 个人的体重，身高：0,1.7
>>>
```

从引例中看到，Python 中提供了两种基本的循环结构：for 循环和 while 循环。for 循环常用于已知循环次数的场合；while 循环常用于未知循环次数场合。

3.3.2 for 语句

1.for 语句

在一般程序设计语言中，for 语句常称为计数型循环语句；而在 Python 语言中，for 语句实质是一个通用的序列迭代器，可以遍历任何序列中的元素。for 语句形式如下：

for 变量 in 序列：

语句块

该语句的作用是用序列中的元素逐一赋值给变量，对每一次的成功赋值都执行一次语句块（循环体）。当序列遍历完，即每一个元素都用过了，则循环结束，接着执行 for 语句的下一条语句。

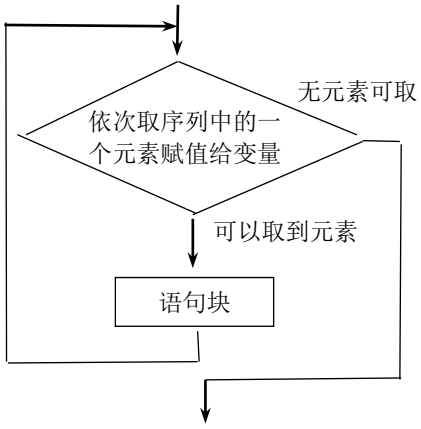


图 1.3.8 for 语句实现的流程

注意：

（1）序列可以是：

常用的 range()函数；组合数据类型，包括字符串、列表、元组、集合和字典等。

（2）循环的次数由序列中的成员个数决定。

（3）书写规则序列后要有冒号、循环体要右缩排。

执行的流程图如图 1.3.8 所示。

【例 3.11】循环控制的序列是 range()函数。将可打印的字符和其 ASCII 码制成表格输出，每行打印 7 个字符。

分析：

①ASCII 码中，空格到“~”是可以打印的字符，其余为不可打印的控制字符。可打印的字符的编码值为 32~126，利用 chr()函数获得编码值对应的字符。

②每行显示 7 个字符项，通过设置一个计数器变量 n，每打印一项加 1；当满了 7 项换行。

代码如下，运行结果如图 1.3.9 所示：

程序如下：

```
n=0
for asc in range(32,127):
    print('%s=%d'%(chr(asc),asc),end='\t')
    n=n+1
    if n %7==0:print()
```

程序运行：

```
 =32    !=33    "=34    #=35    $=36    %=37    &=38
'=39    (=40    )=41    *=42    +=43    ,=44    -=45
.=46    /=47    0=48    1=49    2=50    3=51    4=52
5=53    6=54    7=55    8=56    9=57    :=58    ;=59
<=60    ==61    >=62    ?=63    @=64    A=65    B=66
C=67    D=68    E=69    F=70    G=71    H=72    I=73
J=74    K=75    L=76    M=77    N=78    O=79    P=80
Q=81    R=82    S=83    T=84    U=85    V=86    W=87
X=88    Y=89    Z=90    [=91    \=92    ]=93    ^=94
_=95    `=96    a=97    b=98    c=99    d=100   e=101
f=102   g=103   h=104   i=105   j=106   k=107   l=108
m=109   n=110   o=111   p=112   q=113   r=114   s=115
t=116   u=117   v=118   w=119   x=120   y=121   z=122
{=123   |=124   }=125   ~=126
```

图 1.3.9 ASCII 码对照表

【例 3.12】循环控制的序列是字符串。输入一字符串，检验其输入的是否全部是数字字符。

分析：从字符串中逐一取出字符，判断其若是非数字字符则该字符串是非数字字符串。取字符有两种方法：遍历法或索引法。

方法一、遍历法

```
str=input("输入字符串:")
isnumeric=True
for c in str:
    if c not in "0123456789":
        isnumeric=False
if isnumeric:
    print(str,"是数字字符串")
else:
    print(str,"含有非数字字符")
```

方法二、索引法

```
str=input("输入字符串:")
isnumeric=True
for i in range(len(str)):      #len()字符串长度
    if str[i] not in "0123456789":
        isnumeric=False
if isnumeric:
    print(str,"是数字字符串")
else:
    print(str,"含有非数字字符")
```

程序运行：

```
输入字符串:123e567
123e567 含有非数字字符
>>>
输入字符串:1234567
1234567 是数字字符串
>>>
```

【例 3.13】循环控制的序列是列表。随机生成 10 个 1~100 之间的整数，求最小、最大和平均值。

分析：利用 sample()函数产生指定范围内、指定个数的随机数；然后通过遍历列表，逐一比较、累加，求得最小、最大和平均值。

```
from random import *
lst=sample(range(1,101),10)      #sample()生成 10 个在 1，100 之间的随机数
print(lst)
min1=100; max1=0
sum=0
for i in lst:
    if i>max1: max1=i
    if i<min1: min1=i
    sum+=i
print("max=",max1," min=",min1," avg=",sum/10)
```

程序运行显示：

```
[73, 59, 51, 36, 41, 81, 45, 97, 100, 39]
max= 100    min= 36    avg= 62.2
>>>
```

思考：为什么设置初值 min1=100; max1=0，而不是初值：min1=0; max1=100？

2.range 函数

range()函数是 Python 的内置函数，用于创建一个整数序列，常用于 for 语句中，实现如同其他语言 for 语句的循环控制的功能。range()函数有如下三种形式：

（1）range(start,end,step)

整数序列表以起始值为 start、步长为 step、结束值为 end，但不包括 end。例：

```
>> list(range(1,10,3))          # 产生从 1 开始、以步长为 3、不到 10 的数字序列。
[1,4,7]
```

（2）range(start,end)　　#缺省 step 为 1

```
>> list(range(1,10))          # 产生从 1 开始、以步长为 1、不到 10 的数字序列，例：
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

（3）range(end)　　#缺省 start 为 0，缺省 step 为 1，例：

```
>>list(range(10))              # 产生从 0 开始到 9 的数字序列
[0,1,2,3,4,5,6.7.8.9]
```

3.3.3 while 语句

while 语句常用于控制循环次数未知的循环结构，语句形式为：

while　表达式:

　　语句块

语句作用：判断表达式的值，若为 True，则执行语句块(循环体)；否则退出循环，执行 while 语句的下一句语句。语句执行的流程如图 1.3.10 所示：

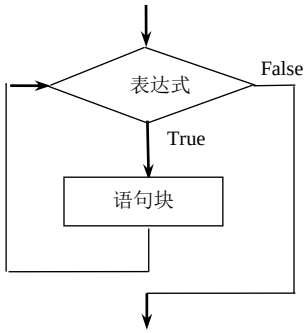


图 1.3.10 while 语句流程图

【例 3.14】据统计 2018 年年末我国总人口 13.95 亿人，自然增长率 3.81‰。若按此增长率，多少年后我国人口翻倍？

分析：解此问题两种方法：

（1）直接调用对数函数求得

可根据公式：$27.9=13.95(1+0.00381)^n$，得：n=log(2)/log(1.00381)

调用内置数学函数对数可求得，但求得的年数不为整数，也得不到实际的人数。

（2）利用循环求得

根据增长率求得每年的人数，当人口数没有翻倍重复计算。

程序代码和运行效果如下:

直接调用对数函数求得	利用循环结构求得
import math n=math.log(2)/math.log(1.00381) if int(n)!=n: 　　n=int(n)+1　#n 若非整数，取整加 1 年 print("用对数求得的年数为：%d"%(n))	x=13.95 n=0 while x<27.9: 　　n+=1 　　x=x*(1+0.00381) print("循环求得年数为：%d 人数为：%f 亿"%(n,x))
程序运行： 　　用对数求得的年数为：183	程序运行： 循环求得的年数为：183 人数为：27.977053 亿

【例 3.15】求两自然数 m 和 n 的最大公约数。

求最大公约数在数学上有多种方法，最方便的是短除法。计算机解决此问题也有多种方法。

（1）枚举法　该方法求最大公约数比较简单且容易理解，可通过递增和递减方式实现。

递减算法思路

①比较两个数 m,n，将小数放入变量 min_mn 中；

②判断 min_mn 是否该两数的公因子，是则 min_mn 为两数的最大公约数，结束；否则 min_mn 递减重复判断是否是两数的公因子。程序如下：

```
m,n=eval(input("输入 m,n"))
 min_mn = min(m,n)
while not(m % min_mn == 0 and n % min_mn== 0):          # gcd 不是 m、n 的公因子则循环
    min_mn = min_mn - 1
print("最大公约数是：",min_mn)
```

程序运行：

```
输入 m,n 24,10
最大公约数是：2
>>>
```

思考：递增法思想是从 1 开始依次递增求两个数的最大公因子，代码如何实现？

（2）碾转相除法　也称为欧几里德算法，特点效率高，算法思想是：

①对于已知两数 m，n，使得 m>n；

②m 除以 n 得余数 r；

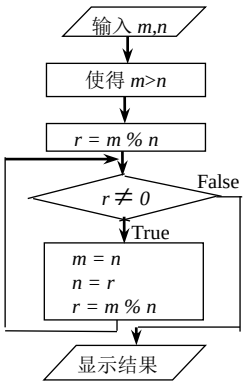
③若 r=0 则 n 为求得的最大公约数，算法结束；若 r≠0，则令 m←n，n←r，重复执行步骤②。

流程图参见图 1.3.11(a)，程序如下：

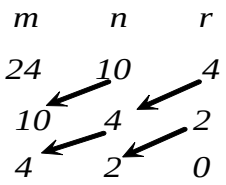
```
m,n=eval(input("输入 m,n"))
if m<n:                                          # 若 m<n，则交换，使得 m>=n
    m,n=n,m
r=m % n
while r!=0:
    m=n
    n=r
    r=m % n
print("最大公约数是：",n)
```

程序运行：

```
输入 m,n 24,10
最大公约数是：2
```



(a)



(b)

图 1.3.11　辗转相除法求最大公约数流程图和示例

当 m，n 输入的值为 24，10 时，显示最大公约数为 2，如图 1.3.11(b)所示。

注意：python 的 math 库中提供求最大公约数 gcd()函数，调用并加以验证自编的求最大公约数程序。

3.3.4 循环结构的嵌套

在一个循环体内又包含了完整的循环结构称为循环结构的嵌套。循环结构的嵌套对 for 语句和 while 语句均适用。

【例 3.16】打印九九乘法表(运行界面参见图 1.3.12)。

分析：打印九九乘法表，利用两重循环可方便地实现。其中：外循环控制变量 i 作为被乘数，也决定了乘法表的行数；内循环控制变量 j 作为乘数，也决定了乘法表的列数；同时还要考虑输出的形式，即一行输出后要换行。程序如下：

```
for i in range(1,10):
    for j in range(1,10):
        print("{}*{}={:2d}".format(i,j,i*j),end=" ")
    print()
```

运行结果：

```
1×1= 1 1×2= 2 1×3= 3 1×4= 4 1×5= 5 1×6= 6 1×7= 7 1×8= 8 1×9= 9
2×1= 2 2×2= 4 2×3= 6 2×4= 8 2×5=10 2×6=12 2×7=14 2×8=16 2×9=18
3×1= 3 3×2= 6 3×3= 9 3×4=12 3×5=15 3×6=18 3×7=21 3×8=24 3×9=27
4×1= 4 4×2= 8 4×3=12 4×4=16 4×5=20 4×6=24 4×7=28 4×8=32 4×9=36
5×1= 5 5×2=10 5×3=15 5×4=20 5×5=25 5×6=30 5×7=35 5×8=40 5×9=45
6×1= 6 6×2=12 6×3=18 6×4=24 6×5=30 6×6=36 6×7=42 6×8=48 6×9=54
7×1= 7 7×2=14 7×3=21 7×4=28 7×5=35 7×6=42 7×7=49 7×8=56 7×9=63
8×1= 8 8×2=16 8×3=24 8×4=32 8×5=40 8×6=48 8×7=56 8×8=64 8×9=72
9×1= 9 9×2=18 9×3=27 9×4=36 9×5=45 9×6=54 9×7=63 9×8=72 9×9=81
```

图 1.3.12　九九乘法表运行界面

思考：若要分别打印呈下三角和上三角的九九乘法表，如图 1.3.13 和 3.14 的结果，程序如何作微小的改动？

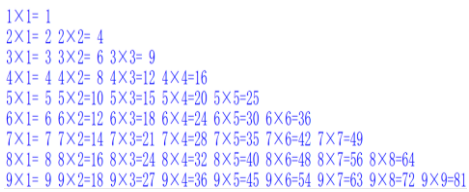


图 1.3.13　呈下三角的九九乘法表

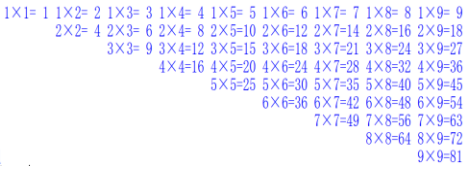


图 1.3.14　呈上三角的九九乘法表

提示：外循环行数不变，内循环改变循环的初值或终值、输出格式控制就可。

多重循环的循环次数等于每一重循环次数的乘积。对于多重循环的每一个循环变量变化规律的理解，读者只要以手表上时、分、秒三根针构成的三重循环的变化模拟，即当内循环秒针走满一圈时，分针加一，秒针又从头开始走；当分针走满一圈时，时针加一，分针、秒针从头开始；以此类推，时针走满一圈，即 12 小时，循环结束；整个循环执行的次数为：12*60*60。

对于循环语句使用，要注意以下事项：

①内循环变量与外循环变量不能同名。

②外循环必须完全包含内循环，不能交叉。

③若循环体内有 if 语句，或 if 语句内有循环语句，也不能交叉。

3.3.5 循环结构的其他语句

循环结构的其他语句表示在循环体内可以出现的语句，分为跳转类语句和 else 子句。

1.跳转语句

在 Python 循环体结构中，可以使用跳转语句来改变循环执行的路径，分别是 break 语句、continue 语句和 pass 语句。

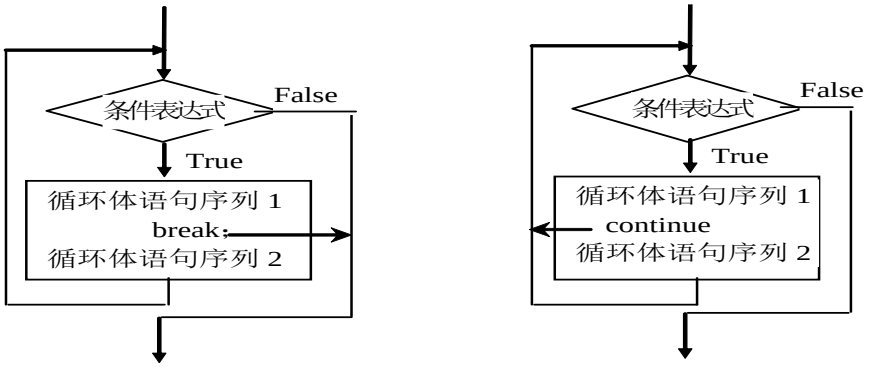
（1）break 语句

break 语句的作用是结束循环，执行循环语句后的下一个语句。break 语句有时又称为循环的断路语句。

（2）continue 语句

continue 语句的作用是跳过本次循环体内 continue 后面的语句，并开始下一次循环。continue 语句有时又称为循环的短路语句，是指只对本次循环短路，并不终止循环。

两条语句出现在循环体内，常和 if 语句一起使用实现有条件地跳转，两者区别见流程图 1.3.15(a)和(b)所示。



(a) break 语句　　(b) continue 语句

图 1.3.15 循环结构中两种语句区别图示

【例 3.17】表 3.1 的两个程序段，显示了 break 和 continue 语句的区别，读者可自行分析。

表 3.1　break 和 continue 语句的区别

循环体内用 break 语句	循环体内用 continue 语句

for i in range(20,0,-1):
　if i % 6 == 0:
　　break
　print(i,end=" ")
	for i in range(20,0,-1):
	　if i % 6 == 0:
	　　continue
	　print(i,end=" ")

运行结果： 20 19	运行结果： 20 19 17 16 15 14 13 11 10 9 8 7 5 4 3 2 1
功能：当找到能够被 6 整除的最大的数时不再循环。	功能：显示 20 到 1 之间非 6 的倍数的数

（3）pass 语句

pass 语句是不执行任何操作，实质是空语句；作用就是占据位置，当某处需要有语句但又不产生任何操作时可使用 pass 语句。

【例 3.18】模拟打字机效果。

分析：可利用空循环起到延迟作用，实现模拟打字机效果。程序如下：

```
for c in "welcome":
    print(c,end=" ")
    for j in range(1,10000000):    # 延迟作用，起到打字机效果
        pass
```

2. else 子句

在其他语言中，else 子句主要用在 if 选择结构中；而在 Python 中，else 子句可以出现在循环结构中，也可以出现在异常处理结构中。

在循环结构中出现 else 子句，表示循环正常结束后执行 else 后的语句块。语句形式：

```
for 循环变量 in 序列：
    循环体
else：
```

```
while 表达式：
    循环体
else：
    语句块
```

语句块

【例 3.19】统计列表中数据的均值。程序如下：

```
sum=0
n=0
for i in [66,88,98,76,89,55,67,54]:
    sum+=i
    n+=1
    print(i,end=" ")
else:
    print("\n 共有%d 个数，平均值为%5.2f"%(n,sum/n))
```

程序运行：

```
66  88  98  76  89  55  67  54
共有 8 个数，平均值为 74.12
>>>
```

此例看似没有 else 子句也可有相同效果，但当在循环体中有 break 语句时，else 子句的作用就显现，尤其在循环的嵌套的场合使得代码更精炼。

【例 3.20】求 2～100 之间的素数，并且每行显示 8 个素数。

分析：素数就是除 1 和本身以外，不能被其他任何正整数整除的数。

（1）根据此定义，要判别某数 m 是否为素数最简单的方法就是依次用 j=2~ m-1 去除，只要有一个能整除，m 不是素数，退出循环；若都不能整除，则 m 是素数。

（2）对 2～100 之间的素数，只要增加一个外循环，m=2,3,…，100 就可。

程序如下：

方法一、一般程序设计语言处理

```
countm=0
for m in range(2,101):
    tag=True                  # 假定m为素数
    for j in range(2,m):
        if m % j ==0:
            tag=False         #m能被j整除不是素数
            break             # 退出本次循环，处理下一个m
    if tag:                   # 若tag为True则m是素数显示
        print(m,end="\t")
        countm+=1
        if countm % 8 ==0:
            print()
```

方法二、用 else 子句

```
countm=0
for m in range(2,101):
    for j in range(2,m):
        if m % j ==0:
            break
    else:
        print(m,end="\t")
        countm+=1
        if countm % 8 ==0:
            print()
```

程序运行：

```
2       3       5       7       11      13      17      19
23      29      31      37      41      43      47      53
59      61      67      71      73      79      83      89
97
>>>
```

从上例求素数的两种方法对比可知，方法二在循环结构中使用 else 子句可使代码更精

炼、算法更利于理解。

思考：将上例方法一中的 break 语句改为 continue 语句，运行结果变化吗？

3.4 综合应用

本章介绍了构成结构化程序设计的三种基本结构：顺序结构、选择语句、循环语句，它们是程序设计的基础，对今后编程非常重要，希望读者熟练掌握。

对初学者来说，从本章开始，编程工作量明显增多，要调试通过一个程序有时要花很多时间。经验告诉我们，学习程序设计没有捷径可走，只有多看多练、通过上机调试，发现问题，解决问题，才能真正理解、掌握所学的知识。

本节介绍相关的应用和常用算法。

1 数字之美——数字图

【例 3.21】利用循环结构显示有规律的数字字符图，如图 1.3.16 所示。

```
1×8+1=9
12×8+2=98
123×8+3=987
1234×8+4=9876
12345×8+5=98765
123456×8+6=987654
1234567×8+7=9876543
12345678×8+8=98765432
123456789×8+9=987654321
```

图 1.3.16 有规律数字图

```
1
12
123
1234
12345
123456
1234567
12345678
123456789
```

图 1.3.17 有规律数值

分析：

（1）如何构建有规律的数值（如图 1.3.17 所示）以及如何控制输出格式。有规律数值生成通项：t=t*10+i　1<=i<=9；输出格式控制，左边的空格数与行数有关：(11-i)* " "，效果。

（2）整个数字图显示就根据可变数值和不变字符逐项输出就可。

程序如下：

```
t=0
for i in range(1,10):
    t=t*10+i
    print("{}{}×8+{}={}".format((11-i)*" ",t,i,t*8+i))
```

思考：若要输出如图 1.3.18 所示的数字图，如何实现？

```
1×1=1
11×11=121
111×111=12321
1111×1111=1234321
11111×11111=123454321
111111×111111=12345654321
1111111×1111111=1234567654321
11111111×11111111=123456787654321
111111111×111111111=12345678987654321
```

```
9×9+7=88
98×9+6=888
987×9+5=8888
9876×9+4=88888
98765×9+3=888888
987654×9+2=8888888
9876543×9+1=88888888
98765432×9+0=888888888
```

图 1.3.18 有规律数字图例

进一步思考：从上例可以理解如何将各位数组成一个有规律的数值，这需要利用循环和循环体的通项表达式：t=t*10+i。当 i 为循环变量时就构成如图 1.3.17 或图 1.3.18 右边的有

规律数值；当 i 为常量时，就构成图 1.3.18 左边的有规律数值。同样如何将一个数值逐一分离，例如：1234 分离输出为 4,3,2,1 或者构成反序的数值，算法如下：

```
x=int(input("输入 x："))
fx=0
while x>0:
    d=x % 10                    # 从右往左开始分离每一位
    x=x//10
    print(d,end=" ")
    fx=fx*10+d                  # 将每一位连接成一个反序数
print("\n 反序数为：{}".format (fx))
```

程序运行显示：

```
输入 x：1234
4 3 2 1
反序数为：4321
>>>
```

2.部分级数和——求圆周率 π

求部分级数和是用计算机求解初等数学问题近似解的常用方法，如求 π、e、sinx 等。解决的方法关键是根据部分级数和展开式找规律写通项。

【例 3.22】 利用格里高利公式求圆周率 π，当通项的绝对值小于 10^{-6} 时停止计算，公式为:

$$\frac{\pi}{4} \approx 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$$

分析：本例涉及到程序设计中重要的运算求累加问题，通项是 t=1/n，并且有正负号切换；结束条件$\left|\frac{1}{n}\right| < 10^{-6}$。

程序如下：

```
p=0
n=t=sign=1
while abs(t)>1e-6:
    p=p+t
    n=n+2
    sign=-sign
    t=sign/n
p=p*4
print("π={0:8.6f}".format(p))
```

运行运行显示：

```
π =3.141591
```

3.枚举法

枚举法亦称穷举法或试凑法。它的基本思想是利用计算机具有高速运算的特点将可能出现的各种情况一一罗列测试，直到所有情况验证完。若某个情况符合题目的条件，则为本题的一个答案；若全部情况验证完后均不符合题目的条件，则问题无解。枚举法是一种比较耗时的算法，利用了计算机快速运算的特点可方便地求解。枚举的思想可解决许多问题。

【例 3.23】计算机来判断肇事车辆。交通事故中肇事车辆撞人后逃逸，警方在现场找到 **3** 位目击证人询问肇事车辆 **5** 位数的车牌号码。甲说只看清最左两位为 **27**；乙说只看清最后位是 **3**；丙说牌号是 **67** 的倍数。

分析：利用枚举法求解。

（1）确定需要枚举的数字有几位，车牌号是：27XX3，枚举范围 X 均为是 0~9；

（2）枚举的几位数字间的关系，即：number=27003+i3*100+i2*10

（3）验证是否是问题的解？，即：if number % 67==0

程序代码如下：

```
for i3 in    range(0,10):
    for i2 in    range(0,10):
        number =27003+i3*100+i2*10
        if number % 67==0:
            print("肇事车辆号码为：", number)
```

程序运行显示：肇事车辆号码为： 27403

从上例可看到，枚举法能有效解决问题的关键在于三点：

（1）确定枚举的范围，尽量避免扩大枚举范围；

（2）确定满足的条件，把所有可能的条件一一罗列；

（3）枚举解决问题效率不高，因此，为提高效率，根据解决问题的情况，尽量减少嵌套循环层数或每层循环次数。

思考：古代百元买百鸡问题是典型的枚举法来求解的问题。假定小鸡每只 0.5 元，公鸡每只 2 元，母鸡每只 3 元。现在有 100 元钱要求买 100 只鸡，即：

0.5x+2y+3z=100

x+y+z=100

三个未知数两个方程，有若干个解，可用三重或两重循环列出所有可能的购鸡方案。

4.迭代法——求高次方程的根

迭代法又称为递推法，其基本思想是把一个复杂的计算过程转化为简单过程的多次重复。每次重复都在旧值的基础上递推出新值，并由新值代替旧值。

【例 3.24】用迭代法求 $x=\sqrt[3]{a}$，a=1~10。求立方根的迭代公式为：

$$x_{i+1}=\frac{2}{3}x_i+\frac{a}{3x_i^2}$$

分析：x_0 的初值是任意的，迭代到$|x_{i+1}-x_i|<\varepsilon=10^{-5}$为止；算法流程图如图 1.3.19 所示。方法是给定一个初值，利用迭代公式求得新值，比较新值与初值的差，若小于所要求的精度，则新值为求得的根；否则用新值替代初值，再重复利用迭代公式求得新值。

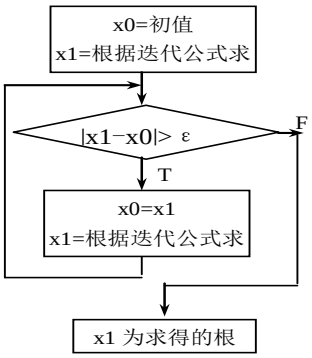


图 1.3.19 算法流程图

程序如下：

```
for a in range(1,11):
    x0=3
    x1=2/3*x0+a/(3*x0*x0)
    while abs(x1-x0)>0.00001:
        x0=x1
        x1=2/3*x0+a/(3*x0*x0)
    print(a,"立方根为",round(x1,4))        # round(x1,4)    保留 4 位小数
```

程序运行显示如图 1.3.20 所示。

```
1 立方根为 1.0
2 立方根为 1.2599
3 立方根为 1.4422
4 立方根为 1.5874
5 立方根为 1.71
6 立方根为 1.8171
7 立方根为 1.9129
8 立方根为 2.0
9 立方根为 2.0801
10 立方根为 2.1544
```

图 1.3.20 运行结果

习题

一、选择题

1. Python 提供了结构化程序设计的三种基本结构，这三种基本结构是____。

A.递归结构、选择结构、循环结构

B.选择结构、过程结构、顺序结构

C.过程结构、输入输出结构、转向结构

D.选择结构、循环结构、顺序结构

2.结构化程序设计的基本思想之一是“单入口和单出口”的控制结构，不符合此思想的语句是_____。

A.if 语句　　B.for 语句

C.while 语句　　D.break 语句

3.用 if 语句表示分段函数 $f(x)=\begin{cases}\sqrt{x+1}, & x>=1\\ x^2+3, & x<1\end{cases}$，不正确的是____。

A.
```
f=x*x+3
if x >=1:
    f= math.sqrt (x+1)
```

B.
```
if x >=1 :
    f=math.sqrt(x+1)
if x <1:
    f=x*x+3
```

C.
```
if x >=1 :
    f= math.sqrt (x+1)
f=x*x+3
```

D.
```
if x < 1 :
    f=x*x+3
else:
    f= math.sqrt(x+1)
```

4.下面程序段求两个数中的大数，____不正确。

A.
```
if x > y : max = x
if y>= x : max=y
```

B.
```
if x > y : max = x
else : max = y
```

C.
```
max = x
if y >= x : max = y
```

D.
```
if y >= x : max = y
max = x
```

5.下面 if 语句统计满足性别为男、职称为副教授及以上、年龄小于 40 岁条件的人数，不正确的语句是____。

A.if sex == "男" and age < 40 and (duty== "教授" or duty== "副教授"): n=n+1

B.if sex == "男" and age < 40 and (duty== "教授" and duty== "副教授"): n=n+1

C.if sex == "男" and age < 40 and "教授" in duty: n=n+1

D.if sex == "男" and age < 40 and duty.index("教授")>=0: n=n+1

6.已知循环语句块：

```
for i in range(1,10):
    pass
print(i)
```

循环体的语句块即 pass 语句执行的次数和循环结束后 i 显示的值分别是_____。

A.9、10　　B.10、11

C.9、9　　D.10、10

7 已知循环语句块：

```
i=1
while(i<10):
    pass
    i=i+1
print(i)
```

循环体的语句块执行的次数和循环结束后 i 显示的值分别是_____。

A.9、10　　B.10、11

C.9、9　　D.10、10

8.下面 for 语句中不能输出 10 和 20 的序列是_____。

```
for x in _____:
    print(x)
```

A.range(10,20)　　B.(10,20)

C.{10,20}　　D.[10,20]

9.下面用于循环结构中的语句，_____语句实质是个占位语句。

A.break　　B.continue

C.pass　　D.else

10. 以下代码的执行结果为_____。

```
for i in range(10,-2,-2):
    print(i,end=″,″)
```

A.9,7,5,3,1,-1,　　B.10,8,6,4,2,0,

C.10,8,6,4,2,0,-2　　D.8,6,4,2,0,-2

二、填空题

1. 结构化程序设计的三种基本结构分别是 _____ 、_____、_____ 。

2. 下面程序运行后若输入 100，则结果是_____，该程序的功能是_____ 。

```
x=int(input("输入 x:"))
y=""
while x!=0:
    a=x % 2
    x=x//2
    y=chr(48+a)+y
print(y)
```

3. 下面程序运行后的结果是_____， 该程序的功能是_____。

```
x,y=242,44
z=x*y
while x!=y:
    if x>y:
        x=x-y
    else:
```

```
        y=y-x
    print(x,z/x)
```

三、程序填空题

1. 当 C 字符串变量中第三个字符是"C"时，显示"Yes"，否则显示"No"。

```
if    (1)    :
    print("Yes")
else:
    print("No")
```

2. 输入当前年份，判断它是否为闰年，并显示相应信息。判断闰年的条件是：年份能被 4 整除但不能被 100 整除，或者能被 400 整除。

```
d =int(input("输入年份"))
if   (1)   or d % 400 == 0 :
    print(d,"是闰年")
else:
    print(d,"是平年")
```

3.利用 if 语句计算分段函数

$$y = \begin{cases} x^2 + 3x + 2 & \text{当} x > 20 \\ \sqrt{3x} - 2 & \text{当} 10 \le x \le 20 \\ \frac{1}{x} + |x| & \text{当} x < 10 \end{cases}$$

```
import math
x = int(input("输入 x 的值："))
if x>20:
    y = x * x + 3 * x + 2
elif   (1)   :
    y = 1 / x +abs(x)
else:
    (2)
print("y=", y)
```

4.下面的程序段是检查输入的算术表达式中圆括号是否配对，并显示相应的结果。

```
s=input("输入表达式,直到输入=：表示结束>")
count=0
for c in s:
    if   c== "(":
```

```
            count = count + 1
        elif    c==")":
            ___(1)___
if ___(2)___:
    print("左右括号配对")
elif ___(3)___ :
    print("左括号多于右括号",count,"个")
else:
    print("右括号多于左括号",-count,"个")
```

四、问答题

1.有哪几种基本控制结构？

2. 如下 if 语句：

```
if  表达式：语句块
```

请问其中表达式可以是哪些类型的表达式？

3. 如下 for 语句：

```
for  变量 in 序列：
```

请问序列可以是哪些数据类型？

4. 如下语句：

```
while  True：print(“Ok”)
```

请问输出了多少个“Ok”？

5. 简述 continue 与 break 的区别。

6. 简述 for 循环与 while 循环的异同点。

第四章 函数

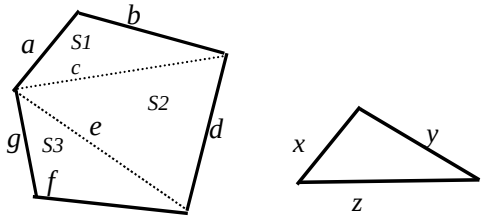
函数是完成某个特定任务的程序语句组合，程序设计中引入函数主要有两个作用：一是实现程序的模块化，便于管理和阅读；二是实现代码复用，提高开发效率。

函数有两种，一种是开发环境提供的函数，称内部函数，如 Python 提供的 abs、max、print，以及一些类库中提供的函数，如 math 库中的 sqrt、sin 等，这些函数用户直接调用即可。还有一种是用户根据问题而自定义的函数。本章就用户自定义函数的语法及调用展开讨论，希望通过本章的学习，读者可以通过定义函数提高程序的模块化编写能力。

4.1 函数定义与调用

4.1.1 引例-多边形面积的计算

【例 4.1】已知五边形的各条边和对角线的长度，计算其面积，见图 1.4.1(a)。



(a)多边形　　(b)三角形

图 1.4.1　多边形面积的计算

计算多边形面积，可将多边形分解成若干个三角形。对图 1.4.1(b)所示的三角形，已知三条边边长为 x、y、z，其面积的计算公式如下：

$$area = \sqrt{c(c-x)(c-y)(c-z)}, \qquad 其中 c = \frac{1}{2}(x+y+z)$$

本例中需要求三个三角形的面积，使用的公式相同，程序的代码相同，仅计算数据不同。如果将求三角形面积的代码定义为一个函数，调用它三次就可完成计算任务。

```
import math
def triangle(x,y,z):
    c=(x+y+z)/2
    area=math.sqrt(c*(c-x)*(c-y)*(c-z))
    return area

s=input("请输入输入 a、b、c、d、e、f、g 边长,以逗号间隔")
a,b,c,d,e,f,g=eval(s)
tri1=triangle(a,b,c)
tri2=triangle(c,e,d)
```

```
tri3=triangle(e,f,g)
area=tri1+tri2+tri3
print(area)
```

上面程序中，定义了计算三角形面积的函数 triangle，它是用户自定义的函数，通过三次调用 triangle 完成多边形面积的计算，使得程序的代码更加简洁，容易阅读和管理。

4.1.2 函数的定义和调用

1. 函数的定义

定义函数由关键字 def 开始，其语法格式为：

```
def  函数名([形式参数表]):
     函数体
```

说明：

①函数名必须符合标识符定义

②形式参数表由逗号间隔的一组形式参数组成（无参数时只保留小括号即可），形式参数简称形参，用于接受调用函数向函数传递数值或返回结果。

③函数体由一组语句组成，书写时右缩，体现从属关系。函数执行过程中，遇 return 语句，执行控制权返回到调用函数；无 return 语句时，则执行到函数最后。

④return 语句的格式为：

```
return  表达式                          #  表达式可以省略
```

return 的作用是，如果表达式不省略，则表达式的值被返回给调用函数。表达式省略或函数无 return 语句时，函数本身返回 None。

【例 4.2】 编写求一元二次方程实根的函数 quadricEquation(a,b,c)，并调用它求解 $2x^2+7x-3=0$ 的根。

分析：一元二次方程由三个参数 a、b、c 确定，可能求得 2 个实根，也可能求不出实根。考虑列表可以包含多个数值，因此可以将两个根组合成一个列表返回，当无实根时，返回空列表。故此定义函数如下：

```
import math
def quadricEquation(a,b,c):
    delta=b*b-4*a*c
    if delta>=0:
        root1=(-b+math.sqrt(delta))/(2*a)
        root2=(-b-math.sqrt(delta))/(2*a)
        return [root1,root2]
    else:
        return []
print('求方程  2x^2+7x-3=0  的根')
x,y,z=2,7,-3
roots=quadricEquation(x,y,z)            #函数有返回值，函数出现在表达式中
print('roots=',roots)
```

程序的运行结果为：

求方程　2x^2+7x-3=0 的根

roots= [0.386, -3.886]

本例中的函数无论走哪路 if 分支，都通过 return 语句返回，语句：

```
roots=quadricEquation(x,y,z)
```

使得变量 roots 保存函数的返回结果。

【例 4.3】编写程序，打印图 1.4.2 所示图形

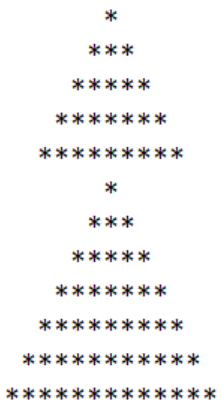


图 1. 4. 2 例 4. 3 输出图形

分析：图形由两个三角形组成，只是构成三角形的行数不同，所以可编写函数输出指定行数 n 的图形，该函数无需返回值，所以 return 语句可有可无。代码如下：

```
def pict(n):
    i=0
    while i<n:
        print("{}{}".format(" "*(10-i),"*"*(2*i+1)))
        i += 1

pict(4)                                  #制作 4 行的图，函数无返回值，调用以语句形式调用
pict(6)                                  #再制作 6 行的图
```

再如编写函数实现对任意给定的实数列表，将其偶数项元素加 1,奇数项元素减 1。由于列表是一种带分量的复合数据类型，函数中只要不对列表本身赋值，则列表可以将数值的改变带回实参，所以函数无需返回值，故定义函数如下：

```
def handleList (x):              # x 是一个列表
    for i in range(len(x)):
        if i% 2==0:
            x[i] +=1
        else:
            x[i] -=1
li=[1,2,3,4,5]
handleList(li)                   #函数无返回值，直接写函数名
print(li)
```

程序的执行结果为：

[2, 1, 4, 3, 6]

上述代码中，因为 python 变量与值采用绑定方式，函数体内只要不对列表变量 x 赋值（变量没有重新绑定），列表有能力将其含有的值返回给调用函数，所以不需要 return 语句。

对无 return 语句或 return 语句后无表达式的函数，因为函数返回 None，所以调用这种函数时，直接写函数名即可。例如，对函数 handleList 的调用语句改写成：

```
print(handleList(li))
```

则程序的执行结果为：

None

[2, 1, 4, 3, 6]

None 是函数的返回值，由于函数无返回值，所以其输出是无意义的。

2. 函数调用

函数调用与内部函数相同，调用函数的形式为：

函数名(实在参数表)

其中：

①实在参数简称实参，可以是常量、表达式、变量。

②一般情况下，实参与形参顺序一一对应，且类型与函数体内对参数的处理逻辑匹配，其作用是用实参的值初始化形参。

③为保证语义完整，有返回值的函数，函数调用应出现在表达式中（例 4.2），对于无返回值的函数，则应以语句形式出现（例 4.3）。

函数调用时的执行过程如下：

（1）计算实参的值

（2）实在参数的值传递给相应的形式参数

（3）控制转移到被调用函数，执行函数体中的语句

（4）遇到 return 语句，将 return 后表达式的值作为函数值返回到调用函数，或执行到函数体结束，将控制转回调用函数。

（5）继续执行调用函数中的后继语句。

以例 4.2 为例，其调用过程示意图如图 1.4.3 所示：

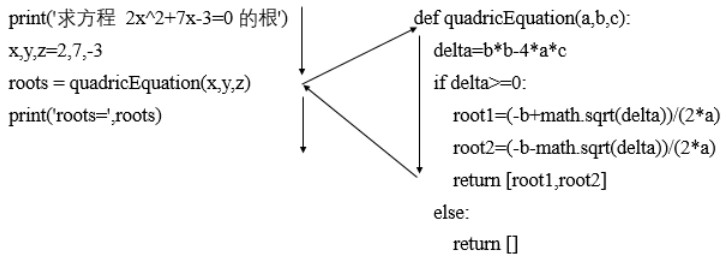


图 1.4.3 函数 quadricEquation 的调用过程

4.2 参数及传递

调用函数时，主调函数和被调用函数之间通常存在数据传递，即将实参传递给形参，然后执行函数体。Python 的形参分为三种类型：位置参数、默认值参数、可变长度参数。不同类型同时出现时，会给实参与形参的结合带来一些差异。

4.2.1 位置参数及默认值参数

1. 位置参数

形参定义时标识符前后无任何修饰的参数就是位置参数，位置参数也称必选参数。在函数调用时，必须有对应的实参。

如例 4.2 中的一元二次方程函数，函数头的定义为：

def quadricEquation(a,b,c):

参数 a,b,c 都是必选参数，调用该函数必须有三个实参。例如下面代码少给一个参数：

```
x,y,z=1,8,3
roots=quadricEquation(x,y)
```

运行时，系统的信息如下：

TypeError: quadricEquation() missing 1 required positional argument: 'c'

即系统认为对应于“c”的位置参数没有提供。

再如多给一个参数：

```
x,y,z=1,8,3
roots=quadricEquation(x,y,z,10)
```

系统提示如下信息：

TypeError: quadricEquation() takes 3 positional arguments but 4 were given

即函数 quadricEquation 带有 3 个位置参数，但调用时给了 4 个，也是不合法的。

2. 默认值参数

在函数定义时，直接被赋值的参数称为默认值参数，默认值参数又称可选参数。调用函数时，对应默认值形参的实参，可有可无。无对应实参时，取其默认值；有对应实参时，则取实参值。语法规定，默认值参数必须放在位置参数的后面。格式如下：

```
def 函数名(必选参数 1,……,必选参数 n,可选参数 1=值 1,……,可选参数 n=值 n):
    函数体
```

【例 4.4】设计函数计算下面级数的部分和

$$s = 1 + x + \frac{x^2}{2!} + \cdots + \frac{x^n}{n!}$$

要求最后一项的绝对值小于 eps，即：$\left|\frac{x^n}{n!}\right| < eps$。若用户不指定精度，则 eps 取 10^{-6}。

分析：根据题意，eps 可采用默认值参数。所以可定义函数如下：

```
def s(x, eps=1e-6):
    w=0
    t=1
    n=1
    while abs(t)>=eps:
        w +=t
        t *= x/n
        n +=1
    return w
print(s(1))                    #省略默认值参数调用
print(s(1,1e-8))               #指定默认值参数的值
```

程序运行结果为：

2.7182815255731922

2.718281826198493

可以发现两次运行结果，在百万分之一位后开始有差异。

3. 形参名称对应

函数参数较多时，按位置一一对应方式易导致程序可阅读性差，调用容易出错。此时可在调用函数时直接指定形参的值，形式为：形式参数名=表达式，这种调用方式叫做形参名称对应调用，它不必考虑形参定义时的先后次序。

一元二次方程求解函数三个参数的名字分别为 a,b,c，采用指定形参名字传递的代码如下：

```
x,y,z=1,10,3
roots=quadricEquation(a=x, c=z, b=y)        # 指定形参名字
print('指定形参名字：',roots)
```

请特别注意，这种调用方式中，一旦一个参数开始使用了形参名字调用，后续的所有参数都必须使用该方式。例如下面的语句：

```
x,y,z=1,10,3
roots=quadricEquation(a=x, z, y)            # 指定形参 a，但 b 和 c 不指定。
```

SyntaxError: positional argument follows keyword argument

系统认为，a=x 关键字调用方式，其后面也必须采用这样的方式，但现在跟随了位置参数。

而下面的调用则是合法的，因为 c=z 指定形参名字后，b 也被指定。

```
x,y,z=1,10,3
roots=quadricEquation(x,c=z,b=y)      # c=z 开始形参关键字，后续 b=z 也指定
```

4.2.2 可变长参数

某些情形下，需要处理的数据的个数是不定的，如下面的示例代码求两组数的最大值。

```
ma1=max(10,123.5,-34,-1000,12.34)
print(ma1)
ma2=max(89.1,45.1,-89)
print(ma2)
```

可以看到，两次传递给 max 函数的数据个数是不同的，但程序都实现了正确的计算。此时显然不能再用位置参数和默认值参数的方式定义函数，python 的解决方案是可变长度参数。

1. 定义及使用规则

可变长参数意指参数对应的实参个数可变。这类参数的定义采用两种方式：

①形式参数的前面带*，它将对应的一组实参转化为一个元组。

如下面代码，定义了可变长参数 arg，它搜集主函数传递的任意多个参数，函数的输出证明，实参被组成了一个元组。

```
def test(*arg):
    print(arg)
test(1,10.5,4)
```

程序运行结果为：(1, 10.5, 4)

②参数的前面带**，对应的实参以“标识符=值”的方式传递。该类可变长参数将对应的实参，以“标识符”为键，以“值”为值，转化为字典， **参数也被称为关键字参数。如下面的代码，函数的输出证明搜集的实参形成字典。

```
def test(**arg):
    print(arg)
test(a=1,b=10.5,c=4)
```

代码输出为：{'a': 1, 'b': 10.5, 'c': 4}

可变长参数必须在形参表的最后出现。所以 python 函数形参表中各种类型参数出现的次序依次为：位置参数、默认值参数、可变长度参数。

【例 4.5】 定义函数，给定一个字符串指令（求和、求平均、累乘），用可变长参数对后续的任意个实数进行相应的处理。代码如下：

```
def calcu(cmd='求平均',*data):    # data 收集多个实参形成是一个元组
    print(data)
    if cmd=='求平均':
        s=sum(data)/len(data)
        return s
    elif cmd=='求和':
        return sum(data)
    else:
        s=1
        for i in data:
            s *=i
        return s
res=calcu('求和',3,3,100)
print(res)
```

程序的运行结果为：

(3, 3, 100)

106

calcu 函数的定义中， data 是可变长参数，它搜集实参中第二个参数开始的所有参数，形成一个元组。

【例 4.6】定义函数，收集实参形成字典，并输出实参的名字和值。

```
def keyArgFunc(**keyarg):          # keyarg  是一个字典
    for x in keyarg:                          #  取字典中的每个键
        print( x + ": " + str(keyarg[x]))      # keyarg[x]取键对应的值
keyArgFunc(no=10,name='李彤',spec='学生',age='20')
```

程序的运行结果如下：

no: 10

name: 李彤

spec: 学生

age: 20

上述函数定义了一个可变长关键字参数 keyarg，调用时，no=10,name='李彤'等被关键字参数收集形成字典，所以就得到了所见的运行结果。

最后要注意的一点是，在位置形参、默认值形参与可变长关键字都存在的函数中，其调用规则不符合指定形参名字方式。因为一旦一个参数使用形参名字调用，后续参数也必须用这种模式，但可变长度形参，对应着一组实参。如例 4.5，如果将调用语句改写为 res=calcu(cmd='求和',3,3,100)，则语法要求后续的三个实参（3，3，100）也必须按形参名字传递，这与函数形参*data 的定义是不符的。

2. 实参的序列解包

对序列类型的实参，在其前加*，称为实参解包，可以配合可变长形参使用。

例如将一元二次方程的三个系数保存在一个序列中，通过实参解包调用函数求解，程序代码如下：

```
def quadricEquation(a,b,c):                    #参见例 4.2
……
L=[2,7,-3]                                     # L 是列表
print(quadricEquation (*L))                    # L 前加*，传递前解包
```

运行机制为，实参*L 解包后去掉列表的方括号，形成对应于形参 a,b,c 的三个实参。

【例 4.7】利用实参解包实现矩阵转置。

对二维列表而言，如 X=[[1,2,3],[4,5,6],[7,8,9]],其解包的结果是三个列表：[1,2,3]、[4,5,6]、[7,8,9]，利用 zip 函数，将三个列表的对应元素组合在一起形成元组，这就实现了将矩阵的列转为对应的行，再将元组转换为列表即可实现函数功能。实现矩阵转置的程序代码为：

```
X=[[1,2,3],[4,5,6],[7,8,9]]
Xt=list(map(list,zip(*X)))          # list、map、zip 函数请参见第五章
print(Xt)
```

程序的执行结果如下：

[[1, 4, 7], [2, 5, 8], [3, 6, 9]]

4.2.3 形参对实参的影响

Python 中实参和形参间传递，全部采用引用的方式，即给对象绑定标签的方式。按数据类型，可以分成简单类型和复杂类型形参进行讨论。

1. 简单类型形参

简单类型的形式参数，要对其值进行变更，必须对其进行赋值，导致形参重新绑定到其它数值上，所以不会影响实参的值。

【例 4.8】寻找三个数 x，y，z 中最大者。代码如下：

```
def getMax(x,y,z,Max):
    if x<y:
        x,y=y,x
    if x<z:
        x,z=z,x
    Max=x
    print('函数求得最大=',Max)
x,y,z=10,47,-1
Max=x
getMax(x,y,z,Max)
print('主函数中 Max=',Max)
print('x={},y={},z={}'.format(x,y,z))
```

程序的运行结果如下：

函数求得最大= 47

主函数中 Max= 10

x=10,y=47,z=-1

从程序运行的结果看，函数中实现了寻找最大值，但最大值没有通过形参 Max 返回给调用函数，同时，函数中对 x, y, z 的值进行了变更，但主函数中它们的值并没有改变。

2. 复杂类型形参

复杂类型是指变量本身含有分量，如序列及后续课程内容中的面向对象编程中的对象。这类形参值的改变又可分为两种情况，形参本身值和形参分量值的改变。

①形参本身值改变

形参本身值的改变，使得形参变量标签被重新绑定，所以不会影响实参的值。

【例 4.9】证明形参本身值改变不影响实参，编写函数求得斐波那契数列的前 n 项

```
def fibs(ans,n):
    for i in range(n-2):
        ans =ans+[ ans[-1]+ans[-2] ]
    print(ans)
start=[0,1]
fibs(start,5)
print(start)
```

程序的运行结果如下：

[0, 1, 1, 2, 3]

[0, 1]

可以发现，函数中实现了求取斐波那契数列，但主函数的中的 start 值并没有改变。其原因在于函数中对形参 ans 进行了重新赋值。

②形参分量值改变

形参分量值改变，不会导致形参本身的重新绑定，所以会导致实参的变更。

【例 4.10】编写函数验证形参分量的改变对实参的影响，求取数值列表每项的平方

```
def power2(li):
    for i in range(len(li)):
        li[i]=li[i]**2
    print('函数内：',li)                          # 验证函数内部形参值的变化

start=[1,2,3,4,5]
power2(start)
print('主程序：',start)
```

程序的执行结果如下：

函数内： [1, 4, 9, 16, 25]

主程序：[1, 4, 9, 16, 25]

可以发现，由于函数中只是对形参的分量进行赋值，不响应形参本身的绑定，所以形参导致了实参值的变更。

总结而言，形参对实参值的改变遵循以下原则：

①函数体的实现语句中，如果形参出现在“=”的左侧，导致其地址变更（标签再挂），则后续操作中，形参值的改变不影响实参的值。

②形参是列表、对象等复杂类型，形参分量值的变更，将导致对应实参的变更。因为这种操作不会影响形式参数的绑定。

4.2.4 变量的作用域

变量的作用域，是指程序代码对变量使用的有效范围。在函数外定义的变量，被称为全局变量，函数内部也可以定义变量，称为局部变量。

1. 全局变量

全局变量定义后可以被整个程序后续代码使用。可以分为两种情形：隐式使用和显式声明使用。

①隐式使用

如果在函数中使用了一个变量的值，而该变量在函数中并未定义，但调用函数前，有同名的全局变量存在，则函数中使用的是全局变量。

【例 4.11】隐式使用全局变量

```
def fx(n):
```

```
    z=100*x                 # 函数内未定义 x，但调用函数前，定义了全局的 x
    print("z=",z,'x=',x)

x=1                         #全局变量 x
fx(3)
print ("全局的 x=", x)
```

看程序的运行结果：

z= 100 x= 1

全局的 x= 1

可以发现，函数中使用的 x，是全局变量 x。

请特别注意，在函数内，变量处于赋值符的左边，则是函数内定义的局部变量，出现在赋值符“=”的右侧，则是引用全局变量。

②显式声明指定全局变量

为避免全局变量在函数中未定义就使用而引起的不确定性，可通过 global 关键字强调使用全局变量。

【例 4.12】显式使用全局变量

```
def fx(n):
    global x                # 明确使用全局变量 x
    z=100*x
    print("z=",z,'x=',x)
    x=1000                  # x 是全局的
    print('x=',x)
x=1
fx(3)
print ("全局的 x=", x)
```

程序的运行结果:

z= 100 x= 1

x= 1000

全局的 x= 1000

此时，即使函数中将变量 x 处于赋值符的左边，x 仍是全局变量，会影响其在主程序中的取值。

2. 局部变量

函数中定义的变量，包括形式参数，都是函数的局部变量，只可在其定义的函数内访问。

【例 4.13】局部变量覆盖全局变量及局部变量的全局访问测试

程序中定义了全局变量 x，在函数 fx 内定义了局部变量 x，函数体内它将覆盖全局变量 x，主函数中尝试访问局部变量 z。代码如下：

```
def fx(n):
    x=1000*n                # 函数内，局部的 x
    print("局部的 x=",x)
    z=10
```

```
x=1                          # 全局 x
fx(3)
x+=1
print ("全局的 x=", x)        # 主程序，使用全局的 x
print("访问局部变量 z=",z)
```

程序的运行结果如下：

局部的 x= 3000

全局的 x= 2

print("访问局部变量 z=",z)

NameError: name 'z' is not defined

可以发现全局的 x 和局部的 x 互不影响。但主程序中对 z 的访问导致程序出错。

4.3 lambda 函数

lambda 函数指通过 lambda 表达式定义的函数，是对逻辑特别简单的函数的快速实现。lambda 表达式是一个匿名函数，即没有函数名的函数，可以在某些场合直接使用，如对象列表的排序等。lambda 表达式的格式为：“ ”

lambda 形参表:表达式

如果给 lambda 表达式赋予名称，它就是 lambda 函数，其格式为：

函数名=lambda 形参表:表达式

其中，表达式的值是函数的返回值，函数的调用方式仍然是：函数名(形参表)。

【例 4.14】 用 lambda 函数计算给定列表的平均值和标准偏差。

```
aver=lambda L: sum(L)/len(L)
sigma=lambda L,aver:math.sqrt(sum([(i-aver)**2 for i in L])/(len(L)-1))
Li=[1,2,100,45]
average=aver(L)
print(sigma(L,average))
```

程序的执行结果如下：

37.0

46.74041791283714

上面例子中定义了两个 lambda 函数，aver 函数实现求给定列表 L 的平均值，sigma 函数实现求列表 L 的标准偏差。

4.4 递归函数

递归是程序设计的一种重要方法，递归函数具有结构清晰、可读性强的特点。

4.4.1 什么是递归

在日常生活中，递归是一种常见的现象。例如，两面相对的镜子所产生的被称为德罗斯特效应的“像中像”。

在数学中，许多问题都是通过递归定义的。例如，斐波那契数列的递归定义为：

$$\text{fibs(n)} = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ fibs(n-1) + fibs(n-2) & n \geq 3 \end{cases}$$

即 fibs(n)可以分解为 fibs(n-1)和 fibs(n-2)两个字问题。在程序设计中，递归是一种与迭代相对应的设计思想。当一个问题可以转化为规模较小的同类子问题时，就可以使用递归。递归函数是指在定义时直接或间接调用自身的函数。递归分成两种类型：

① 直接递归：一个函数直接调用自身。例如，函数 f 调用自身，记作 f ⟶ f

② 间接递归：一个函数调用其他函数，而其他函数又调用该函数。例如，函数 f 调用函数 g，而函数 g 又调用函数 f，记作 f ⟶ g ⟶ f。

在实际应用，常见的是直接递归，间接递归较少应用。

【例 4.15】编写递归函数 fact(n)，求阶乘 n!程序。

假定求 n!的函数为 fac(n)，根据 n!的定义可知递推公式为：

$$\text{fact(n)!=} \begin{cases} 1 & \text{n=1} \\ \text{n * fact(n-1)} & \text{n>1} \end{cases}$$

因此，可以编写如下 fac(n)函数并且加以调用：

```
def fact(n):
    if n==1:                          # 递归结束条件
        return 1
    else:
        return n*fact(n-1)            # fact 函数调用自己计算(n-1)!
n=int(input())
result=fact(n)                        # 调用 fact 函数
print(result)
```

在函数 fact 定义中，n==1 是递归结束条件（又称递归边界条件）。若没有递归结束条件，递归将无限进行下去；当 n>1 时，调用 fact(n-1)求(n-1)!，然后再求 n * fact(n - 1)。

分析程序执行过程，如图 1.4.4 所示，递归的过程可以分成两个子过程：递推和回归，如图 1.4.5 所示。

① 递推　即 fact(4)调用 fact(3)，fact(3)调用 fact(2)，fact(2)调用 fact(1)，fact(1)直接计算。

② 回归　当 fact(1)计算完毕后，先由 fact(1)计算得到 fact(2)，再由 fact(2)计算 fact(3)，最后由 fact(3)得到 fact(4)。

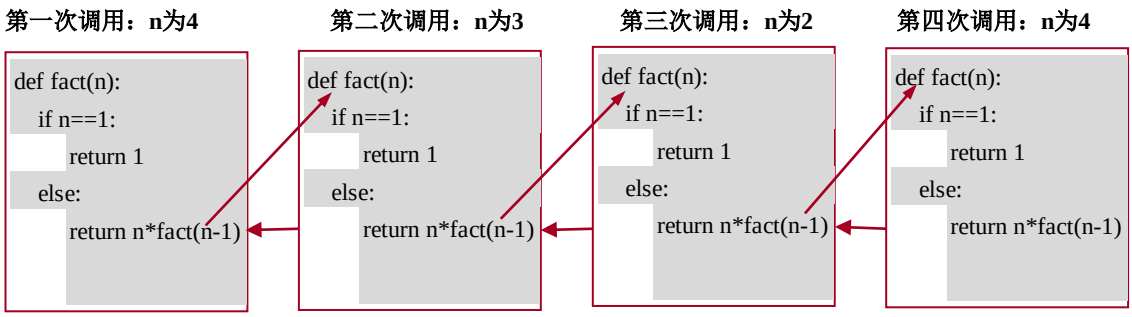


图 1.4.4　求 4!的执行过程

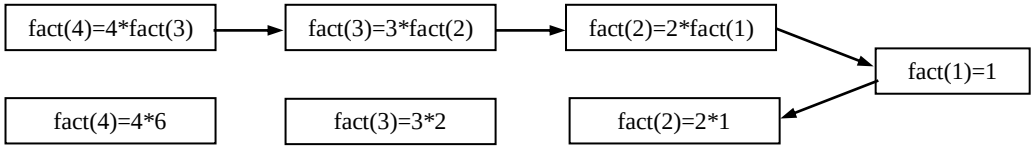


图 1.4.5　求 4!的递推和回归过程

从问题求解的角度来说，计算 n!问题可以分解成更小的问题（子问题）：计算(n-1)!问题，当计算(n-1)!问题解决后，计算 n!问题也就解决。由于计算(n-1)!是计算 n!的子问题，并且算法是相同的，仅仅是参数值不同，因此可以用递归解决。

4.4.2 递归函数的设计

递归函数的设计过程一般分成三个步骤：

1．问题分解

不是所有问题都可以或适合用递归求解。一个问题是否适合用递归求解，可以从两个方面思考。

① 一个问题很小时，是否有一个直接的结果，即递推的结束条件。

② 当一个问题不小时，是否可以将其分解为一个规模较小的问题，且可以一直分解下去。

以字符串逆转为例，就符合上述 2 个条件。

当字符串 s 中只有一个字符时（最小化时），其逆转就是本身。

当字符串 s 中有多个字符时，其每次操作是字符串的分片 s[1:]逆转的结果和第一个字符 s[0]的+运算，而 s[1:]比 s 本身少了一个字符，规模缩小了。

2．归纳出递归模式

递归模式是根据问题规模的逐步缩小而总结出来的逻辑关系，是递归程序算法实现的关键点。如字符串的逆转问题。设字符串 s="ABC"，将其转化为"CBA"的过程，可归纳出字符串的递归模式如下：

子字符串 s[1:]的逆转与 s[0]的+运算

3.编写代码

根据递推公式，遵循一般函数的定义语法编写递归函数，特别强调递归的终止条件及问

题分解规则。

【例 4.16】 编写函数实现字符串的逆转

```
def rev(s):
    if len(s)==1:
        return s
    else:
        return rev(s[1:])+s[0]
print(rev("ABCDEF"))
```

【例 4.17】用递归方法编程求两个正整数的最大公约数。

求最大公约数的递推公式为：

$$gcd(m,n)!= \begin{cases} n & m \,\%\, n = 0 \\ gcd(n, m \,\%\, n) & m \,\%\, n \neq 0 \end{cases}$$

分析：求 m 和 n 的最大公约数 gcd(m,n)可以分解成子问题：求 n 和 gcd(n, m % n)的最大公约数。由于求解子问题和整个问题的算法是相同，仅仅是参数不同，如图 1.4.6 所示，因此可以用递归方法解决。递归的结束条件为 m % n=0，此时最大公约数为 n。

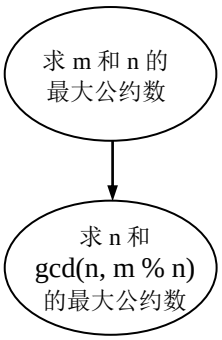


图 1.4.6　求最大公约数问题

程序如下：

```
def gcd(m,n):
    if (m % n==0):                          # 递归结束条件
        return n
    else:
        return gcd(n,m % n)                 # 递归调用
m,n=eval(input())
result=gcd(m,n)
print(result)
```

【例 4.18】汉诺塔问题，又称为“世界末日问题”。该问题是：有 A、B、C 三根柱，A 柱上从下到上放了半径由大到小的六十四片圆金片，要求借助 B 柱，把这些金片全部由 A 柱移到 C 柱，一次只能移一片。移动时，金片的大小次序不能颠倒，小片永远只能放在大片上面。盘子的搬迁的过程如图 1.4.7 所示：

汉诺塔移动的过程，是借助 C 柱移动到 B 柱，再借助 A 柱移动到 C 柱，其过程循环往复，因而可以总结出如下的递归步骤：

①当只剩下一个盘片时，n=1，递归结束

②将上面的 n-1 块盘子，借助 C 柱，移动到 B 柱（子问题 1）

③将最下面的第 n 块盘子，从 A 柱移动到 C 柱

④借助 A 柱，将 n-1 块盘子，从 B 柱子移动到 C（子问题 2）

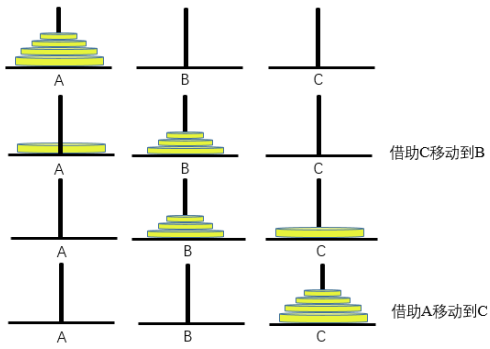
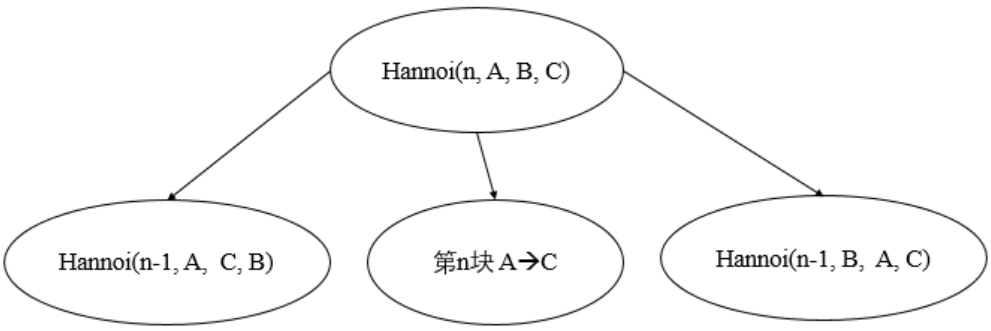


图 1.4.7 汉诺塔盘片的移动逻辑

经过问题分解，n 个盘子的移动被分解移动 2 个 n-1 块盘子和移动第 n 块盘子的问题，因而可以可以用递归方法实现，如图 1.4.8。



n-1 块盘子 A→B
子问题 1

n-1 块盘子 B→C
子问题 2

图 1.4.8 汉诺塔问题分解

设移动总盘数为 n，A、B、C 分别用字符“A”、“B”、“C”表达。函数返回移动的次数，汉诺塔移动的递归函数可定义如下。

```
def Hanoi(n, A, B , C):
    if n == 1:
        print(A+"→"+C)
        return 1
    else:
        t1=Hanoi(n - 1, A, C, B)          #n-1 盘子，借助于 C，从 A 移到 B
        print(A + "→" + C)                # 最后一个盘子，从 A 移动到 C
        t2=Hanoi(n - 1, B, A ,C)          # n-1 个盘子，借助于 A，从 B 移到 C ，
        return t1+t2+1
```

```
times=Hanoi(3,"A","B","C")                    # 借助列表的值，函数中改变可传递回来
print('移动次数=',times)
```

主函数的调用中，设盘子总数为 3，程序的运行结果如下：

A→C

A→B

C→B

A→C

B→A

B→C

A→C

移动次数= 7

【例 4.19】 根据 4.4.1 描述的斐波那契数列每项的计算公式，编写递归函数求解数列的任意指定项。

分析：4.4.1 节的公式中已经完整描述递归的终止条件及递归的问题分解，按递归公式可以编写函数代码如下：

```
def fibs(n):
    if n==1:
        return 0
    elif n==2:
        return 1
    else:
        return fibs(n-1)+fibs(n-2)

print(fibs(2))
```

代码运行结果为 1.

4.5 综合应用

函数是实现程序模块化设计的基础，是面向对象程序设计的前导内容，是程序设计中锻炼问题抽象的重要步骤。本节通过一些综合案例进一步加强读者对函数设计的理解。

1. 二分法求高次方程的根

二分法是求解一元高次方程根的常用方法之一，参见图 1.4.9，设函数 f(x)连续，如果通过测试找到两个点 a，b，使得 f(a)*f(b)<0，则可以断定，函数在[a，b]中必定存在一个根。求解过程为：取中间值 m=(a+b)/2，判断 f(m)的符号与 f(a)是否同号，如果同号则令 a=m，否则令 b=m，不断缩小求解区间，并且保持 f(a)*f(b)<0，当 a 和 b 无限接近时，例如 abs((a-b)/a)<1e-6，则认为找到了解。

化学中弱酸弱碱溶液 pH 求解问题是一个一元高次函数，以求解浓度为 c mol/L 的任意一元弱酸溶液 pH 值为例，该类溶液都可以根据弱酸的解离方程、水溶液质子平衡、水的离

解常数等化学平衡知识，经过推导，将氢离子浓度 x 表达为如下方程：

$$x^3 + K_a x^2 - (K_a c + K_w)x - K_a K_w = 0$$

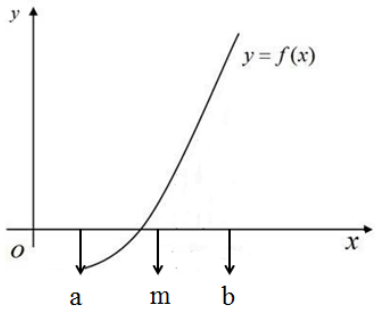


图 1.4.9 二分法求高次方程的根示意图

其中 ka 是一元弱酸的电离常数，对给定的酸是固定值，如醋酸的值为 $10^{-4.56}$， Kw 是水的电离常数，为 10^{-14}。

【例 4.19】求解指定浓度的醋酸溶液的 pH 值，将高次方程和二分法分别用函数实现

该问题可以归纳为两个函数，一个是高次方程本身，它是 x 和浓度 c 的函数。为调用函数方便起见，规定函数中使用的 c 是全局变量，所以可以定义如下：

```
def fx(x):
    global c                    #  使用全局变量
    ka=10**(-4.56)              #醋酸的 ka
    kw=1e-14
    kakw=ka*kw
    f=x**3 + ka * x**2 -(ka * c + kw)*x-kakw
    return f
```

问题中的另一个函数是二分法求解逻辑，它接收给定的区间[a，b]、高次方程函数作为二分法函数的参数，根据算法原理求根并返回。所以可以定义函数如下：

```
def biAlgorithm(a,b,func):      # func 对应高次方程函数，作为二分法的参数
    fa=func(a)                  # func 对应 fx，fx 中使用了全局的 c，简化处理
    while (abs(a-b)>1e-10):
        m=(a+b)/2
        fm=func(m)
        if (fa*fm>0):
            a=m
        else:
            b=m
    return a
```

在上面两个函数支持下，主程序的任务是获取溶液的浓度，找到合理的区间，并传递给二分法函数。所以主程序定义如下：

```
fa=1
fb=1                            # fa 和 fb 可以随意取值，保证进入下面的 while 循环即可
c=float(input('请输入醋酸浓度:'))
while(fa*fb>0):                 # while 循环寻找合理区间
    a=float(input('请输入左区间：'))
```

```
        b=float(input('请输入右区间'))
        fa=fx(a)
        fb=fx(b)

    answer=biAlgorithm(a,b,fx)
    import math
    pH=-math.log10(answer)
    print(pH)
```

程序的运行过程及结果如下：

请输入醋酸浓度:0. 01

请输入左区间：0

请输入右区间 1

pH= 3.291394771168757

2. 递归制作树形分型图

树形分形图的图元由一个主树干和树干上生长的 3 个子枝形成，其问题分解步骤是：

（1）从主干底部计算，分别在 1/3，1/2，2/3 的位置上，以左转 30 度、右转 30 度、左转 30 度的角度，分别绘制树枝 A_1B_1、A_2B_2、A_3B_3,三个树枝的长度分别为主干的 0.5、0.5、0.5，如图 1.4.10 所示。

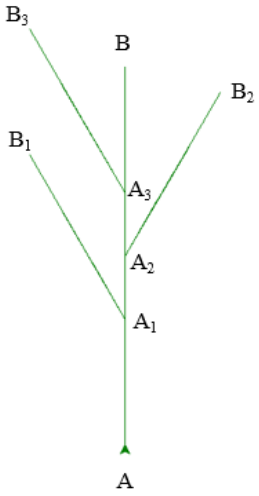


图 1.4.10 树形图元

（2）将每个树枝作为主树干，重复上述步骤 1 的工作。

（3）终止条件：当树干的长度小于 20 时，递归终止。

（4）turtle 模块

为利用制图展现程序的递归过程，采用小乌龟 turtle 模块实现递归程序。turtle 以一个小海龟的爬行轨迹来完成制图，小海龟的初始状态为头向右，通过旋转、前进、后退、提笔、落笔等函数实现制图。应用 turtle 制图时，先在程序中通过下列语句引用

```
import turtle
```

之后，可以参阅表 1.4.1 完成制图。

表 1.4.1 turtle 制图函数

函数	作用
turtle.color(color)	设置轨迹颜色为 color
turtle.penup()/pu()	提笔
turtle.pendown()/pd()	落笔
turtle.right(x)	向右旋转 x 度
turtle.left(x)	向左旋转 x 度
turtle.forwardward(x)	沿当前方向前进 x 像素
turtle.backward(x)	沿当前方向后退 x 像素

turtle.goto((x,y))	从当前位置走到(x,y)位置

【例 4.20】 结合 turtle 制图原理与树形图案的实现原理，设计递归函数，制作树形分型图。

分析：由于制图过程中多次左右旋转角度，使程序的逻辑较难控制，为简单起见，每制作一个子枝时，都采取回到当前主干的底部再制作其他子枝的方案。所以小乌龟爬行的方案制定如下：

画当前 AB 主干，回到主干底部

前进 1/3，左转 30°，画 A_1B_1 树枝，长度为主干的 0.5，再右转 30°，退 1/3，回树干底部

前进 1/2，右转 30°，画 A_2B_2，长度为主干的 0.5，再左转 30°，退 1/2，回到树干底部

前进 2/3，左转 30°，画 A_3B_3，长度为主干的 0.5，再右转 30°，退 1/2，回到树干底部

树干长度小于 20 时，停止递归

设计函数的代码如下：

```
import turtle as tt
import turtle
def draw_brach(brach_length):
    if brach_length>20:
        #画树主干
        turtle.forward(brach_length)
        turtle.backward(brach_length)          #返回底部
        #画低位树枝
        turtle.forward(brach_length/3)
        turtle.left(30)
        draw_brach(brach_length*0.5)
        turtle.right(30)
        turtle.backward(brach_length/3)        #回树枝最底部
        # 绘中枝
        turtle.forward(brach_length*0.5)
        turtle.right(30)

        draw_brach(brach_length*0.5)
        turtle.left(30) # 返回
        turtle.backward(brach_length*0.5)      #回树枝最底部
        # 绘上枝
        turtle.forward(brach_length*2/3)
        turtle.left(30)
        draw_brach(brach_length*0.5)
        turtle.right(30)
        turtle.backward(brach_length*2/3)      #回树枝最底部
```

```
turtle.left(90)                          # 本来向右，改为向上，位置屏幕中央
turtle.penup()                           #提笔
turtle.backward(150)                     #向下退 150 像素
turtle.pendown()                         #落笔
turtle.color('green')
draw_brach(400)                          # 主树干 400 像素
turtle.exitonclick()                     # 点击鼠标退出制图界面
```

设定树枝最短长度为 20，制作的分形图如图 1.4.11 所示。

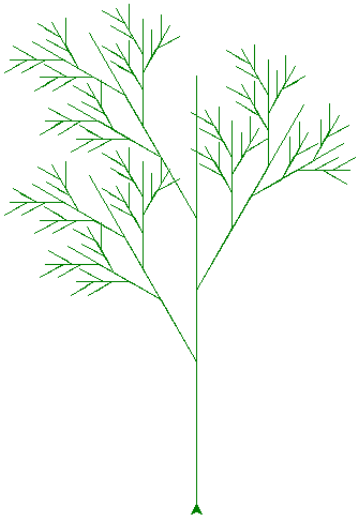


图 1.4.11 树枝最短长度为 20 的树形分型图

3. 统计字符串 s 中每个字符出现的次数

分析：函数接受给定的字符串 s，首先通过 set 函数找到不重复的字符列表 L，通过遍历 L，对每个字符 c，用字符串的 count 方法统计 c 出现的次数，也整理成一个列表 freq，返回 L 和 freq 即可实现任务。

【例 4.21】 编写函数 charFreq,它接受字符串 s，返回字符串中每个独立字符及其出现的次数。

```
def charFreq(s):
    chars=list(set(s))                   # 整理不重复的字符
    chars.sort()
    freq=[]                              #记录字符出现次数
    for char in chars:
        v=s.count(char)
        freq=freq+[v]
    return chars,freq

str1=input('请输入字符串：')
charList,freqList=charFreq(str1)
print(charList)
print(freqList)
```

程序运行的过程如下：

请输入字符串：He is a teacher of this university.His name is Yange

[' ', '.', 'H', 'Y', 'a', 'c', 'e', 'f', 'g', 'h', 'i', 'm', 'n', 'o', 'r', 's', 't', 'u', 'v', 'y']

[9, 1, 2, 1, 4, 1, 6, 1, 1, 2, 6, 1, 3, 1, 2, 5, 3, 1, 1, 1]

习题

一、选择题

1. 以下描述函数定义时函数名及参数的格式，正确的是_____

A 函数名符合标识符定义即可，后必须跟小括号，参数逗号间隔放小括号内，后跟冒号

B 函数名不能使用函数库里的函数，否则报语法错

C 函数没有参数的时候，可以省略函数名后的小括号

D 函数没有参数的时候，也必须有小括号，小括号内放置一个逗号。

2. 当定义含有位置参数、默认值参数、可变长参数的函数时，参数的顺序必须遵守_____

A 位置参数、默认值参数、可变长参数

B 默认值参数、位置参数、可变长参数

C 可变长参数、位置参数、默认值参数

D 默认值参数、可变长参数、位置参数

3. 设函数 f 没有 reutrn 语句，则下列调用函数的语句，正确的是_____

A y=f(……)

B print(f(……))

C f(……)

D f(……)=y

4. 定义函数头如下：f(x,y,z='C')，设参数逻辑实现时以 x,y 为浮点数，z 是字符串类型，函数有返回值。则以下调用语句，正确的是_____。

A f(1)

B s=f(1.0,2,3)

C print(f(1,2.0,'A'))

D print(f(x=1.0,2,'A'))

5. 设定义函数 fit(k,n_component=1)，假设函数有返回值,则下面调用语句不正确的是_____

A print(fit(1)) B print(fit(1,2))

C print(fit(k=1,n_component=2)) D print(fit(k=1,1))

6. 下列函数定义错误的原因是_____

f(X,*paras,z=1)

A 参数的前面不能有*号

B 参数 X 不能大写

C 按类型，参数书写顺序必须是 X,z 和*paras

D z=1 错误，不能给参数常数值

二 填空题

1. 运行并记录函数的结果

x=12.6789

round(x) 结果：_____

round(x,2) 结果：_____

2. 记录代码的输出结果

a,b=2,4

s=eval('a*a+10*b')

print("s=",s) 结果：_____

3. 记录 math 库中函数的结果

x=2

y=3

pow(x,y) 结果：_____

exp(x) 结果：_____

sqrt(x) 结果：_____

sin(x) 结果：_____

log10(x) 结果：_____

degrees(x) 结果：_____

4. 记录函数的结果

s="Tongji"

s.upper() 结果：_____

s.islower() 结果：_____

s.isnumeric() 结果：_____

t="When I was young "

t.split(" ") 结果：_____

s.count("n") 结果：_____

s.replace("n", "t") 结果：_____

5. 下列函数执行后，程序的执行结果分别为_____

```
def foo(t):
    t=1+1
    return t
t=1
print(foo(t),t)
```

6. 下列函数执行后，程序的执行结果分别为_____

```
def foo(t):
    t[0]=t[0]+1

t=[1,2]
foo(t)
print(t)
```

7. 下列代码的执行结果为_____

```
k,v=3,-1
s=eval('k**2+10*v')
print("s=",s)
```

8. 定义函数如下：

```
def  average(*Li):
    return sum(Li)/len(Li)
y=average(1,2,9,8)
print(y)
```

程序的执行结果为_____

9. 下面程序利用牛顿迭代法求方程 $f(x)=x^3-4x^2+5$ 的根，请将程序补充完整，牛顿迭代法请自行查阅。

```
def f(x):                                   # 原函数
    y = x**3-4*x*x+5
    return y
def f1(x):                                  #函数的一阶导数
    y=3*x*x-8*x
    return y
def nt(x):                                  # 牛顿迭代法
    while True:
        x0=___(1)_____
        y1=f(x0)
        y2=___(2)_____
        x= x0-y1/y2
        if(abs(f(x))<1e-6):
            ___(3)_____
    return(x)
x0 = 100
root=___(4)_____
print("Root=%7.5f"%(root))
```

10. 辗转相除法求两个整数 m,n(m>n)的最大公约数，其原理是取 r 为 m 和 n 的余数，如果 r=0,则 n 就是最大公约数，程序终止，否则令 m=n，n=r，然后再继续求新的 m 和 n 的最大公约数，所以可以用递归法实现，请填写空格中的代码使程序完整。

```
def gcd(m, n):
    if ___(1)____:
        return ____(2)
    else:
```

```
        r=__(3)____
        m=n
        return   gcd(___(4)___)
print(gcd(12,18))
```

三．简答题

1．简述为何要编写函数，用户自定义函数给编程带来哪些优势？

2．请简单描述用户自定义函数的格式，形式参数有哪几种类型？各在何种情况下使用？

3．函数调用参数传递时，实参与形参有几种匹配传递方式，使用时需要注意哪些规则？

4．lambda 函数的定义格式是什么？在什么情况下适合使用 lambda 函数。

5．用递归法求解问题的关键步骤有哪些？如何分析一个问题是否适合用递归方法求解？

第五章 列表、元组、字典与集合

在各种类型的计算机应用中，数据常常被组织成线性表、队列、堆栈、树、图等数据结构。在这些数据结构中，应用最为广泛的是线性表，在 Python 中可以用列表和元组实现。列表和元组是最常见的序列类型，相当于其他程序设计语言中的数组。列表和元组的基础知识在第二章“数据表示与处理”中已有过初步的介绍，本章将深入、系统地进一步讨论。另外，由于字典和集合是无序序列，本章将一起介绍。

5.1 列表与元组

列表是 Python 处理数据的利器，它提供的方法可以快速完成数据的操作：分片、排序、统计等等。由于列表元素的数据类型没有限制，所以列表有很强的适应性和灵活性。例如，列表的元素也可是列表类型，因而可以容易表达科学计算中的矩阵。元组与列表相似，主要在操作上面有限制，所以被认为是轻量级的列表。

5.1.1 引例-学生成绩计算

下面首先通过一个引例说明为什么要使用列表，然后展开相应的知识。

【例 5.1】输入 100 位学生的成绩，求所有学生的平均成绩和标准差。

标准差的计算公式为：$\text{std} = \sqrt{\sum_{k=1}^{n}(x_k - \bar{x})^2/(n-1)}$

分析：如果不使用列表存储学生成绩，那么学生成绩就要输入两次。因为第一次输入在计算平均成绩时没有保存下来，所以计算标准差时要再一次输入。

```
import math
n=100                                   #学生人数
total=0                                 #记录所有学生的成绩之和
for i in range(n):
    x=int(input("请输入成绩："))         #为计算平均值输入成绩
    total += x
ave=total/n                             #计算平均成绩
s=0                                     #记录标准差
for i in range(n):
    x=int(input("请输入成绩："))         #为计算标准差输入成绩
    s=s+(x-ave)**2
std=math.sqrt(s/(n-1))                  #计算标准差
print('总平均=',ave,'标准差=',std)
```

如果使用列表存储学生成绩，那么只需要输入一次就可以计算平均成绩和标准差。

```
import math
n=100
Scores=[]                                   # 记录学成成绩的列表
for i in range(n):
    x=int(input("请输入成绩："))
    Scores.append(x)                        #追加当前输入成绩到列表中
total=0
for x in Scores:                            #遍历列表计算成绩之和
    total=total+x
ave=total/n
s=0
for x in Scores:
    s=s+(x-ave)**2                          # 用列表存储的数据计算标准差
std=math.sqrt(s/(n-1))
print('总平均=',ave,'标准差=',std)
```

当学生成绩用列表存储时，可以使用列表提供的方法和内置函数直接计算，使代码更加简洁。

```
import math
n=100
Scores=[]
for i in range(n):
    x=int(input("请输入成绩："))
    Scores.append(x)
ave=sum(Scores)/len(Scores)                 #利用内置函数操作列表计算平均成绩
lst=[(x-ave)**2 for x in Scores]            #列表推导式计算每个成绩与平均成绩差的平方
std=math.sqrt(sum(lst)/(n-1))
print('总平均=',ave,'标准差=',std)
```

上面的代码段中，Scores 是一个列表，它可以存储大量的数据，为计算提供了存储支撑，列表还支持列表推导式快速生成新的列表 lst，支持使用内置函数 sum、len 完成计算，提高编程效率。

5.1.2 列表操作

列表操作是指对列表的数据进行管理、查询、运算等操作方法，在前面章节中已经涉及，现在系统地介绍。

1．遍历列表

遍历列表指通过循环引用列表的每个元素。遍历列表有三种方式：枚举方式、下标方式、枚举加索引方式。

（1）枚举方式

枚举方式是指将列表中的每个元素直接取出进行处理，其使用形式为：

```
for item in L：
    对 item 进行处理
```

说明：按列表中元素的顺序，每执行一次循环体就取一个元素，直到取完所有的元素。

（2）索引方式

采用 for 语句配合 range 函数完成，range 函数根据列表的长度生成列表的所有索引值，其格式为：

```
for i in range(len(L)):
    对 L[i]进行处理
```

（3）枚举加索引方式

遍历列表时有些场合需要同时知道列表元素的索引和值，就应借助 enumerate 函数（enumerate 函数在第 5.1.4 中详细介绍）实现，其使用形式为：

```
for i, item in enumerate(L):
    对 i 和 item 进行处理
```

说明：循环时，i 和 item 存在对应关系，item 是第 i 个元素的值。例如，假定执行如下的代码：

```
L=[['李成',80],['赵华',61],['华连',91]]
print('带序号的枚举遍历结果：')
for i, item in enumerate(L):
    print('第',i,'项：',item)
```

则代码运行结果为：

```
第 0 项： ['李成', 80]
第 1 项： ['赵华', 61]
第 2 项： ['华连', 91]
```

列表遍历的三种方式，各有优势。第一种方式最简单，直接按顺序枚列表中的元素；第二种方式枚举列表元素的索引，根据索引访问列表元素；第三种枚举列表元素的索引和值。这三种方式各有场合，当遍历的目的是对列表元素本身值变更时，应该采用第二或第三种方式。

如下面代码尝试将列表的每个元素平方，分别用枚举方式和下标方式。

```
R=[1,3.47,8.21,65]
for item in R:                  #枚举方式
    item=item**2
print(R)
for i in range(len(R)):         #下标方式
    R[i]=R[i]**2
print(R)
```

运行结果为：

```
[1, 3.47, 8.21, 65]
[1, 12.0409, 67.4041, 4225]
```

可以发现，枚举方式列表每个元素的值没有改变，但使用下标方式则达到了目的。因此，在遍历列表时，如果需要改变元素的值，则应该按下标索引的方式完成。

2．删除列表

使用 del 函数可以删除列表，其使用形式为：

del 列表

例如，执行下面的代码：

```
Li=[1,4,9]
del Li                                    # 删除列表
print(Li)
```

运行时显示“NameError: name 'Li' is not defined”，表示 Li 被删除后，它被从内存清理了，不再存在列表 Li。

3．列表分片

（1）分片

列表分片的基础请参阅第二章字符串的介绍，这里在前述基础上对分片作进一步明确。列表的分片格式为：

列表名称[start:end:step]

说明：两个冒号，至少出现一个，如写成[:]的形式，此时该冒号是指 start 和 end 之间的冒号，结果为列表的拷贝。

分片操作中，为了得到有意义的结果，在步长为正数时，end 应大于 start；步长为负数时，start 应大于 end。否则，得到的是空列表[]。

（2）分片赋值

分片赋值指将列表指定位置分片的元素值变更为另一个列表的值。这种赋值能力可以方便删除列表原有的部分元素或插入一组新的元素。例如，将一个空列表付给一个分片，就可以将该分片包含的元素全部删除，而将一个列表赋值给一个空分片，则可以将其插入到列表中。请见下面的示例代码：

```
L=[1,2,3,4,5]
L[1:1]=[-9,-8,-7]
print(L)
```

结果为：[1, -9, -8, -7, 2, 3, 4, 5]

L[1:1]是一个空表，将该空表赋值为[-9,-8,-7]，实现将新元素插入原列表的指定位置。

也可以将指定的分片赋值空表，实现元素的删除，如下面的代码：

```
L=[1, -9, -8, -7, 2, 3, 4, 5]
L[1:4]=[]
print(L)
```

结果为：[1, 2, 3, 4, 5]

（3）多维列表的分片

无论列表有几维，列表的分片只支持最外层的分片，对于二维列表而言，如果需要在列方向上实现分片是比较遗憾的。如下面的二维列表分片：

```
L=[[1, -9], [-8, -7], [2, 3], [4, 5],10]
L1=L[1:3]
print(L1)
```

结果如下：[[-8, -7], [2, 3]]

4．列表运算

列表运算符不多，只有+、+=和*、*=、成员运算符 in。其中+、*运算符请参阅第二章的描述，需要提醒的是，对列表而言，下面两种语句的计算结果虽然相同，但实现机制不同。

```
a=a+b
a +=b
```

其区别在于，第一种方式 a 列表的首地址变更，而第二种方式，a 保持原来的首地址。

in 运算符用于判断一个元素在列表中是否存在，存在时返回 True，否则返回 False。其格式为：

元素 in 列表

【例 5.2】已知一个列表中有某些元素重复，编程去掉重复元素，形成一个新列表。

分析：首先生成一个空列表，然后遍历原来的列表，每从原列表中取出一个元素后，判断其在新表中是否存在，不存在就将其加到新列表，遍历完原列表后，新列表就是所求结果。程序的代码如下：

```
L1=[1,3,6,3,3,7,8,1,1,1]
newL=[]
for i in L1:
    if not i in newL:
        newL +=[i]
print(newL)
```

程序运行结果如下：[1, 3, 6, 7, 8]

5.1.3 列表对象方法

列表作为一种对象，有自己的方法，即列表自身拥有的函数。利用这些方法可以快速完成相应的操作，例如追加、删除、排序、插入等。

列表方法的使用形式如下：

```
列表名称.方法(参数)
```

下面介绍列表常用方法。

1．append 方法

append 方法用于在列表的最后追加一个新元素，其使用形式为：

列表名称. append(新元素)

【例 5.3】已知斐波那契数列的前两个数据为 0 和 1，计算其前 10 项。

分析：假定 fib 存储有斐波那契数列的前 n 项，fib[-1]和 fib[-2]表示最后两项，所以新的一项的值应为 fib[-1]+fib[-2]。

```
fib=[0,1]
for i in range(2,10):                    #表中已存在 2 项，所以从 2 开始
    fib.append(fib[-1]+fib[-2])
print (fib)
```

代码运行结果为：[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

2．reverse 方法

reverse 方法将列表逆转，是无参数方法，其使用形式为：

列表名称.reverse()

见下面的示例代码：

```
L=[1,2,3,'laoyang']
L.reverse ()
print(L)
```

结果为：['laoyang', 3, 2, 1]

3．sort 方法

当列表中所有元素之间都是可比较时，可以用 sort 方法进行排序。sort 方法的使用形式为：

列表名称.sort(key=None, reverse=False)

说明：

① sort 方法使用时一般不需要参数，数据按升序排序。例如，执行如下的代码：

```
lst =[8,2,3,17.8]
lst.sort()
print(lst)
```

输出结果为：[2, 3, 8, 17.8]，数据按从小到大的顺序排序了。

② 排序时列表所有元素之间都应是可比较的，否则会报错。例如，执行如下的代码：

```
lst =['Tongji University', 1907, 'Fudan University', 1905]
lst.sort()
```

系统提示如下错误信息：

TypeError: '<' not supported between instances of 'int' and 'str'

意思是‘<’不支持整数和字符串实例间的比较。

③ 参数 reverse 决定排序的顺序是升序还是降序。若 reverse 为 False，则升序，否则降序。reverse 的缺省值为 False。例如，语句 lst.sort(reverse=True)将 lst 列表中的元素降序排序。

④ 当对复杂列表进行排序时，通常需要使用参数 key 指定排序关键字。参数 key 通常是一个函数，列表所有元素根据该函数值的大小进行排序。例如，假定有列表 lst=[['李成',80],['赵华',61],['华连',91]]，若要按成绩排序，则应用使用下列语句：

lst.sort(key=lambda item:item[1])

其中，key 指定的函数是一个 lambda 函数。key 指定的函数可以是自定义函数、内置函数等。例如，

```
def fun(x):                                    # 定义函数
    return x[1]
lst =[['李成',80],['赵华',61],['华连',91]]
lst.sort(key=fun)                              # 指定自定义函数排序
print(lst)
```

上述代码中 key 指定了一个自定义函数 fun。

4．count 方法

count 方法用于统计列表中指定元素出现的次数，其使用形式为：

列表名称.count(元素值)

说明：统计结果是 count 方法的返回值。例如，下面的代码使用 count 方法统计 lst 列表中“上海”出现的次数：

```
lst=["杭州","苏州","上海","苏州","无锡","上海","苏州"]
n=lst.count('上海')
print(n)
```

【例 5.4】人类发现，蛋白质是由 20 种氨基酸（每种氨基酸用一个大写字母表示）组成的长链，还发现，氨基酸的组合有保守性（即某些氨基酸组合在一起的频率比较高）。现给定一条蛋白质，以 3 个氨基酸为单位，统计其中每种组合出现的次数。假定氨基酸序列如下：

MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDGTQDNLSGAEKAVQVKVKALPDAQFEVVHSLAKWKRQTLGQHDFSAGEGLYTHMKALRPDEDRLSPLHSVYVDQWDWERVMGDGERQFSTLKSTVEAIWAGIKATEAAVSEEFGLAPFLPDQIHFVHSQELLSRYPDLDAKGRERAIAKDLGAVFLVGIGGKLSDGHRHDVRAPDYDDWSTPSELGHAGLNGDILVWNPVLEDAFELSSMGIRVDADTLKHQLALTGDEDRLELEWHQALLRGEMPQTIGGGIGQSRLTMLLLQLPHIGQVQAGVWPAAVRESVPSLL

分析：首先将给定的氨基酸序列作为一个字符串，将每三个氨基酸切片保存到一个列表中，最终通过列表遍历的方式，统计出每种组合出现的次数。

程序代码如下：

```
seq='MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDGTQDNLSGAEKAVQVKVKALPDAQFEVVHSLAKWKRQTLGQHDFSAGEGLYTHMKALRPDEDRLSPLHSVYVDQWDWERVMGDGERQFSTLKSTVEAIWAGIKATEAAVSEEFGLAPFLPDQIHFVHSQELLSRYPDLDAKGRERAIAKDLGAVFLVGIGGKLSDGHRHDVRAPDYDDWSTPSELGHAGLNGDILVWNPVLEDAFELSSMGIRVDADTLKHQLALTGDEDRLELEWHQALLRGEMPQTIGGGIGQSRLTMLLLQLPHIGQVQAGVWPAAVRESVPSLL'
li=[]                                   #保存分片的结果
length=3                                #3 个氨基酸的组合
for i in range(len(seq)-(length-1)):
    segments=seq[i:i+length]
    li.append(segments)
counts=[]                               #记录每种组合的次数，元素与 segments 有一一对应关系
segments=[]                             #去掉重复后的氨基酸组合
for seg in li:
    if not seg in segments:
        segments.append(seg)            #记录一个新的组合
        n=li.count(seg)
        counts.append(n)                #记录组合出现的次数
for i in range(len(counts)):
    print(segments[i],counts[i])
```

程序的输出结果为每个独立的三个氨基酸片段出现的次数。

5．index 方法

index 方法用于查找指定元素在列表中第一次出现的索引值，其使用形式为：

列表名称.index(元素值)

说明：

① 查找结果是 index 方法的返回值。如下面的代码，找出元素“宁波”第一次出现的位置。

```
lst=["上海","杭州","苏州","宁波","南京","宁波"]
n=lst.index('宁波')
print(n)
```

结果为 3，即“宁波”第一次出现在列表中的第 3（从 0 开始计数）位置。

② 将上述程序再加两个语句

```
pos2=lst.index("北京")
print(pos2)
```

程序的运行结果为：ValueError: '北京' is not in list。

说明当列表不存在指定的元素值时，系统会报错。

index 方法配合 count 方法可以完成更复杂的任务，例如要找出例 5.4 中蛋白质 3 氨基酸组合中出现最多次数的氨基酸组合及其对应的次数。在统计了每种组合出现的次数后，可首先通过 max 函数寻找列表 counts 的中的最大值，然后找出该最大值的索引。因为 counts、segments 两个列表的元素是一一对应的，所以输出索引指定的 segments 元素，就是出现频数最大的 3 氨基酸组合。程序追加的代码如下：

```
maxCount=max(counts)   #max 为 Python 内置函数，可直接使用
index=counts.index(maxCount)
maxSeg = segments[index]
print(maxSeg, maxCount)
```

代码的运行结果如下：IAK 2。

思考题：index 方法有时会出现异常而导致程序崩溃，请你结合列表的 in 操作符，思考如何正确使用 index 方法。

6．pop 方法

pop 方法返回列表指定位置元素并将其从列表中删除，其使用形式为：

```
列表名称.pop(index=-1)
```

说明：index 参数省略时返回最后元素并删除。

pop 方法使列表有堆栈的特性。堆栈具有数据先进先出的特点。可以将列表竖起来看，如下面的列表：

x=[4, 10, 1, 97, -5, -7]

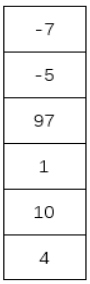


图 1.5.1 将列表看作堆栈

元素 4 是最先进入列表的，所以它在栈底，-7 是最后进入堆栈的，所以它在栈顶，其示意图如图 1.5.1。

如果在操作一个列表时，仅允许使用 pop 方法和 append 方法，则列表就是堆栈。

下面实例代码将列表 x 的栈顶元素弹出。

```
x=[4, 10, 1, 97, -5, -7]
n=x.pop()
```

```
print(n)
print(x)
```

代码运行结果如下：

-7

[4, 10, 1, 97, -5]

7. 自学内容

除了上述描述的方法外，列表还有很多其他的方法，如 clear 方法将表清空，remove 方法移除元素值为指定值的第一个元素，insert 方法在指定位置插入新元素，extend 将其他列表的元素逐个追加到当前列表中等。这些方法，从实现目的上讲，都可以使用分片赋值或直接给列表赋值的方法实现，有兴趣的读者，可以自行查阅资料尝试学习。

5.1.4 操作列表的内置函数

部分内置函数可以直接操作列表，这些函数包括：加和函数 sum，元素个数函数 len，最大、最小值函数 max、min，将指定函数映射到列表每个元素的函数 map 等等。本节挑选其中的部分内容加以介绍。

1．len 和 sum

len 返回列表的元素个数，sum 返回数值列表的和。其使用形式分别为：

```
len(列表名称)
sum(列表名称)
```

【例 5.5】 从键盘上输入一组数，数值间用逗号间隔，计算这组数的平均值和标准偏差。

分析：程序获得一组以逗号间隔的数，通过字符串的 split 函数分割后得到字符串列表，其每个元素是一个数字组成的字符串，通过 float 函数将每个元素转化为实数，在此基础上可以完成计算任务。程序代码如下：

```
data=input('请输入一组数，数间以逗号间隔：')
data=data.split(',')                          # 按逗号将数据分割
for i in range(len(data)):                    # 将每个数字字符串转化为对应的实数
    data[i] = float(data[i])
aver=sum(data)/len(data)                      # 求平均值
std=0
for i in data:
    std += (i-aver)**2
import math
std = math.sqrt(std/(len(data)))
print('平均值={:.3f},标准偏差={:.3f}'.format(aver,std))
```

2．max 和 min

max 和 min 函数从列表中返回最大值和最小值。其使用形式分别为：

```
max(列表名称)
min(列表名称)
```

【例 5.6】将数值列表的每个元素的数值调整到 0-1 之间。

分析：先求列表的最大值 ma 和最小值 mi，然后将列表的每个元素 item 按如下公式处理。

item=(item-mi)/(ma-mi)

代码如下：

```
L=[12,34,21.3,45,19,87.2,76.4]
mi=min(L)
ma=max(L)
L=[(i-mi)/(ma-mi)  for i in L]
print(L)
```

输出结果为：[0.0, 0.293, 0.124,0.439,0.093,1.0,0.856]

最大最小值函数并非只可用于数值列表，只要列表的元素之间是可比较的，则都可以用其求取最大最小值。例如：

```
L=['aaa','zzz','cfd']
print(max(L))
```

代码运行的结果为：'zzz'

3．map

map 函数用于将指定函数作用于列表的每个元素，其使用形式为：

```
map(函数,列表名称)
```

说明：

① 函数可以是内置函数，也可以是用户自定义函数。

② map 函数返回的结果是 map 对象，可用 list 函数将 map 对象转换为列表以便观察其值。

下面的代码将给定的数字字符串列表，转换为实数值组成的列表。

```
L=['123.6','24','63.1']
z=map(float,L)
L=list(z)
print(L)
```

代码运行结果为：[123.6, 24.0, 63.1]

【例 5.7】修改例 5.5，利用 map 函数将数字字符串列表转换为数值列表，计算其均值和标准偏差。

分析：键盘输入的字符串经 split 函数分割后得到数字字符串列表，用 map 函数将 float 函数映射到列表在转换为列表可实现任务，程序代码如下：

```
data=input('请输入一组数，数间以逗号间隔：')
data=data.split(',')                          # 按逗号将数据分割
data=list(map(float,data))
aver=sum(data)/len(data)                      # 求平均值
std=0
for i in data:
    std += (i-aver)**2
```

```
import math
std = math.sqrt(std/(len(data)))
print('平均值={:.3f},标准偏差={:.3f}'.format(aver,std))
```

代码的运行过程和结果为：

请输入一组数，数间以逗号间隔：12,34,21.3,45,19,87.2,76.4

平均值=42.129, 标准偏差=27.142

4．zip 和 enumerate

zip 函数用于操作多个列表，enumerate 函数操作一个列表，二者都返回迭代对象。

(1) zip 函数

zip 函数将多个列表的对应元素整合为元组，并返回以这些元组为元素的 zip 对象。其使用形式为：

zip(列表名称 1,列表名称 2,…,列表名称 n)

说明：

① 所有列表的元素个数应该等同

② zip 对象不能直接显示其包含的值，需要用 list 函数转换为列表再显示。

例如将两个元素个数相同的列表的对应元素组成元组，作为列表的元素形成新列表，代码如下：

```
names=['张仟', '石戎', '万行']
ages=[23, 21, 22]
ziped=zip(names,ages)
print(ziped)
print(list(ziped))
```

代码的运行结果为：

<zip object at 0x0000023E0BABB808>

[('张仟', 23), ('石戎', 21), ('万行', 22)]

输出结果的第一行，说明 zip 对象无法直接显示数据，第二行则可以看出，zip 将两个列表的对应元素组合在一起。

利用 zip 函数可使多个列表的遍历更具可阅读性。下面代码用 zip 实现学生姓名和年龄的对应遍历显示。

```
names=['张仟', '石戎',  '万行']
ages=[23, 21, 22]
for name,age in zip(names,ages):
    print(name,age)
```

程序的运行结果为：

张仟 23

石戎 21

万行 22

（2）enumerate 函数

enumerate 函数将一个列表的元素与其下标索引组成元组。其使用形式为：

enumerate (列表名称)

下面代码将列表的元素值及其对应的索引一起输出。

```
names=['张仟', '石戎',  '万行']
enu=enumerate(names)
print(enu)
print(list(enu))
```

代码运行结果如下：

<enumerate object at 0x0000023E0BA94FC0>

[(0, '张仟'), (1, '石戎'), (2, '万行')]

（3）迭代类型对象的使用

zip 和 enumerate 对象属于迭代类型，使用时需要注意的是：这类对象遍历完最后一个元素后，其使命即宣告结束，继续对其进行操作，虽然无语法错误，但程序结果往往超乎设计者的意外。如下面的代码所示：

```
names=['张仟', '石戎',  '万行']
ages=[23, 21, 22]
ziped=zip(names, ages)
print("第一个 print 的输出：{}".format(list(ziped)))
print("第二个 print 的输出：{}".format(list(ziped)))
for name,age in ziped:
    print(name,age)
```

上述代码中，设计者想对 ziped 对象输出两次，最后又想通过 for 循环对其再次遍历，能得到怎样的结果呢？

第一个 print 语句中，list 函数对 ziped 进行第一次遍历，输出结果是正常的。执行第二个 print 语句时，由于此时 ziped 已经完成遍历，所以，其输出结果应该是空表。而对于 for 循环，因 ziped 已处于迭代最后位置，所以 for 循环无结果输出。

运行上述程序，其输出结果如下：

第一个 print 的输出：[('张仟', 23), ('石戎', 21), ('万行', 22)]

第二个 print 的输出：[]

5.1.5 列表推导式

列表推导式是 Python 提供的一种简洁式的快速对列表进行遍历、过滤进而达到重组的方案。其语法格式为：

[结果表达式 for 表达式 1 in 列表 1 [if 条件 1

for 表达式 2 in 列表 2 if 条件 2

……

for 表达式 n in 列表 n if 条件 n]

]

说明：

① 结果表达式可以为任何类型。

② 根据 for 循环计算“结果表达式”，并以“结果表达式”作为元素形成新的列表。

③ 条件用于选择符合条件的数据。

例如，对列表 lst=[['李成','男',80],['赵华','男',61],['华连','女',91],['贾茜','女',75], ['程泽','男',95],['彭鹏','男',89],['张艺','女',100],['黎玲','女',99]]，求女生成绩列表，其列表推导式为：

girlScores=[stu[2] for stu in lst if stu[1]=='女']

【例 5.8】从鸢尾花数据集中抽取山鸢、变色鸢尾两种花各 3 个样本，每株样本用萼片的长和宽、花瓣的长和宽 4 个特征，山鸢的类别用 0 表达，变色鸢尾用 1 表达。特征数据用二维列表 X 存储，类别用一维列表 y 存储，根据列表 y 值，将山鸢的三个样本挑选出来形成新的列表。

分析：通过下标索引遍历列表，y 元素为 0 的样本对应的 X 同位置的元素就是需要过滤的样本，程序的代码如下：

```
X=[[5.1, 3.5, 1.4, 0.2],  [4.9, 3. , 1.4, 0.2],  [4.7, 3.2, 1.3, 0.2],
   [7. , 3.2, 4.7, 1.4],  [6.4, 3.2, 4.5, 1.5],  [6.9, 3.1, 4.9, 1.5]]
y=[0,0,0,1,1,1]
result=[X[i] for i in range(len(X)) if y[i]==0 ]
```

【例 5.9】 实现矩阵的平铺。矩阵是二维列表，平铺是指将矩阵的后续行，连接到前行上，形成一维列表。

分析：按行遍历二维列表可以用两个 for 循环完成，外层循环访问行，内存循环负责行中的每个元素，将内循环访问的元素作为列表的元素可完成任务，故可用下面的列表推导式实现任务：

```
X=[[1, 2, 3, 4],  [5, 6, 7, 8],  [9, 10, 11, 12]]
res=[item for row in X for item in row ]
print(res)
```

程序的运行结果为：[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

【例 5.10】 用列表推导式实现矩阵的转置。矩阵的转置是指矩阵的行变列，列变行。

分析：设定按列遍历二维列表的循环，第 i 次循环，取列表每行的第 i 个元素组成列表，将它作为列表推导式的一个元素可实现任务。所以，矩阵转置的列表推导式方案如下：

```
X=[[1, 2, 3, 4],  [5, 6, 7, 8],  [9, 10, 11, 12]]
Xt=[[row[i] for row in X ]                #每个元素是一个列表，由矩阵的某列组成
     for i in range(len(X[0])) ]          # 循环以矩阵的列展开
print(Xt)
```

程序的输出结果为：[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]

5.1.6 元组

元组被认为是轻量级的列表，其类型为 tuple。对列表的很多操作也适用于元组，如遍历、切片、下标的使用，+、*、in 运算等。元组被看作一个不可变更的列表，系统开销小，操作速度快，对无需修改的数据有一定的保护作用，所以有其自身优势。元组和列表之间，可以通过 list 和 tuple 函数互相转换，元组的初步知识请回顾 2.5.2 节。

1．元组的创建

元组的创建除了用()直接表达或用 tuple 函数外，任何以逗号间隔的一组数值、变量、表达式，都被组织成元组。如下面语句：

```
b=12.3
a=b/3,1,2                                    #逗号间隔的值,自动形成元组
print(a)
```

典型的两变量值交换语句

```
x,y=y,x
```

其实质是将 y,x 组成元组，再通过序列解包赋值给 x,y，达到值交换的目的。

2．元组元素操作及分片

元组元素通过“元组[下标]”使用。但由于元组元素的初值不可变更，所以对元组元素赋值是不允许的。

当元组元素的类型是复合型数据时，元组元素的属性值可以改变。如下面的代码示例。

```
Li=([11, 22], 33,88)
Li[0].append(-55)
print(Li)
```

元组 Li 的第一个元素是列表，此处调用了列表的 append 方法为其增加了一个元素，元组元素的属性值改变了，但元组 Li 的第一个元素的首地址并没有改变。代码的运行结果如下：

([11, 22, -55], 33, 88)

但采用如下的语句，则是不允许的：

Li[0] +=[-55]

程序运行结果为：

TypeError: 'tuple' object does not support item assignment

意思为：元组对象不支持表项的赋值。

这是由于该语句试图改变第一个元素的存储首地址，被 Python 拒绝。

元组的分片与列表的分片采用同样的规则，只是返回结果的类型仍然是元组。

3．操作元组的内置函数

对列表操作的函数，都可以用于元组，例如 sum，len，max，min，map 等，其操作与列表类似。

【例 5.11】求元组的最大最小值、元组的和

```
Li=(11, 22,33,44,55,66,77,88)
maxNum=max(Li)
minNum=min(Li)
s=sum(Li)
lenth=len(Li)
print("最大={},最小={},和={},元素个数={}".format(maxNum,
      minNum,s,lenth))
Li=('11', '22','33','44','55','66','77','88')
L1=tuple(map(float,Li))
print(L1)
```

程序的运行结果如下：

最大=88,最小=11,和=396,元素个数=8

(11.0, 22.0, 33.0, 44.0, 55.0, 66.0, 77.0, 88.0)

5.2 字典与集合

与列表和元组相比，字典和集合是无序的数据序列。字典（dict）是按“键/值”对存储的一种数据结构，它通过关键字，快速查找对应的信息，集合则强调元素间不重复。关于字典和集合的基础内容可参阅 2.5.4。

数据采用列表存储与采用字典存储会导致查询方法和效率有很大差异。如将学生名字和成绩用列表存储时

```
name_Score=[('李成',80),('赵华',61),('华连',91),('高薇',98),('彭池',71),('钟毓',77)]
```

则查询某个同学的成绩时，需要通过循环找到该同学然后给出其成绩。但数据用字典存储时

```
name_Score={'李成':80,'赵华':61,'华连':91,'高薇':98,'彭池':51,'钟毓':77}
```

则可直接通过 name_Score[姓名]的方式获得学生的成绩。

5.2.1 字典的创建

创建字典最直接的方式是将数据按格式写进{}或用 dict 函数，除此之外，还有多种方式生成字典。

1．逐项建立

先建立空字典，然后逐步添加新项完成，其使用格式为：

```
字典[键]=值
```

例如下面的代码，通过逐步增加表项建立字典。

```
Dcountry={}
Dcountry['中国']='北京'
Dcountry['德国']='柏林'
```

```
Dcountry['美国']='华盛顿'
print(Dcountry)
```

结果为：

{'中国': '北京', '德国': '柏林', '美国': '华盛顿'}

由于键是唯一的，所以，采用“字典[键]=值”方式增加字典项时，如果原来已经存在指定的键，则运行结果会替换原来的键值对。

2．用 dict 函数转换

dict 函数常用来将合适的序列数据转换为字典。

① 二维的列表或元组，只要能清晰地表达“键/值”关系，就可以通过 dict 函数转换成字典。例如：

```
data=[['中国','北京'],['德国','柏林'],['美国','华盛顿']]
Dcountry=dict(data)
print(Dcountry)
```

② 两个元素相同的列表或元组，结合 zip 函数，也可以经 dict 函数转化为字典。如下面例子：

```
names=['张仟', '石戎', '万行']
ages=[23, 21, 22]
dict1=dict(zip(names,ages))
print(dict1)
```

代码的运行结果为：{'张仟': 23, '石戎': 21, '万行': 22}

5.2.2 字典项操作

字典对象拥有很多方法，对字典 d，表 1.5.1 中列出了词典操作的常用方法。

表 1. 5. 1 字典操作的常用方法

操作方法	说明
len(d)	返回 d 中“键/值”对的数量，即表项数目
d.keys()	返回字典所有的键
d.values()	返回字典所有的值
d.items()	返回所有键值对
d.clear()	清空字典内容
d.get(key, default=None)	返回字典键 key 的值，如果键在字典中不存在，返回 default 指定的值
d.pop(key,[default]	若字典中存在键 key，则删除并返回 key 对应的值，不存在则返回 defalut
d[k]	出现在表达中，返回关联到键 k 上的值
d[k] = v	将值 v 关联到键 k 上

del d[k]	删除键 k 的项
k in d	判断 k 是否是 d 中的一个键

遍历字典，可以通过遍历字典的键完成，也可以用遍历 items 方法的返回值完成。如下面的代码通过遍历键实现了字典的遍历。

```
DzipCode={'北京':100000,'上海':200000,'南京':210000,'深圳':518000}
DzipCode['广州']=510000
for key in DzipCode:                                    #字典遍历，通过遍历键获得值
    print(key,DzipCode[key])
print("项数",DzipCode.items())
print("所有的键",list(DzipCode.keys()))
print("所有的值",list(DzipCode.values()))
print("上海的邮政编码：",DzipCode['上海'])
```

代码的运行结果为：

北京 100000

上海 200000

南京 210000

深圳 518000

广州 510000

项数 dict_items([('北京', 100000), ('上海', 200000), ('南京', 210000), ('深圳', 518000), ('广州', 510000)])

所有的键 ['北京', '上海', '南京', '深圳', '广州']

所有的值 [100000, 200000, 210000, 518000, 510000]

上海的邮政编码： 200000

【例 5.13】将例 4.21 统计英文句子每个字符出现的次数的函数，形成以字符为键，次数为值的字典并返回。

分析：因为统计过程是逐个字符进行的，所以可以从空字典出发，每访问一个字符形成一个字典项的方式完成任务，程序的代码如下：

```
def charFreq(s):
    chars=list(set(s))                  # 整理不重复的字符
    chars.sort()
    freq={}                             #建立空字典
    for char in chars:
        times=s.count(char)             #记录字符出现次数
        freq[char]=times                #为字典增加一项
    return freq                         # 返回字典
str1=input('请输如字符串：')
charFreq=charFreq(str1)                 #调用函数
print(charFreq)
```

5.2.3 集合运算

创建集合的方法请参见第 2.5.3 节，本节仅介绍集合的运算。集合运算可以通过运算符进行，表 1.5.2 列出了集合运算的运算符及实例运算结果。

表 1.5.2 集合运算符

运算符	意义	示例	结果　设初值为：s1=set((1,3,7,8))，s2=set((2,8,10))
-	差集	s1-s2	{1, 3, 7}
&	交集	s1 & s2	{8}
|	并集	s1 | s2	{1, 2, 3, 7, 8, 10}
^	对称差集	s1 ^ s2	{1, 2, 3, 7, 10}

验证表 1.5.2 实例结果的代码如下：

```
s1=set((1,3,7,8))
s2=set((2,8,10))
print('差集：',s1-s2)
print('交集：',s1 & s2)
print('并集：',s1 | s2)
print('对称差集：',s1 ^ s2)
print('检验元素存在：', 7 in s2)
print('集合元素个数：',len(s1))
```

集合运算也可以用集合对象的方法完成。设有集合对象 s 和 s1，表 1.5.3 列出了集合对象的部分方法。

表 1.5.3 集合对象的方法

方法	意义
s.add(x)	为集合增加新元素 x
s.update(x)	x 必须是序列类型，x 的每个元素成为 s 的一项，若 x 是字典，取字典的键作为元素
s.remove(x)	移除元素 x，若元素不存在，则报错
s.discard(x)	移除元素 x，若元素不存在，系统不报错
s.clear()	清空集合
s.difference(s1)	同 s-s1
s.intersection(s1)	s 与 s1 的交集，同 s & s1
s.issubset(s1)	s 是否是 s1 的子集，是则返回 True
s.union(s1)	s 与 s1 的并集，同 s | s1
s.symmetric_difference(s1)	返回两个集合中不重复的元素集合,同 s ^ s1

通过集合对象的方法实现集合运算的代码如下：

```
s1=set((1,3,7,8))
s2=set((2,8,10))
```

```
print('差集：',s1.difference(s2))
print('交集：',s1.intersection(s2))
print('并集：',s1.union( s2))
print('对称差集：',s1.symmetric_difference( s2))
```

5.3 综合应用

本章重点讲解了序列数据的结构及相应的处理方法，结合循环、函数，学习者能解决更加复杂的问题。本节通过几个综合例子，向学习者展示如何利用序列数据解决实际问题。

1．数据分拣

数据分拣是指根据某种条件，从已知数据序列中，挑选出符合要求的元素。

【例 5.14】 设原始考试数据是所有班级混合考试的列表，现在想将某个班级同学成绩分拣出来。已知条件为：混考成绩表及班级同学的名单。

分析：班级同学名单是混考成绩表的子集，故可通过数据序列的 in 运算符实现分拣。求解问题的思路如下：

①整理学生成绩到一个对象列表 studentList

②整理班级学生名单到列表 C，C 列表中只记录学号

③令结果列表 R=[]

④遍历列表 studentList

⑤ studentList 的每个元素，通过 in 运算符确认其在列表 C 中，则将其追加到 R 中

⑥输出 R

完整的程序代码如下：

```
studentList=[]
studentList.append(('1200001','许枫',65,84,61))
studentList.append(('1200002','周杰',76,65,72))
studentList.append(('1200003','李丽',87,76,91))
studentList.append(('1200004','江红燕',76,90,78))
C=['1200002','1200003']
R=[]
R=[stu for stu in studentList if stu[0] in C]                    # 分拣
for stu in R:
    print("{}\t{}\t{}\t{}\t{}".format(stu[0],stu[1],
          stu[2],stu[3],stu[4]))
```

程序的运行结果如下：

```
1200002  周杰        76        65        72
1200003  李丽        87        76        91
```

2．矩阵相乘及函数实现

矩阵可以用二维列表表示，两个矩阵 A 和 B 相乘时遵循如下的原则：

左矩阵的列数等于右矩阵的行数，结果矩阵的行数等于左矩阵的行数，列数等于右矩阵

列数。

结果矩阵的第 i 行 j 列的元素，是左矩阵的第 i 行和右矩阵第 j 列，对应元素相乘的加和，如下式。$C_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$

矩阵相乘的示意图如图 1.5.2 所示：

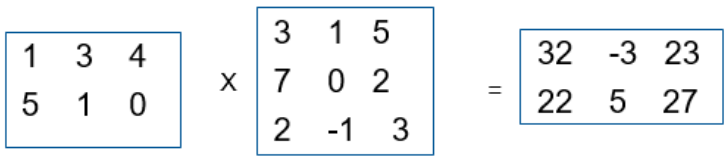


图 1.5.2 矩阵相乘示意图

【例 5.15】根据矩阵计算的公式，编写程序，计算 C=A×B。

程序代码如下：

```
A=[[1, 3,4],[5,1.0,0]]                  # 2*3 的矩阵
B=[[3,1,5],[7,0,2],[2, -1, 3]]          # 3*3 矩阵
C=[ ]                                   #结果矩阵
row=len(A)                              # 取 A 矩阵的行数
col=len(B[0])                           # 取 B 矩阵的列数
for i in range(row):
    oneRow=col*[0]                      # 结果矩阵的一行，每个元素的值设 0
    for j in range(col):
        s=0.0
        for k in range(len(B)):         #计算结果矩阵的第 i 行第 j 列元素
            s += A[i][k]*B[k][j]
        oneRow[j]=s
    C.append(oneRow)
print(C)
```

程序的运算结果为：

[[32.0, -3.0, 23.0], [22.0, 5.0, 27.0]]

矩阵相乘、转置、格式化输出应用非常普遍，将其函数化可大大提高编写矩阵计算程序的效率。下面的例子求矩阵 X 的协方差矩阵（X 的转置和 X 相乘的结果）。它利用了矩阵的转置、矩阵的相乘运算。

【例 5.16】编写函数，分别实现矩阵相乘、转置、格式化输出

```
def matDot(A,B):                        # 计算 C=A*B，返回 C
    C=[ ]                               #结果矩阵
    row=len(A)                          # 取 A 矩阵的行数，也是 C 的行数
    col=len(B[0])                       # 取 B 矩阵的列数，也是 C 的列数
    for i in range(row):
        oneRow=col*[0]                  # 结果矩阵的一行，每个元素的值设 0
        for j in range(col):
            s=0.0
            for k in range(len(B)):     #计算结果矩阵的第 i 行第 j 列元素
```

```
                    s += A[i][k]*B[k][j]
                oneRow[j]=s
            C.append(oneRow)
        return C
    def printMat(A):                            #矩阵的格式化输出
        for row in A:
            for item in row:
                print(item,'\t',end='')
            print()
    transpose=lambda X:list(map(list,zip(*X)))    # 矩阵转置函数
```

在以上三个函数的支持下，求取矩阵的协方差矩阵的主程序代码如下：

```
X=[[1,2,3],[4,5,6]]
Xt=transpose(X)
matCoe=matDot(Xt,X)
printMat(matCoe)
```

程序的运行结果为：

17.0　　22.0　　27.0

22.0　　29.0　　36.0

27.0　　36.0　　45.0

3．感知器算法分类建模

美国植物学家埃德加. 安德森收集了不同类别的鸢尾花的花瓣、萼片数据形成了著名的鸢尾花数据集。从数据集中分别选择出 10 朵山鸢尾和 10 朵变色鸢尾花瓣的长、宽作为数据集，以长为横轴、宽为纵轴，将 20 朵鸢尾花作为数据点在二维平面上显示出来，如图 1. 5. 3 所示。图中可见两种花分布在不同的区域。现在问，能不能通过机器学习自动建立图示直线的方程将两类样本分开？

令花瓣长为 x_1，宽为 x_2，则分类直线的方程可表达为：

$$a_1x_1+a_2x_2+b=0$$

山鸢尾样本在直线的上方，变色鸢尾样本在直线的下方，所以将任一样本带入方程后，必定满足下面不等式。

山鸢尾：$a_1x_1+a_2x_2+b>0$

变色鸢尾：$a_1x_1+a_2x_2+b<0$

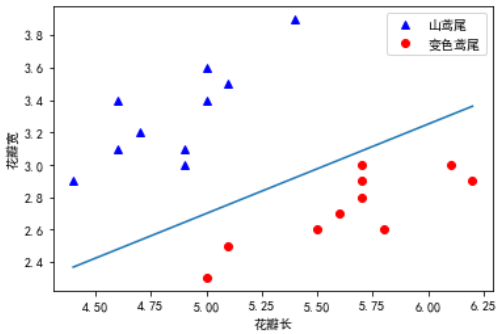


图 1.5.3　山鸢尾和变色鸢尾样本分布

鸢尾花分成两类，山鸢尾用+1 表示，变色鸢尾用-1 表示，用变量 y 存储。如果直线方程不合理，则会导致部分样本误判，如果样本 i 误判，则它必定满足下面不等式：

$y^{(i)}(a_1x_1^{(i)}+a_2x_2^{(i)}+b)<0$

式中，上标(i)表示第 i 个样本，下同。

感知器算法以误判样本来调节方程的系数，在给定初值 a_1、a_2、b 及学习速率 r 后，对于误判样本 i，系数调节的公式如下：

$a_1=a_1+ryx_1^{(i)}$

$a_2=a_2+ryx_2^{(i)}$

$b=b+ry^{(i)}$

直线方程的好坏通过如下的函数表达：

$Loss(a1,a2,b)=\Sigma \max(0, -y^{(i)}(a_1x_1^{(i)}+a_2x_2^{(i)}+b))$

该函数被称为损失函数，对于判断正确的样本，损失为 0，所以对函数值无贡献，而误判的样本，其对应的表达式值必定为负数，取负号后为正数，因此被计入损失函数。损失函数值越大，说明误判的样本多，直线方程不合理。当损失函数为 0 时，说明所有样本分类正确。

【例 5.17】设计感知器算法程序，根据给定的鸢尾花数据，求解感知器判别模型方程

分析：首先定义损失函数，损失函数返回每个样本的损失函数贡献值及函数值，主程序则根据损失函数返回值调节方程的系数，直到所有样本正确分类，算法逻辑如下：

（1）损失函数定义

① 给定记录样本特征的二维列表 x，记录样本分类值的一维列表 y，系数 $a=[a_1,a_2]$，截距 b

② 将每个样本带入公式$(a_1x_1+a_2x_2+b)*y$，计算其损失函数贡献值，放入列表 temp

③ 根据 temp 和损失函数公式计算损失函数值 loss

④ 返回 loss 和 temp

（2） 感知器算法逻辑

① 给定系数初值，a=[1.0,1.0]; b=1.0; 学习速率 rate=0.1

② 指定迭代次数，进入循环

③ 根据 a，b，rate, 调用损失函数获得损失函数值和每个样本损失贡献值

④ 如果损失函数等于 0，或者到达最大迭代次数，退出循环，程序终止

⑤ 找误判样本，按公式调节系数，转③进入下次循环

感知器算法程序代码如下：

```
def lossF(x,y,a,b):                          # x 是 2 维列表，每行一个样本, y 是样本分类符
    temp=[]
    for one in x:
        temp1=[one[i]*a[i] for i in range(len(one))]
        temp1=sum(temp1)+b                   # 每个样本的 a1x1+a2x2+b 的取值
```

```
            temp.append(temp1)
        temp2=[]
        for i in range(len(temp)):
            temp2.append(temp[i]*y[i])          # 每个样本函数值与真值的乘积
        loss=0
        for i in temp2:
            loss += max(0,-i)                   # 计算损失函数，每项取负值，即误判样本
        return loss,temp2                       # temp2 返回有用，用于找误判，调节系数
# 数据准备, x1 是山鸢尾，x2 是变色鸢尾
x1=[[5.1, 3.5 ], [4.9, 3. ], [4.7, 3.2], [4.6, 3.1], [5. , 3.6],
    [5.4, 3.9], [4.6, 3.4], [5. , 3.4 ], [4.4, 2.9], [4.9, 3.1]]
x2=[[5.5, 2.6 ], [6.1, 3. ], [5.8, 2.6], [5. , 2.3], [5.6, 2.7],
    [5.7, 3. ], [5.7, 2.9], [6.2, 2.9], [5.1, 2.5], [5.7, 2.8]]
x=x1+x2                                         #合成一个矩阵
y=[-1, -1, -1, -1, -1, -1, -1, -1, -1, -1]      #样本分类标识
y1=[1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
y += y1
a=[1.0,1.0];b=1.0;rate=0.01                     #给系数初值
for i in range(100):
    loss,temp=lossF(x,y,a,b)                    #计算损失函数
    print(loss)
    if loss==0.0:
        break
    for j in range(len(temp)):
        if temp[j] <0:                          #找误判样本
            a[0] +=rate*y[j]*x[j][0]            #更新系数
            a[1] +=rate*y[j]*x[j][1]
            b +=rate*y[j]
print("loss=",loss)

print("model is {:10.3f}x1{:+10.3f}x2{:+10.3f}".format(a[0],a[1],b))
```

程序运行的部分结果如下：

loss= 31.774599999999957

loss= 1.7453000000000035

loss= 33.645599999999966

loss= 0

model is 0.088x1 -0.399x2 +0.700

5．数据协同过滤

数据协同过滤在商业推介中应用比较广，其原理是将积累的客户营销数据与目标客户比对，找到与目标客户最相似的客户，以相似客户的喜好向目标客户推介。

【例 5.18】某超市积累了一组近两年客户对奶制品的销售数据，包括品牌及选购次数。现有一新客户，请用数据协同过滤的原理，向新客户推介奶制品。

分析：本例中将“爱好相似客户”定义为购买了产品种类重叠最多的客户。计算现有客户群每个客户购买的产品集合与目标客户购买产品集合的交集，取交集最大的客户作为目标

客户的爱好相似客户，将其购买的产品推介给目标客户。

首先将用户购买记录整理成字典格式：每个客户为键，其购买记录作为值。购买记录本身也形成一个字典，以产品为键，购买次数为值。通过将每个客户的购买记录形成集合与目标客户的购买记录集合作交集运算，达到推荐产品的目的。程序实现的代码如下：

```
userData={'客户 1':{'伊利安慕希酸奶':3,'蒙牛纯甄':6,'光明莫斯利安':3,'三元':3,'蒙牛冠益乳':3},\
          '客户 2':{'君乐宝':5,'安佳':5,'光明研简':5,'优诺':3,'兰格格':5,'光明莫斯利安':3},\
          '客户 3':{'兰格格':5,'欧德堡':4,'三元':3,'光明如实':2,'伊利畅轻':5,'伊利安慕希酸奶':7},\
          '客户 4':{'蒙牛冠益乳':3,'三元':3,'光明如实':2,'伊利畅轻':5,'伊利安慕希酸奶':7}}

目标客户={'蒙牛冠益乳':3,'三元':3,'伊利安慕希酸奶':3}          #目标客户的购买记录
commonLen=0
for k,v in userData.items():
    theCommonMilk=set(v.keys()) & set(目标客户.keys())          #一个客户与目标客户购买产品的交集
    if commonLen<len(theCommonMilk):                          #记录交集最多的客户作为相似客户
        相似客户=k
        commonMilk=theCommonMilk                              #记录最相似客户的交集

print('喜好相似的客户为：',相似客户)
print('共同产品：',commonMilk)
# 推荐产品
userMilk=set(userData[相似客户].keys())-commonMilk            #相似客户的产品集合-交集，作为推荐
print('推荐产品',userMilk)
```

程序的运行结果如下：

喜好相似的客户为： 客户 4

共同产品：{'伊利安慕希酸奶', '三元', '蒙牛冠益乳'}

推荐产品 {'伊利畅轻', '光明如实'}

【例 5.19】莫尔斯电码采用了短脉冲和长脉冲(分别为点和点划线)来编码字母和数字。将莫尔斯电码的编码规则组成字典，输入英文字母组成的英文句子，根据字典将句子翻译成莫尔斯电文，电文不同字母间用空格间隔。部分莫尔斯编码如下：

字符	电码	字符	电码	字符	电码	字符	电码
A	.-	B	-...	C	-.-.	D	-..
E	.	F	..-.	G	--.	H	....
I	..	J	.---	K	-.-	L	.-..
M	--	N	-.	O	---	P	.--.
Q	--.-	R	.-.	S	...	T	-
U	..-	V	…-	W	.--	X	-..-
Y	-.--	Z	--..	.	.-.-.-	?	..--..
空格	/						

分析：首先形成字符与对应编码的字典，读入英文句子后，通过遍历每个字符，根据字

典输出其对应的莫尔斯电码。

程序代码如下：

```
Dict={'A':'.-','B':'-...','C':'-.-.','D':'-..',\
'E':'.','F':'..-.','G':'--.','H':'....',\
'I':'..','J':'.---','K':'-.-','L':'.-..',\
'M':'--','N':'-.','O':'---','P':'.--.',\
'Q':'--.-','R':'.-.','S':'...','T':'-',\
'U':'..-','V':'…-','W':'.--','X':'-..-',\
'Y':'-.--','Z':'--..','.':'.-.-.-','?':'..--..',' ':'/'}
sentence=input('please input a sentence:')

sentence=sentence.upper()
for ch in sentence:
    code=Dict.get(ch)
    print(code,' ',end='')
```

习题

一. 选择题

1. 已知列表 L=[46,37 ,10]，令 L1=L,L[1:2]=[65,34],则 L1 的结果是什么？

A [46,65,34,10]　B [65,34,37,10]　C [46,37,10]　D [37,10,65,34]

2. 关于列表排序方法 sort()，下面说法正确的是_____

A　只要列表的元素之间可以比较，其 sort 方法都有效

B　Sort 能自动适应列表的各种类型元素完成排序

C　只有元素是数值类型的列表，才能调用其 sort 方法

D　只有元素是字符类型的列表，才能调用其 sort 方法

3. 遍历列表 L 时，同时取列表元素的值和元素的索引值，最好的方法是使用_____

A　for i,v in L:　　B　for i,v in zip(L):

C　for i,v in enumerate(L):　　D　for i,v in len(L):

4. 向列表 L 追加一个整数元素 1，然后输出整个列表，应该使用_____。

A　print(L.append(1))　B　print(L+1)

C　print(L+[1])　　D　print([L]+[1])

5. 字典和集合在初始化为非空序列时，都使用_____。

A　{}　B　[　]　C　()　D　' '

6. 为字典 di 增加一个新的表项“张制:78”，应该使用语句_____

A　di[张制]=78　B di['张制']=78

C　di{张制}=78　D di{'张制'}=78

7. 遍历字典 d 的键、值对，下面哪组语句不能实现_____

A　for k,y in d:　B　for k in d:　C　for k,v in d.items():　D for k in d.keys():

8. 有列表记录了学生两门课的考试成绩，scores=[['宋安',80,91],['周璐',61,89],['汤琳',81,67],['孙忠',95,99]]，现在想按学生两门课的总分为准将列表按升序重排，使用的语句为_____

A scores.sort(key=lambda o:o[1]+o[2])　　B scores.sort(key=lambda p:sum(p))

C scores.sort(key o:o[1]+o[2])　　D scores.sort(key=lambda obj:obj[2])

二．填空题

1. 列表、元组、字典用不同的括号表达，它们分别是_____、_____、_____。

2. 列表、元组、字典、集合的函数分别是____、_____、_____、_____。

3. 已知列表 L=[5,1,24,65,10,4],现在需要将其中 24,65 两项数据删除，若采用分片实现，其语句应该是_____。

4. 对给定的列表 L，若采用分片的方法将其逆转，实现该功能的语句为_____。
5. 采用列表表达式将列表 L=[5,1,24,65,10,4]中的偶数值的元素剔除，形成新的列表的语句为_____。
6. 最简单的获取字典 d 的所有键的语句为_____。
7. 教师甲和教师乙的学生名单分别存储在集合 S1 和 S2 中，寻找同时选修两教师课的学生名单的语句为_____。
8. 运行并记录结果

   X=[1,2,3,5,9,11]

   X[-1] 结果：_____

   X[::-1]结果：_____

   X[0:7:2]结果： _____
9. 定义 X=(1,2,3,5,9,110),运行下面代码记录结果

   X[-1] 结果：_____

   X[::-1]结果：_____

   X[0:7:2]结果： _____
10. 运行并记录结果

    L=[12,34,23,12]

    L.append(30) 结果：_____

    L.reverse() L 的值：_____

    L.sort() L 的值：_____

    sum(L) 结果：_____

    len(L)结果：_____

    L=[1,2,3,4,5]

    L1=L

    L1[1]=-10

    L 的值为：_____

    list(range(1,10,2))的结果：_____
11. 已知学生组 1：s1=set(('沈月月','王华','邓享东','杨丹一','居雷'))和学生组 2：s2=set(('沈月月','王海波','邓享东','杨丹一','陆龙剑'))，请运行下列代码并记录结果：

    s1 & s2 的结果：_____

    s1-s2 的结果：_____

    s1 | s2 的结果：_____
12. 数列 1/2，2/3，3/5，5/8，8/13，…….，其特点是第 n 项的分子是第 n-1 项的分母，第 n 项的分母是第 n-1 项的分子与分母之和，下面程序以每行 5 个元素的方式，输出数列的

前 15 项。请将程序补充完整。

```
a=1
b=2
__(1)____
for i in range(15) :
    print(str(a)+'/'+str(b),end ="\t")
    count += 1
    if    ___(2)___ :
        print()
    temp=__(3)__
    a=b
    b=__(4)_____
```

13. 以下程序实现购物结算时十万元以下货币金额的大写转换，如输入 4521，将转换为['肆', '千', '伍', '佰', '贰', '拾', '壹', '元']，请将程序补充完整。

```
def convert(num):
    s={'0':'零','1':'壹','2':'贰','3':'参','4':'肆','5':'伍','6':'陆','7':'柒','8':'捌','9':'玖'}
    danwei=['元','拾','佰','千','万']
    num=list(____(1)_____)
    num.reverse()
    result=[]
    for i,j in ___(2)___:
        result.append(danwei[i])
        result.append(__(3)___)
    ____(4)_____
    return result
num=int(input("请输入结账金额（由 0-9 的数字组成）："))
resultnum=convert(num)
print(resultnum)
```

三．简答题

1. 列表的元素可以属于不同的数据类型，请讨论为何列表的部分方法虽然存在，但却不一定能使用，如列表的 sort 方法就不能随意使用。
2. 列表和元组都是序列数据，请简单讨论回答为何列表的很多方法如 append、sort 等，元组对象则没有，但两者都有 index、count 方法。
3. 列表推导式是生成列表的一种方法，对数据过滤很有效，请简述列表推导式的格式，通过何种语法实现数据过滤？
4. 列表、元组在某些状态下可以转换为字典，请列举将列表（或元组）转换为字典的多种方法（提示：列表的形式可以多样，如两个有相同个数元素的列表，或一个列表的每个元素是一个列表等形式）。
5. 请罗列三种字典遍历的方式，遍历时输出每个键-值对的方法。

第六章　文件

文件是有名的一组相关信息的集合。在计算机系统中，所有的程序和数据都是以文件的形式存放在计算机的外存储器（如磁盘等）上，操作系统也是以文件为单位对数据进行管理。因此，对程序设计初学者来说，文件基本知识及其读写方法也是必须要掌握的。

6.1 概述

本节下面首先通过一个引例说明为什么要使用文件以及如何读写文件的，然后介绍文件的基本知识。

6.1.1 引例——统计学生成绩

【例 6.1】输入一批学生的学号、姓名和成绩，统计学生人数、总成绩和平均成绩。

方法 1：将读入的学生数据存储在列表中。

```
# 学生数据存储在列表中
stuList=[]
n=int(input("请学生人数："))
for i in range(n):
    No=input("请输入学号：")
    Name=input("请输入姓名：")
    Mark=int(input("请输入成绩："))
    stuList.append([No,Name,Mark])
Total=0
for i in range(n):
    Total=Total+stuList[i][2]
Average=Total/n
print("人数：",n)
print("总成绩",Total)
print("平均成绩",Average)
```

这样的解决方案存在一个缺点：每次统计学生成绩都必须输入所有学生的数据。因为程序一旦运行结束，列表所占有的内存空间被释放了，列表中存储的学生数据也就丢失。若要永久保存，学生成绩可以保存在文件中，因为文件能在外存储器上长期保存。

方法 2：将读入的学生数据存储在如图 1.6.1 所示的 Scores.txt 文件中。

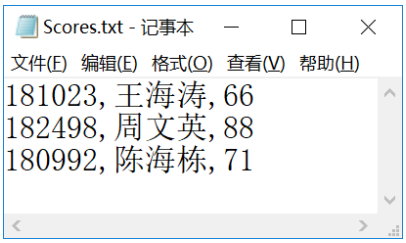


图 1.6.1　学生成绩文件

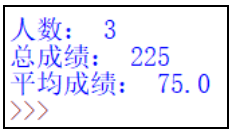


图 1.6.2　学生成绩统计结果

```
n=int(input("请学生人数："))
fScores=open(r"D:\Test\Scores.txt","w")                    # 打开文件供写数据
for i in range(n):                                         # 输入三个同学的成绩
    No=input("请输入学号：")                                # 输入学号
    Name=input("请输入姓名：")                              # 输入姓名
    Mark=int(input("请输入成绩："))                         # 输入成绩
    fScores.write(No + "," + Name +"," + str(Mark) + "\n") # 写入文件
fScores.close()                                            # 关闭文件
```

若要统计文件中的成绩，则打开文件，然后读出数据，再进行处理，最后关闭文件。

```
Statistics=open(r"D:\Test\Scores.txt","r")                 # 打开文件供读数据
n=0                                                        # 变量 n 用于统计人数
Total=0                                                    # 变量 Total 用于统计总成绩
while True:                                                # 循环读数据
    Student=Statistics.readline()                          # 读一行，即读一个学生数据
    if not Student:                                        # 判断数据是不是读完
        break                                              # 若数据读完则退出循环
    n=n+1                                                      # 学生人数加 1
    Temp=Student.split(",")                                # 根据“,”分隔符切分数据成列表
    Mark=int(Temp[2])                                      # 取出成绩
    Total=Total+Mark                                       # 累加成线
Average=Total/n                                            # 计算平均成绩
print("人数：",n)
print("总成绩：",Total)
print("平均成绩：",Average)
Statistics.close()                                         # 关闭文件
```

图 1.6.2 是对如图 1.6.1 所示的含有三个学生数据的文件进行处理的结果。比较两种方法可以得出结论：数据若要长久保存供重复使用，则可以保存在文件中。

6.1.2 文件和文件路径

一个磁盘上的文件成千上万，为了有效地管理和使用文件，通常将磁盘上所有文件组织成树状结构，也就是在磁盘上创建文件夹（目录），在文件夹下再创建子文件夹（子目录），然后将文件分门别类地存放在不同的文件夹中，如图 1.6.3 所示。这种结构像一棵倒置的树，树根为根文件夹（根目录），树中每一个分枝为文件夹（子目录），树叶为文件。在树状结构中，用户可以将一个项目的文件放在一个文件夹中，也可以按文件类型或用途将文件分类存放。

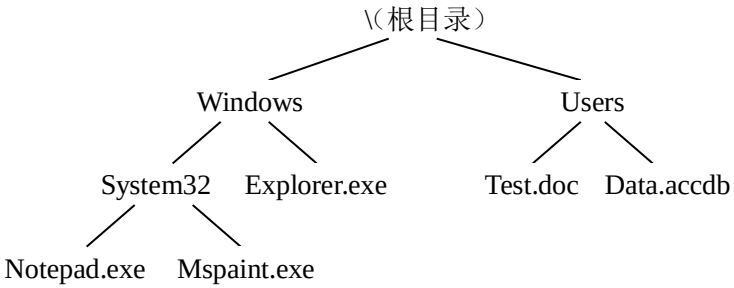


图 1.6.3　树型目录结构

当一个磁盘的目录结构被建立后，所有的文件可以分门别类地存放在所属的文件夹中，接下来的问题是如何访问这些文件。若要访问某个文件，程序中必须描述清楚文件的位置，即给出文件路径，以便操作系统可以查找到所需要的文件。

文件路径分有两种：

① 绝对路径：从根目录开始到某个文件的路径。例如，Notepad.exe 和 Test.doc 文件的绝对路径分别为 C:\Windows\System32\Notepad.exe 或和 C:\Users\Test.doc。

② 相对路径：从当前目录开始到到某个文件的路径。如果当前目录为 System32，则 Data.accdb 文件的相对路径为..\..\User\Data.accdb（用“..”表示上一级目录）。

在路径中，文件夹名称之间的分隔符可以是“\”，也可以是“/”。因为 Windows 中使用“/”作为分隔符，但是 MS DOS 中使用“\”，为了支持 MS DOS 的使用习惯，所以也支持“\”。但是 Python 中“\”表示转义字符的开始，所以表示路径的字符串应该避免出现歧义。下面是三种正确的用法。以 C:\Users\Test.doc 为例。

◆ 用“/”作为分隔符，如"C:/Users/Test.doc"。

◆ 用“\\”表示一个“\”，例如"C:\\Users\\Test.doc"。

◆ 字符串之前用“r”或“R”可以取消“\”的转义功能，成为一个普通字符，如 r"C:\Users\Test.doc"。

6.1.3 文件分类

在计算机系统中，文件种类繁多，处理方法和用途也各不相同。文件的分类标准主要有下列两种：

1．按文件的内容分类，可分为程序文件和数据文件。程序文件存储的是程序，包括源程序和可执行程序，例如 Python 的源程序文件（.py）、C++源程序文件（.cpp）、可执行程序文件（.exe）等都是程序文件。数据文件存储的是程序运行所需要的各种数据，例如文本文件（.txt）、Excel 工作簿（.xlsx）都是数据文件。

2．按存储信息的形式分类，可分为文本文件和二进制文件。文本文件存放的是各种数据的 ASCII 代码，可以用记事本打开；二进制文件存放的是各种数据的二进制编码，不能用记事本打开，必须由专用程序打开。

文本文件具有行结构，如图 1.6.4 所示。每一行的结束都有换行符“\n”，图中用“↵”表示。因此，在 Python 中对文本文件的读写通常采用两种方式：

- 一次性地读写文件的所有内容。
- 以行为单位，一行一行地按顺序进行读写。也就是说，读出时从头到尾按顺序一行一行地读，写入时也一样。

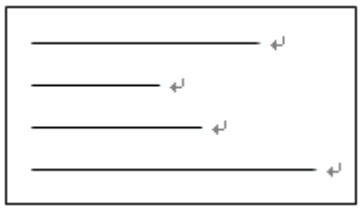


图 1.6.4　顺序文件的结构

6.1.4 文本文件编码

文本文件中的字符都是以某种类型编码存储的。打开文件时应指定编码类型，否则读写时会出现错误。例如，假定 SHI.TXT 文件中含有汉字，编码类型是 UTF8，若执行下面语句：

```
fp=open(r"C:\Test\SHI.TXT","r")
s=fp.read()
print(s)
```

则出现下列错误信息：

```
UnicodeDecodeError: 'gbk' codec can't decode byte 0xa4 in position 4: illegal multibyte sequence
```

这是 Unicode 解码错误。因为 SHI.TXT 文件的编码类型是 UTF8，而语句 open 函数打开文件时没有指定文件编码类型，默认为 ANSI，所以出现了错误。那么，如何文件的编码类型有哪几种类型？如何确定文件的编码类型？

用记事本打文本文件，选择“文件丨另存为”命令，在对话框下面的“编码”组合框显示当前文件采用的编码类型，打开组合框可以看到所有支持的编码类型，如图 1.6.5 所示。若要转换文件的编码类型，则使用“文件丨另存为”命令，选择所需要的编码类型。

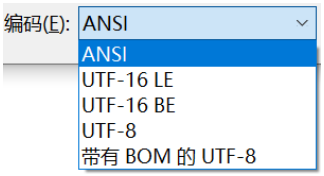


图 1.6.5　记事本“另外为”对话框中“编码”组合框

下面介绍三种常用的编码：

（1）ANSI　中文 Windows 中，西文字符采用 ASCII 码，占一个字节；中文字符采用 GBK 码，占两个字节。例如，“2021 上海 SH”，ANSI 编码（十六进制形式）为：

编码：32　30　32　31　C9　CF　BA　A3　53　48

字符：2　0　2　1　上　海　S　H

（2）UTF-16 LE　针对 Unicode 的一种字符编码。它的特点是所有字符均占两个字节，低字节在前高字节在后。例如，字符“A”的 ASCII 值为 41H，UTF-16 LE 编码是 41 00H,；中文字符的 UTF-16 LE 码与 GBK 没有关系。例如，“2021 上海 SH”，Unicode 编码（十六进制形式）为：FF FE 32 00 30 00 32 00 31 00 0A 4E 77 6D 53 00 48 00。

编码：32 00 30 00 32 00 31 00 0A 4E 77 6D 53 00 48 00
字符： 2 0 2 1 上 海 S H

其中：文件开始两个字节“FF FE”表示编码类型为 UTF-16 LE；

（3）UTF-8 针对 Unicode 的一种可变长度字符编码。它使用 1~4 字节为每个字符编码，ASCIl 字符占 1 个字节，中文字符占 3 个字节。例如，“2021 上海 SH”，UTF-8 编码（十六进制形式）为：

编码：32 30 32 31 E4 B8 8A E6 B5 b7 53 48
字符：2 0 2 1 上 海 S H

打开文件时，一般都通过 encoding 参数指定文件的编码类型。encoding 参数的取值见表 1.6.1 所示。

表 1.6.1 encoding 参数的常用值

编码类型	enconding 对应的值 （不区分大小写）
ANSI	MBCS、ANSI、GB2312、GBK
UTF-16LE	UTF-16LE
UTF-8	UTF、UFT8、U8、UTF-8、UTF_8

6.1.5 文件访问流程

一般来说，访问文件流程分三个步骤：打开文件、读写文件、关闭文件，如图 1.6.6 所示。

对文件的操作主要有两类：一是读操作，也称为输入，将数据从文件（存放在外存上）读入到变量（内存）供程序使用；二是写操作，也称为输出，将数据从变量（内存）写入文件（存放在外存上）。

打开文件时，系统为文件在内存中开辟了一个专门的数据存储区域，称为文件缓冲区。将数据写入文件时，先是将数据写入文件缓冲区暂存，等到文件缓冲区满了或文件关闭时才输出到文件。反之，从文件读数据时，先是将数据送到文件缓冲区，然后再提交给变量，如图 1.6.7 所示。这样处理的目的是为了减少直接读写外存的次数，节省操作时间。

文件操作结束后一定要关闭文件，因为有部分数据仍然在文件缓冲区，若不关闭文件会有数据丢失现象发生，尽管大多数情况下操作系统会自动关闭文件。



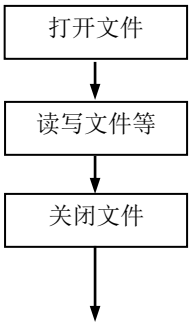


图 1.6.6　文件处理一般流程

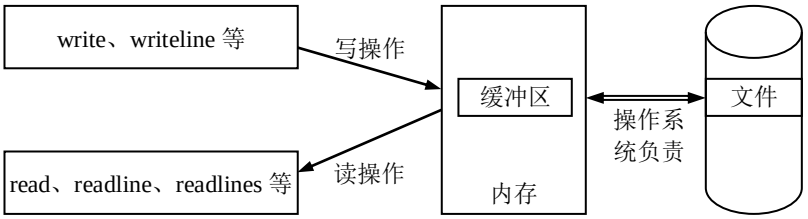


图 1.6.7　文本文件的读写与文件缓冲区

6.2 文本文件

在 Python 中，文本文件的读写有三个特点：

① 读写文件时，可以一次性读写文件所有内容，也可以行为单位进行读写。

② 以行为单位进行读写的规则也简单，就是按顺序进行访问。读的时候从头到尾一行一行地按顺序读，写入时也一样，不可以跳过前面的数据而直接读写某一行数据。

③ 写文件时，各种类型的数据都需要转换成字符串后才能写入文件。

6.2.1 文本文件的打开与关闭

1．打开文件

在对文件进行操作之前，必须打开文件，同时通知操作系统对文件所进行的操作是“读”还是“写”。打开文件的函数是 open，其常用形式如下：

```
fp = open(file, mode='r', encoding='utf-8')
```

说明：

① file 是要读写的文件名，可以是字符串常量，也可以是字符串变量。如"D:/Test/Scores.txt"、"D:\\Test\\Scores.txt"或 r"D:\Test\Scores.txt"。

② mode 指明打开文件的模式，是可选参数，其值如表 1.6.2 所示。

表 1.6.2　mode 参数

mode	作用	说明
'r'	读模式	文件必须存在
'w'	写模式	若文件不存在，则创建文件；若文件存在，则清空文件内容
'x'	创建文件	若文件已经存在，则失败
'a'	追加模式	若文件存在，则向文件的末尾追加内容； 若文件不存在，则创建新文件
'b'	二进制模式	可以与其他模式结合使用
't'	文本模式（默认）	可以与其他模式结合使用
'+'	读 / 写模式	可以与其他模式结合使用

例如，如果要打开 D:\Python 目录下一个文件名为 Students.txt 文件，供写入数据，则语句应为：

```
fp = open(r"D:\Python\Students.txt", "w")
```

③ encoding 指定文件的字符编码方式，缺省值是"utf-8"，常用值见表 1.6.1。

④ open 函数的返回值是文件对象，文件的读写和关闭都需要文件对象的方法实现的。

⑤ 文件对象内置了一个迭代器，因此可以把文件对象当作由文件所有行所组成的列表一样遍历文件。例如，

```
f=open("C:\\Test\\t1.txt","r")
for line in f:
    print(line, end="")
f.close()
```

2．关闭文件

当结束各种读写操作以后，还必须关闭文件，否则会造成数据丢失等现象。因为通常数据是被送到缓冲区，关闭文件时才将缓冲区中数据全部写入文件。关闭文件所用的方法是 close，其使用形式如下：

```
文件对象.close()
```

例如，语句 fp.close()关闭 fp 文件对象所代表的文件。

6.2.2 文件文件的读写

1．读文件

从文本文件中读出数据常用的方法有 read、readline、readlines。

（1）read 方法

功能：将文件的所有内容作为一个字符串一次性读出。

常用格式：变量=文件对象.read()

说明：read 方法的返回值是字符串，它包含文件所有的内容。

例如，下列程序将 D:\Test\Scores.txt 读出并且输出。

```
fp=open("D:\Test\Scores.txt","r")
all=fp.read()
print(all)
```

（2）readline 方法

功能：从文本文件中读出一行数据，并将它作为函数的返回值。

常用格式：变量=文件对象.readline()

说明：readline 的返回值是一字符串，字符串的内容是文件的一行，包含换行符。

例如，下列程序将 D:\Test\Scores.txt 读出并且输出。

```
fp=open(r"D:\Test\Scores.txt","r")
while True:
    str=fp.readline()
    if not str:
```

```
            break
        print(str, end="")
    fp.close()
```

（3）readlines 方法

功能：把文件中的多行数据一次性读入一字符串列表。

常用格式：变量=文件对象.readlines()

格式：变量=fp.readlines()

说明：readlines 的返回值是一列表，列表的元素是文件的一行，包含换行符。

例如，读取并且输出文本文件 Scores.txt 中的内容。

```
fp=open(r"D:\Test\Scores.txt","r")
lines=fp.readlines()
print(lines)
fp.close()
```

程序运行结果为：['181023,王海涛,66\n', '182498,周文英,88\n', '180992,陈建栋,71\n']

2．写文件

将数据写入文本文件常用的方法是 write、writelines。

（1）write

格式：文件对象.write(字符串)

说明：

① 一次只能写一个字符串。若要写入多个数据，只能使用多个语句。例如，语句 fp.write("185125","徐文静","88")是错误的，应写成：

```
fp.write("185125")
fp.write("徐文静")
fp.write("88")
```

② 参数只能是字符串，使用其他类型的数据均会出错。例如，fp.write(88)是错误的，而应写成 fp.write("88")

③ 写数据时不会自动换行，若要换行，应添加换行符："\n"。例如，fp.write("88\n")

④ 尽管写的是字符串，但是不像其他程序语言一样写的字符串自动添加双引号。

```
fp=open(r"D:\Test\Sushi.txt","w")
lines=["竹外桃花三两枝，","春江水暖鸭先知。","蒌蒿满地芦芽短，","正是河豚欲上时。"]
for line in lines:
    fp.write(line+"\n")
fp.close()
```

（2）writelines 方法

功能：将存放在列表中的多行数据一次性写入文本文件

格式：文件对象.writelines(字符串列表)

说明：若要换行，则应在列表元素的末尾添加换行符。

```
fp=open(r"D:\Test\Sushi.txt","w")
lines=["竹外桃花三两枝，\n","春江水暖鸭先知。\n","蒌蒿满地芦芽短，\n","正是河豚欲上时。\n"]
fp.writelines(lines)
```

```
fp.close()
```

程序运行结果如图 1.6.8 所示。

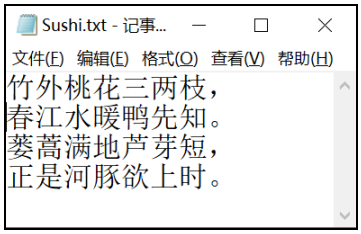


图 1.6.8　writelines 运行结果

【例 6.2】用 readlines 方法读出例 6.1 中文件的数据并且实现统计功能。

```
Statistics=open(r"C:\Test\Scores.txt","r")                    # 打开文件供读数据
Total=0                                                       # 变量 Total 用于统计总成绩
Students=Statistics.readlines()                               # 一次读出所有的行
n=len(Students)                                               # 统计人数
for item in Students:                                         # 对列表 Students 元素进行循环处理
    Temp=item.split(",")                                      # 根据“,”分隔符切分一行的数据
    Mark=int(Temp[2])                                         # 取出成绩
    Total=Total+Mark                                          # 累加成绩
Average=Total/len(Students)                                   # 计算平均成绩
print("人数：",n)
print("总成绩",Total)
print("平均成绩",Average)
Statistics.close()                                            # 关闭文件
```

6.2.4 异常处理

在文件处理过程中，经常会出现各种各样错误或异常，例如，以"r"模式打开一个不存在的文件。为了不影响程序的正常运行，避免因为错误或异常导致程序中止运行，程序中可以使用 try:...except...语句进行异常处理。

需要读者注意的是，错误和异常导致的结果往往相似，但是两个不同的概念。一般来说，错误是指语法错误和逻辑错误，而异常是指一个事件，该事件会在程序运行过程发生，从而影响程序的正常执行。不管是错误还是异常，发生时程序都需要捕获和处理它，否则程序会终止运行。

1. try...except...简单形式

try...except...语句的简单使用形式为：

```
try:
        <语句块 1>
except: <异常类型>:
        <语句块 2>...
```

使用 try...except...语句的关键是了解异常的类型，不知道异常类型就无法使用 try...except...语句。了解异常类型的方法是在调试时人为制造异常，从提示信息中可以发现

异常类型。例如，假定 D:\Test\Data.txt 文件不存在，则执行 fp=open(r"D:\Test\Data.txt","r") 语句时会显示：

```
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    fp=open(r"D:\Test\Data.txt","r")
FileNotFoundError: [Errno 2] No such file or directory: 'C:\\Test\\Data.txt'
```

其中指出异常类型为“FileNotFoundError”。

【例 6.3】为例 6.2 设置异常处理。

```
n=int(input("请学生人数："))
try:
    fScores=open(r"D:\Test\Scores.txt","w")                    # 打开文件供写数据
    for i in range(n):                                         # 输入三个同学的成绩
        No=input("请输入学号：")                               # 输入学号
        Name=input("请输入姓名：")                             # 输入姓名
        Mark=int(input("请输入成绩："))                        # 输入成绩
        fScores.write(No + "," + Name +"," + str(Mark) + "\n") # 写入文件
    fScores.close()                                            # 关闭文件
except FileNotFoundError:
    print("文件不存在")
```

【例 6.4】分班程序：n 个同学分成若干个班，一个班有 k 个同学。k 是键盘输入的，若输入的是 0，则程序会终止运行。添加异常处理语句处理除数为 0 的情况。

```
try:
    n=int(input("请输入学生人数："))
    k=int(input("请输入班级人数："))
    if n%k==0:
        m=n//k
    else
        m=n//k+1
    print("班级数=",m)
except ZeroDivisionError:
    print("输入的班级人数为零")
```

2. try...except...复杂形式

一个语句块可能会发生多种错误或异常，此时需要分门别类捕捉和处理，就应使用 try...except...语句的复杂形式，其语法格式如下：

```
try:
    <语句块 0>
except   <异常类型 1>:
    <语句块 1>
except   <异常类型 2>:
    <语句块 2>
……
except   <异常类型 n>:
    <语句块 n >
except:
    <语句块 n+1>
```

```
    [else:
        <语句块 n+2>]

    [finally:
        <语句块 n+3>]
```

说明：

① 这种形式的异常处理语句指定了各种异常发生的情况下所执行的语句块，最后一个except 子句没有指定异常类型，表示处理剩下的异常类型。

② else 后的语句块是对 try 后语句块正常执行后的一种追加处理，与 for 和 while 语句中的 else 子句相同。

③ 不论会不会发挥异常，finally 后的语句块都是执行，一般是将执行<语句块 1>的一些收尾工作放在这里。

【例 6.5】修改例 6.4，使之还能够处理 n 和 k 是非整数的情况。

```
n 个同学分成若干个班，一个班中个同学，
try:
    n=int(input("请输入学生人数："))
    k=int(input("请输入班级人数："))
    if n%k==0:
        m=n//k
    else
        m=n//k+1
    print("班级数=",m)
except ValueError:
    print("输入了非整数")
except ZeroDivisionError:
    print("输入的班级人数为零")
except:
    print("其他错误")
```

显然，使用 try...except...语句可以提高程序的稳定性和可靠性，但是也不能过度使用。因为过度使用会降低程序的可读性，增加维护的难度。因此，一般来说，均在关键的少数地方使用 try...except...语句。

6.2.4 CSV 格式

CSV（Comma-Separated Values，逗号分隔值）格式是一种常用的文本格式，其特点是存储表格数据时用逗号作为分隔符（也可以使用其他字符），如图 1.6.9 所示。CSV 格式应用非常广泛，常用于程序之间传递表格数据。目前很多应用软件都支持 CSV 格式。例如，使用 MS Excel 可以打开 CSV 文件，也可以另存为 CSV 文件，但是另存为 CSV 文件后其他数据格式信息全部丢失。

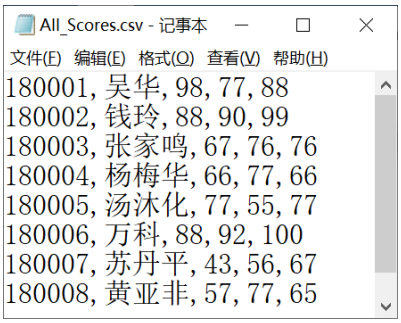


图 1.6.9　CSV 格式文件

因为直到 2005 年 CSV 才有标准，所以难以统一，多种 CSV 格式都在使用，它们自成体系，相互不兼容，导致处理 CSV 文件时会遇到麻烦。对于 Python 程序员来说，最好有一套自己的标准并且遵循，这样才不会犯低级错误。

CSV 文件具有下列特点：

① 纯文本文件，扩展名为.csv

② 像普通文本文本一样具有行结构，没有空行。

③ 数据之间通常用逗号作为分隔符，但是也可以不用逗号，因此 CSV 也称字符分隔值。

④ 字符串数据没有双引号

尽管 Python 提供了 CSV 库可以使用，但是因为 CSV 格式非常简单，程序员可以自己编写程序或函数，这样更显灵活和个性化。

【例 6.6】图 1.6.7 所示的 CSV 文件一行的数据由学号、姓名以及数学、外语、计算机三门课程成绩组成。读出文件中的数据，统计所有同学的平均总成绩。

```
fp=open(r"D:\Test\All_Scores.csv","r")
lines=fp.readlines()
Total=0
Count=0
for line in lines:
    temp=line.split(",")
    Total=Total+int(temp[2])+int(temp[3])+int(temp[4])
    Total=Total+1
    Count=Count+1
Average=Total/Count
print("平均总成绩",Average)
```

6.3 jieba 中文分词

中文分词是指将一个汉字序列切分成一个个独立的词。在英文中，单词之间是以空格、标点符号作为自然分界符的，使用字符串的 split 方法就可以完成分词；在中文中，词与词之间没有一个形式上的分界符，分词是一件非常困难的事情。然而中文分词又是非常重要，是中文信息处理的基础，因而需要使用 jieba 库。

jieba 是一个用于中文分词的第三方库，使用之前必须安装。安装命令的形式为：

```
pip install jieba
```

1．中文分词模式

jieba 库的中文分词模式有三种：精简模式、全模式、搜索引擎模式。

① 精简模式

精准模式尝试将句子最精确地分词，适合文本分析。精简模式通常使用 lcut 方法实现，其返回值是列表。例如：

```
import jieba
lst =jieba.lcut('小明本科毕业于同济大学，后在耶鲁大学深造')
print(lst)
```

程序运行结果为：['小明', '本科毕业', '于', '同济大学', '，', '后', '在', '耶鲁大学', '深造']

② 全模式

全模式把句子中所有可以成词的词语都扫描出来，速度非常快，但其分词得到的结果有可能有歧义。精简模式通常也是使用 lcut 方法实现，其返回值也是列表，但是指定形式参数 cut_all 的值为 True。例如：

```
import jieba
lst=jieba.lcut('小明本科毕业于同济大学，后在耶鲁大学深造',cut_all=True)
print(lst)
```

程序运行结果为：['小', '明', '本科', '本科毕业', '毕业', '于', '同济', '同济大学', '大学', '', '', '后', '在', '耶鲁', '耶鲁大学', '大学', '深造']

从输出结果可以可以发现，全模式比精确模式得到的词的组合多了很多。

③ 搜索引擎模式

在精确模式的基础上，对长词再次切分，提高召回率，适合于搜索引擎分词。搜索引擎模式通常使用 lcut_for_search 方法实现，其返回值是列表。例如：

```
import jieba
list1=jieba.lcut_for_search('小明本科毕业于同济大学，后在耶鲁大学深造')
print(list1)
```

程序运行结果为：['小明', '本科', '毕业', '本科毕业', '于', '同济', '大学', '同济大学', '，', '后', '在', '耶鲁', '大学', '耶鲁大学', '深造']

【例 5.21】统计红楼梦小说中主要人物的出现次数。

分析：首先是读入红楼梦.txt 文件，是需要将读入的文本进行分词，然后进行中文分词，最后对主要人物进行统计，统计结果存储在字典中。由此程序代码如下：

```
import jieba
file = open(r"C:\Test\红楼梦.txt", "r", encoding='utf-8')
txt=file.read()                                    #读文件
wordsList=jieba.lcut(txt)                          #  分词结果
actors=[('贾宝玉',"宝玉"),("林黛玉","黛玉"),("薛宝钗","宝钗"),
        ("王熙凤","凤姐"),("贾母","老太太"),("袭人",),("探春",),
        ('贾琏',),("王夫人","夫人")]               # actors 是人物列表，每个人物使用了元组
dictActors={}                                      #  拟建立的字典
for actor in actors:                               #从人物表中查分词
```

```
        if len(actor)==2:
            count1=wordsList.count(actor[0])
            count2=wordsList.count(actor[1])
            dictActors[actor[0]]=count1+count2
        else:
            count1=wordsList.count(actor[0])
            dictActors[actor[0]]=count1+count2

    items = list(dictActors.items())                    # 将字典表项转化成列表
    items.sort(key=lambda x:x[1], reverse=True)         # 按频次从大到小排序
    for i in range(len(items)):                         # 输出统计结果
        word, count = items[i]
        print ("{}\t{}".format(word, count))
```

程序的最终运行结果如下：

```
贾宝玉      3665
贾母        2091
王熙凤      1201
王夫人      1110
林黛玉      697
贾琏        659
袭人        580
薛宝钗      572
探春        420
```

2．文本关键词抽取

使用 jieba.analyse 中的 extract_tags 方法可以提取文本中出现的关键词，其使用格式为：

```
extract_tags(文本, topK=20, withWeight=False, allowPOS=( ))
```

说明：

① topK 参数指定按权重提取的关键字个数，默认 20。

② withWeight 指定是否返回关键字的权重。

下面的代码获取了红楼梦小说中权重最大的 5 个关键词。

```
import jieba.analyse as analyse
file = open(r"F:\teach\python 语言\例子数据\红楼梦.txt", "r", encoding='utf-8')
txt=file.read()
lists=analyse.extract_tags(txt, topK=5, withWeight=False, allowPOS=( ) )
print(lists)
```

程序运行结果为：['宝玉', '凤姐', '贾母', '王夫人', '老太太']

6.4 图像文件

图像文件是典型的二进制文件。Python 中常用的图像处理库是 PIL（Python Image Library）。它拥有 21 个处理图像的类，如 Image、ImageFilter、ImageEnhance 等，支持图像

的存储（格式转换）、处理、显示等功能。

PIL 是 Python 的第三方库，使用之前必须安装。安装库的名字是 pillow，安装命令为：

```
pip  install  pillow
```

6.4.1 图像基本处理

Image 是 PIL 中最重要的类，可以用来实现图像处理的所有基本功能。Python 处理图像是从导入 Image 开始的。下面首先通过一个例子说明使用 PIL 包处理图像的基本过程，然后再介绍 Image 类。

【例 6.7】将彩色图像转换成黑白图像

```
from PIL import Image                          # 导入 PIL 库的 Image 模块
im = Image.open(r"D:\Test\Tulips.jpg")         # 使用 open 方法打开图像文件，返回图像对象
im.show()                                      # 显示图像对象 im 中的图像
im = im.convert("L")                           # 转换为灰度 8 位图像
im.show()                                      # 显示图像
im.save(r"D:\Test\New-Tulips.jpg")             # 保存图像文件
```

程序运行结果如图 1.6.10 所示。

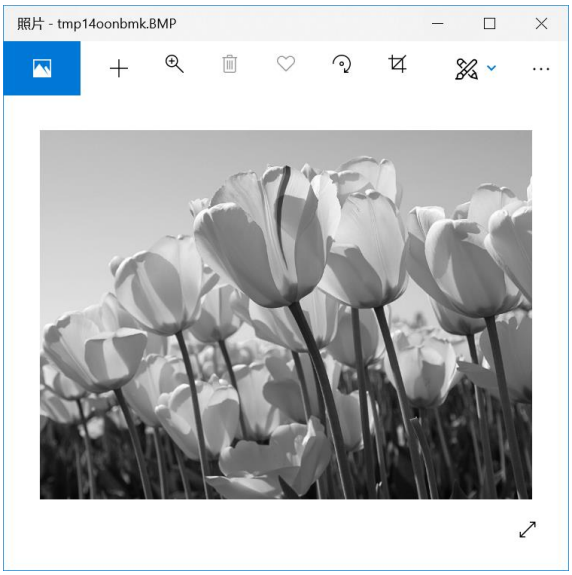


图 1.6.10　Tulips.jpg 转换后的效果

从上述例子可以看到，PIL 处理图像的基本过程是：

① 导入 PIL 库的 Image 等模块

② 使用 open 方法打开图像文件，返回图像对象，以后通过图像对象实现图像处理的。

③ 图像显示或处理

④ 若需要，则保存图像

1．图像打开、显示、转换和保存

（1）图像打开

Image 类的 Open 用于打开图像文件，返回一个 Image 对象，其使用形式为：

```
变量=open(文件名)
```

说明：

① open 的返回值一个 Image 对象，图像文件打开后所有的操作都是通过 Image 对象实现的；

② Image 对象具有 size、format、mode 等属性。

- size 属性　一个元组，由图像的宽度和高度两个数据组成;
- format 属性　图像文件格式，常见的包括 JPEG、PNG、GIF 等格式;
- mode 属性　颜色模式，如表 1.6.3 所示等。

表 1.6.3　图像颜色模式

格式	说明
1	1 位像素，表示黑和白，但是存储的时候每个像素仍然为 8bit
L	8 位像素，黑白图像
P	8 位像素，使用调色板映射到其他模式
RGB	3x8 位像素，真彩色
RGBA	4x8 位像素，有透明通道的真彩色
CMYK	4x8 位像素，印刷四色模式
YCbCr	3x8 位像素，彩色视频格式
I	32 位整型像素
F	32 位浮点型像素

例如，下列代码显示 Tulips.jpg 图像的 size、mode 和 format 属性

```
from PIL import Image
im = Image.open(r"D:\Test\Tulips.jpg")
print(im.size)
print(im.mode)
print(im.format)
```

程序结果如下：

```
(1024, 768)
RGB
JPEG
```

（2）图像显示

Image 对象的 show 方法用于显示图像，其使用形式为：

```
图像对象.show()
```

（3）保存图像

Image 对象的 save 方法用于保存图像，其使用形式为：

```
图像对象.save(文件名, 格式)
```

说明：save 方法的格式参数表示以何种图像格式存储，其值与 Image 对象的 fomat 属性相同，但使用时应加引用，如"JPEG"。若缺省则根据文件扩展名表示的格式存储。

（4）转换图像

Image 对象的 convert 方法用于转换图像格式，其使用形式为：

图像对象.convert(格式)

说明：图像格式如表 1.6.3 所示。

（5）获取指定位置的像素值

Image 对象的 getpixel 方法用于获取指定位置的像素值。如果图像为多通道，则返回一个元组。其使用形式为：

图像对象.getpixel((x,y))

例如，下列代码显示 Tulips.jpg 图像中位置（100,80）像素值。

```
from PIL import Image                          # 导入 PIL 库的 Image 模块
im = Image.open(r"D:\Test\Tulips.jpg")         # 使用 open 方法打开图像文件，返回图像对象
print(im.getpixel((100,80)))
```

程序结果如下：

(156, 204, 253)

表示位置（100,80）像素的红绿蓝三种颜色的值分别为 156、204、253。

（6）处理像素

Image 对象的 point 方法用于处理图像的像素。其使用形式为：

图像对象.point(函数)

例如，下列代码首先将 Tulips.jpg 转换成 8 位灰度图像，然后将灰度值 127 以上的转换成 255（白），127 以下的转换成 0（黑）。结果如图 1.6.11 所示。

```
from PIL import Image
im = Image.open(r"D:\Test\Tulips.jpg")
im = im.convert("L")
im = im.point(lambda x: 255 if x > 127 else 0)
im.show()
```

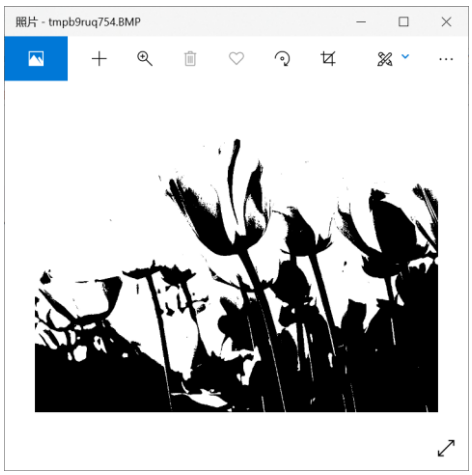


图 1.6.11　用 point 方法处理后的黑白图像

2. 缩略图、缩放、旋转、翻转

Image 类可以制作缩略图，也可缩放、旋转、翻转图像，所用方法如表 1.6.4 所示。

表 1.6.4　图像的基本操作方法

功能	方法	示例	说明
缩略图	thumbnail	img2=img1.thumbnail((128,128))	制作 128×128 的缩略图
缩放	resize	img2=img1.resize((200,200))	将图像缩放到 200×200
旋转	rotate	img2 = img1.rotate(-45)	顺时针旋转 45 度 若数值为正，则逆时针旋转
翻转	transpose	img2=img1.transpose(Image.FLIP_LEFT_RIGHT)	左右翻转
		img2=img1.transpose(Image.FLIP_TOP_BOTTOM)	上下翻转
		img2=img1.transpose(Image.ROTATE_90)	逆时针 90
		img2= img1.transpose(Image.ROTATE_180)	逆时针 180
		img2=img1.transpose(Image.ROTATE_270)	逆时针 270

【例 6.8】将图像左右翻转。

```
from PIL import Image
img1 = Image.open(r"D:\Test\Tulips.jpg")
img2 = img1.transpose(Image.FLIP_LEFT_RIGHT)      #左右翻转
img2.save("New_Tulips.jpg")
```

3. 色彩通道的分离和合并

色彩通道的分离是指将 RGB 图像的 3 个颜色通道分离开来，生成三个列表，所有的方法是 split，常用使用形式为：

```
r,g,b=im.split()
```

色彩通道的合并是分离的逆操作，即将通过 3 个颜色通道合并成一个新的图像，所有的方法是 imerge，常用使用形式为：

```
om=Image.merge("RGB",(r,g,b))
```

【例 6.9】将图像中的红色和蓝色两个颜色通道进行交换。

```
from PIL import Image
im = Image.open(r" D:\Test\birdnest.jpg")
im.show()
r,g,b=im.split()
om=Image.merge("RGB",(b,g,r))
om.show()
```

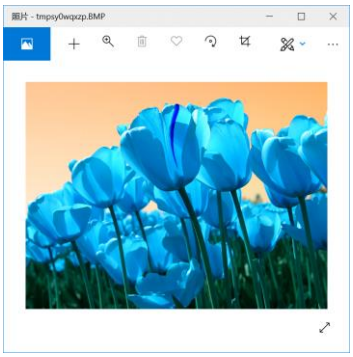


图 1.6.12　红、蓝两个颜色通道交换后的图像

6.4.2 图像的滤波和增强

1. 图像滤波

在图像处理中，经常需要对图像进行平滑、锐化、边界增强等滤波处理。在使用 PIL 图像处理库时，可以通过 Image 类中的成员函数 filter()来调用滤波函数对图像进行滤波，而滤波函数则通过 ImageFilter 类来定义的。filter()函数的使用形式为：

```
im.filter(滤波函数)
```

其中，常用的滤波函数如表 1.6.5 所示。

表 1.6.5　常用的滤波函数

滤波函数	滤波效果说明
ImageFilter.BLUR	模糊效果
ImageFilter.CONTOUR	轮廓效果
ImageFilter.DETAIL	细节效果，即图像变得清晰
ImageFilter.EDGE_ENHANCE	边界增强效果
ImageFilter.EMBOSS	灰色浮雕效果
ImageFilter.FIND_EDGES	边界效果，即寻找边界滤波
ImageFilter.SMOOTH	平滑效果
ImageFilter.SHARPEN	锐化效果

【例 6.10】图像浮雕处理。

```
from PIL import Image
from PIL import ImageFilter as IF
im = Image.open(r"D:\Test\Tulips.jpg")
im1 = im.filter(IF.EMBOSS)
im1.show()
```

程序结果如图 1.6.13 所示。

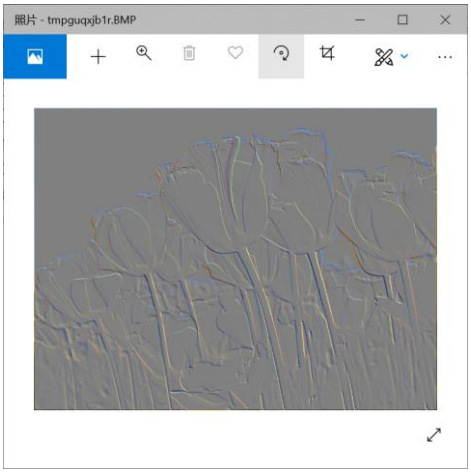


图 1.6.13　图像浮雕效果

2. 图像增强

PIL 模块中的 ImageEnhance 类专门用于图像的增强处理，可以增强（或减弱）图像的亮度、对比度、色度、锐度等。

ImageEnhance 类中所有功能都是一个类（派生类），所以用法与一般模块不同。例如，要使得某个图像亮度增加一倍，则应首先用 im 实例化一个 ImageEnhance.Brightness 对象（假定被变量 en 引用），然后 en 调用 enhance 方法增强图像，返回值是增强后的图像对象（假定被变量 im1 引用），最后 im1 调用 show 方法显示增强后的图像。代码如下：

```
from PIL import Image
from PIL import ImageEnhance
im=Image.open(r"D:\Test\Tulips.jpg")
en=ImageEnhance.Brightness(im)
im1=en.enhance(2)
im1.show()
```

ImageEnhance 的图像增强功能对应所用的类以及用法如表 1.6.6 所示。

表 1.6.6　ImageEnhance 的图像增强方法

功能	所用的类	用法示例
亮度增强	ImageEnhance.Brightness	en=ImageEnhance.Brightness(im) im1=en.enhance(2) im1.show()
色度增强	ImageEnhance.Color	en=ImageEnhance.Brightness(im) im1=en.enhance(2) im1.show()
对比度增强	ImageEnhance.Contrast	en=ImageEnhance.Brightness(im) im1=en.enhance(2) im1.show()
锐度增强	ImageEnhance.Sharpness	en=ImageEnhance.Brightness(im) im1=en.enhance(2) im1.show()

6.5 综合应用

【例 6.11】生成 Fibonacci 数列的前 10 个数据写入文件 D:\Test\Fibonacci.txt，格式如图 1.6.14 所示。

分析：Fibonacci 数列又称黄金分割数列，因数学家列昂纳多·斐波那契（Leonardoda Fibonacci）以兔子繁殖为例子而引入，故又称为“兔子数列”，指的是这样一个数列：1、1、2、3、5、8、13、21、34……在数学上，Fibonacci 数列以如下被以递推的方法定义：F(1)=1，F(2)=1，F(n)=F(n-1)+F(n-2)（n>=3）。

程序：

```
fp=open(r'D:\Test\Fibonacci.txt','w')
f1=1
fp.write("Fib(1)=%d\n"%f1)
f2=1
fp.write("Fib(2)=%d\n"%f2)
for i in range(3,11):
    f3=f1+f2
```

```
        fp.write("Fib(%d)=%d\n"%(i,f3))
        f1=f2
        f2=f3
    fp.close()
```

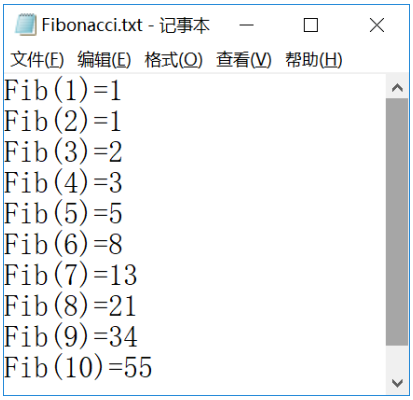


图 1.6.14　Fibonacci 数列的前 10 个数据

【例 6.12】设计一个文件加密程序，如图 1.6.15 和图 1.6.16 所示。它将 D:\Test\Source 文件中的内容加密，加密后的内容存入 D:\Test\Target。加密方法是：A→F、a→f、B→G、b→g……Y→D、y→d、Z→E、z→e，即每一个字母加一个密钥（5），其他字符不变。

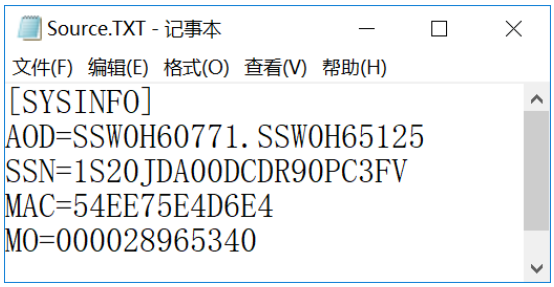


图 1.6.15　源文件

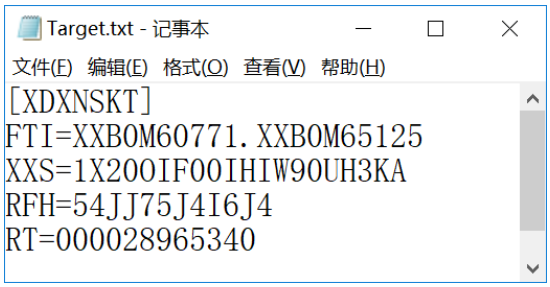


图 1.6.16　加密文件

程序：

```
    fp1=open(r'D:\Test\Source.txt','r')
    fp2=open(r'D:\Test\Target.txt','w')
    lines=fp1.readlines()
    for line in lines:
        newline=""
        for ch in line:
            if "A"<=ch<="U" or "a"<=ch<="u":
                ch=chr(ord(ch)+5)
            elif "V"<=ch<="Z" or "v"<=ch<="z":
                ch=chr(ord(ch)+5-26)
            newline=newline+ch
        fp2.write(newline)
    fp1.close()
    fp2.close()
```

【例 6.13】数据分拣。现有一个总成绩文件和一个班级名单文件，如图 1.6.17 和 1.6.18 所示，数据分拣就是根据班级名单中的学号从总成绩文件把班级同学的成绩分拣出来，组成

一个新的文件，如图 1.6.19 所示。

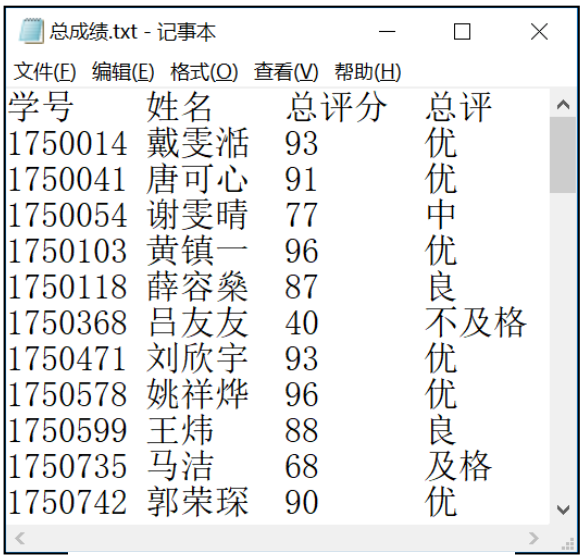


图 1.6.17　学生总成绩

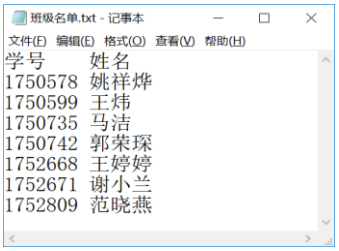


图 1.6.18　班级名单文件

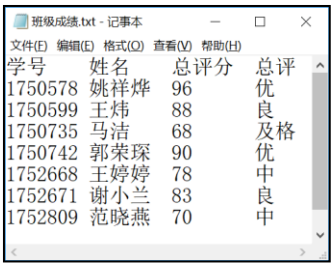


图 1.6.19　班级成绩文件

```
TotalScoreFile=r"D:\test\总成绩.txt"
myClassListFile=r"D:\test\班级名单.txt"
myClassScoreFile=r"D:\test\班级成绩.txt"
fmyClass=open(myClassListFile,"r")
myClassStudentList=[]
lines=fmyClass.readlines()
for line in lines:
    temp=line.split("\t")
    studentNo=temp[0]
    myClassStudentList.append(studentNo)
fmyClass.close()
fTotal=open(TotalScoreFile,"r")
Title=fTotal.readline()
fmyClassScore=open(myClassScoreFile,"w")
fmyClassScore.write(Title)
for line in fTotal:
    temp=line.split("\t")
    thisStudentNo=temp[0]
    if thisStudentNo in myClassStudentList:
        fmyClassScore.write(line)
fTotal.close()
fmyClassScore.close()
```

【例 6.14】微球金纳米颗粒覆盖度计算。一个 SiO2，上面覆盖了金颗粒(小白点)，如图 1.6.20 所示，计算覆盖度。

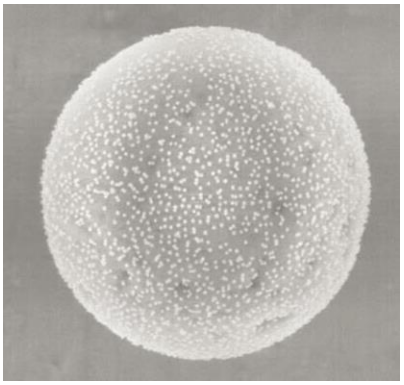


图 1.6.20　覆盖金颗粒的 SiO2

分析：从照片上可以看到，金颗粒（小白点）周围有明显边界，球的边缘附近，白色背景，看不清金纳米颗粒。

解决方案：对图像进行二值化处理，找到合理阈值，再计算像素点加和除以圆的总像素点。

```
from PIL import Image
im = Image.open(r"D:\Test\edegeDetect2.jpg")
im = im.convert("L")
im = im.point(lambda x: 255 if x > 200 else 0)
im.show()                                          # 2 值化后
count1=0
w,h = im.size                                      # 获得图像的宽度和高度，像素单位
for i in range(w):
    for j in range(h):
        pixel=im.getpixel((i,j))                   # i,j 位置的像素
        if pixel != 0:
            count1 +=1    #
print(count1/(w*h)*100)
```

习题

一、选择题

1．文件按存储信息的形式分类，可分为_____。

A．系统文件和用户文件　　B．文本文件和二进制文件

C．程序文件和数据文件　　D．磁盘文件和 U 盘文件

2．在 Python 中，从计算机内存中将数据写入文件，称为_____。

A．输入　　B．输出

C．修改　　D．删改

3．若要指定 D:\Data 文件夹下的 T.txt，应可以使用_____。

A．r'D:\Data\T.txt'　　B．'D:\Data\T.txt'

C．r'D:\\Data\T.txt'　　D．'D:\\Data\T.txt'

4．假定 T.txt 是采用 UTF8 编码，则正确的语句是_____。

A．fp=open("T.TXT","r")

B．fp=open("T.TXT",encoding="UTF-8","r")

C．fp=open("T.TXT","r",encoding="ANSI")

D．fp=open("T.TXT","r",encoding="UTF-8")

5．使用 open 函数打开文件时，_____方式不能打开一个不存在的文件。

A．"r"　　B．"w"

C．"a"　　D．"wb"

6．下面代码对文本文件“Text.txt”操作后，变量 s 的数据类型是_____。

```
MyFile=open('Text.txt','r')
s=MyFile.readline()
```

A．字符串　　B．列表

C．元组　　D．字典

7．下面代码对文本文件“Text.txt”操作后，变量 s 的数据类型是_____。

```
MyFile=open('Text.txt','r')
s=MyFile.readlines()
```

A．字符串　　B．列表

C．元组　　D．集合

8．某个学生的学号、姓名和成绩分别为 185125、张颖和 77，则作为一行写入文件对象 fp 所代表的文件，正确的语句应为_____。

A．
```
fp.write("185125","张颖",77)
```

B．
```
fp.write("185125","张颖","77")
```

C．
```
fp.write("185125")
fp.write("张颖")
fp.write(77)
```

D．
```
fp.write("185125")
fp.write("张颖")
fp.write("77")
```

9．fp 是文件对象，下面语句中 s 的数据类型应是_____。

```
fp.writelines(s)
```

A．字符串　　B．列表

C．元组　　D．字典

10．若要将数据添加到一个文件名为“C:\T1.txt”的文本文件，则应使用下列_____程序段打开文件。

A．
```
s=r"C:\T1.txt"
fp=open(s,"a")
```

B．
```
s = "C:\T1.txt"
fp=open("s","a")
```

C．
```
fp=open("C:\T1.txt","a")
```

D． fp=open(C:\\T1.txt,"a")

11．程序的运行过程中经常会出现各种各样错误或异常，为了不影响程序的正常运行，避免因为错误或异常导致程序中止运行，程序中可以使用_____语句进行处理。

A．if

B．while 和 break

C．for 和 continue

D．try...except...

12．在下列关于 try...except...使用的说法中，错误的是_____。

A．若最后一个 except 子句没有指定异常类型，表示处理剩下的异常类型。

B．else 后的语句块是对 try 后语句块正常执行后的一种追加处理，与 for 和 while 语句中的 else 子句相同。

C．finally 后的语句块是在异常发生的情况下执行的。

D．使用 try...except...语句的关键是了解异常的类型，不知道异常类型就无法使用 try...except...语句

13．在下列关于 jieba 的说法中，错误的是_____。

A．jieba 中文分词有三种模式，jieba.lcut(s)属于精简模式。

B．jieba 也可以用于英文文章的分词。

C．使用 jieba 之前需要安装，安装命令是 pip install jieba。

D．jieba.lcut(s)分词的结果是一个列表。

14．jieba 的分词模式有三种，下面_____不属于 jieba 的分词模式。

A．精简模式　　B．全模式

C．匹配模式　　D．搜索引擎模式

15．执行_____函数可进行文档的关键词抽取。

A．jieba.analyse.extract_tags()　　B．jieba.lcut()

C．jieba.analyse.lcut()　　D．jieba.extract_tags()

16. 执行语句 from PIL import Image 后，要打开指定的图像文件，从语法上来说，下面_____是正确的。

A．im = Image.open(r"E:\pyteach\digit.jpg")

B．im = open(r"E:\pyteach\digit.jpg")

C．im = Image.open("E:\pyteach\digit.jpg")

D．im = open("E:\pyteach\digit.jpg")

17．假定有：im=Image.open(r"C:\Test\tu1.jpg")，则在下列关于 PIL 的说法中，错误的是_____。

A．语句 im.show()用于显示图像

B．语句 im.convert("L")是将图像转换成 8 位灰度图像。

C．语句 im = im.point(lambda x: 255 if x > 196 else 0)将图像变成真彩色图像

D．属性 im.format 的值是图像的格式

18．PIL 库中专门用于图像的增强处理类是_____。

A．Image

B．ImageEnhance

C．ImageFilter

D．pillow

二、填空题

1．在 Windows 中，文件路径分有两种：绝对路径和_____路径。

2．若文件内容编码类型为 ANSI，则中文字符采用 GBK 码，占_____个字节。

3．_____是针对 Unicode 的一种可变长度字符编码，其特点是使用 1~4 字节为每个字符编码。

4．_____语句用于处理异常.

5．_____格式是一种常用的文本格式，其特点是存储表格数据时常常使用逗号作为分隔符。

6．jieba 库的_____方法用于中文分词，其返回值是一个列表。

7．用于提取关键字的 extract_tags 方法位于 jieba 库的_____子库中。

8．PIL 是 Python 的第三方库，使用之前必须安装。安装库的名字是_____。

9．Image 对象的 show 方法用于显示图像

10．Image 对象的_____方法用于获取指定位置的像素值。

11．可以通过 Image 类中的成员函数_____来调用滤波函数对图像进行滤波。

12．PIL 模块中的_____类专门用于图像的增强处理，可以增强（或减弱）图像的亮度、对比度、色度、锐度等。

13．统计某项选举活动的投票结果。假定有 3 个候选人甲、乙、丙，每个选举人投一票，投票情况为：甲、甲、丙、甲、乙、乙、甲、乙、甲、丙、乙、丙、甲、丙、丙。统计每位候选人的得票数，并按照得票数的降序排序后写入文件中，数据格式如图 1.6.21 所示。

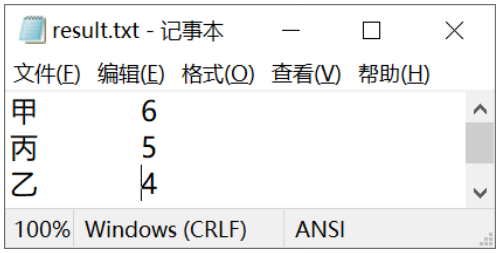
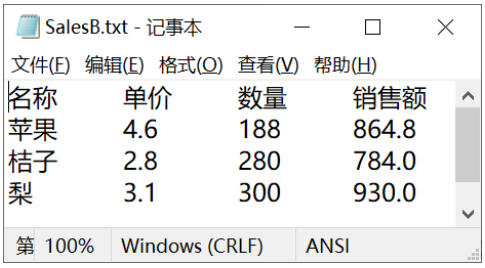


图 1.6.21　结果文件数据格式

```
candidate=['甲','乙','丙']
vote=['甲','甲','丙','甲','乙','乙','甲','乙','甲','丙','乙','丙','甲','丙','丙']
n=len(candidate)
```

```
count=[0]*n
for i in range(n):
    count[i]=vote.count(candidate[i])
result=____(1)____(zip(candidate,count))
result.sort(key=lambda x:____(2)____,reverse =True)
f=open(r'D:\result.txt','w')
for candi in result:
    s=candi[0]+'\t'+____(3)____(candi[1])+'\n'
    f.____(4)____(s)
f.close()
```

14．SalesA.txt 文件中存入了水果的名称、单价和数量，如图 1.6.22 所示。从文件中读出数据，计算销售额，按照销售额降序排列后，写入到文件 SalesB.txt 中，文件内容如下图所示。




```
fp1=open( r"c:\Test\SalesA.txt", "r" )
fp2=open( r"c:\Test\SalesB.txt", ____（1）____)
Titles=fp1.readline()
NewTitles=Titles.rstrip()+"\t"+"销售额"+"\n"
fp2.write(NewTitles)
data1=fp1.readlines()
data2=[]
for item in ____（2）____:
    item=item.rstrip()
    lst=item.split()
    value=float(lst[1])*float(lst[2])
    newitem=item+"\t"+str(value)+"\n"
    data2.append(____（3）____)
fp2.writelines(____（4）____)
fp1.close()
fp2.close()
```

三、问答题

1．什么是文件？文本文件与二进制文件有什么区别？

2．写出满足下列条件的 open 函数的使用形式。

（1）建立一个新的文本文件 new.txt，供用户写入数据。

（2）打开一个老的文本文件 old.txt，用户将从该文件读出数据。

（3）打开一个老的文本文件 append.txt，用户将在该文件后面添加数据。

3．常用文本编码有哪些？如何打开文件时如何指定文本编码？

4．请用三种不同的方法，将文本文件 Text.dat 中的所有内容读出输出在显示器上，要求写出程序代码片段。

5．请说明 read、readline 和 readlines 方法之间的区别。

6．请说明 write 方法和 writelines 方法的区别。

7．为什么不使用 close 语句关闭文件可能会导致文件数据的丢失？

8．CSV 文件有什么特点？如何读写 CSV 文件？

9．请简述 jieba 三种分词模式的区别？

10．PIL 中的 point 函数的作用是什么？

第七章　面向对象程序设计基础

前面几章介绍了结构化程序设计（SP，Structure Programming）的内容和方法，本章将介绍面向对象程序设计（OOP，Object Oriented Programming）的一些基础知识。面向对象程序设计是以软件的形式对现实世界的对象及其属性及行为进行建模，从而提供一种更加自然直观的程序设计方式。

7.1　面向对象程序设计概述

面向对象的编程方法是伴随着软件工程技术的快速发展和软件规模的日益扩大而产生的，它很好地解决了结构化程序设计中数据和操作分离带来的问题，从而更加有利于程序的维护，对数据的操作更加安全、方便。另一方面，客观世界本身就是由各种各样的对象构成的，每种对象都有各自的属性和行为，不同对象之间的相互作用和联系构成了不同的系统。而面向对象的方法就是从现实世界的实体出发，以对象为基本单位，分析、设计和实现一个系统，这种编程思想更符合人们的认知规律。

7.1.1 面向对象的基本概念

面向对象的程序设计从根本上解决了结构化程序设计的缺陷，方法是将数据和所有与这些数据相关的模块一起打包在一个称为“类”的软件构件中。该软件构件中的数据对其内部的所有模块来说，相当于全局变量，操作起来非常方便；而对该构件以外的其它模块来说，通过对这些数据访问权限的设置让其称为不能被访问的数据，从而保障了数据的安全。下面介绍面向对象方法中的一些基本概念。

1．对象

现实世界中存在着各种各样的实体，包括具体的事物和抽象的事物。这些实体就代表了一个个对象。如一名学生、一门课程、一个圆形都是一个个对象。

每个对象有自己的特征和作用在该对象上的行为。如一个学生对象，具有学号、姓名、性别、成绩等特征，具有选课、查询成绩、退课等行为。当然，一个学生对象也具有身高、体重、视力等特征；也具有吃饭、睡觉等行为。

2．类

在现实世界的问题领域中，类是对一组对象中共有的属性特征和行为进行的抽象。它的定义中包含了两方面的内容：

(1) 该类对象的数据描述，包括数据名称及类型。

(2) 该类对象的行为描述，包括具有哪些行为，这些行为是如何实现的。

3．抽象

一个类是一组对象的抽象。抽象是通过特定的实例抽取共同特征以后形成概念的过程。抽象强调主要特征，忽略次要特征；强调给出与应用相关的特性，抛弃不相关的特性。

例如，作为一种抽象，游览线路图提供了给定地理区域中的特征，以便于游客游览。这些特征通常包括主要街道、标志性建筑及游客感兴趣的景点，而不会包括诸如每棵树木，每盏交通信号灯及每家店铺等次要信息。再比如，要处理学生的成绩信息，就应该抽取出与此问题相关的学生的学号、姓名、成绩等特征，而抛弃与问题无关的学生的身高、体重、视力等特征。

7.1.2 类的基本特征

类具有封装性、继承性和多态性三大特征，在这三大特征支持下，数据操作更加安全，程序维护更加方便，代码复用效率进一步提高，处理上也更加灵活。

1．封装性

在面向对象的程序设计中，为了保护对象的状态不受外界影响，而将对象的状态和其行为打包在一个软件构件中，这种特性被称为封装性，它是面向对象程序设计的基础。

这好比飞机上的“黑匣子”，“黑匣子”内部封存了飞机停止工作前的一段时间内的语音对话和飞行高度、速度、航向等飞行参数，但这些内部细节是被保护起来的，对外是隐而不见的。只有通过相应的设备连通黑匣子上的接口才可以获取黑匣子中的信息。使用黑匣子的用户不必关心黑匣子的内部组成和工作原理，只需要关心各种参数指标（属性）和各种接口的使用（方法）。

又如，我们在使用手机时，也从不需要了解其内部工作原理是怎样的，只需要通过显示屏、按键、听筒这些接口设备即可操作手机。

2．继承性

在工作和生活中，继承性无所不在。例如，要设计一款新电脑，有两个途径：一是从头开始，费时费力；二是对现有的型号加以改进，事半功倍。工程师一般是从原有型号的基础上进行改进，增加一些功能，新的电脑很快设计出来了。旧款电脑派生出了新款电脑，新电款脑继承了旧款电脑的功能，这就是继承和派生的实例。另外，像子承父业、遗产继承都是继承和派生的实例。

在面向对象程序设计中，可以在原有类的基础上定义新的类。原有的类称为基类（或称父类），新的类称为派生类（或称子类）。继承性为代码的复用提供了技术支持，更方便程序的扩充。

3．多态性

多态性一词最早用于生物学，是指同一种族的生物体具有相同的特性。在面向对象程序

设计中，多态性是指同一消息作用在不同对象之上产生不同的行为，它使得程序实现更加灵活方便。例如，教室中有教师和学生两种不同类的对象，当上课铃响时，教师打开投影仪和教案准备讲课，学生则打开笔记本和教材准备听课。也就是说，当上课铃响这一消息传来时，教师和学生这两类不同的对象产生了不同的行为。

7.2 类和对象

Python 中有许多已经定义好的类（如列表、元组等），也允许用户自定义类。在这一节中，我们将介绍类的定义和实例化，目的是让大家能更好地理解、使用 Python 或第三方所提供的各种各样的类。

7.2.1 类的定义

Python 中，类是通过 class 保留字定义的，一般形式如下：

```
class 类名:
    '''类文档字符串'''
    类体
```

与定义函数时一样，定义类时可以写一行类文档字符串，用于提供帮助信息。类体需要缩进，用于定义类的属性、方法等。

例 7.1　定义一个表示学生的类 Student，并编写一个应用该类的主程序。

```
# 定义Student类
class Student:
'''定义学生类Student'''
    # SetValue方法（函数）用于定义该类对象的数据属性
    def SetValue(self,a,b,c):
        self.No=a
        self.Name=b
        self.Score=c
    # Disp方法（函数）用于输出该类对象的数据属性值
    def Disp(self):
        print(self.No)
        print(self.Name)
        print(self.Score)
p=Student()                          # 创建类Student的实例（对象）
p.SetValue("1900001", "王涛", 82)    # 对象p调用SetValue方法（函数）定义自己的数据成员
p.Disp()                             # 调用Disp方法（函数）输出对象p中数据成员的值
```

下面按照代码的执行顺序说明类是如何定义和使用的。

① 语句 p=Student()用于创建 Student 类的一个实例（对象），并且赋值给 p，由 p 引用这个实例。

② 类 Student 中定义了 SetValue 和 Disp 函数，类中的函数被称为方法。实例（对象）

创建后，方法被绑定到实例（对象）中，可以通过实例（对象）调用。

③ 语句 p.SetValue("1900001", "王涛", 82)是 p 调用自己的 SetValue 函数定义自己的数据成员，调用时三个实在参数分别赋值给 a、b、c。

④ SetValue 函数的第一个参数是 self，是对象自身的引用。

⑤ p.Disp()输出 p 中的三个数据成员。



图 1.7.1　程序运行结果

从上面的例子可以看到：

（1）定义类的关键字为 class，class 后是类名。

（2）类的方法就是指类中定义的那些函数，形式上除了第一个参数是必须是 self 以外其他的与普通函数相同。一般形式为：

对象.方法名(实在参数表)

（4）类的成员

类的成员可以简单地分为私有成员、保护成员和公有成员，他们的形式和访问权限如表 1.7.1 所示。

表 1.7.1　类的成员

类的成员	形式	示例	访　问　权　限
私有成员	以两个下划线开头	__x	只能在类的内部可以访问，在类的外部不能直接访问，
保护成员	以一个下划线开头	_x	只能在类和派生类中可以访问，在类的外部不能直接访问
公有成员	没有以下划线开头	Name	在类的内部可以访问，在类的外部也可以访问。

对于初学者来说，类的数据成员一般都简单地处理为公有成员。当基本掌握了面向对象程序设计思想后再精细地区别私有成员、保护成员和公有成员。

（5）可以使用 isinstance 内置函数测试一个实例是否为某个类的实例，也可以使用 type 内置函数相看实例类型。例如，在例 7.1 最后添加如下代码：

```
print(isinstance(p,Student))
print(type(p))
```

则显示结果如下：

```
True
<class '__main__.Student'>
```

表示 p 引用的实例是 Student。

例 7.2 定义表示点的类 PyPoint，拥有 SetValue 函数和 Distance 函数。SetValue 函数用于定义对象的 x、y 数据成员，Distance 用于计算对象到原点的距离函数。

```
import math
class MyPoint:
    '''定义表示平面上点的类MyPoint'''
```

```
    def SetValue(self,x,y):
        self.x=x
        self.y=y
    def Distance(self):
        return(math.sqrt(self.x**2+self.y**2))
d=MyPoint()
d.SetValue(3,4)
print(d.Distance())
```

若将上述程序中的数据成员改为私有成员，则存在错误。

```
import math
class MyPoint:
    '''定义表示平面上点的类MyPoint'''
    def SetValue(self,x,y):
        self.__x=x                                  # __x为私有的，只能在类中可以访问
        self.__y=y                                  # __y为私有的，只能在类中可以访问
    def Distance(self):
        return(math.sqrt(self.__x**2+self.__y**2))  # __x和__是私有的，在类中可以访问
d=MyPoint()
d.SetValue(3,4)
print(d.__x,d.__y)                  # 错误：__x和__是私有的，在类的外部不能访问
print(d.Distance())
```

7.2.2 构造函数

Python 中，如果类的每一个实例（对象）都需要完成相同的初始化工作，则可以定义__init__函数。__init__函数是一个特殊的函数，在创建实例（对象）时由 Python 自动调用，在 C++、VB.NET 等面向程序设计语言中都有类似函数，尽管形式有所不同，但均被称为构造函数。

Python 中构造函数的特点：

① 构造函数名是__init__，第一参数是 self，是代表实例（对象）自身；

② __init__函数除了 self 参数以外，可以拥有其他的形式参数；

③ __init__函数可以重载，即可以定义多个参数个数不同的构造函数，但是第一个构造函数必须是 self；

③ 构造函数是在创建实例（对象）时由系统自动调用，程序中不能直接调用。

例 7.3　为例 7.1 中的类 Student 添加构造函数，使之能执行如下的语句。

```
p=Student("1900001", "王涛", 82)
```

构造函数如下：

```
def __init__(self,a,b,c):
    self.No=a
    self.Name=b
    self.Score=c
```

当类实例化时，Python 首先创建实例对象，然后检查是否定义了__init__函数。如果定义了则调用__init__函数，调用时将实例对象作为实参传递给 self，将实例化时的实在参数传递给相应的形式参数，实例化时的参数要与__init__的形式参数匹配。例如，定义了上面的的__init__函数，则下面的语句都是错误的

```
p=Student()
p=Student("1900001", "王涛")
```

因为实例化时的参数个数与定义时的形式参数数量不匹配。

例 7.4 定义表示一元二次方程的类 Equation，含有构造函数和求解方程根的函数。

```
import math
class Equation:
    def __init__(self,x,y,z):
        self.a=x
        self.b=y
        self.c=z
    def solve(self):
        delta=self.b**2 - 4.0*self.a*self.c
        if (delta>=0):
            root1=(-self.b+math.sqrt(delta))/(2.0*self.a)
            root2=(-self.b-math.sqrt(delta))/(2.0*self.a)
            ans=[root1,root2]
            return ans
        else:
            return []
equ=Equation(1,-3,2)
roots=equ.solve()
print(roots)
```

例 7.5　表示圆的类 Circle。圆有三个数据：圆心（x，y）和半径 r，能够求圆的面积。

```
import math
class Circle:
    def __init__(self,x,y,r):
        self.x=x
        self.y=y
        self.r=r
    def area(self):
        return math.pi*self.r**2
p=Circle(0,0,2)
print(p.area())
```

7.2.3 类的实例及其成员的访问

1．类的实例化

定义了类以后，就可以创建类的实例（对象）。类的实例化的一般形式为：

变量=类(实在参数表)

说明：

① 若没有定义构造函数，则实例化时不应有实在参数表；若定义了构造函数，则一定要有实在参数表。

② 将实例（对象）赋值给一个变量，实质是将实例（对象）的地址赋予对象变量，变是与实例的关系是引用，如图 1.7.2 所示。

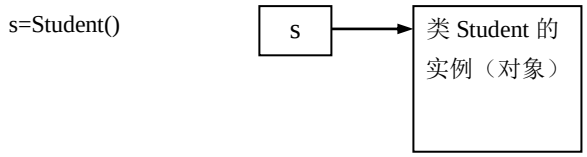


图 1.7.2　对象变量

2．对象成员的访问

对象成员是通过对象变量才能访问其成员。其一般形式为：

数据成员：对象变量.数据成员名

方法：对象变量.方法(<实在参数表>)

7.3 继承和派生

继承性是面向对象程序设计的重要特性。利用继承性，可以实现代码重用，提供无限重复利用程序资源的途径，节省程序开发的时间和资源。

所谓继承，就是在已有类的基础上构造新的类，新类继承了原有的类的数据成员、属性、方法和事件。已有的类称为基类，新的类称为派生类。从集合的角度来说，派生类是基类的子集。例如，若在类的基础上定义表示学生干部的派生类 StudentLeader，则 StudentLeader 类是 Student 的子集，如图 1.7.3 所示。若用图来表示基类与派生类的关系，则用带箭头的直线从派生类指向基类。

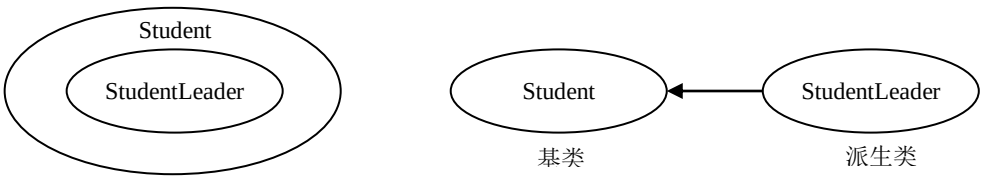


图 1.7.3　基类与派生类的关系

在 Python 中，一个派生类可从一个基类派生，称为单继承；也可从多个基类派生，称为多继承。

图 1.7.3 描述了形状类的继承层次结构关系，形状类是个基类，它包括二维形状和三维形状，称这两个类是形状类的派生类，而梯形和三角形又属于二维形状，称它们是二维形状类的派生类。在该继承关系里，每个派生类只有一个基类，因而都是一种单继承的关系。而图 1.7.4 所示的为计算机系组成人员的继承层次结构中，系主任既是教师，又是行政管理人

员，因而系主任类是从这两个类派生而来的，是一种多继承关系。限于篇幅，本节只介绍单继承。

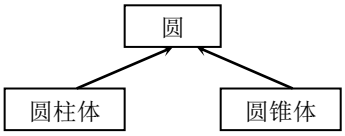


图 1.7.3　形状类的继承

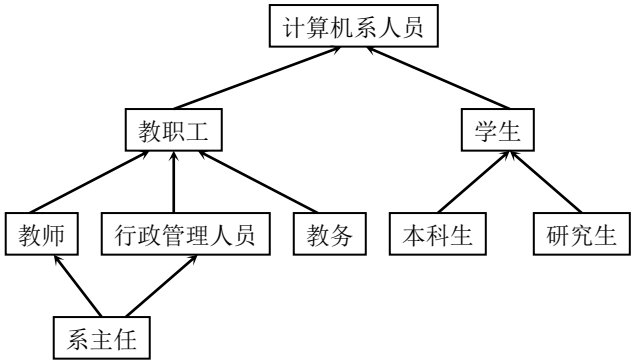


图 1.7.4　计算机系组成人员的继承层次结构

1．引例

首先看一个学生干部类(StudentLeader)的定义。

```
class StudentLeader:
    # SetValue定义对象的属性
    def SetValue(self,a,b,c,d):
        self.No=a                    # 学号
        self.Name=b                  # 姓名
        self.Score=c                 # 成绩
        self.Duty=d                  # 职务
    # Disp用于输出对象的数据成员值
    def Disp(self):
        print(self.No)
        print(self.Name)
        print(self.Score)
        print(self.Duty)
```

学生干部类与学生类的区别是增加了一个表示职务的 Duty 数据成员。

对比例 7.1 所定义的 Student 类可以发现，StudentLeader 类的许多信息都与他相同，只是在该类中增加了一个 Duty 属性，类中的方法也基本不同。若能利用继承机制使用已有 Student 类的成员，则可避免代码的重复编写，采用的方法是将 StudentLeader 类定义为 Student 类的派生类。

例 7.6　定义派生类 StudentLeader。

```
class Student:
    def SetValue(self,a,b,c):
        self.No=a
        self.Name=b
        self.Score=c
    def Disp(self):
        print(self.No)
```

```
            print(self.Name)
            print(self.Score)
class StudentLeader(Student):
    def New_SetValue(self,a,b,c,d):
            Student.SetValue(self,a,b,c)
            self.Duty=d
    def New_Disp(self):
            Student.Disp(self)
            print(self.Duty)
sLeader=StudentLeader()
sLeader.New_SetValue("1900001","王涛",82,"班长")
sLeader.New_Disp()
```

2．派生类的定义

在 Python 中，定义派生类的形式非常简单，只需要在派生类的类名后的括号中加上父类的名字即可。一般形式如下：

```
class   派生类名(基类名):
    "类文档字符串"
    类体
```

3．派生类的构造函数

派生类继承了基类的所有成员，但是基类的构造函数是不能继承的。因此，若需要对派生类对象进行初始化，则需要定义新的构造函数。定义派生类的构造函数的一般形式：

```
__init__(派生类构造函数总参数表)
    基类名.__init__(基类构造函数参数表)
    定义派生类对象属性数据成员初始化
End Sub
```

例 7.7 为例 7.6 中的 StudentLeader 类添加构造函数，并且编写程序测试。

```
class Student:
    def __init__(self,a,b,c):
            self.No=a
            self.Name=b
            self.Score=c
    def Disp(self):
            print(self.No)
            print(self.Name)
            print(self.Score)
class StudentLeader(Student):
    def __init__(self,a,b,c,d):
            Student.__init__(self,a,b,c)
            self.Duty=d
    def New_Disp(self):
            Student.Disp(self)
            print(self.Duty)
sLeader=StudentLeader("1900001","王涛",82,"班长")
```

```
sLeader.New_Disp()
```

7.4 迭代器和生成器

Python 之所被认为是美丽的程序设计语言，原因之一是已将迭代器、生成器等功能内置在语言中，这些功能的使用有时会简化程序设计的复杂度。例如，处理大量数据时常遇到计算机内存不足的问题，使用迭代器或生成器后就没有必要将所有数据一次性都放入内存中，而是根据处理所需调入数据，减少了计算机内存的负载，这就是迭代器和生成器的作用。

7.4.1 迭代器

1．可迭代对象

可迭代对象是能够一次返回其一个成员的对象。简单地说，任何可以用 for 循环遍历的对象都是可迭代对象，例如：

```
# iterables
sample = ["复旦大学", "上海交通大学", "同济大学","华东师范大学"]
for i in sample:
    print(i)
```

这里，sample 可以用 for 循环遍历，所以是一个可迭代对象。因此，字符串、列表、元组、字典、集合都是可迭代对象。从语法上来说，for 循环语句中 in 后面应该是一个可迭代对象。可迭代对象为什么可以用 for 遍历？因为可迭代对象内部有一个迭代器。

2．迭代器

迭代器就是用来从可迭代对象中进行取值的循环方法，必须用__iter__()和__next__()实现。拥有__iter__方法的对象是可迭代的，拥有__next__()方法的对象是迭代器。下面通过一个例子说明__iter__()和__next__()是如何实现的。

【例 7.7】自定义一个迭代器，实现斐波那契数列。

```
class Fibs():
    def __init__(self,max):
        self.max=max
        self.n,self.a,self.b=0,0,1
    #定义__iter__方法
    def __iter__(self):
        return self
    #定义__next__方法
    def __next__(self):
        if self.n<self.max:
            tmp=self.b
            self.a,self.b=self.b,self.a+self.b
            self.n+=1
            return tmp
        raise StopIteration                          # 停止迭代
```

```
for item in Fibs(10):
    print(item,end=" ")
```

程序运行结果为：1 1 2 3 5 8 13 21 34 55。

说明：

① __init__()方法是类构造函数，调用类时会首先执行该函数。n、a、b 的初值设置为 0、0、1。

② __iter__()和__next__()方法使这个类变成了迭代器。

③ __iter__()方法返回迭代器对象并对迭代进行初始化。由于类对象本身是迭代器，因此它返回自身；

④ __next__()方法从迭代器中返回当前值，并改变下一次调用的状态。

7.4.2 生成器

1．生成器

生成器也是迭代器，但更加优雅。使用生成器，可以实现与迭代器相同的功能，但不必在类中编写__iter__()和__next__()函数。生成器就是一个能够完成与迭代器相同功能的简单函数。

```
def Fibs(n):
    a,b=0,1
    while a<n:
        value=a
        a,b=b,a+b
        yield value
for item in Fibs(10):
    print(item,end=" ")
```

这个生成器函数和常规函数是有区别的。普通函数使用 return 关键字返回值，但是这个生成器函数使用 yield 关键字返回值。这就是生成器函数与常规函数不同的地方，除了这种区别其他完全相同的。

yield 关键字的工作方式类似于普通的 return 关键字，但有额外的功能：它能记住函数的状态。因此，下次调用生成器函数时，它不是从头开始，而是从上次调用中停止的位置开始。

2．生成器表达式

为了简化编写生成器函数的工作，Python 提供了生成器表达式，就像列表生成式一样。唯一的区别是生成器表达式包含在圆括号内。例如，

```
square_gen = (x*x for x in range(1,10))
for i in square_gen:
    print(i)
```

最后，通过比较产生 10000 个整数的列表推导式和生成器表达式所占用的内存空间说明

为什么要使用生成器。

```
import sys
mylist = [i for i in range(10000000)]                    # 列表推导式所产生的列表
print('列表所占用内存的大小：',sys.getsizeof(mylist))
mygen = (i for i in range(10000000))                     # 生成器表达式
print('生成器表达式所占用内存的大小：',sys.getsizeof(mygen))
```

程序运行结果为：

```
列表所占用内存的大小： 81528056
生成器表达式所占用内存的大小： 120
```

7.5 模块、库、包

1．模块

随着代码量的不断增多，程序的维护成了一件困难的事情。为了解决这一问题，通常会进行拆分，将具有类似功能的代码放同一文件中。这样每个文件包含的代码就相对较少。在 Python 中，一个.py 文件称为一个模块（module），每一个模块中往往定义了一些变量、函数、类等。例如，图 1.7.X 中的 Test1.py 和 Test2.py 都可以称为模块，模块名分别为 Test1 和 Test2。

```
""" Test1.py 模块 """
import Test2
Angle=float(input("请输入一个角度："))
Answer=Test2.MySin(Angle)
print(Answer)
```

```
""" Test2.py 模块 """
import math
def MySin(x):
    s = 0
    t = x
    n = 1
    while (math.fabs(t) >= 1e-6):
        s=s+t
        n=n+1
        t=-t*x*x/(2 * n - 1)/(2 * n - 2)
    return s
```

使用模块的好处如下所示：

1．提高代码的可维护性

2．提高代码的可复用性。当一个模块编写完成之后，可以被其他地方引用。

3．使用模块还可以避免变量名、函数名、类名等命名冲突。相同名字的函数、变量名、类名等可以放在不同的模块中，但要避免与内置函数名冲突

2．包

模块虽然解决了函数名、变量名和类名等命名冲突，但是模块之间也会存在命名冲突的情况。为了解决模块命名冲突问题，Python 引入了按目录来组织模块的方法。包（package）就是一个内含模块或子包的有层次的文件目录结构，其中一定要__init__.py 模块。例如，可

以将 Test1 和 Test2 模块组合成 Test 包，如图 1.7.X 所示。

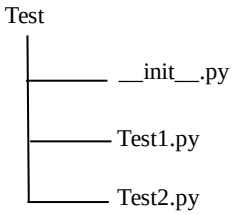


图 1.7.X　Test 包的目录结构

包目录具有如下两个特点：

① 每个包目录中必须含有__init__.py 模块，否则 Python 会将视为普通目录，而非一个包。__init__.py 文件内容可以为空，也可以有相应的代码。

② 一个包目录中还可以存在多个目录，称为子包，组成多层级的包。

3．库

库（lib）是具有相关功能模块的集合。库是一个抽象概念，任意一个模块或包都可以称为库。库通常分成三类：标准库、第三方库以及自定义模块。

◆ 标准库　Python 自带的库。math、random 等模块都是其中的一模块。

◆ 第三方库　就是第三方发布的具有特定功能的模块。Jieba、Matplotlib、NumPy、pandas、scikit-learn、TensorFlow、PyTorch、Keras、SciPy 等。

◆ 自定义模块　用户自己可以自行编写模块，然后使用。

7.6 综合应用

本章简要地介绍了类的定义以及类的继承性，下面介绍 3 个综合应用案例，希望能帮助读者理解本章的内容。

例 7.8　定义一个时间类 MyDate，具有年、月、日三个属性，以及一个求第二天日期的函数和输出日期的函数。

```
import datetime                                          # 导入datetime库
class MyDate:
    def __init__(self):
        t=datetime.datetime.now()                        # 获取当前时间
        self.MyYear=t.year
        self.MyMonth=t.month
        self.MyDay=t.day
    def NextDate(self):
        TotalDays=[[31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31], \      # \是续行符
                   [31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]]
        self.MyDay = self.MyDay + 1
        leap = ((self.MyYear % 400==0) or (self.MyYear%4==0 and self.MyYear%100!=0))
        if self.MyDay>TotalDays[leap][self.MyMonth - 1]:
            self.MyDay = 1
            self.MyMonth = self.MyMonth + 1
            if self.MyMonth>12:
                self.MyMonth=1
```

```
                self.MyYear=self.MyYear + 1
        def Disp(self):
            print(self.MyYear)
            print(self.MyMonth)
            print(self.MyDay)
    d=MyDate()
    d.NextDate()
    d.Disp()
```

例 7.9　从例 7.5 派出新类--圆柱体类 Cylinder，它具有新属性 h（高度）和计算体积的 volume 函数。

```
    import math
    class Circle:
        def __init__(self,x,y,r):
            self.x=x
            self.y=y
            self.r=r
        def area(self):
            return math.pi*self.r**2
    class   Cylinder(Circle):
        def __init__(self,x,y,r,h):
            Circle.__init__(self,x,y,r)
            self.h=h
        def volume(self):
            return self.area()*self.h
    p=Cylinder(0,0,2,3)
    print(p.area())
    print(p.volume())
```

习题

一、选择题

1．在面向程序设计中，类有三个基本特征，不属于三个基本特征的是_____。

A．封装性　　B．继承性

C．多态性　　D．交互性

2．在面向程序设计中，对象具有三要素，不属于三要素的是_____。

A．属性　　B．方法

C．类　　D．事件

3．有如下类的定义：

```
class person:
    def __init__(self,name):
        self.name=name
    def show(self):
        print(self.name)
```

下面代码能正常执行的是_____.

A．h=person('张三');h.show()　　B．h=person();h.show('张三')

C．h=person();h.show()　　D．h=person('张三');h.show('张三')

4．在 Python 中，构造函数的名称是_____。

A．与类的名称相同　　B．new

C．New　　D．__init__

5．定义类时，以两个下划线开头的是_____。

A．公有成员　　B．私有成员

C．保护成员　　D．构造函数

6．设计一个类的代码时，类中每个方法的第一个参数都必须是_____。

A．this　　B．it

C．me　　D．self

7．在以下说法中，错误的是_____。

A．一个基类可以派生出很多个派生类

B．一个类只能创建一个对象

C．派生类除了可以继承基类的属性和方法，还可以定义自己的属性和方法。

D．类定义中可以没有__init__方法。

8．在以下说法中，错误的是_____。

A．任何可以用 for 循环遍历的对象都是可迭代对象。

B．生成器函数不一定要使用 yield 关键字返回值。

C．生成器表达式包含在圆括号内。

D．使用迭代器或生成器后可以节省内存。

9．在以下说法中，错误的是_____。

A．一个.py 文件称为一个模块（module）。

B．每个包目录中必须含有__init__.py 模块。

C．任意一个模块或包都可以称为库。

D．使用自定义模块时不需要用 import 导入。

二、填空题

1．类具有_____、继承性和多态性三大特征。

2．在类中，私有成员是以_____个下划线开头。

3．构造函数的名称是_____函数。

4．所谓继承，就是在已有类的基础上构造新的类，新类继承了原有的类的数据成员、属性、方法和事件。已有的类称为_____，新的类称为派生类

5．拥有__iter__方法的对象是可迭代的，拥有_____方法的对象是迭代器

6．生成器函数使用_____关键字返回值。

7．包（package）就是一个内含模块或子包的有层次的文件目录结构，其中一定要有_____模块。

三、问答题

1．什么是类？什么是对象？类和对象的关系是什么？

2．面向对象程序设计的主要特性是什么？

3．构造函数有什么特点？如何编写构造函数？

4．类的方法实质是什么？

5．什么是派生？派生类的构造函数如何编写？

6．self 代表什么？

第 8 章　数据可视化

数据可视化是指用图形图表的方式呈现数据，可以直观清晰地展示出数据包含的信息，有助于分析数据中隐含的变化规律或发展趋势，是数据处理过程中非常重要的一个环节。数据可视化广泛应用于销售数据的可视化，统计数据的可视化和机器学习数据的可视化等。实现数据可视化的第三方库主要包括 Matplotlib、Seaborn、ggplot、Bokeh 等，Matplotlib 是 Python 中使用最广泛的可视化库，其他许多可视化库大多是建立在其基础上或者直接调用该库，本章将介绍 Matplotlib 库的使用和常见图表的绘制方法。

8.1 Python 图表绘制基础

下面通过某公司销售数据的图表化来了解图表绘制的一般过程。

8.1.1 引例—销售量柱状图

【例 8.1】绘制某公司某年度的销售柱状图（又叫条形图），显示某产品各季度的销售量，效果如图 1.8.1 所示。

分析：首先准备好绘制柱状图所需的数据，这里是某产品的销售量，可以是列表类型，也可以是元组类型，分别控制各个柱形的高度，然后确定每个柱形的宽度和位置，最后调用 matplotlib.pyplot 库中的 bar 函数绘制柱状图。

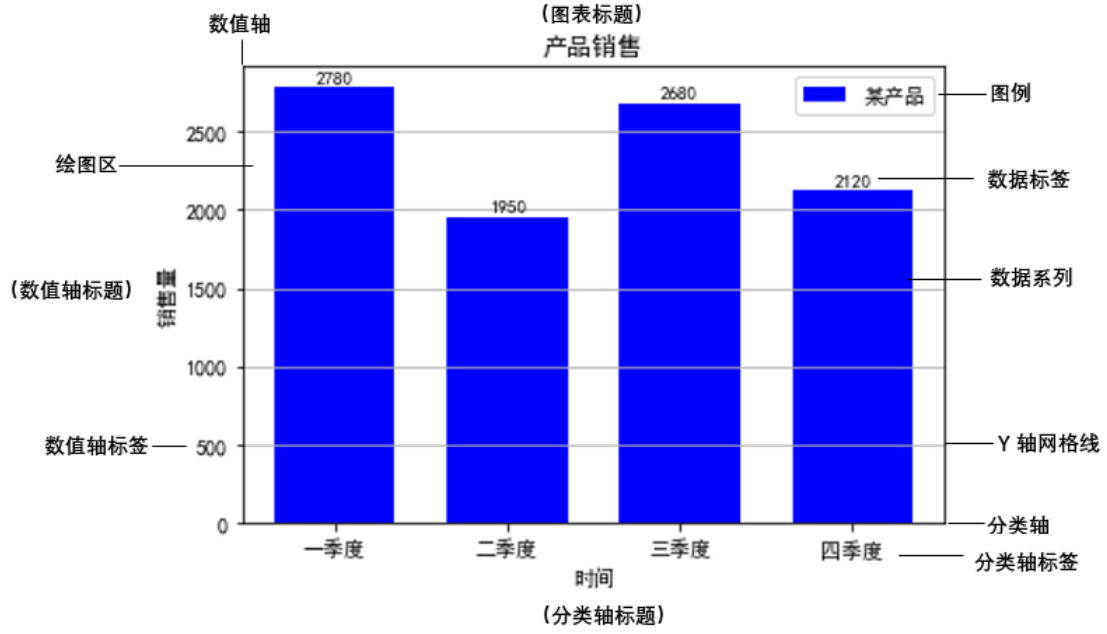


图 1.8.1　销售柱状图

① 导入库

```
import matplotlib.pyplot as plt                    #导入 pyplot 子库
```

② 新建绘图区

```
plt.figure(figsize=(6,4))                          #绘图区大小为 6*4 英寸
plt.rcParams['font.sans-serif']=['SimHei']         #设置显示汉字（黑体）
```

③ 准备数据

```
seazons=['一季度', '二季度', '三季度','四季度']      #设置分类轴显示文本
sales=[2780,1950,2680,2120]                        #设置某产品的销售量数据
index=[0,1,2,3]                                   #index 控制分类轴刻度
barW=0.7                                           #barW 控制条形的宽度
```

④ 设置图表属性

```
plt.title("产品销售")                               #设置图表标题
plt.xlabel("时间")                                 #设置 X 坐标轴标签
plt.ylabel("销售量")                               #设置 Y 坐标轴标签
plt.xticks(index,seazons)                          #设置分类轴标记
plt.grid(axis="y")                                 #显示横向网格线
```

⑤ 绘制图表

```
plt.bar(index,sales,barW,color='b',label='某产品')   #绘制柱状图
for a,b in zip(index,sales):                       #显示数值标注
    plt.text(a,b+30, '%.0f'%b, ha='center', fontsize=9)   #ha 设置水平对齐方式
```

⑥ 显示图例

```
plt.legend(loc='upper right')                      #在右上角显示图例
```

⑦ 显示图表

```
plt.show( )
```

从引例中看到，Python 中图表的绘制过程比较简单，关键是要准备好绘制所需的数据，控制好各图表元素的显示，掌握图表绘制函数的调用方法。

8.1.2 Matplotlib 绘图基础

Matplotlib 是 Python 环境下常用的一个绘图库，可生成指定分辨率的图形文件，满足各种出版级图形的要求，官网下载地址为：https://matplotlib.org/。

1.子绘图库

（1）pyplot　Matplotlib 库下的一个模块，又叫子绘图库，能够绘制出各种常见的图表类型，包括二维图表和三维图表，还可同时绘制多个子图。

（2）pylab　也是 Matplotlib 的子绘图库，同样可以绘制二维和三维图表，区别是 pylab 中包括了许多 numpy 和 pyplot 模块中常用的函数，方便用户快速进行计算和绘图，更适合在 IPython 交互式环境中使用。

在开始绘制图表前，先用“import matplotlib.pyplot as plt”导入绘图库，后面用别名 plt 来指代 matplotlib.pyplot 子绘图库。

2.图表的组成

Matplotlib 的图表区包含三个层次：容器层，辅助显示层和图像层。

（1）容器层　由 Canvas（画布）、Figure（绘图区）和 Axes（坐标系）三个部分组成，Canvas 位于容器层的最底部（由系统自动创建），Figure 则建立在 Canvas 上面，而 Axes 则建立在 Figure 上面。一个 Figure 对象内可以包含多个 Axes 对象，每个 Axes 对象都是一个独立的坐标系，绘图过程中所有图像都是基于坐标系绘制的。

（2）辅助显示层　包括 title (标题)、axis (坐标轴)、axis label (坐标轴标题)、tick (坐标

轴刻度)、tick label (坐标轴刻度标签)、spines (图表边框线)、grid (网格线)、legend (图例)、facecolor (前景色)等。

（3）图像层　调用各个绘图函数绘制的柱状图、折线图、饼图、散点图等图像。

注意：辅助显示层和图像层都必须创建在 Axes 对象上。

3.绘图区

绘图区是图表绘制的基础，既可以由系统自动创建，也可以根据需要调用 Figure()函数来创建，还可以将一个绘图区划分为多个子绘图区，各个子绘图区拥有自己独立的坐标系，可以在指定的子绘图区上绘制图表。

（1）自动创建绘图区

在 Matplotlib 中可以直接调用图表绘制函数，系统将自动创建一个绘图区，默认大小为 6.4*4.8 英寸，即 figsize=(6.4,4.8)，分辨率为 100，即 dpi=100，背景为白色，即 facecolor=“white”。

例如：

```
import matplotlib.pyplot as plt
plt.plot((3,5))              #自动创建一个绘图区，绘制一条从(0,3)到(1,5)的直线
```

（2）手动创建绘图区

如果需要指定绘图区的大小、分辨率和背景颜色等参数，那么先调用 figure()函数创建一个绘图区，然后再调用图表绘制函数。

例如：

```
import matplotlib.pyplot as plt
plt.figure(figsize=(4,3),dpi=72)        #绘图区指定为 4*3 英寸，分辨率为 72
plt.plot((6,2))                         #绘制一条从顶点(0,6)到顶点(1,2)的直线
```

常用绘图区控制函数如下：

① figure 函数，用于生成一个全局绘图区，figure 函数的使用形式为：

```
figure(num=None,figsize=None,dpi=None,facecolor=None,**kwargs)
```

说明：

● 所有参数均可省略，创建的绘图区上没有建立坐标系，默认不显示该绘图区，可以调用函数 plt.show( )来显示。

例如:

```
>>>plt.figure( )
```

在 IDLE 中显示执行结果为<Figure size 640x480 with 0 Axes> ，即绘图区大小为 640*480 像素，无坐标系。

● 参数 figsize 用于指定绘图区大小，dpi 指定分辨率，facecolor 指定绘图区的颜色。

例如：

```
>>>plt.figure(figsize=(4,3),facecolor='b')
#指定绘图区大小为 4*3 英寸，背景色为蓝色
>>>plt.show( )      #显示绘图区，效果如图 1.8.2 所示。
```

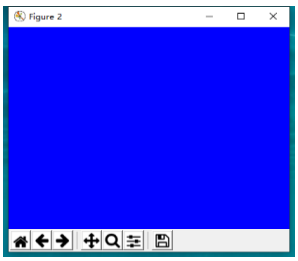


图 1.8.2 全局绘图区

② axes 函数，用于创建一个坐标系风格的绘图区并显示，axes 函数的使用形式为：

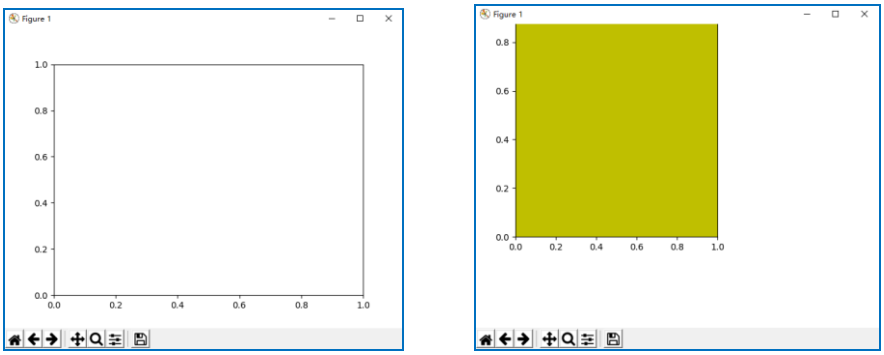
```
axes((left, bottom, width, height), facecolor='w')
```

说明：所有参数均可省略，也可以在参数中指定绘图区的位置与大小，设置绘图区的颜色等。

例如:

```
>>> plt.axes( )                                  #显示效果如图 1.8.3(a)所示
>>> plt.axes((0.1,0.3,0.5,0.8),facecolor='y')    #显示效果如图 1.8.3(b)所示
>>> plt.show( )                                  #显示坐标系风格的绘图区
```

注意：坐标系的原点默认在绘图区的左下角（0，0），图 1.8.3(b)的坐标原点设在了绘图区的（0.1，0.3）处。如果绘图区大小为 640*480 像素，这里的（0.5，0.8）则对应宽高为 320*384 像素。



(a) 默认坐标系位置和大小　　(b) 指定坐标系位置和大小

图 1.8.3　IDLE 中坐标系风格的绘图区

（3）创建子绘图区

在一个全局绘图区中可以同时绘制多个子图，首先创建多个子绘图区，每个子绘图区拥有独立的坐标系，然后在指定的子绘图区中绘制图表。

常用子绘图区控制函数如下：

① subplot 函数，用于在全局绘图区中创建子绘图区并显示指定子绘图区，subplot 函数的使用形式为：

```
subplot(nrows, ncols, index, **kwargs)
```

说明：

● nrows 表示子绘图区的行数，ncols 表示子绘图区的列数，index 表示当前子绘图区的编号，按照从左到右，从上往下的顺序从 1 开始依次编号。

● 如果 nrows、ncols 和 index 的值都小于 10 的话，可以把三个参数的值缩写为一个整数，如 subplot(323)和 subplot(3,2,3)都表示有 3 行 2 列共 6 个子绘图区，显示第 3 个子绘图区。

例如：

```
>>>plt.subplot(1,1,1)    #表示有 1 行 1 列，绘制在第 1 个子绘图区
>>>plt.subplot(232)      #有 2 行 3 列共 6 个子绘图区，只显示第 2 个子绘图区
>>>plt.subplot(346)      #有 3 行 4 列共 12 个子绘图区，只显示第 6 个子绘图区
```

● kwargs 参数用于接收图表辅助显示对象的值，例如以下代码创建 2 行 2 列共 4 个子绘图区，在 3 个子绘图区中分别绘制 3 条不同的直线，代码运行结果如图 1.8.4 所示。

```
import matplotlib.pyplot as plt
import numpy as np
fig=plt.figure()                                    #该语句可以省略
plt.subplot(221,title="No 1",xticks=np.arange(6))   #设置图表标题和 X 轴刻度范围
plt.plot((2,4),(3,6),c='g')
plt.subplot(223,xlabel="Number",ylabel="Value")     #设置坐标轴标签
plt.plot((1,2),(4,3),c='b')
```

```
plt.subplot(224, facecolor='#00FFFF')                    #设置子绘图区填充颜色为青色
plt.plot((1,2),(3,4),c='y')
plt.subplots_adjust(wspace=0.4,hspace=0.6)               #调整各子绘图区之间的间隔
plt.show()
```

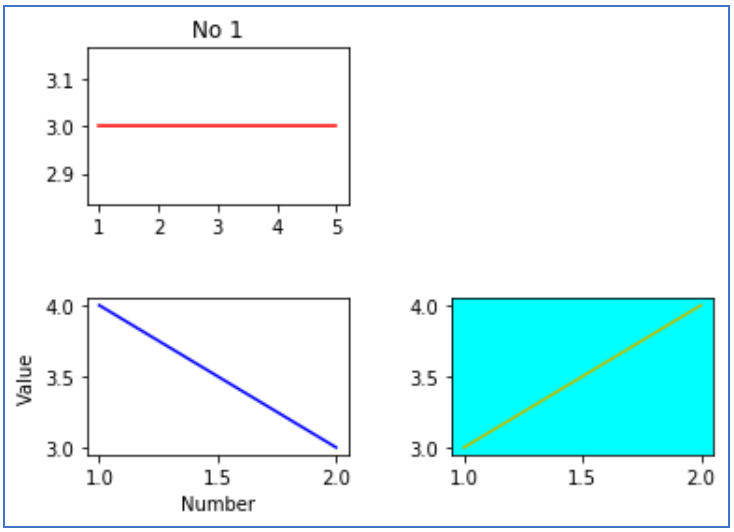


图 1.8.4 在指定子绘图区绘制线条

② add_subplot 函数，用于给已有全局绘图区添加子绘图区，add_subplot 函数的使用形式为：

```
add_subplot(nrows, ncols, index, **kwargs)
```

说明：参数 nrows 和 ncols 用于指定子绘图区的行数与列数，仅显示 index 参数值对应编号的子绘图区。

例如：创建一个 2 行 2 列的全局绘图区，在第 1 行第 1 列显示一条红色折线，在第 2 行第 2 列显示随机生成的 10 个绿色圆点，参考代码如下：

```
import matplotlib.pyplot as plt
import numpy as np
fig=plt.figure(figsize=(6,4))                #创建一个指定大小的全局绘图区
fig.add_subplot(221)                         #添加第 1 个子绘图区
plt.plot((1,6,2),c='r',marker='o')           #在第 1 个子绘图区绘制红色折线与圆点标记
fig.add_subplot(224)                         #添加第 4 个子绘图区
x=np.random.randint(0,100,10)                #随机生成 10 个坐标点的 x 坐标序列
y=np.random.normal(0,100,10)                 #随机生成 y 坐标序列
plt.scatter(x,y,c='g')                       #在第 4 个子绘图区绘制随机生成的坐标点
plt.show( )
```

上面程序执行效果如图 1.8.5 所示。

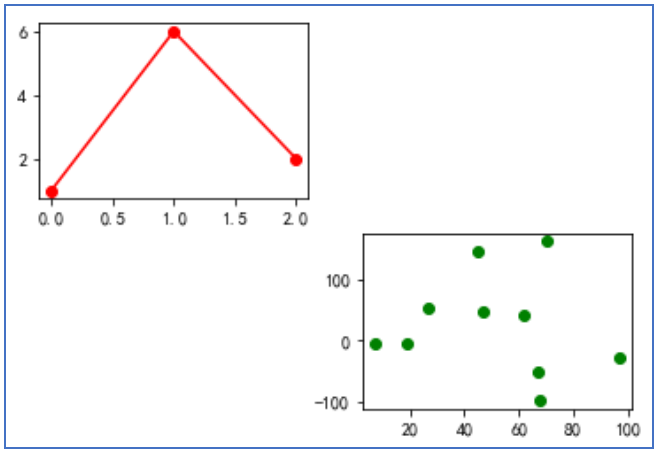


图 1.8.5　添加并显示在指定子绘图区

③ subplots 函数，用于创建一个全局绘图区，然后添加多个子绘图区并显示所有子绘图区，subplots 函数的使用形式为：

subplots(nrows, ncols, **fig_kw)

例如：

```
import matplotlib.pyplot as plt
fig,ax = plt.subplots(nrows=2,ncols=3,figsize=(8,4),dpi=100)
    #创建 2 行 3 列共 6 个子绘图区，全局绘图区大小为 8*4 英寸，分辨率为 100
plt.subplots_adjust(wspace=0.3,hspace=0.3)      #调整子绘图区之间的间隔
ax[0][2].plot((1),'rD')          #在第 0 行第 2 列的子绘图区绘制一个红色菱形标记
ax[1][0].plot((1,2),'b')         #在第 1 行第 0 列的子绘图区绘制一条蓝色线条
plt.show( )
```

上面程序执行后显示的子绘图区效果如图 1.8.6 所示。

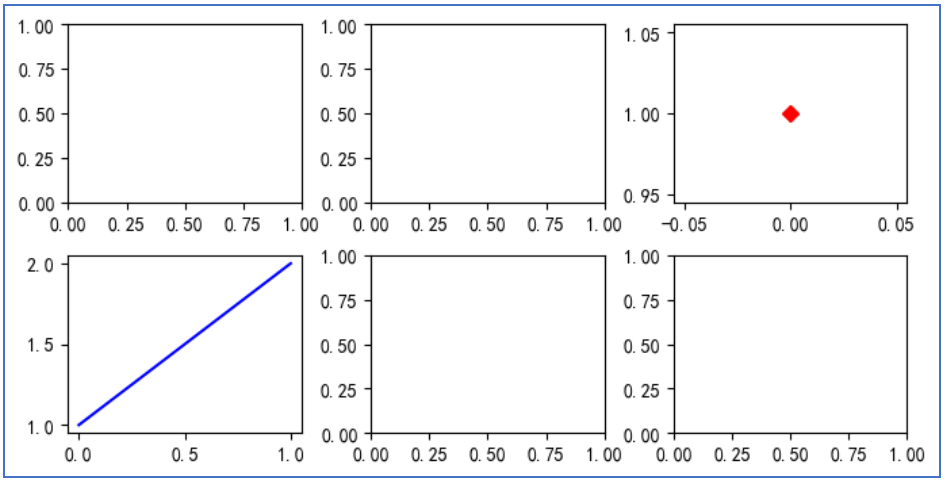


图 1.8.6　显示所有子绘图区

注意：

- 当子绘图区的行数与列数值均大于 1 时，用 ax[i][j]可以访问第 i+1 行第 j+1 列对应的坐标系，如 ax[0][0]表示的是第 1 行第 1 列的坐标系。

如果将上面代码中的 ax[0][2]与 ax[1][0]都替换成 plt，那么红色菱形标记和蓝色线条都绘制到了最后一个子绘图区。

- 当子绘图区的行数或列数只有一个值等于 1 时，可以通过 ax[i]来访问第 i+1 个坐标系。

例如：

```
import matplotlib.pyplot as plt
fig,ax=plt.subplots(nrows=3,ncols=1,figsize=(4,3))    #创建 3 行 1 列共 3 个子绘图区
ax[1].plot((1,2,3))        #绘制在第 2 行对应的子绘图区
fig,ax=plt.subplots(nrows=1,ncols=4,figsize=(6,4))    #创建 1 行 4 列共 4 个子绘图区
ax[2].plot((5,3))          #绘制在第 3 列对应的子绘图区
```

如果改用 plt.plot((1,2,3))和 plt.plot((5,3))来绘制前面的线条，那么线条都绘制到了当前绘图区的最后一个子绘图区。

- 当子绘图区的行数与列数值均等于 1 时，绘图区只有一个坐标系，既可以通过 plt 来访问该坐标系，也可以通过 ax 来访问。

例如：

```
fig,ax=plt.subplots(nrows=1,ncols=1,figsize=(3,2))    #创建 1 行 1 列共 1 个子绘图区
ax.scatter([5,3,8],[1,3,5],marker="D")                #绘制在当前坐标系
```

#上一行代码可以改写成 plt.scatter([5,3,8],[1,3,5],marker="D")，输出结果完全相同。

下面代码通过 ax 来访问图表的各个元素并绘制一个柱状图。

```
import matplotlib.pyplot as plt
fig,ax=plt.subplots(figsize=(6,4))                  #ax 用于访问指定坐标系
plt.rcParams['font.sans-serif']=['SimHei']          #SimHei 指定中文字体为黑体
subjects=['语文', '数学', '英语','体育']              #设置分类轴显示的文本内容
score= [80,98, 92,70]                               #某同学的分数
ax.set_title("考试成绩")                            #设置子绘图区的标题
ax.set_xlabel("科目")                               #设置子绘图区 X 分类轴的标签
ax.set_ylabel("分数")                               #设置子绘图区 Y 数值轴的标签
ax.set_xticks([0,1,2,3])                            #设置 X 分类轴的标记数据
ax.set_xticklabels(subjects)                        #设置 X 分类轴标记显示文本内容
ax.set_yticks([0,30,60,90,120])                     #设置 Y 数值轴的标记数据
ax.bar([0,1,2,3],score,label="Name")                #绘制柱状图
ax.legend(loc='upper right')                        #显示图例
```

8.1.3 图表绘制函数

pyplot 子库中提供了许多绘制图表的函数，可以绘制常见的各种图表，当绘图区只有一个坐标系时，一般直接调用 plt 的图表绘制函数，默认绘制在当前坐标系，常见图表绘制函数见表 1.8.1。

表 1.8.1 常见图表绘制函数

函　数	描　述
plot([x],y, [fmt],**kwargs)	绘制线条或标记
bar(x, height, width=0.8,**kwargs)	绘制柱状图
pie(x, explode=None,**kwargs)	绘制饼图
scatter(x, y, s=None, c=None, marker=None, cmap=None,**kwargs)	绘制散点图
hist(x,bins=None,range=None, ,rwidth=None, **kwargs)	绘制直方图

图表由多个元素组成，包括坐标轴、图表与坐标轴的标题、坐标轴的刻度和标签、图例

和数据标签等。每个图表元素的输出都可以调用相应的函数或方法来实现，常见图表设置函数见表 1.8.2 所示。

表 1.8.2 图表设置函数

函　数	描　述
plt.axis( )	显示当前坐标轴的范围，默认输出为：(0.0, 1.0, 0.0, 1.0)
plt.axes( )	显示坐标系风格的绘图区
plt.xlim(left,right) plt.ylim(bottom,top)	设置坐标轴的取值范围 如 plt.xlim(left=1,right=5)　　#设置 X 轴的取值范围为[1,5] 如 plt.ylim(bottom=3,top=6)　　#设置 Y 轴的取值范围为[3,6]
plt.text(x,y,s)	在指定位置添加注释文本 如 plt.text(0.5,0.5,'Max')　　#在两个坐标轴的中间输出字符串“Max”
plt.grid( )	设置绘图区的网格显示 如 plt.grid(True)　　#显示两个轴向的网格线 如 plt.grid(axis='both')　　#显示两个轴向的网格线 如 plt.grid(axis='x')　　#只显示 x 轴的纵向网格线
plt.title(label)	在指定位置显示图表标题 如 plt.title("Title",loc='center')　　#居中显示标题文本
plt.xlabel(xlabel) plt.ylabel(ylabel)	设置 X 轴的标题，如 plt.xlabel(“分类轴”) 设置 Y 轴的标题，如 plt.ylabel(“数值轴”)
plt.xticks(ticks,labels) plt.yticks(ticks,labels)	设置分类轴和数值轴的刻度及其标签文本 如 plt.xticks(range(4), ('一季度', '二季度', '三季度', '四季度')) 如 plt.yticks(range(3), ('10','20','30'))
plt.legend(loc)	设置图例显示 如 plt.legend(loc='upper right')　　#图例显示在右上角 如 plt.legend(bbox_to_anchor=(0.5,1))　#通过 xy 轴的相对位置定位
plt.fill(x,y,color)	填充由坐标点构成的连续封闭区域或函数曲线与坐标轴之间的区域 如 plt.fill([3,2,4,3],[5,1,1,5],color='b')　　#显示一个用蓝色填充的三角形

补充：rcParams，又叫 rc 参数，用于设置图表各个属性的默认值，如绘图区的大小、图像分辨率、线条样式、坐标轴、字体等。由于 Matplotlib 库默认设置的是英文字体，不支持中文字符的显示，所以需要手动设置字体属性或修改 rcParams 中的字体属性，使汉字能够正常显示。

- 设置中文字体，正常显示汉字

方法 1：通过 rcParams 设置指定中文字体。

```
plt.rcParams['font.sans-serif']=['SimHei']　　# SimHei 指定中文字体为黑体
```

其他常见中文字体有：SimSun（宋体）、KaiTi（楷体）、Microsoft YaHei（微软雅黑体）等。

方法 2：在需要输出汉字的函数参数中增加 fontproperties 属性，但是只能显示当前函数调用中需要显示的汉字。

```
plt.xlabel(‘X 轴标签’,fontproperties='Microsoft YaHei',fontsize=12)
```

- 设置坐标轴参数，正常显示负号

```
plt.rcParams['axes.unicode_minus']=False
```

- 设置绘图区大小、图像分辨率、线条样式、线条宽度等

```
plt.rcParams['figure.figsize'] = (8,6)        #设置绘图区大小为 8*6 英寸
plt.rcParams['figure.dpi'] =72                #设置图像的分辨率为 72dpi
plt.rcParams['lines.linestyle'] = '--'        #设置线条样式为长虚线
plt.rcParams['lines.linewidth'] = 5           #设置线条宽度为 5
```

8.2 二维图表

二维图表中只有 x 和 y 两个坐标轴，绘制的图形是一种平面图形。

8.2.1 绘制线条

plot 函数用于绘制线条和标记，不仅可以绘制折线图、曲线图，还可以制作出类似散点图的效果。plot 函数的使用形式为：

```
plt.plot([x],y, [fmt],**kwargs)
```

说明：

① x 和 y 通常是列表或元组等序列。x 中存储所有顶点的 x 坐标序列，y 中存储所有顶点的 y 坐标序列。例如，图 1.8.7 所示的折线图由 4 个顶点组成，坐标分别为（1，2）、（3，4）、（2，6）和（2，1），所以参数 x 的值应为[1, 3, 2, 2]，参数 y 的值应为[2, 4, 6, 1]，plot 函数的具体调用形式为：

```
plt.plot([1,3,2,2],[2,4,6,1])
```

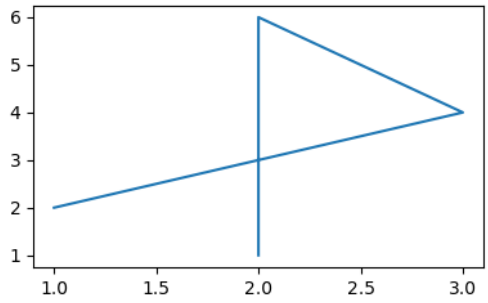


图 1.8.7　plot 函数绘制的折线图

② 参数 x 可以缺省，缺省值为列表[0, 1, 2, …, n-1]，此处 n 表示顶点数量。例如，函数 plt.plot([0, 1, 2, 3], [2, 5, 3, 6])可以简写为 plt.plot([2, 5, 3, 6])。

③ 参数 fmt 是一个字符串变量，用于定义图表的基本属性，如颜色（color）、标记（marker）、线型（linestyle）等。参数 fmt 是由各个属性的单个字母或符号缩写组合而成的字符串，组成形式如下（其排列顺序可以改变）：

```
'[颜色][标记][线型]'
```

例如，'g'表示绿色，'Dr'表示菱形标记红色，'bo-'表示蓝色圆点实线。用于表示颜色、线型、标记的单个字母或符号见表 1.8.3、表 1.8.4、表 1.8.5。

注意：颜色也可以赋值为十六进制格式的数值，如 color='#FF0000'等价于 color='r'。

表 1.8.3　表示颜色的字母缩写

缩写	**颜色**
'r'	red
'g'	green
'b'	blue
'c'	cyan
'm'	magenta
'y'	yellow
'w'	white
'k'	black

表 1.8.4　表示线型的字母缩写

线型缩写字母	说明	线型
'-'	1 个减号	实线
'--'	2 个减号	长虚线
': '	冒号	短虚线
'None'		不画线
'-.'	减号加点	点划线

表 1.8.5　marker 标记

标记	描述	标记	描述	标记	描述	标记	描述
‘o’	●（圆圈）	‘.’	●（圆点）	‘_’	–水平线	‘v’	▼（下三角形）
‘D’	◆（菱形）	‘s’	■（正方形）	‘8’	●八边形	‘<’	◀（左三角形）
‘h’	⬢（六边形）	‘*’	★（五角星）	‘p’	⬟五边形	‘>’	▶（右三角形）
‘H’	⬣（六边形）	‘d’	◆（菱形）	‘,’	· 像素点	‘^’	▲（上三角形）
‘+’	＋（加号）	‘X’	✖（粗线叉）	‘x’	×（细线叉）	‘| ‘	|（垂直线）
‘1 ‘	⋎（下三叉）	‘2 ‘	⋏（上三叉）	‘3 ‘	≺（左三叉）	‘4 ‘	≻（右三叉）

④ kwargs 是一个关键字参数（dict 类型），用于接收 0 个或多个由属性与属性值组成的键值对。若颜色属性赋值为“green”、“black”等单词形式，则不能用 fmt 参数来组合赋值，必须对单个颜色属性赋值。

例如：绘制一条从(3,1)到(6,5)的粗细为 2 的绿色短虚线且两端显示菱形标记的命令为：

```
plt.plot([3,6],[1,5], color='green', marker='D', linestyle =':',linewidth=2)
```

可将颜色、线型和线宽的属性名缩写为：

```
plt.plot([3,6],[1,5], c='green', marker='D', ls =':',lw=2)
```

注意：颜色、标记和线型可以采用组合字符串"gD:"，但不能写成"greenD:"。

【例 8.2】绘制三角函数 y=sin(x)和 y=cos(x)的曲线图。

方法 1：调用 math 库中的三角函数，利用 plot 函数输出标记来绘制曲线，效果如图 1.8.8（a）所示。

参考代码如下：

```
import math                                    #导入数学函数库
import matplotlib.pyplot as plt                #导入绘图库
plt.title("三角函数曲线")                       #显示图表标题
plt.ylim(-1,2)                                 #设置 y 轴的取值范围
plt.rcParams['font.sans-serif']=['SimHei']     #设置中文字体
plt.rcParams['axes.unicode_minus']=False       #显示负号
```

```
x=-math.pi
while x<=math.pi:
    sin=math.sin(x)              #计算当前坐标 x 对应的 sin 函数值
    cos=math.cos(x)              #计算当前坐标 x 对应的 cos 函数值
    plt.plot(x,sin,marker='*')   #在 sin 函数的每个坐标点输出一个星形标记
    plt.plot(x,cos,marker='D')   #在 cos 函数的每个坐标点输出一个菱形标记
    x+=0.1
plt.show()
```

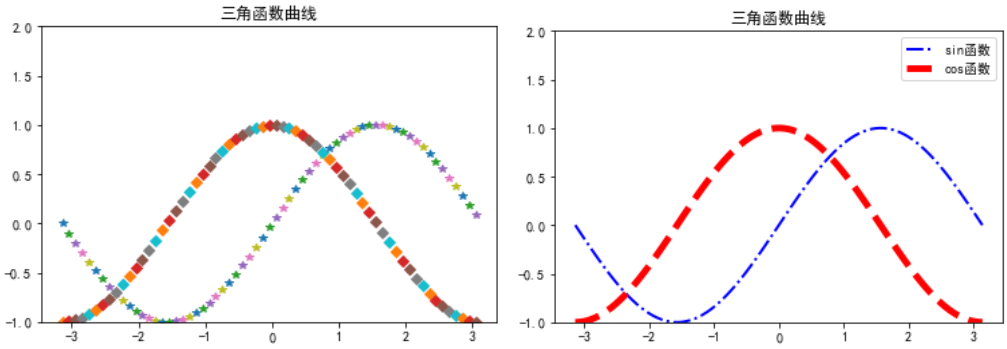
注意：上面代码中绘制图表的 plot 函数放在循环内可以正常输出图表，但是当循环次数越来越多时，图表绘制速度会越来越慢，甚至无法正常输出。建议利用 python 的列表将绘制的数据计算好并保存，然后再绘制图表，优化后的程序代码如下：

```
import math                                  #导入数学函数库
import matplotlib.pyplot as plt              #导入绘图库
plt.title("三角函数曲线")                    #显示图表标题
plt.ylim(-1,2)                               #设置 y 轴的取值范围
plt.rcParams['font.sans-serif']=['SimHei']   #设置中文字体
plt.rcParams['axes.unicode_minus']=False     #显示负号
x=-math.pi;   xx=[]
sinx=[];   cosx=[]
while x<=math.pi:
    sinx.append(math.sin(x))         #sinx 列表添加当前坐标点的 sin 函数值
    cosx.append(math.cos(x))         #cosx 列表添加当前坐标点的 cos 函数值
    xx.append(x)                     #xx 列表添加当前坐标点的 x 坐标值
    x+=0.1
plt.plot(xx,sinx,marker='*')         #绘制 sin 函数的曲线
plt.plot(xx,cosx,marker='D')         #绘制 cos 函数的曲线
plt.show()
```

方法 2：引入 numpy 库，利用其数学函数快速生成 x 轴上的数据系列，并计算对应的函数值，无需通过循环即可生成每个坐标点的数值序列，结合 numpy 库来绘制图表将更加简单方便，效果如图 1.8.8（b）所示。

参考代码如下：

```
import numpy as np                  #导入 numpy 库
import matplotlib.pyplot as plt     #导入绘图库
x=np.linspace(-np.pi,np.pi,100)     #快速生成 x 轴上（-π,π）之间的 100 个数
sin,cos = np.sin(x), np.cos(x)      #计算相应的 sin 函数值和 cos 函数值
plt.title("三角函数曲线")           #显示图表标题
plt.ylim(-1,2)                      #设置 y 轴的取值范围
plt.rcParams['font.sans-serif']=['SimHei']                          #设置中文字体
plt.rcParams['axes.unicode_minus']=False                            #显示负号
plt.plot(x,sin,color='b',linewidth=2, ls='-.',label='sin 函数')     #绘制 sin 函数曲线
plt.plot(x,cos,color='r',linewidth=5, ls='--',label='cos 函数')     #绘制 cos 函数曲线
plt.legend(loc=‘upper right’)                                       #右上显示图例文字
plt.show( )
```



（a） 绘制标记　　（b）绘制线条

图 1.8.8　三角函数的曲线图

【例 8.3】 绘制中国 1979 年至 2018 年的国民生产总值（GDP）的变化趋势图，统计数据见表 1.8.6，将所有数据存放到”gdp.txt”数据文件中，绘制的图表效果如图 1.8.9 所示。

分析：折线图适合显示随时间变化的连续数据，类别沿水平轴均匀分布，垂直轴方向显示相应的数据值，一般调用 plot 函数绘制折线图，也可以调用 scatter 函数绘制散点图，当点比较密集时，就可以达到折线图的效果。

表 1.8.6　1979~2018 年中华人民共和国国内生产总值（GDP）统计数据

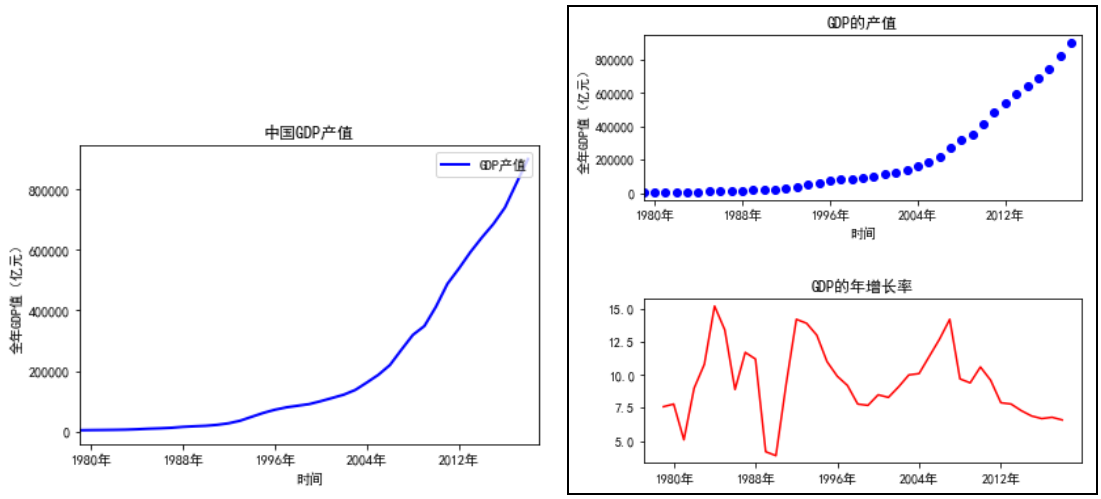
时间	全年（亿元）	时间	全年（亿元）	时间	全年（亿元）	时间	全年（亿元）
2018 年	900309	2008 年	319245	1998 年	85196	1988 年	15180
2017 年	820754	2007 年	270092	1997 年	79715	1987 年	12175
2016 年	740061	2006 年	219439	1996 年	71814	1986 年	10376
2015 年	685993	2005 年	187319	1995 年	61340	1985 年	9099
2014 年	641281	2004 年	161840	1994 年	48638	1984 年	7279
2013 年	592963	2003 年	137422	1993 年	35673	1983 年	6021
2012 年	538580	2002 年	121717	1992 年	27195	1982 年	5373
2011 年	487940	2001 年	110863	1991 年	22006	1981 年	4936
2010 年	412119	2000 年	100280	1990 年	18873	1980 年	4588
2009 年	348518	1999 年	90564	1989 年	17180	1979 年	4101

方法 1：读取文本文件“gdp.txt”中的数据（数据中无标题行）并绘制图表，效果如图 1.8.9（a）所示。

参考代码如下：

```
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif']=['SimHei']          #设置中文字体
fp=open("gdp.txt","r")
dt=fp.readlines()                    #从文本文件中读取全部 GDP 数据到列表中
n=len(dt)                            #获得列表中记录个数
gdpYear=[' '] * n                    #初始化年份列表
gdpValue=[0] * n                     #初始化产值列表
for i in range(0,n):
    dt[i]=dt[i].rstrip('\n')         #去除每一行字符串末尾的换行符
    dt[i]=dt[i].split('\t')          #按制表符将年份和产值分隔开
    gdpYear[i] =dt[i][0]             #将年份列表取出
    gdpValue[i]=int(dt[i][1])        #将 GDP 产值列表取出并转换成整型
```

```
plt.title("中国 GDP 产值")                    #设置图表的标题
plt.xlabel('时间')                            #设置 X 轴的标题
plt.ylabel("全年 GDP 值（亿元）")             #设置 Y 轴的标题
plt.xlim(0,40)                                #设置 X 轴的取值范围
plt.xticks(range(1,40,8))                     #设置 X 轴的刻度
plt.plot(gdpYear,gdpValue,color='b',linewidth=2,label="GDP 产值")
plt.legend(loc='upper right')                 #显示图例
plt.show( )
```



（a）绘制折线　　(b) 绘制散点图和折线图

图 1.8.9　中国 GDP 产值和增长率图

方法 2：引入 pandas 库，读取电子表格文件“gdp-1.xlsx”中的数据，该文件增加了 GPD 的年增长率数据，用 2 个子图分别绘制 GDP 产值的散点图以及年增长率的折线图，效果如图 1.8.9（b）所示。

注意：电子表格中的第一行为数据的标题行，分别为年份、产值和增长率，利用 pandas 库提供的 DataFrame 结构可以很方便地读取到表格中每一行和每一列的数据。本例中首先用 import pandas as pd 导入 pandas 库，然后用 dt=pd.read_excel("gdp-1.xlsx")来读取电子表格文件中的数据，最后通过 dt['年份'],dt['产值'], dt['增长率']分别获取表格中的三列数据。

参考代码如下：

```
import matplotlib.pyplot as plt
import pandas as pd                           #导入 pandas 库，后面用 pd 别名来引用
plt.rcParams['font.sans-serif']=['SimHei']    #设置中文字体
dt=pd.read_excel("gdp-1.xlsx")                #从电子表格文件中读取 GDP 数据
fig,axes = plt.subplots(nrows=2,ncols=1,figsize=(6,6))
plt.subplots_adjust(hspace=0.6)                   #调节子绘图区之间的垂直间距
plt.subplot(211)                                  #绘制在第 1 个子绘图区
plt.title("GDP 的产值")                           #设置子图表 1 的标题
plt.xlabel('时间')                                #设置子图表 1 的 X 轴标题
plt.ylabel("全年 GDP 值（亿元）")                 #设置子图表 1 的 Y 轴标题
plt.xlim(0,40)                                    #设置子图表 1 的 X 轴取值范围
plt.xticks(range(1,40,8))                         #设置子图表 1 的 X 轴的刻度
plt.scatter(dt['年份'],dt['产值'],color='b')      #绘制 GDP 产值的散点图
plt.subplot(212)                                  #绘制在第 2 个子绘图区
```

```
plt.title("GDP 的年增长率")                    #设置子图表 2 的标题
plt.xticks(range(1,40,8))                      #设置子图表 2 的 X 轴的刻度
plt.plot(dt['年份'],dt['增长率'],color='r')     #绘制 GDP 增长率的折线图
plt.show( )
```

8.2.2 绘制饼图

pie 函数用于绘制饼图，饼图适合显示一个数据系列中各项数值与其总和的占比关系，pie 函数的使用形式为：

```
plt.pie(x,explode=None)
```

说明：

① x 是一个列表或元组序列，x 中的每一个数值与其总和的比例等于饼图中对应扇形块所占的比例。

② explode 也是一个列表或元组序列，其长度与参数 x 一致，其中每个数值用于设置对应扇形块从整个饼图中分裂出来的偏移量。

【例 8.4】绘制一个饼图，显示各等级成绩的占比情况，效果如图 1.8.10 所示。

分析：首先需要准备好各等级成绩所占百分比的数据，然后设置图表中各个组成元素的值，比如图表标题，每一块的颜色，每一块分裂出来的偏移量列表，图例的标签文本及图例显示位置，数据标记值及其显示格式等。

参考代码如下：

```
import matplotlib.pyplot as plt
score = [0.16,0.28,0.32,0.14,0.1]                  #各等级的成绩数据
plt.rcParams['font.sans-serif']=['SimHei']         #设置中文字体
plt.title("成绩分布图")                             #图表标题
labels = ['优秀', '良好', '中', '及格','不及格']     #数据标记
explode = (0.2, 0, 0, 0, 0)                        #第 1 块分裂出来
plt.axis("equal")
plt.pie(score, explode=explode, autopct='%4.1f%%', labels=labels)
        # autopct='%4.1f%%'控制数据标记的格式化显示，显示 1 位小数
plt.legend(bbox_to_anchor=(1, 0.6))                #显示图例
plt.show( )
```

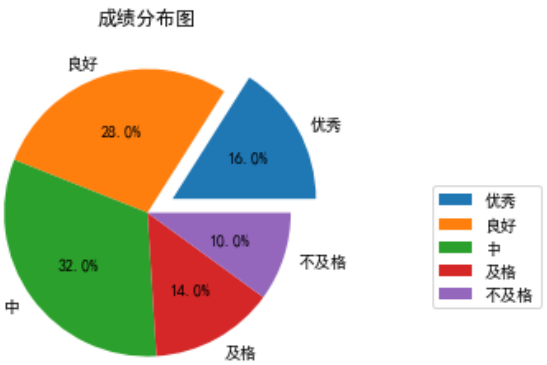


图 1.8.10　饼图

8.2.3 绘制散点图

scatter 函数用于绘制散点图，散点图适合展现两个变量之间的关系，用两组数据构成多个坐标点，通过坐标点的分布来判断两个变量之间是否存在某种关联，或分析坐标点的分布情况。scatter 函数的使用形式为：

```
plt.scatter(x, y, s=None, c=None, marker=None, cmap=None,**kwargs)
```

说明：

① x 和 y 通常是列表或元组等序列。x 中存储所有顶点的 x 坐标序列，y 中存储所有顶点的 y 坐标序列。

② 参数 s 用于指定标记大小，默认为 36.0。如果指定 s=10，那么所有标记点的大小都是 10。如果要显示不同大小的标记，可以给 s 赋值一个列表或元组序列，其长度必须与标记点的数量一致。

例如：

```
import matplotlib.pyplot as plt
x=[1,5,9,3] ;  y=[6,13,20,32]                    #设置 4 个顶点的坐标序列
plt.scatter(x,y,s=(36,120,60,10))                #4 个标记的大小不同
```

③ 参数 c 用于标记每个顶点的颜色，如果 c='r'，表示所有标记的颜色都是红色。如果要显示不同颜色的标记，则给 c 赋值一个颜色字符串或者一个表示颜色的序列或元组，其中字符串的长度或序列的长度必须与标记点的数量一致。

例如：

```
import matplotlib.pyplot as plt
from numpy import random
x=random.randint(10, size=5)      #随机生成 5 个数值赋给 x 坐标序列
y=random.randint(20, size=5)      #随机生成 5 个数值赋给 y 坐标序列
plt.scatter(x,y,c='rgbgb')        #5 个标记的颜色分别为“红绿蓝绿蓝”
```

注意：参数 c 如果赋值一个表示颜色的字符串，每个字母必须对应表示的是一种颜色，c='rgbgb'可以改写成 c=['r','g','b','g','b']。如果同时有参数 cmap=”rainbow”（指定为彩虹色系），则颜色由 c 参数决定，cmap 不起作用。

参数 c 也可以赋值一个数值列表或元组，相同数值用相同的颜色显示，这时可以通过参数 cmap 来指定一种配色方案，如 plt.scatter(x,y,c=[0,0,2,0,1],cmap='ocean')。

在聚类信息可视化时可以将表示类别信息的标注值序列赋给参数 c，使同一个类别的顶点用相同的颜色来显示。

【例 8.5】绘制一个散点图，显示鸢尾花数据集中的数据分布情况。

分析：iris 数据集由 Fisher 于 1936 年收集整理，是常用的分类实验数据集，包含 3 类鸢尾花（setosa：山鸢尾花，versicolor：变色鸢尾花，virginica：维吉尼亚鸢尾花），共 150 个数据样本，每个类别有 50 条数据记录，每条记录包含 4 个属性，分别是 sepal length(cm)：花萼长度，sepal width (cm)：花萼宽度，petal length (cm)：花瓣长度，petal width (cm)：花瓣宽度。

```
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"]=["SimHei"]
iris=load_iris()                  #载入鸢尾花数据集
dt=iris.data                      #获得数据集中 150 条数据记录
```

```
tg=iris.target                    #tg 用于存放每个样本的类别数据
x=dt[:,0]                         #x 存放所有样本的花萼长度
y=dt[:,2]                         #y 存放所有样本的花瓣长度
label="山鸢尾花\n 变色鸢尾花\n 维吉尼亚鸢尾花"          #图例文本
plt.scatter(x,y,c=tg,marker="^",cmap= "viridis",label=label)      绘制散点图
plt.legend()
plt.show()
```

上面代码的运行效果如图 1.8.11 所示。

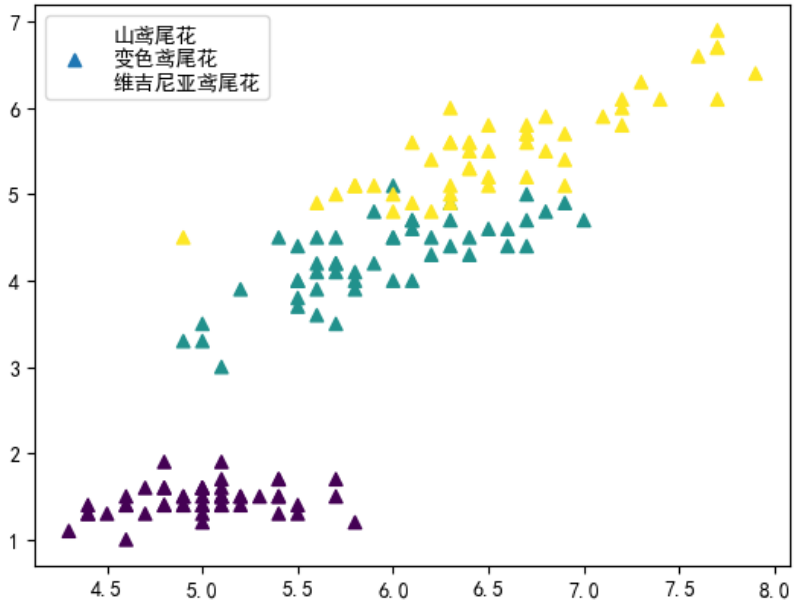


图 1.8.11　散点图

8.2.4 绘制直方图

hist 函数用于绘制直方图，直方图是一种统计报告图，用一系列高度不等的纵向条纹或线段表示数据的分布情况，一般用横轴表示数据类型，纵轴表示当前类别的统计结果。hist 函数的使用形式为：

```
plt.hist(x,bins=None,range=None, ,rwidth=None, **kwargs)
```

说明：

① x 是一个列表或元组序列，直接存放原始数据序列。

② bins 参数指定直方图条形的个数，对应原始数据的统计类别个数，默认值为 10，每个条形图的高度对应当前类别的统计结果。

③ range 参数指定直方图在 X 轴上数值的显示范围。

④ rwidth 参数指定每个条形的宽度。

例如，随机生成 50 个 100 以内的整数，显示 30 到 100 之间的数据分布情况。

```
import matplotlib.pyplot as plt
from numpy import random
data=random.randint(100,size=50)
plt.hist(x=data,bins=10,range=[30,100],rwidth=0.75)
```

程序的运行结果如图 1.8.12 所示。

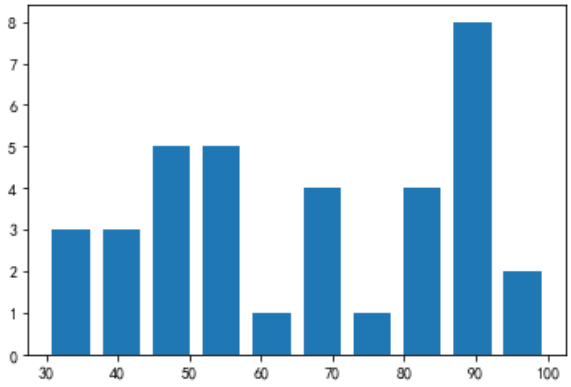


图 1.8.12　随机生成 50 个整数的直方图

【例 8.6】绘制指定图像文件的直方图，效果如图 1.8.13 所示。

分析：图像直方图的横轴表示灰度等级数（0 表示最黑，255 表示最白），纵轴的高度表示对应当前灰度等级像素点数目的多少，常用于分析图像的灰度分布是否合理，Photoshop 图像处理软件中提供了“色阶”调整，显示了图像直方图，对于缺乏层次感的图像（该亮的地方不亮，该暗的地方不暗），可直观地调整该直方图的暗部与亮部，达到增强图像对比度的目的。

要绘制指定图像文件的直方图，首先打开图像文件，读取到每一个像素点的数据，保存到一个三维数组 im 中，其中第 1 维和第 2 维定位像素点的坐标，第 3 维对应各个颜色通道的值，如 im[i,j,0]保存的是第 i 行第 j 列的红色通道值，im[i,j,1]保存的是第 i 行第 j 列的绿色通道值, im[i,j,2]保存的是第 i 行第 j 列的蓝色通道值。针对 RGB 彩色图像，可以采用下面的公式计算每个像素点的灰度值：

$$Gray= R*0.299 + G*0.587 + B*0.114$$

注意：灰度值的变化范围为[0,255]，总共有 256 个灰度等级。

参考代码如下：

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image                          #导入 Image 库
im=np.array(Image.open("Tulips.jpg"))
n=int(im.size/3 )
hd=n*[0]                                       #hd 列表用于存放每个像素点的灰度值
row=im.shape[0]
col=im.shape[1]
for i in range(row):
    for j in range(col):
        v=im[i,j,0]*0.299+im[i,j,1]*0.587+im[i,j,2]*0.114#计算当前像素点的灰度
        hd[i*col+j] =v                                   #保存每一个灰度值
plt.hist(hd,256,color='k')                               #绘制直方图
    # hd：各像素点的灰度值列表，256 是横轴灰度等级数，color 设置填充黑色
plt.show( )
```

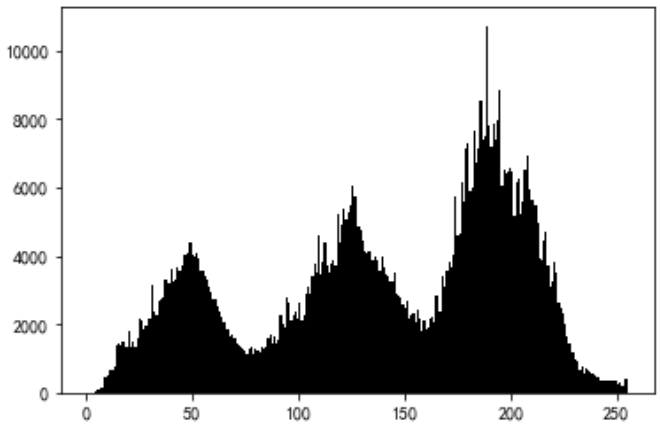


图 1.8.13　图像的直方图

8.3 三维图表

三维图表包含了 xyz 三个坐标轴，可以绘制三维立体图形，如三维线条、三维曲面、三维几何体等。

8.3.1 三维图表基础

1. 三维绘图子库

Matplotlib 的三维图表主要由 mplot3d 模块实现，由于绘制的三维图表是在二维画布上展示，所以同样需要导入 pyplot 子库。在 mpl_toolkits.mplot3d.axes3d 子库中包含了实现各种三维图表的类和方法，所以绘制三维图表前，一般用下面语句导入三维绘图子库。

```
from mpl_toolkits.mplot3d.axes3d import Axes3D
或 from mpl_toolkits.mplot3d import Axes3D
```

2. 三维绘图区

与二维图表绘制过程一样，首先调用 figure（）函数创建一个绘图区，然后调用 Axes3D() 函数创建三维坐标系；或者通过 gca 函数获取当前坐标系，并通过参数 projection='3d'指定绘制的是三维图表。

例如：

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()                                  #创建全局绘图区
ax = Axes3D(fig)                                    #创建三维坐标系
plt.show()
```

程序执行结果如图 1.8.14 所示。

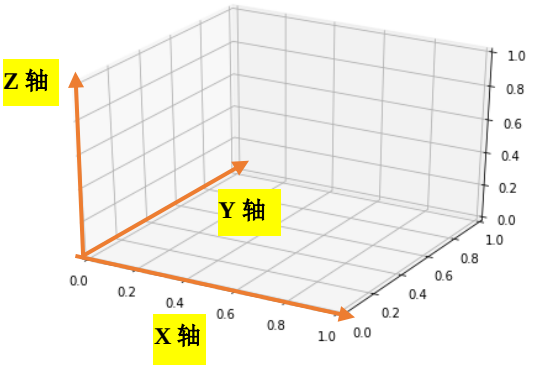


图 1.8.14　三维绘图区及其坐标系

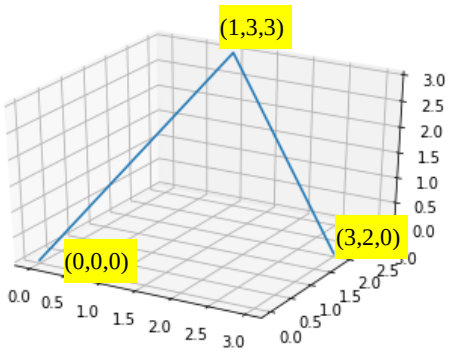


图 1.8.15　绘制三维折线

或者改用 gca 函数设置三维坐标系风格，下面程序代码执行效果如图 1.8.15 所示。

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()                          #创建全局绘图区
ax  = fig.gca(projection='3d')              #设置三维坐标轴
ax.plot((0,1,3),(0,3,2),(0,3,0))            #绘制一条三维折线
plt.show()
```

3. 三维子绘图区

三维绘图区中可以添加多个三维子绘图区，与二维子绘图区的添加方法基本一样，都是调用 add_subplot 函数，但是必须增加一个函数参数 projection='3d'。

例如：

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(12,5))            #创建全局绘图区
ax1 = fig.add_subplot(121,projection='3d')  #添加三维子绘图区 1
ax1.plot((0,3),(0,3),(0,3),ls='--',lw=5)    #绘制三维线条
ax2 = fig.add_subplot(122,projection='3d')  #添加三维子绘图区 2
ax2.plot((0,3,0),(3,3,0),(3,0,0),c='r',marker="D")  #绘制三维折线与标记
plt.show()
```

上面代码执行效果如图 1.8.16 所示。

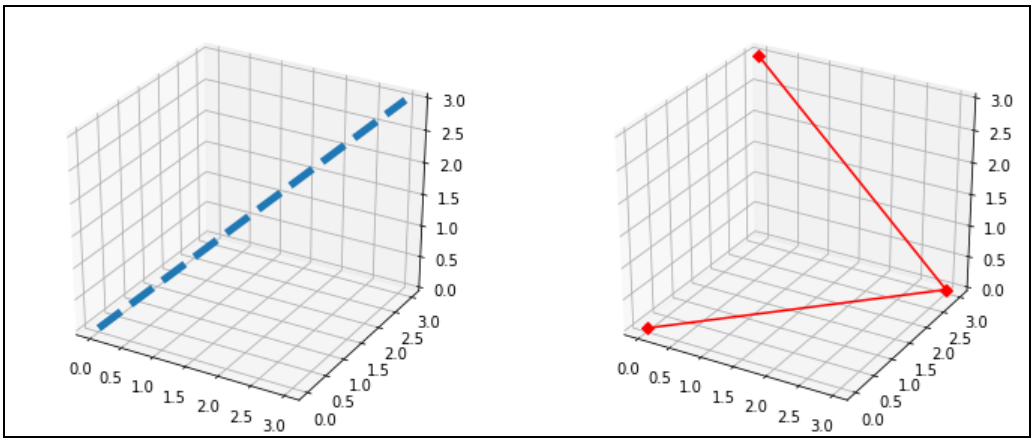


图 1.8.16　三维子绘图区

8.3.2 绘制三维图表

1. 绘制三维线条

三维线条的绘制方法与二维图表中的线条绘制方法基本一样，调用 plot 函数时需要给出 x、y、z 三个维度方向的坐标序列。

【例 8.7】绘制一个三维螺旋线，效果如图 1.8.17 所示。

参考代码如下：

```
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import matplotlib.pyplot as plt
fig=plt.figure()
ax=Axes3D(fig)
#下面生成螺旋线在 xyz 三个坐标维度上的坐标点序列
z=np.arange(-5*np.pi,5*np.pi,0.1)
x,y=np.sin(z), np.cos(z)
#下面设置三个轴向的刻度范围与刻度标记个数
ax.set_xticks(np.linspace(-1,1,4))
ax.set_yticks(np.linspace(-1,1,3))
ax.set_zticks(np.linspace(-20,20,5))
plt.plot(x,y,z,c='b')                    #绘制三维线条
plt.show( )
```

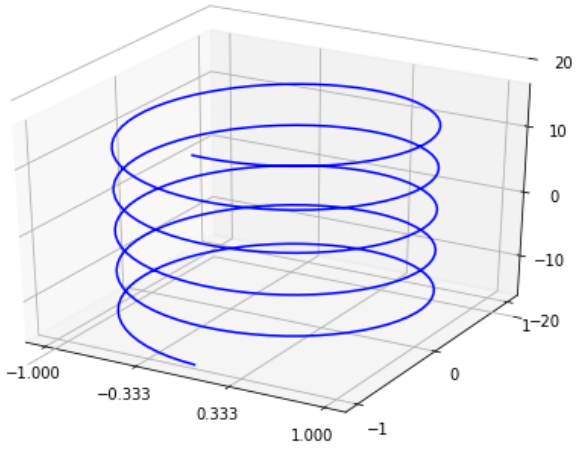


图 1.8.17　三维螺旋线

2. 绘制三维对象

Matplotlib 通过不同的填充方式来绘制不同的三维实体对象。如填充三角面、填充四边形或填充网格面等。

【例 8.8】 绘制一个四棱锥，效果如图 1.8.18 所示。

分析：首先给出四棱锥的 5 个顶点的坐标，然后调用函数 plot_trisurf（），用三角面进行填充即可绘制出一个四棱锥。

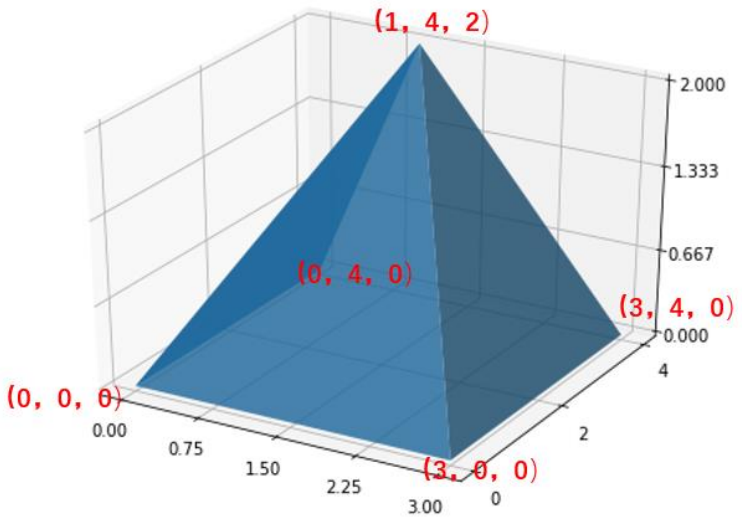


图 1.8.18　四棱锥

参考代码如下：

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
x= [0,3,3,0,1]                          #指定 XYZ 三个轴向 5 个坐标点的值
y= [0,0,4,4,4]
z= [0,0,0,0,2]
ax.set_xticks(np.linspace(0,3,5))       #设置三个轴向的刻度范围
ax.set_yticks(np.linspace(0,4,3))
ax.set_zticks(np.linspace(0,2,4))
ax.plot_trisurf(x, y, z,alpha=0.8)   #绘制 5 个顶点并以三角面填充
plt.show( )
```

【例 8.9】 绘制一个空心的立方体，由顶底前后左右六个面组成。

分析：首先需要创建该立方体各个顶点的坐标，然后再调用 plot_surface（）将 6 个面绘制出来，最终形成一个空心的立方体。

① 导入必要的库，调用 numpy 中的函数生成 xy 平面上的 25 个网格坐标点。

```
import matplotlib.pyplot as plt
import numpy as np
x=np.linspace(1,5,5)          #生成的 x 序列为：array([1., 2., 3., 4., 5.])
y=np.linspace(1,5,5)          #生成的 y 序列为：array([1., 2., 3., 4., 5.])
xx,yy=np.meshgrid(x,y)        #调用 meshgrid 生成 xy 平面上的 25 个网格坐标点
plt.plot(xx,yy,"bo")          #用蓝色圆点标注各个坐标点的位置
plt.show()
```

以上代码执行的结果如图 1.8.19 所示。

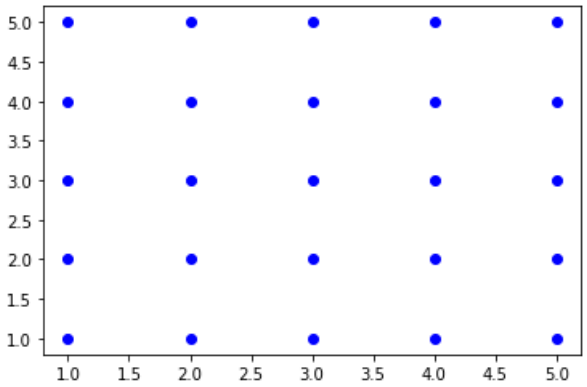


图 1.8.19　网格坐标点

② 创建一个三维坐标系的绘图区，设置各个坐标轴的刻度范围

```
from mpl_toolkits.mplot3d import Axes3D
fig=plt.figure()
ax=Axes3D(fig)
ax.set_xlim(1,5)                    #设置三个坐标轴向的刻度范围
ax.set_ylim(1,5)
ax.set_zlim(1,5)
```

③ 生成 xy 平面上 25 个网格点坐标对应的 z 坐标的值

```
z = np.zeros(25)+5          #z 的值是 array([5., 5.…… 5.])，共 25 个
zz= z.reshape(5, 5)         #将 z 修改为二维向量
```

④ 绘制立方体的 6 个面

```
ax.plot_surface(xx,   yy,   zz,   color='r', alpha=0.7)        #顶面
ax.plot_surface(xx,   yy   ,zz-4,color='g', alpha=1    )       #底面
ax.plot_surface(zz-4,xx,   yy,   color='b', alpha=1)           #后面
ax.plot_surface(zz,   xx,   yy,   color='c', alpha=0.7)        #前面
ax.plot_surface(yy,   zz,   xx,   color='m', alpha=1    )      #右面
ax.plot_surface(yy,   zz-4,xx,   color='y', alpha=0.7)         #左面
```

以上代码执行的结果如图 1.8.20 所示。

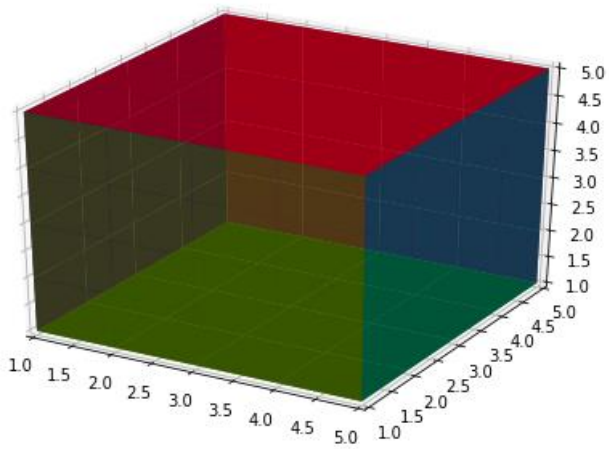


图 1.8.20 空心立方体

【例 8.10】绘制一个由网格面组成的三维几何体对象。

分析：首先生成 XY 平面上的网格点坐标，利用函数 z=sin($\sqrt{x^2+y^2}$)获得 Z 轴上对应

的坐标值，再调用 plot_surface（）绘制网格面，输出一个三维几何体。

参考代码如下：

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm                    #颜色风格控制库
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = Axes3D(fig)
ax.set_xlim(-5,5)
ax.set_ylim(-5,5)
ax.set_zlim(-1,1)
X,Y = np.arange(-5, 5, 0.1),np.arange(-5, 5, 0.1)
X,Y = np.meshgrid(X, Y)                      #形成 XY 平面上的网格点
Z = np.sin(np.sqrt(X**2 + Y**2))             #生成 Z 轴方向对象的坐标值
ax.plot_surface(X,Y,Z,rstride=3,cstride=5,cmap=cm.rainbow)
                    #rstride 与 cstride 控制每个网格面行列方向的跨度大小
plt.show( )
```

以上代码执行的结果如图 1.8.21 所示。

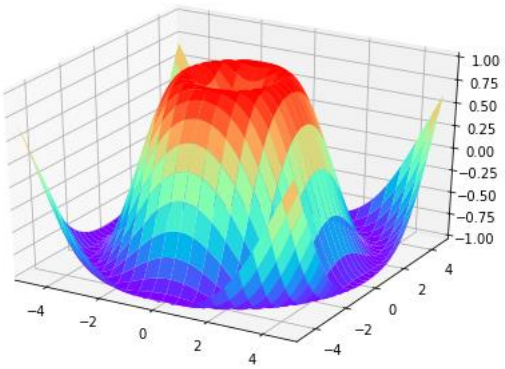


图 1.8.21　三维网格对象

8.4 综合应用

本章主要介绍了 Python 语言中 Matplotlib 绘图库的使用，包括图表的组成，绘图区的控制，二维图表和三维图表的绘制方法等，下面综合运用本章所学内容，制作所需图表。

【例 8.11】请分别用 plot 函数和 scatter 函数绘制如图 1.8.22 所示的图案，生成图案的参数方程为：

$$\begin{cases} r=\sin(a*\theta) \\ x=r*\cos(\theta) \\ y=r*\sin(\theta) \end{cases}$$

其中：θ 是[0，2π]之间的弧度值，a 可以是一个任意实数，a 的取值决定了图案的形状。

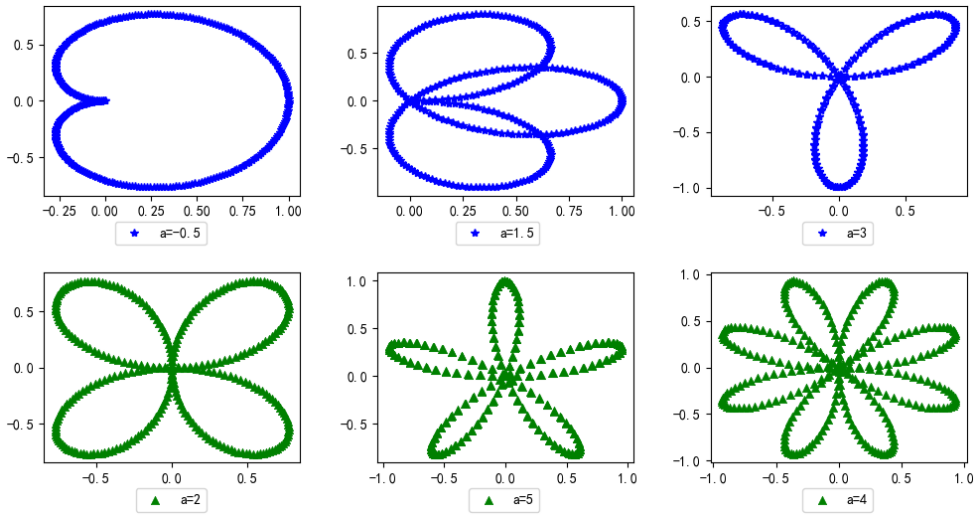


图 1.8.22　绘制图案

方法 1：用标准的数学库函数来实现，利用循环控制生成每一个坐标点的值。

参考代码如下：

```
import    math
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus']=False                    #正常显示负号
fig,ax = plt.subplots(nrows=2,ncols=3,figsize=(12,6))       #设置子绘图区
plt.subplots_adjust(wspace=0.3,hspace=0.4)                  #调整子绘图区的间隔
a= [-0.5,1.5,3]                        #设置 a 的取值序列，控制第 1 行的三个图案
for i in range(3):                     #控制第 1 行的三个子绘图区
    th=0;    xx=[];    yy=[]
    while th<=2 * math.pi:             #用循环控制生成所有坐标点的取值序列
        r=math.sin(a[i]*th)
        xx.append(r*math.cos(th))
        yy.append(r*math.sin(th))
        th+=0.03
    ax[0][i].plot(xx,yy,'b*',label="a=" + str(a[i]))          #用 plot 函数绘制标记
    ax[0][i].legend(bbox_to_anchor=( 0.7,-0.1))               #指定图例位置
a=[2,5,4]
for    i in range(3):                  #控制第 2 行的三个子绘图区
    th=0; xx=[]; yy=[]
    while th<=2* math.pi:
        r=math.sin(a[i]*th)
        xx.append(r*math.cos(th))
        yy.append(r*math.sin(th))
        th+=0.03
    ax[1][i].scatter(xx,yy,c="g",lw=0.5,marker="^",label="a=" + str(a[i]))
                                                          #用 scatter 函数绘制散点图
    ax[1][i].legend(bbox_to_anchor=( 0.7,-0.1))
```

方法 2：引入 numpy 库，快速生成数据序列并绘制图表。

参考代码如下：

```
import    numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus']=False          #正常显示负号
fig,ax = plt.subplots(nrows=2,ncols=3,figsize=(12,6))
plt.subplots_adjust(wspace=0.3,hspace=0.4)
a= [-0.5,1.5,3]
th=np.arange(0,2*np.pi,0.02)                      #快速生成弧度值序列
for i in range(3):
    r=np.sin(a[i]*th)
    x,y=r*np.cos(th),r*np.sin(th)                 #计算坐标点序列
    ax[0][i].plot(x,y,'b*',label="a=" + str(a[i]))    #绘制坐标点标记
    ax[0][i].legend(bbox_to_anchor=( 0.7,-0.1))   #通过图例显示当前 a 的取值
a=[2,5,4]
for   i in range(3):
    r=np.sin(a[i]*th)
    x,y=r*np.cos(th),r*np.sin(th)
    ax[1][i].scatter(x,y,c="g",lw=0.5,marker="^",label="a=" + str(a[i]))
    ax[1][i].legend(bbox_to_anchor=( 0.7,-0.1))
```

【例 8.12】绘制微信好友性别统计柱状图和好友签名的词云图。

分析：词云就是对网络文本中出现频率较高的关键词予以视觉上的突出，渲染出符合指定形状的关键词分布图像，从而过滤掉大量的文本信息，使用户可以快速领略文本的主旨大意，下面一起学习词云图的制作。

本例需要用到以下几个库：itchat（用于登录微信）、jieba（用于分词处理）、wordcloud（用于生成词云图）、PIL（用于图像处理）。

先登录自己的微信账号，获取所有朋友信息，提取需要的签名数据，然后利用结巴库进行分词处理，再用 wordcloud 生成词云图并输出图像。

制作过程：

① 导入必要的库

```
import itchat
import numpy as np
import pandas as pd
import jieba
import os
import matplotlib.pyplot as plt
from wordcloud import WordCloud
import PIL.Image as Image
```

② 登录微信，获取所有朋友的数据

```
itchat.login()              #弹出微信二维码图片，在手机上扫码并确认登录
friends = itchat.get_friends(update=True)          #获取好友信息
```

③ 获取自己的昵称，在当前目录下创建一个用自己昵称命名的文件夹，然后将其设置为当前文件夹

```
NickName = friends[0].NickName#获取自己的昵称
```

```
    cdr = os.getcwd()                       #获取当前目录
    if not(os.path.exists(NickName)):       #判别昵称文件夹是否已存在
    os.mkdir(NickName)                      #则创建一个名字为昵称的新文件夹
    os.chdir(cdr+'\\' + NickName)           #切换到昵称文件夹
```

④ 获取好友的性别信息，统计各性别人数，然后绘制各性别好友数量的柱状图

```
    df = pd.DataFrame(friends)              #把好友数据处理成 DataFrame
    Sex = df.Sex                            #获取好友性别信息
    Sex_count = Sex.value_counts()          #统计人数，男：1、女：2、未知：0
    plt.figure(figsize=(10,5))
    plt.rcParams['font.sans-serif'] = ['SimHei']
    plt.subplot(121)
    x=[0,1,2]
    plt.xticks(x,("男",'女','未知'))
    barW=0.5
    plt.bar(x,Sex_count,barW,color='b',label='性别')  #绘制柱状图
    plt.legend(loc='best')
```

⑤ 获取好友签名信息，并将其写入文本文件中保存

```
    Signatures = df.Signature               #获取好友签名信息
    text = ' '.join(Signatures)             #将好友签名数据保存到 text 变量中
#下面将所有好友的签名信息写入昵称文件夹下的 sig.txt 文件中
    file_name = NickName+'sig.txt'
    with open(file_name,'w',encoding='utf-8') as f:
        f.write(text)
    f.close()
```

⑥ 用结巴库对好友签名数据进行分词处理

```
    wordlist=jieba.cut(text,cut_all=True)   #对好友签名信息进行分词处理
    wordSS = ' '.join(wordlist)                 #将分词结果转换成字符串
```

⑦ 设置词云图的各项参数并生成词云图，保存为图片文件

```
    mask = np.array(Image.open('wx.png'))#定义词云图显示的遮罩形状
    wcd = WordCloud(
        font_path='C:/Windows/Fonts/simhei.ttf',
        background_color ='white',
        mask=mask,                          #设置遮罩图片
        max_words=1000,                     #最多显示词数
        max_font_size=100    )              #字体最大值
    myWD = wcd.generate(wordSS)             #根据签名数据生成词云图
    myWD.to_file(NickName+'.jpg')           #将词云图保存为图片文件
    plt.subplot(122)
    plt.imshow(myWD)                        #显示词云图
    plt.axis('off')                         #不显示坐标轴
    plt.show()                              #显示图像
```

以上代码执行的结果如图 1.8.23 所示。

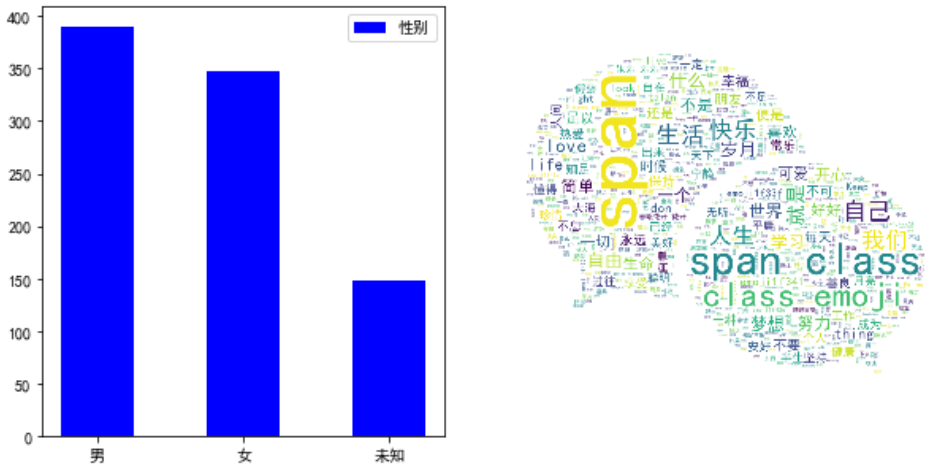


图 1.8.23　柱状图和词云图

习题

一、选择题

1.创建并显示 2 行 3 列共 6 个子绘图区，以下代码正确的是_____。

A. fig,axes = plt.subplot(nrows=2,ncols=3,figsize=(4,3),dpi=100)

B. fig,axes = plt.subplots(nrows=2,ncols=3)

C. fig,axes = plt.subplots(nrows=3,ncols=2)

D. fig,axes = plt.subplot(nrows=2,ncols=3)

2.语句 plt.plot([1,5],c='g')绘制的是_____。

A. 从点（0，0）到点（1，5）的一条绿色直线

B. 从点（0，0）到点（1，5）的一条绿色曲线

C. 从点（1，5）到点（0，0）的一条绿色直线

D. 从点（0，1）到点（1，5）的一条绿色直线

3.绘制一条从(1, 1)到(2, 3)最后到(3, 2)的折线，正确的语句是_____。

A. plt.plot((3,2),(2,3),(1,1))

B. plt.plot([1,3,2],[1,2,3])

C. plt.plot((1,1),(2,3),(3,2))

D. plt.plot([1,2,3], [1,3,2])

4.语句 plt.plot((3,5),(0,1),c='b')绘制的是[　　]。

A. 从点（3，5）到点（0，1）的一条蓝色直线

B. 从点（3，5）到点（0，1）的一条蓝色曲线

C. 从点（3，0）到点（5，1）的一条蓝色直线

D. 从点（0，3）到点（1，5）的一条蓝色直线

5.在 Matplotlib 中，以下_____用于绘制饼图。

A. plt.scatter()

B. plt.hist()

C. plt.pie()

D. plt.bar()

6.绘制一个散点图，只有(1, 1)、(2,3)和(3, 5)三个点，正确的语句是_____。

A. plt.scatter([1,2,3], [1,3,5],c="b",marker="o")

B. plt.scatter((1,1),(2,3),(3,5),c="b",marker="o")

C. plt.scatter([1,3,5],[1,2,3],c="b",marker="o")

D. plt.scatter((3,5),(2,3),(1,1),c="b",marker="o")

7.在 matplotlib 中，绘制由三角形构成曲面单元的三维图形时，应使用_____方法。

A. plot

B. scatter

C. plot_surface

D. plot_trisurf

8. 在 Matplotlib 中，以下_____用于绘制直方图。

A. plt.plot()

B. plt.hist()

C. plt.pie()

D. plt.polar()

二、填空题

1.用于创建一个坐标系风格的绘图区域并显示绘图区的函数是_____。

2.Matplotlib 的图表区包含三个层次：容器层，辅助显示层和_____。

3.pyplot 是 Matplotlib 的子库，能够绘制出各种常见的图表类型，包括_____和三维图表，还可同时绘制多个子图。

4.plot 函数用于绘制_____和_____，不仅可以绘制折线图、曲线图，还可以制作出类似散点图的效果。

5._____适合显示一个数据系列中各项数值与其总和的占比关系。

6.请补充下面程序中空白处的代码，绘制一个如图 1.8.24 所示的五角星。

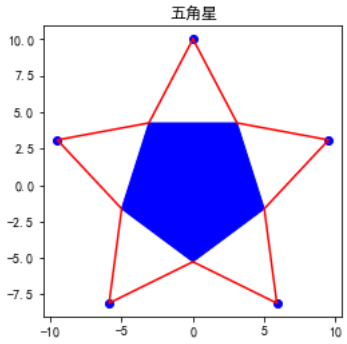


图 1.8.24 五角星

```
import matplotlib.pyplot as plt
import numpy as np
fig=plt.figure(figsize=(4,4),dpi=72)
plt.rcParams["font.sans-serif"]=["SimHei"]
plt.rcParams["axes.unicode_minus"]=False
rw=10                                  #假设外接圆的圆心坐标为（0，0），半径为 10
k=np.arange(5)                         #k 设为各个顶点的序号，array([0, 1, 2, 3, 4])
x1=rw*np.sin(2*np.pi/5*k)              #生成外接圆上五个顶点的 x 坐标序列
y1=rw*np.cos(2*np.pi/5*k)              #生成外接圆上五个顶点的 y 坐标序列
plt._____(1)_____                      #用蓝色圆点标记外接圆上的五个顶点
rn=rw*np.sin(np.pi/10)/np.sin(np.pi/5)       #计算内接圆的半径
x2=rn*np.sin(2*np.pi/5*k+np.pi/5)            #生成内接圆上五个顶点的 x 坐标序列
y2=rn*np.cos(2*np.pi/5*k+np.pi/5)            #生成内接圆上五个顶点的 y 坐标序列
plt.fill(_____(2)_____)                      #用蓝色填充内接五边形
x=[x1[0],x2[0],x1[1],x2[1],x1[2],x2[2],x1[3],x2[3],x1[4],x2[4],x1[0]]
y=[y1[0],y2[0],y1[1],y2[1],y1[2],y2[2],y1[3],y2[3],y1[4],y2[4],y1[0]]
plt. _____(3)_____                           #绘制红色的五角星
plt.title ("五角星")
plt.show( )
```

三、简答题

1.简述 Matplotlib 可以绘制的常见图表类型。

2.简述 Matplotlib 图表的主要组成部分。

3.如何在指定的子绘图区中绘制图表？有哪几种方法？

4.当子绘图区的个数大于 1 时，如何访问指定子绘图区的坐标系？
5.绘制三维图表时一般用哪些方法生成三维实体对象？
6. 直方图适合分析什么样的数据？图像直方图有什么作用？

实验一　Python 程序设计基础

一、实验目的

1．掌握 Python 开发环境的下载和安装；

2．通过简单的程序设计熟悉 IDLE 和 Anaconda 开发环境；

3．掌握简单 Python 程序设计的基本组成；

4．掌握 Python 输出/输出数据的基本方法。

二、实验内容

1．在自己的电脑上，安装 Python 开发环境。

① 安装 IDLE 的开发环境。说明：Python 官网网址为 https://www.python.org

② 安装 Aanaconda 的开发环境。说明：Anaconda 官网网址为 https://www.anaconda.com

2．下面程序的功能是从键盘输入一个整数，然后计算它的平方根并且输出，保留两位小数。请在 IDLE 中调试该程序。注：下列程序存在四个错误。

```
import "math"
x=input("请一个整数：")
y=sqrt(x)
print("平方根是：", "y")
```

3．编写一个程序，要求输入直角三角形的斜边和一条直角边，求另外一条直角边、周长和面积。

4．编写一个计算一个学生三门课平均成绩的程序。要求学生成绩从键盘输入。

5．编写一个华氏温度与摄氏温度的换算程序。要求输入摄氏温度，计算并且输出华氏温度。

```
华氏温度=9/5*摄氏温度+32
```

实验二　顺序结构

一、实验目的

1．掌握数值、字符串、布尔类型数据的表示和常规处理。

2．各类表达式的正确书写规则。

3．掌握常用函数的使用。

3．掌握 input 函数的使用。

4．掌握 print 函数和格式的使用。

二、实验内容

1．数学函数的使用。在 IDEL 交互方式运行下，导入数学库 math，输入对应变量的值，完成如下表达式的计算并显示计算结果。已知 x=5，y=6，z=7。

（1）$\dfrac{\sqrt{(3x+y)/z}}{(xy)^4}$

（2）$\sin 45^{\circ}+\dfrac{e^{10}+\ln 10}{\sqrt{x+y+1}}$

2．字符串函数使用。已知字符串 s=" Python Programming Language"，在 IDLE 交互方式运行下，利用字符串函数，完成如下功能处理：

（1）利用 lower()函数将 s 字符串全部转换成小写字母并显示。

（2）利用 upper()函数将 s 字符串全部转换成大写字母并显示。

（3）利用 count()函数统计 s 字符串中出现字母"n"的个数。

（4）利用 split()函数将 s 字符串按单词分离到列表中。

（5）利用 replace()函数将 s 字符串中出现的空格由","替换。

3．关系和逻辑表达式的书写。编程输入年份，判断该年是否是闰年并显示。

【提示】关键是书写正确的关系表达式。判断闰年应满足以下两个条件之一：

① 该年能被 4 整除但不能被 100 整除；

② 该年能被 400 整除。

4．算术运算符的使用。编程利用 input 函数输入一个 3 位正整数，然后逆序输出。例如，输入 734，输出是 437。

【提示】利用算术运算符进行取余和整除来实现。

5．字符串的切片和连接。编程输入 18 位身份证号码，取出身份证中的出生年月日，如图 2.2.1 所示显示。

【提示】

（1）利用对字符串的切片获得出生年月日；

（2）利用字符串的“+”将切片后的字串以出生年月日的方式显示。

```
请输入18位身份证号：  310110199310154011
     身份证号                出生年月日
310110199310154011      1993年10月15日
```

图 2.2.1 运行效果

。

实验三　选择结构

一、实验目的

1. 掌握逻辑表达式的正确书写形式。

2. 掌握单分支与双分支条件语句的使用。

3. 掌握多分支条件语句的使用。

4. 利用选择结构解决实际问题。

二、实验内容

1．求一元二次方程的根。

要求：输入一元二次方程的三个系数 a、b、c，根据系数值，可得出如下三种根：

（1）△＞0，两个实根

（2）△=0，重根，即相同得根

（3）△＜0，无实数根

其中：△为判别式，即 b^2-4ac

输出两个实根 x1 和 x2，若没有实根则输出信息：无实根。

2. 在购买某物品时，若所花的的钱 x 在下述范围内，所付钱 y 直接按对应折扣支付：

$$y=\begin{cases} x & x<1000 \\ 0.9x & 1000\le x<2000 \\ 0.8x & 2000\le x<3000 \\ 0.7x & x\ge 3000 \end{cases}$$

【提示】注意计算公式和条件表达式的正确书写。

3. 编一程序输入上网的时间计算上网费用，计算的方法如下：

$$\text{费用}=\begin{cases} 30\text{元基数}, & <10\text{小时} \\ \text{每小时}2.5\text{元}, & 10\sim 50\text{小时} \\ \text{每小时}2\text{元}, & \ge 50\text{小时} \end{cases}$$

同时为了鼓励多上网，每月收费最多不超过 150 元。

【提示】

（1）首先利用多分支 if 语句根据三个时间段分级计算费用，例如若 s 为使用了 80 小时，则计算费用公式 m=30+2.5*40+(s-50)*2；

（2）然后再用单分支 If 语句对 m 超过 150 元的费用限定为 150 元。

4.输入 x，y，z 三个数，按从小到大的次序显示，如图 2.3.1 所示：

```
67   <  98   <  345
>>>
```

图 2.3.1 运行效果

5. 利用计算机解决古代数学问题“鸡兔同笼问题”。即已知在同一笼子里有总数为 M 只鸡和兔，鸡和兔的总脚数为 N 只，求鸡和兔各有多少只？

【提示】鸡、兔的只数通过已知输入的 M，N 列出方程可解，设鸡为 x 只，兔为 y 只，则计算公式为：

x+y=M

2x+4y=N

即：x=M-y

y=N/2-M

但不要求出荒唐的解（例 3.5 只鸡、-4 只兔）。因此，对输入的数据要考虑下面两个条件：

① 对输入的总脚数 N 必须是偶数，否则提示数据错的原因；

② 若求出的头数为负数，提示数据错的原因。

6.学分绩点计算。为评定学生的学习质量，高校一般采用计算学分绩点的办法，表 2.3.1 是某学校成绩与绩点的折算对照表。编程输入某门课程的成绩，利用 if 多分枝结构显示其成绩和绩点。

表 2.3.1　　成绩与绩点折算对照表

成绩	<60	60~69	70~79	80~89	90~100
绩点	0	2	3	4	5

实验四　循环结构

一、实验目的

1. 掌握 for 语句的使用。

2. 掌握 while 语句的各种形式的使用。

3. 掌握如何控制循环条件，防止死循环或不循环。

4. 利用循环结构解决实际问题。

二、实验内容

1. 随机产生一个区间[5,10]内的整数 n，分别打印具有 n 行的有规律字符图，如图 2.4.1(a)和（b）所示。

【提示】

（1）随机数产生要导入 random 库，调用 randint(a,b)函数；

（2）利用序列 range()函数，获得字母编码值，调用 chr()函数将字母编码值转换成字母；

（3）对于图（b）产生奇数随机数，利用 n=random.randrange(5,10,2)实现；每行字符不同可增加一个计数器变量，每输出一行加 1，再转换成字母的编码值。



（a）　　（b）

图 2.4.1 运行效果

【提示】序列为 range()函数，则要利用 chr()函数将字母编码值转换成字母。

2. 检查表达式输入中圆括号配对问题。要求对文本框输入的算术表达式，检验其圆括号配对情况，并给出相应信息如图 2.4.2 所示。

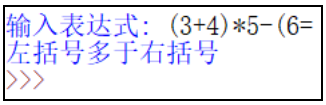


图 2.4.2 运行效果

【提示】利用 for 语句遍历整个表达式，对取出的每个字符判断，对于是左或右括号，

进行相应的计数，所有字符都遍历了，根据计数确定表达式配对情况。

3.输入一字符句子，各单词间空格分隔，显示最长的单词和长度。

【提示】利用字符串的 split 方法，将单词分离到列表中；然后对列表遍历利用字符串 len()函数求出最长单词。

4. 输入任意位的正整数，利用 while 语句将输入的数按逆序显示。即若输入 x=15356，则输出 nx=65351。

【提示】实现的方法是将一个十进制数 x 不断除以十取余、x 整除十，并将余数连接到逆序数中，直到 x 为 0。

5.计算 $S=1+\frac{1}{2}+\frac{1}{4}+\frac{1}{7}+\frac{1}{11}+\frac{1}{16}+\frac{1}{22}+\frac{1}{29}+\mathrm{K}$ 当第 i 项的值<10^{-4}时结束。

【提示】找出规律，第 i 项的分母是前一项的分母加上表示有分母项开始计数，即分母通项为：$t_i=t_{i-1}+i$。

6．计算π的近似值，π的计算公式为：

$$\pi=2\times\frac{2^2}{1\times3}\times\frac{4^2}{3\times5}\times\frac{6^2}{5\times7}\times\cdots\times\frac{(2\times n)^2}{(2n-1)\times(2n+1)}$$

求 n=1000 时的结果，并与数学库提供的常数 pi 进行验证，为提高精度，如何设置 n 的值？

7．编一程序，显示出所有的水仙花数。所谓水仙花数，是指一个 3 位数，其各位数字立方和等于该数字本身。例如，153 是水仙花数，因为$153=1^3+5^3+3^3$。

【提示】解该题的方法有两种：

① 利用三重循环，将三个一位数连接成一个 3 位数进行判断。

② ② 利用单循环将一个 3 位数逐位分离后进行判断。

8．求 $S_n=a+aa+aaa+aaa+\cdots aa\cdots aaa$(*n* 个 *a*)，其中 *a*（*1~9*）、 *n*（*5~10*）的数，通过键盘输入。

例如：当输入 *a*=2，*n*=7 时，

显示：S_n=2+22+222+2222+22222+222222+2222222

【提示】

①为了得到不断重复 a 的 n 位的数 Temp，可用如下程序段实现：

```
temp=0
 for i in range(1,n+1):
    temp=temp*10+a
```

②若不光要显示计算的结果，还要显示产生的表达式，即：

s=2+22+222+2222+22222+222222+2222222= 2469134

则如何实现？也就是在循环体内对字符串 ss 通过：ss=+str(temp)+"+"语句将表达式连接起来，出了循环可以在 print()函数中使用转义字符"\b"去除最后一个"+"。

9.利用枚举法安排期末考试。期末考试在周一到周六的 6 天时间内要考 x、y、z 三门课程，考试课程顺序先考 x，后考 y，最后考 z，规定一天只能考一门，且 z 课程最早安排在周五考。编写程序列出满足条件的所有考试安排方案。

【提示】利用枚举法通过三重循环来解决，关键是每重循环的初值和终值。

10.利用递推法求解猴子吃桃子问题。小猴在第一天摘了若干个桃子，当天吃掉一半多一个；第二天接着吃了剩下的桃子的一半多一个；以后每天都吃尚存桃子的一半多一个，到第 7 天早上要吃时只剩下一个了，问小猴最初共摘下了多少个桃子？

【提示】这是一个“递推”问题，先从最后一天推出倒数第二天的桃子，再从倒数第二天的桃子推出倒数第三天的桃子…

设第 n 天的桃子为 x_n，那么它是前一天的桃子数 x_{n-1} 的二分之一减去 1。

已知：当 n=7 第 7 天的桃子数为 1 ，则第 6 天的桃子由上面递推公式得 4 个，依次类推，可求得第 1 天摘的桃子数。 即：$x_n = \frac{1}{2}x_{n-1} - 1$ 也就是：$x_{n-1} = (x_n + 1) \times 2$

11．猜数游戏。计算机随机产生一个 1~100 之间的整数，由用户去猜，猜中即胜；猜不中，显示提示信息，继续猜，直至猜中，显示相关信息和次数，如图 2.4.3 所示。



图 2.4.3 运行界面

【提示】利用循环来解决，利用随机函数产生一个数，与用户输入的数进行比较，用户输入的数应利用二分法法则提高命中利率。若最多猜 5 次，又如何实现？

实验五　函数

一、实验目的

1．掌握用户函数的定义与调用

2．理解位置参数、默认值参数

3．掌握递归函数的定义和调用

二、实验内容

1．写一个函数，求斐波那契数列的前 n 个数据，要求：返回值是由斐波那契数列的前 n 个数据的列表，调用该函数，并输出序列。

2．编写判断 m 是否为素数的函数 fun(m)，若 m 是素数，则返回 True，否则返回 False，并利用它输出 2~100 之间所有的素数。

3．编写函数 fun(x)，即给定正整数 x，返回其逆序数，例如 1234 的逆序数是 4321，用这个函数输出 1000~9999 之间所有的回文数。（回文数是指顺读和倒读都相同的数，如 5、151、3553 等）

提示：对给定的整数，可以通过不断与 10 相除取余获得其每个数字位，追加到一个列表中，然后将列表中的数字遍历，和值不断乘 10 加新项的方式实现逆序。

4．随机产生 30 个成绩（0~100 之间）放入列表 a 中，设计一个函数 MyFun，将 a 传递给它，再为函数设定一个默认值参数 grade=5。grade 传递 5,4,3,2,1，分别统计优、良、中、及格和不及格的人数。不传递值时，统计优秀者人数。请分别使用不指定关键字、指定关键字，不给默认值参数值三种方式调用函数。

5．编一函数 s(x)，求级数和

$$s = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots\cdots$$

当最后一项的绝对值小于 10^{-6} 时结束。

6．编写二分法求高次方程根的函数，求解方程 $2x^3-4x^2+3x-6=0$ 在区间[-10,10]之间的根。

7．编写函数返回 100~999 之间的所有水仙花数的列表。所谓水仙花数是指一个 3 位数，它的每个位上的数字的 3 次幂之和等于它本身。

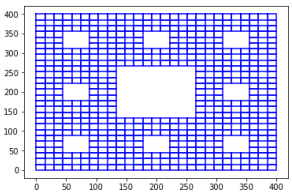
8．定义递归函数 f(n)，其功能是返回斐波那契数列的第 n 项（假定斐波那契数列的前两项是 0 和 1）

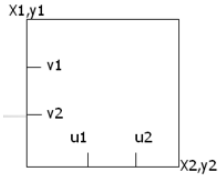
9．将十进制的正整数 d 转换为 r 进制整数(r=2-9),可以分别取商 Q=d//r 和余数 L=d % r，将 Q 再次与 r 求商取余，直到 Q 等于 0 结束。将每次求得的余数 L，按先后次序从右向左排列，就是转换得到的 r 进制的数。请用递归函数，实现将 d 转化为 r 进制的整数，以字符

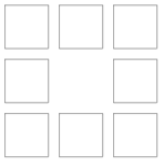
串方式返回结果。

10．编写递归函数，实现整数列表的逆转，并以 L[1,2,3]对其进行调用。

11．请参照 4.5 节树形分型图，用递归方法制作如下的四边形分型图。







提示：从 1 个大的四边形开始，每条边三等分，分成相同的 9 个四边形，然后除中间外的 8 个四边形重复上述过程，直到满足给定的条件底层为止（5 层）。调用设计的递归函数，给定最大四边形边长 500，递归 5 层结束

实验六　列表、元组、字典、集合

一、实验目的

1、 掌握不同数据序列的基本概念、定义、操作及应用场合

2、 掌握列表、元组的分片、列表对象方法的使用

3、 掌握内置函数 sum、len 对列表、元组的操作

4、 掌握列表、元组推导式的用法

5、 掌握字典的创建、遍历

二、实验内容

1、 用 ramdom 库，生成一个数值在-1~1 之间的 10*5 的矩阵，再用列表推导式将其转置。计算矩阵的转置与矩阵本身的乘积。

2、 生成 10 行 5 列的随机数矩阵。每个元素的值范围为：-1~1 之间。（提示：random 函数产生的值在 0-1 之间），再编写代码实现矩阵转置。

3、 输入一个英文句子，统计其中每个字母出现的次数，组合成一个字典。例如{"A":5,"B":0}。要求：大小写字母相同处理。

4、 请编写程序，从键盘上一次输入 10 个数，数间以空格间隔，用 map 函数将数据整理成一个数值列表输出。

5、 建立一个学生姓名与手机号的字典，要求输入格式为：张斌 13401279012，数据自行定义。输入-1 时结束输入过程。然后再通过一个循环，实现通过输入姓名查找手机的功能，输入“xxx”则查找结束

6、 生成一个含有"A"~"Z"26 个字母的列表，再生成一个列表：[1,2,...,26]，将两个列表结合生成以字母为键、数字为值的字典。

7、 用字典完成程序,提示用户输入电话号码,并用英文单词形式显示数字。例如:输入 138 显示：one three eight

实验七　文件

一、实验目的

1．掌握文件文件的特点和使用。

2．掌握文件的打开、关闭和读写操作。

3．学会在应用程序使用文件。

二、实验内容

1．打开一个文本文件 T1.txt，读出文件的内容，然后进行如下处理：

① 统计并且输出文件的行数；

② 将文件的大小写字母互相转换后写入文件 T2.txt，即大写的转成小写的，小写的转换成大写的，其余不变

2．打开如下图 2.7.1 所示的学生成绩文件.txt，统计平均分、最低分、平均分、最高分。

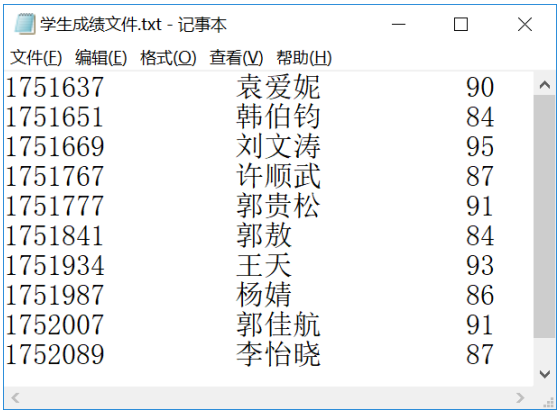


图 2.7.1　所示的学生成绩文件.txt

3．从互联网上下载《红楼梦》的某一回组成文本文件 hlm.txt，然后设计一程序统计林黛玉和贾宝玉两个人名在文件中出现的次数。

4．编写一个职工奖金处理程序。要求如下：

① 输入不超过 10 个职工的工号、姓名和奖金，保存在文件 JJ.txt。文件中的数据格式：

90813,张大海,1080.50 元

② 从 JJ.txt 中读出数据，然后按奖金从低到高排序，保存在文件 NewJJ.txt。

5．编写一个将两个文本文件的内容合并的程序。

实验八　面向对象程序设计基础

一、实验目的

1．掌握类的定义和实例化的方法。

2．学会设计简单的类。

3．掌握利用类的继承关系定义派生类的方法。

二、实验内容

1．设计一个楼房类（Building），包含楼的长、宽、层数及每平米单价属性，并具有求楼房的面积及总价等功能。最后编写利用该类的主程序。

2. 定义一个 identifier 类，具有一个属性 id，存放身份证号，以及如下函数：

（1）getyear 函数用于从身份证号码中提取出生年份；

（2）Disp 方法用于输出身份证号码。

编写利用该类的主程序。

3．设计一个矩形类（Rect），具有长、宽属性，类还具有求解并显示矩形的周长和面积的功能以及求两个矩形面积和的功能。

提示：假定有两个矩形 r1 和 r2，求两矩形面积之和的方法是 Sum，则调用方法为：

r1.Sum(r2)　　r2.Sum(r1)

4．设计一个 Person 类，具有如下的成员：

（1）属性：Name 和 Age，表示一个人姓名和年龄；

（2）方法：Disp，输出各数据成员

最后编写一个使用该类的测试程序。

5．从题 4 派生类 Teacher，并添加下列成员：

（1）属性：No 和 Ta，分别表示教师的工号和教龄；

（2）方法：NewDisp，输出所有成员的值。

最后编写利用该类的主程序。

实验九　数据可视化

一、实验目的

1. 掌握 Python 的绘图区和子绘图区的使用。
2. 熟悉常用图表的绘制方法。
3. 掌握图表常用组成元素的显示输出。

二、实验内容

1.创建一个 2 行 1 列的绘图区并在第 1 行第 1 列绘制函数 f(x)=x^2 的曲线图（x 的取值范围[-1,1]），在第 2 行第 1 列绘制函数 f(x)=1/x 的曲线图（x 的取值范围[0,1]），效果如图 2.9.1 所示。

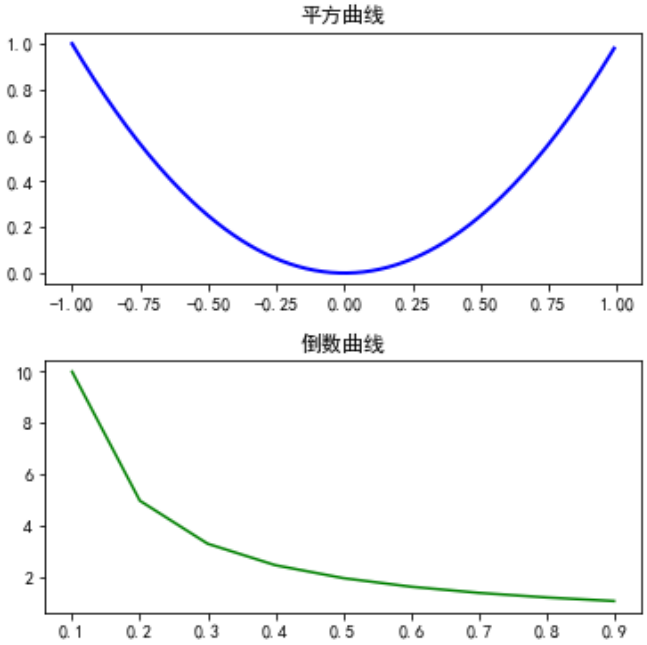


图 2.9.1 平方曲线与倒数曲线

2.调用 scatter 函数绘制正弦函数的曲线，请在曲线中添加一个表示 XY 的轴线，并在 X 轴方向输出刻度标记文本，效果如图 2.9.2 所示。

提示：利用 plot 函数绘制直线，然后在合适位置显示标记字符。

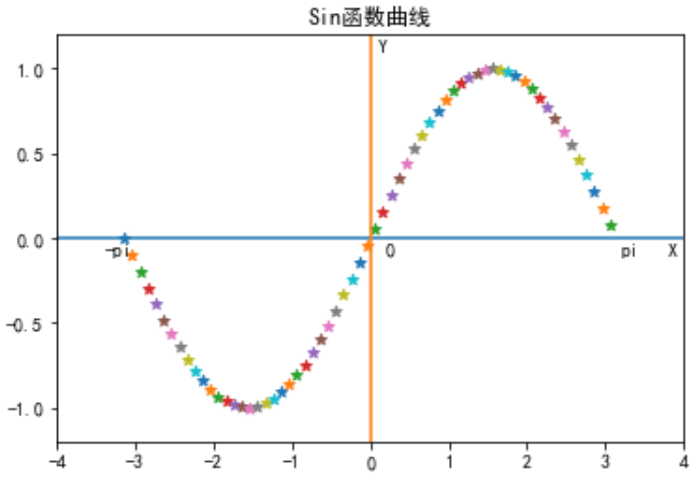
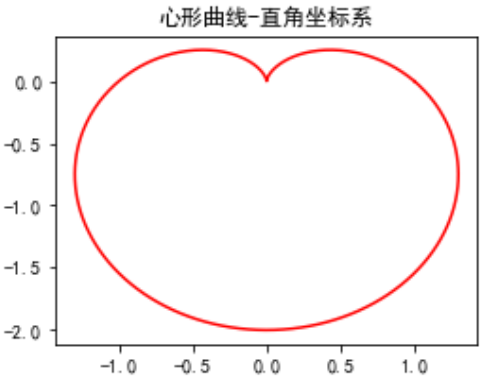


图 2.9.2　sin 函数的曲线图

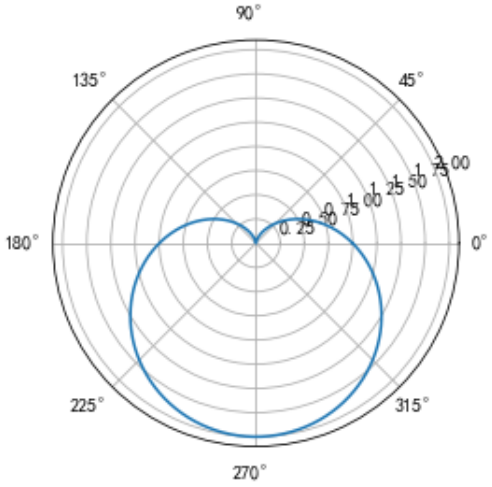
3. 调用 bar()函数绘制教材部分例 8.6 中的图像直方图效果，图像文件可以自己任意指定。

4. 绘制一个心形图案，参考数学函数为 ρ=a(1-sinθ)（θ 的取值范围[0,2π]），x=pcosθ，y=psinθ，在直角坐标系上的绘制图案效果如图 2.9.3（a）所示，在极坐标中的绘制效果如图 2.9.3（b）所示。

提示：极坐标绘制函数参考格式：plt.polar(theta,p)



(a) 直角坐标系



(b) 极坐标系

图 2.9.3　心形图案

5. 绘制一个饼图，显示你每个月各项消费支出的比例，消费支出主要包括：学习用品、日常用品、伙食费、通讯费、娱乐费和其他开支。

6. 读取“三国人名汇总.txt”中的人物名字，读取“三国演义.txt”的全部内容，先统计所有人物的名字在书本中出现的次数，并对出现次数超过 100 次的人物绘制一个柱状图，然后根据人物的词频大小绘制一个三国人名的词云图，效果如图 2.9.4 所示。（数据文件下载链接：https://pan.baidu.com/s/1b81lMIx3In-V64vs7bsHkw，提取码：jg41）

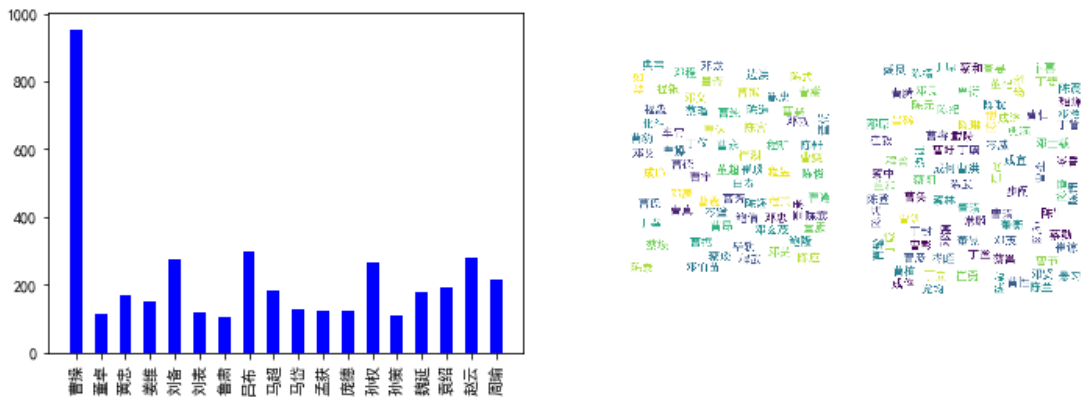


图 2.9.4　柱状图和词云图

7.神经网络模型在训练过程中将显示每一个训练回合所用的时间、训练集上的损失值、训练集上的准确率、验证集上的损失值和验证集上的准确率，把这些数据保存到变量 history,训练结束后对这些数据进行分析，图 2.9.5 显示了前面 5 个回合的训练数据。

```
In  [*]: history=sjbModel.fit(train_images1, train_labels1, epochs=50, batch_size=128,validation_data=(partial_train_images,partial_train_labels))
Epoch 1/50
157/157 [==============================] - 168s 1s/step - loss: 0.7225 - accuracy: 0.6906 - val_loss: 0.4364 - val_accuracy: 0.8307
Epoch 2/50
157/157 [==============================] - 165s 1s/step - loss: 0.3391 - accuracy: 0.8667 - val_loss: 0.3285 - val_accuracy: 0.8570
Epoch 3/50
157/157 [==============================] - 167s 1s/step - loss: 0.2395 - accuracy: 0.9071 - val_loss: 0.3291 - val_accuracy: 0.8577
Epoch 4/50
157/157 [==============================] - 169s 1s/step - loss: 0.1840 - accuracy: 0.9301 - val_loss: 0.1635 - val_accuracy: 0.9405
Epoch 5/50
157/157 [==============================] - 168s 1s/step - loss: 0.1465 - accuracy: 0.9449 - val_loss: 0.1102 - val_accuracy: 0.9615
```

图 2.9.5 神经网络模型训练过程

现将 50 个训练回合产生的数据写入到文件 Epoch.txt 中。

```
#写入数据文件的参考代码如下：
f=open(r'.\epoch.txt',"w")
loss = history.history['loss']                #训练集的损失
acc= history.history['accuracy']              #训练集的准确率
val_loss = history.history['val_loss']        #验证集上的损失
val_acc = history.history['val_accuracy']     #验证集上的准确率
for i in range(len(acc)):
    f.write(str(loss[i]) + '\t')
    f.write(str(acc[i])+ '\t')
    f.write(str(val_loss[i])+ '\t')
    f.write(str(val_acc[i])+ '\t')
    f.write('\n')
f.close()
```

用记事本中打开 Epoch.txt，前面 5 行数据的显示结果如图 2.9.6 所示，其中第 1 列为训练集的损失值，第 2 列为训练集的准确率，第 3 列为验证集上的损失值，第 4 列为验证集上的准确率。

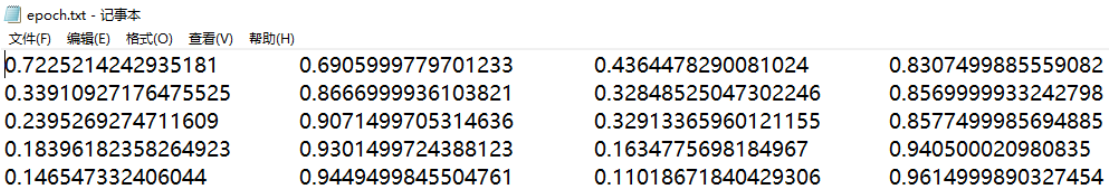


图 2.9.6　训练产生的部分数据

下面请读入 Epoch.txt 文件中的数据，可视化显示训练集与验证集的准确率及损失值的变化趋势，可视化显示的结果将有助于判别本次训练的神经网络模型其拟合情况，输出结果如图 2.9.7 所示。(Epoch.txt 文件的下载链接为 https://pan.baidu.com/s/1KukeCMtSkAhzgDsRVbtwzg，提取码为：iqjy)

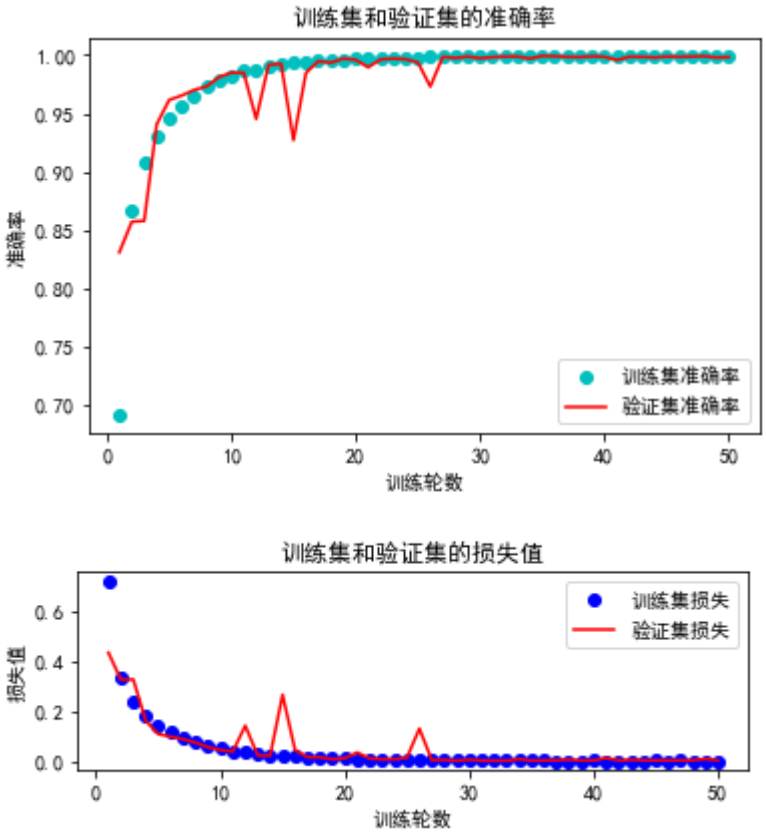


图 2.9.7　训练集与验证集的准确率与损失值可视化结果